

FM-Agent 第二次增量自验证 Bug 清单

生成口径：依据本次 incremental run 写回的 `logic_verification_results/`、`bug_validation/summary.json`、逐条 validator 报告与 extracted-function [SPEC] 汇总。
`confirmed` 仅表示 probe 复现了“实现与生成 SPEC 不一致”，不等于已经确认产品源码存在真实 bug。

总括

本轮是从 `95fd9ed9d4d147aeb6f37cf8aa78d4c908f572f2` 到 `20748637e3dde2ba5b17eeb3812be08f0faf74cc` 的真正增量验证。流水线正常结束并写回 target，耗时 **6247.88 秒**（约 **1 小时 44 分 08 秒**）；运行期间未发现 connection error、timeout、SQLite lock、Python traceback 或自嵌套调用 FM-Agent 的 probe。

项目	本轮结果
基线提交	<code>95fd9ed9d4d147aeb6f37cf8aa78d4c908f572f2</code>
当前提交	<code>20748637e3dde2ba5b17eeb3812be08f0faf74cc</code>
变更源文件	14
新增 / 修改 / 删除函数	16 / 26 / 0
当前 extracted functions	396
与本轮 intent 相关的函数	122
重新生成或更新的 SPEC	91
进入 reasoner 的函数	98
MATCH / MISMATCH / ERROR	25 / 73 / 0
已产出 bug-validator 结论	73 / 73
confirmed / not_confirmed / error	64 / 9 / 0

Reasoner 共产生 **73** 个 bug 候选。Bug validator 已对全部候选形成结果：**64 confirmed**、**9 not_confirmed**、**0 error**，覆盖率为 **100.00%**。原先未落盘的 Go `batch_extract` 已通过定向重跑补齐，并在第一次尝试中得到 `confirmed`。

将生成 SPEC、前置条件、reasoner 推导 POST、源码/调用方和 probe 一起人工复核后，73 条 MISMATCH 可按与 `bug_list.md` 一致的口径分层如下：

审计类别	数量	占 MISMATCH	含义
推理误判	30	41.10%	代码读取错误、反例违反真实输入约束、错误 mock/callee 语义；包含 validator 已 <code>not_confirmed</code> 的 9 项
SPEC 错误	21	28.77%	差异可能复现，但 SPEC 自造了不存在的行为、异常或格式约束
契约待确认	15	20.55%	实现与 SPEC 确有差异，但当前仓库证据不足以判定应当采用哪一种行为
实现缺陷候选	7	9.59%	有独立源码不变量、真实调用链或下游可观察影响支持，值得先补回归测试

因此，高置信度误报下界（推理误判 + SPEC 错误）为 **51/73 = 69.86%**，与全量审计的 **69.81%** 几乎一致，说明高误报主要是 SPEC/推理/validator 方法的系统性问题，而不是本次增量提交突然引入了大量缺陷。若把

尚无产品契约支持的 15 条“契约待确认”也按不可直接报 bug 处理，则当前不应直接修源码的比例为 $66/73 = 90.41\%$ ；可直接进入修复/补测候选队列的为 $7/73 = 9.59\%$ 。

Validator 的 64 条 **confirmed** 中，人工复核为 21 条推理误判、21 条 SPEC 错误、15 条契约待确认和 7 条实现缺陷候选。因而即使只看 **confirmed**，高置信度误报仍至少为 $42/64 = 65.63\%$ 。原因是 validator 通常把同一份 LLM SPEC 当作 expected：它能证明“实现与 SPEC 不同”，却没有独立证明 SPEC 就是正确的产品契约。

为便于按编号人工复核，分类索引如下：

- 推理误判 (30)：002、005、006、010、012、013、018、020、024、025、026、027、029、030、032、037、039、040、044、046、047、048、051、052、053、056、058、060、061、064。
- SPEC 错误 (21)：001、003、007、008、014、015、016、021、022、028、033、034、041、049、054、055、057、068、069、070、072。
- 契约待确认 (15)：004、009、011、017、019、023、031、035、038、042、043、050、059、062、063。
- 实现缺陷候选 (7)：036、045、065、066、067、071、073。具体优先级、独立依据和代码位置见文末“人工复核追加：最可能的实际 Bug”。

解读边界

- 本文完整保留 FM-Agent 生成的 SPEC 和验证证据，但不把 SPEC 自动视为产品 ground truth。
- confirmed** 表示 probe 支持该 SPEC/实现差异；仍需用 README、调用方、维护者意图或独立测试审查 SPEC 是否正确。
- not_confirmed** 表示本轮 probe 未能支持原 mismatch，通常应优先视为推理误判或 SPEC 证据不足。
- 本轮只验证 98 个增量相关/受影响函数，因此不能把这份文档当作 396 个函数的全量验证报告。

Validator 结果索引

编号	Bug ID	审计类别	Validator	触发摘要
001	config-py--Settings::settings_customise_sources	SPEC 错误	confirmed	_LayeredSource.call() 省略了 p Field 默认值，因此像 inject.id 这样的字段是在 settings_customise_sources 返回 source 之外解析的。
002	config-py--_LayeredSource::__call__	推理误判	not_confirmed	_LayeredSource.call 直接返回而不过滤模型默认值，但 self._ TOML + 环境变量填充，因此输出不会出现模型默认值。
003	config-py--_LayeredSource::__init__	SPEC 错误	confirmed	当 path 不存在时，诊断信息打印指定的路径（可能是相对路径），而范要求的绝对路径。
004	main-py--run_pipeline	契约待确认	confirmed	run_pipeline 无条件调用 generate_topdown_layers()，该 resume 参数，当 resume=True fm_agent/ 存在时，会导致先前阶段被重新执行。
005	src--cli_backend-py--build_agent_command	推理误判	confirmed	当 files=['a.txt','b.txt'] 时， AgentCommand.argv 缺少文件

编号	Bug ID	审计类别	Validator	触发摘要
				志，违反了规范要求每个文件路径的上下文附加到 backend 调用的规范。
006	src--cli_backend-py--cli_effort	推理 误判	confirmed	settings.llm.effort 是一个 bytes 类型（b'hello'）；.strip() 返回 bytes（b'hello'），而不是 str，违反了规范的返回类型。
007	src--cli_backend-py--resolve_model_backend	SPEC 错误	confirmed	settings.llm.backend 设置为无法识别的字符串 'foobar' —— _normalize_backend 会返回 None，导致 resolve_model_backend 接收一个非规范标识符。
008	src--domain_knowledge-py--collect_domain_knowledge_paths	SPEC 错误	confirmed	在 cli_paths 中传入一个不存在的路径会导致 ValueError，而不是按照预期跳过该路径。
009	src--domain_knowledge-py--resolve_domain_knowledge_paths	契约 待确认	confirmed	包含 '..' 的相对路径遍历符号链接时，os.path.abspath 按词法将其规范化，因此不存在不存在的路径，即使原始候选路径在文件系统中存在，也导致虚假的 ValueError。
010	src--domain_knowledge-py--stage_domain_knowledge_files	推理 误判	not_confirmed	规范要求无论在哪种平台都使用正斜杠作为路径分隔符；list_staged_domain_knowledge_files 已经在第 124 行执行 replace(os.sep, '/')，因此不存在 bug。
011	src--entry_reasoning_pipeline-py--_extracted_file_to_source_rel	契约 待确认	confirmed	hyphen>0 守卫错误地将以连字符开头的子目录组件（例如 -cpp）排除在识别源文件子目录之外，导致回退返回组件名称而不是正确的源文件名。
012	src--entry_reasoning_pipeline-py--_fqfn_to_ident	推理 误判	confirmed	当最右侧的源文件组件是 FQN 时，parts[i+1:] 为空，因此 parts[i+1:].join(()) 返回空字符串，违反了返回非空字符串的要求。
013	src--entry_reasoning_pipeline-py--_select_functions_by_source	推理 误判	not_confirmed	Bug 声明断言该函数会丢失返回的函数列表，但源代码检查确认在空的 parts 守卫中存在显式 raise，并在函数列表返回前使用显式 return 语句。
014	src--env_check-py--_check_codegraph_version	SPEC 错误	confirmed	当 codegraph 二进制执行成功但无法写入 stdout 时，空字符串在 'if not good' 守卫中为真，导致代码报告 'not installed' 错误，要求的版本不匹配错误。
015	src--extract-py--run_extraction	SPEC 错误	confirmed	is_file_ready 要求严格按照顺序写入文件，但 SPEC + 2 个 INFO 标记，因此每个文件被 2 个 SPEC + 1 个 INFO 的文件被写入，导致文件被覆盖，这违反了规范中“每个文件行”就足够”的要求，[SPEC] 和 [INFO] 标记行”就足够了。
016	src--file_utils-py--_get_phase_files	SPEC 错误	confirmed	当提取函数子目录包含嵌套子目录时，os.walk 先产生根级别文件，再产生子目录文件，从而生成的文件总列表未按规范要求的全局按文件名排序。
017	src--generate_topdown_layers-py--_collect_phase_files	契约 待确认	confirmed	当源文件的 basename 以点开头（例如 '.hidden'）时，last_dot 等于 0，导致无法正确识别源文件。

编号	Bug ID	审计类别	Validator	触发摘要
				> 0 守卫失败，因此代码会查找 '.hidden' 而非规范正确的 '-hidden'。
018	src--generate_topdown_layers-py--_strip_comments_from_source	推理 误判	not_confirmed	验证工具声称该函数只屏蔽字符，而不屏蔽注释。通过对 Python # 和 C /* */ 注释的实证测试，所有均被正确地替换为空格。
019	src--git-py--frozen_worktree	契约 待确认	confirmed	git add -A 会跳过匹配 .gitignore 文件，因此被 gitignored 的未跟踪文件从快照中省略，而规范要求捕获跟踪文件。
020	src--git-py--frozen_worktree::_git	推理 误判	confirmed	_git() 硬编码了 check=True, capture_output=True, text=True，**kwargs 传递给 subprocess.run()，kwargs 中传入这些键中的任何一者为重复的关键字参数引发 TypeError。
021	src--incremental_reasoner-py--_extracted_files_by_method	SPEC 错误	confirmed	访问返回的 defaultdict(list) 上缺少修改返回的列表会永久向映射中值对，违反了规范中“对返回列表不得影响映射”的要求。
022	src--incremental_reasoner-py--_reconcile_extracted_dir	SPEC 错误	confirmed	对不可写子目录中的文件调用 os.remove() 会引发 PermissionError，即终止函数并使文件系统保持部分状态——一个孤立文件被删除，且不存在。
023	src--incremental_reasoner-py--_remove_stale_extracted	契约 待确认	confirmed	_remove_stale_extracted 对每个调用 _reconcile_extracted_dir，移除过时文件但从不移除 func_dir，修剪其上层的空父目录；当多个源文件共享一个公共父目录时，保留在原地。
024	src--incremental_reasoner-py--_update_specs_for_intent	推理 误判	not_confirmed	seed 集合是由 changed_targets(seed.update) 和 relevant_rel_files 填充的；逻辑验证器遗漏了 seed.update(changed_targets.k) 调用。
025	src--incremental_reasoner-py--collect_relevant_function_scope	推理 误判	not_confirmed	Bug 声明 asserted changed_function_scope 并未被纳入；代码检查在第 1 行显示 or any() 子句确实存在，probe 确认即使 LLM 评估返回空选择包含已更改文件的模块。
026	src--incremental_reasoner-py--run_incremental_pipeline	推理 误判	not_confirmed	code_evidence 只引用了目录删除，遗漏了紧随其后的人工产物删除代码 (231-243 行)，该代码通过 glob.glob() 和 os.remove 删除匹配 select_relevant 和 spec_update_* 模式——规范已得到满足。
027	src--languages--c-py--batch_extract	推理 误判	confirmed	batch_extract 使用真值检查 (if None)，显式 None 检查，因此一个非 None

编号	Bug ID	审计类别	Validator	触发摘要
				值的 CodeGraphExtractor 会错 {} 而非提取出的函数。
028	src--languages--c-py--function_spans	SPEC 错误	confirmed	对于有效的 proj_dir 和一个没有文件，function_spans 返回 None 列表，因为对于没有定义的文件 get_function_spans 返回 None，码直接将其传递出去。
029	src--languages--codegraph-py--CodeGraphExtractor::get_functions_by_file	推理 误判	confirmed	向 get_functions_by_file 传递一个 proj_dir 会在返回的字典中产生和系统路径作为键，而不是规范要路径。
030	src--languages--codegraph-py--_bare_function_name	推理 误判	confirmed	当 tail 以 'operator' 开头但其余部含连续的运算符符号（例如 'operatorFoo'）时，代码会落入并返回 'operatorFoo'，而非规范 'operator'。
031	src--languages--codegraph-py--_codegraph_cmd	契约 待确认	confirmed	当 settings.codegraph.bin_dir 是且该路径上存在可执行的 'codegraph' 文件时，_codegraph_cmd() 返回而非绝对路径。
032	src--languages--codegraph-py--_extraction_ident	推理 误判	confirmed	仅包含空白符的作用域限定符组 'foo:: ::func') 从 _bare_function 生空字符串，这些空字符串通过 canonicalize 并在结果中产生空的组件，违反了规范的非空组件
033	src--languages--codegraph-py--_node_fqn_map	SPEC 错误	confirmed	基于 0 的计数器 'c' 直接用作后续要求第 k 个重复使用 1 索引 _k 次出现得到 _1 而非 _2。
034	src--languages--codegraph-py--_qualified_parts	SPEC 错误	confirmed	函数在提取作用域组件之前会剔 qualified_name 中的空白符，因前缀中的前导空白会被丢失： qualified_name=' bar::foo', name 返回 ['bar', 'foo'] 而不是规范期望 'foo']。
035	src--languages--codegraph-py--_warn_on_codegraph_version_mismatch	契约 待确认	confirmed	代码只从配置的版本中去除前导去除命令输出中的 v，导致当两义上相同时（例如配置的 'v1.0' 'v1.0'）发出虚假的 WARNING。
036	src--languages--codegraph-py--try_codegraph_init	实现 缺陷 候选	confirmed	当 force=True 且 .codegraph/codegraph.db 存在 codegraph 可执行文件不在 PATH 中 shutil.rmtree() 会在可执行文件被删除 .codegraph 目录，违反了规范执行文件缺失时不进行文件修改
037	src--languages--cpp-py--function_spans	推理 误判	confirmed	传递 None 作为 proj_dir 导致 CodeGraphExtractor.from_proj_dir os.path.abspath(None) 引发 TypeError 该异常向上传播而非按规范要求 None。

编号	Bug ID	审计类别	Validator	触发摘要
038	src--languages--erlang-py--ElpClient::_handle_server_message	契约待确认	confirmed	当缺少 'status' 键时，params.g 返回 None，覆盖了 _status 而非变。
039	src--languages--erlang-py--ElpClient::_next_message	推理误判	confirmed	_next_message 返回任何非 BaseException 的队列项而不检查为 dict，因此非 dict 的 JSON 值（表）会被返回，违反了规范。
040	src--languages--erlang-py--ElpClient::_send	推理误判	confirmed	stdin 是一个非 None 对象但处于不可用状态；None 检查通过，因此 RuntimeError，而是 write/flush ValueError。
041	src--languages--erlang-py--ElpClient::_wait_for_response	SPEC 错误	confirmed	error=null 的响应导致真值检查返回 result 而非引发 RuntimeError。
042	src--languages--erlang-py--ElpClient::initialize	契约待确认	confirmed	截止时间是在 request() 之后计算而不是在调用入口处；请求延迟超过超时窗口，超出规范限制。
043	src--languages--erlang-py--ElpClient::open_document	契约待确认	confirmed	Path.resolve() 会跟随符号链接，path 是一个符号链接时，open_ 发送的是目标 URI，而不是 path URI。
044	src--languages--erlang-py--ElpClient::request	推理误判	confirmed	将 _MAX_CONTENT_MODIFIED 设置为 0 会导致 range(0) 产生零进而落入 raise AssertionError("unreachable"), 规范允许的结果（return、Timeo RuntimeError）。
045	src--languages--erlang-py--_SourceIndex::build	实现缺陷候选	confirmed	对 source='hello\\vworld', splitlines(keepends=True) 错误。制表符 (\v) 分割，而该字符并非产生了 2 行而不是 1 行。
046	src--languages--erlang-py--_analyze_project_uncached	推理误判	not_confirmed	当没有 .erl 文件时，ErlangAnalysis(functions={}, ed 然通过数据类字段 (default_fact 具有 spans={} —— 逻辑验证器类默认值的情况下分析了提取的生了误报。
047	src--languages--erlang-py--_elp_argv	推理误判	confirmed	当 settings.erlang.command 为 时，调用 .strip() 会引发 Attribut 而非按照规范要求返回一个有效列表。
048	src--languages--erlang-py--_source_for_range	推理误判	not_confirmed	LSP 范围中，同一行的 start cha > end character 1；Python 切片 source[start:end] 返回空字符串，范隐含的空 span。
049	src--languages--erlang-py--_timeout_seconds	SPEC 错误	confirmed	在 ELP_TIMEOUT_SECONDS 情况下，settings.erlang.timeou _DEFAULT_TIMEOUT_SECON

编号	Bug ID	审计类别	Validator	触发摘要
				vs 180) 不一致; 函数返回的是的值, 而非规范要求的默认常量
050	src--languages--go-py--batch_extract	契约待确认	confirmed	当 codegraph 数据库包含带有父 (../) 的 file_path 条目时, p 之外的路径可能出现在返回的字
051	src--languages--go-py--function_spans	推理误判	confirmed	function_spans 在调用 get_function_spans 时硬编码了 'go', 导致对于被 codegraph 索引函数定义的非 Go 文件返回 Non
052	src--languages--javascript-py--function_spans	推理误判	confirmed	function_spans 返回来自 cg.get_function_spans 的原始列 start_idx 排序; 当后端返回无序时, 将违反规范中按 start_idx 升
053	src--languages--rust-py--batch_extract	推理误判	confirmed	batch_extract 原样透传来自 get_functions_by_file 的空列表; 行过滤, 违反了规范中每个值必
054	src--languages--rust-py--function_spans	SPEC 错误	confirmed	function_spans 委托给 get_function_spans, 后者同时: 'function' 和 'method' 种类, 但顶层函数声明; impl 块内的方法而泄漏。
055	src--languages--typescript-py--batch_extract	SPEC 错误	confirmed	batch_extract 委托给 get_functions_by_file, 后者返回函数在内的所有函数, 但规范要求函数定义。
056	src--llm_client-py--_inject_targets	推理误判	confirmed	INJECT_HOST 环境变量设置为的 hosts, 但 settings.inject.hos 函数返回 [] 而不是解析后的 env
057	src--llm_client-py--_messages_to_anthropic	SPEC 错误	confirmed	当存在 3 条以上系统消息时, 代 92 行的迭代 .strip() 过早删除了内容的尾部空白符, 而规范要求有内容, 最后只 strip 最终结果。
058	src--llm_client-py--_metadata_body	推理误判	confirmed	在两次 _metadata_body() 调用 settings.inject.id 会导致返回的 u 生变化, 违反了规范的稳定性保
059	src--llm_client-py--_retry_create	契约待确认	confirmed	传入导致 client.chat.completion 内部引发 TypeError 的输入会触发的 except Exception 处理器, 将时性错误重试 5 次, 而不是立即
060	src--llm_client-py--_stable_user_id	推理误判	confirmed	当 settings.inject.id 是真值非字(整数 5) 时, Python 的 or 原样通串值, 而不是字符串, 违反了规
061	src--opencode_trace-py--_opencode_provider_config	推理误判	confirmed	当 settings.llm 为 None 时, 函数 None.api_key 引发 AttributeError 按规范要求返回 None。

编号	Bug ID	审计类别	Validator	触发摘要
062	src--opcode_trace-py--_start_opcode_process	契约待确认	confirmed	当 command 没有 stdin 文本（command_stdin 返回 None） subprocess.Popen 收到 stdin=None 致子进程继承父进程的 stdin fd，规范要求断开。
063	src--pipeline_setup-py--_deduplicate_phases	契约待确认	confirmed	当一个模块的 source_files 包含子目，且该文件被完全删除时，removed_files 会保留这些重复项。重。
064	src--pipeline_setup-py--_phase_plan_complete	推理误判	confirmed	当 phases.json 是一个包含有效 JSON 的 FIFO（非常规文件）时，_phase_plan_complete 返回 True，是规范要求的 False。
065	src--pipeline_setup-py--_phase_plan_schema_errors	实现缺陷候选	confirmed	打开包含无效 UTF-8 字节的文件时，会引发 UnicodeDecodeError，而非返回错误列表。
066	src--pipeline_setup-py--_phases_cover_current_sources	实现缺陷候选	confirmed	phases.json 中的绝对文件路径使用 os.path.join() 绕过 os.path.exists() 使得 proj_dir 之外的文件被接受，规范要求它们必须在 proj_dir 下。
067	src--pipeline_setup-py--_run_generate_phases	实现缺陷候选	confirmed	当 is_incremental=True 且 phases.json 的 mtime 改变但 _phases_cover_current_sources 为 False 时，OR 运算符将 phase_plan_ready 设为 True，而非等待了不完整的 phases.json。
068	src--pipeline_setup-py--_setup_outputs_complete	SPEC 错误	confirmed	phases.json 存在但包含无效 JSON 时，_phase_plan_complete 检查 JSON 有效性+模式，但规范仅要求文件存在，_setup_outputs_complete 返回 True，而非 True。
069	src--prompts-py--_generate_block_post_condition	SPEC 错误	confirmed	调用 _generate_block_post_condition 时，_llm_json_call 引发 RuntimeError（网络故障）—— 异常未被捕获而传播，而非按规范要求返回 None。
070	src--prompts-py--_parse_spec_check_json	SPEC 错误	confirmed	仅包含空白符的 counterexample/offending_statement 字符串绕过了 MATCH 判定 Value 检查，因为 _nonempty_string 使用 bool(value.strip()) 而非 len(value)
071	src--reasoner-py--_compute_brace_depth_per_line	实现缺陷候选	confirmed	未跟踪多行字符串：第 0 行出现双引号会导致后续行的花括号被排除。
072	src--reasoner-py--_split_into_blocks_braced	SPEC 错误	confirmed	C 函数体花括号在第二行：entry_depth 但第一个块从深度 0 开始，违反分段从 entry depth 开始的要求。
073	src--verification-py--streaming_reasoner	实现缺陷候选	confirmed	在所有 spec_procs 完成时的提示中，会对其余预期文件调用 is_file_ready，就绪文件被跳过且从未提交进行

逐条 MISMATCH 与完整 SPEC

每条均给出本轮生成 SPEC 的全文、reasoner 认定的冲突，以及 bug validator 的实际结论。链接均相对于 `fm_agent/bug_incremental_list.md`。

config-py

INCR-MISMATCH-001 — `config-py--Settings::settings_customise_sources`

- 人工审计：**SPEC 错误**。
- Validator：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：`config-py/Settings::settings_customise_sources.py`。
- Reasoner 结果：`logic_verification_results/config-py/Settings::settings_customise_sources.json`。
- 详细报告：`config-py--Settings::settings_customise_sources.md`。
- Probe：`probe_config-py--Settings::settings_customise_sources.py`。

完整生成 SPEC

```
[SPEC]
单元: config.py

Settings.settings_customise_sources(cls, settings_cls, init_settings, env_settings,
dotenv_settings, file_secret_settings)

前置条件：
- settings_cls 是 BaseSettings 的子类，并通过 pydantic 的 Field 注解声明了类型化字段
- init_settings 是一个 PydanticBaseSettingsSource，携带传递给 Settings() 的关键字参数值
- _CONFIG_PATH 是一个 pathlib.Path，指向 fm-agent.toml 配置文件

后置条件：
- 返回一个包含两个 PydanticBaseSettingsSource 实例的二元组，定义了完整的字段解析优先级链
- 第一个元素（init_settings）具有最高优先级；通过 Settings(...) 关键字参数提供的任何字段值都将直接使用，不会被任何其他来源覆盖
- 第二个元素是一个 _LayeredSource，按以下优先级递减顺序解析每个字段：
    1. 名称作为 _ENV_MAP 中的键出现的过程环境变量
    2. _CONFIG_PATH 处的 TOML 文件中的值
    3. settings_cls 中该字段声明的 pydantic Field 默认值
- env_settings、dotenv_settings 和 file_secret_settings 来源被丢弃；它们不参与字段解析
- Settings 模型的每个字段都恰好通过返回的两个来源之一进行解析

[SPEC]
```

Reasoner 差异

- SPEC 声称：

```
- 返回一个包含两个 PydanticBaseSettingsSource 实例的二元组，定义了完整的字段解析优先级链
- 第一个元素（init_settings）具有最高优先级；通过 Settings(...) 关键字参数提供的任何字段值都将直接使用，不会被任何其他来源覆盖
- 第二个元素是一个 _LayeredSource，按以下优先级递减顺序解析每个字段：
    1. 名称作为 _ENV_MAP 中的键出现的过程环境变量
    2. _CONFIG_PATH 处的 TOML 文件中的值
    3. settings_cls 中该字段声明的 pydantic Field 默认值
- env_settings、dotenv_settings 和 file_secret_settings 来源被丢弃；它们不参与字段解析
- Settings 模型的每个字段都恰好通过返回的两个来源之一进行解析
```

- 推导出的实际行为：

```
该函数返回一个元组 `(init_settings, _LayeredSource(settings_cls, _CONFIG_PATH))`。第一个元素是同一个 `init_settings` 对象（携带有构造函数关键字参数的 `PydanticBaseSettingsSource`）。第二个元素是一个 `_LayeredSource` 实例，当被调用时，返回一个将顶级节名称（类型为 `str`）映射到字段名及其解析值子字典的 `dict`。在该 `_LayeredSource` 结果中，值的获取遵循以下优先级（从高到低）：(1) 通过 `_ENV_MAP` 查找对应 `(section, field)` 对的过程环境变量覆盖值；(2) 位于 `_CONFIG_PATH` 的 TOML 文件中的值。如果 `_CONFIG_PATH` 没有指向一个存在的文件，则不贡献任何 TOML 值，仅出现环境覆盖。关键的是，返回的元组配置 pydantic 设置以使用以下解析顺序：构造函数关键字参数（`init_settings`）> 环境覆盖（通过 `_ENV_MAP`）>
```

TOML 文件 > pydantic `Field` 默认值。形式上,对于 `settings\_cls` 的每一个字段 `f`,在设置构建过程中分配的最终值 `v\_f` 满足: `v\_f = init\_settings.get(f) if f in init\_settings else ( LS.get(section, f) if (section, f) LS.data else default\_field\_value(f) )`,其中 `LS.data` 表示 `\_LayeredSource` 产生的合并字典,`default\_field\_value(f)` 是 pydantic `Field` 注解上声明的 Python 级别的默认值。

- 代码证据:

第 11 行: return (init_settings, _LayeredSource(settings_cls, _CONFIG_PATH))

- 触发条件:

该规约要求 `\_LayeredSource` 使用 环境变量 > TOML > Field 默认值 的优先级解析每个字段,并强制每个字段都通过返回的两个来源之一进行解析。然而,代码中的 `\_LayeredSource` 仅返回环境变量和 TOML 值,并省略了 Field 默认值,导致仅依赖其默认值的字段在返回的两个来源之外进行解析,从而违反了规约。

Bug validator

- 触发摘要: `\_LayeredSource.call()` 省略了 pydantic Field 默认值,因此像 inject.id (默认值为 "") 这样的字段在 settings_customise_sources 返回的两个来源之外被解析。
- Probe 标准输出:

CONFIRMED – inject.id 的 Field 默认值 '' 未出现在 _LayeredSource.__call__() 中 (来源中的节: ['llm', 'runtime', 'scope', 'erlang', 'codegraph']),但 Settings().inject.id 却解析为 ''. Field 默认值在返回的两个来源之外被解析。

INCR-MISMATCH-002 — config-py--_LayeredSource::__call__

- 人工审计: 推理误判。
- Validator: **not_confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: config-py/_LayeredSource::__call__.py。
- Reasoner 结果: logic_verification_results/config-py/_LayeredSource::__call__.json。
- 详细报告: config-py--_LayeredSource::__call__.md。
- Probe: probe_config-py--_LayeredSource::__call__.py。

完整生成 SPEC

```
[SPEC]
单元: config.py

_LayeredSource.__call__(self) -> dict

前置条件:
- 无。

后置条件:
- 返回一个字典,表示 Pydantic BaseSettings 模型的解析后配置。每个顶层键对应于一个 Settings 字段名;其关联值是该嵌套模型字段的子字段值的嵌套字典。
- 返回的字典只包含显式配置的值 (来源于 TOML 文件和/或环境变量);不包含任何模型级别的默认值。
- 对同一实例的多次调用返回同一个字典对象,内容不变。
- 调用总是成功;从不引发异常。
[SPEC]
```

Reasoner 差异

- SPEC 声称:

- 返回一个字典,表示 Pydantic BaseSettings 模型的解析后配置。每个顶层键对应于一个 Settings 字段名;其关联值是该嵌套模型字段的子字段值的嵌套字典。
- 返回的字典只包含显式配置的值 (来源于 TOML 文件和/或环境变量);不包含任何模型级别的默认值。

- 对同一实例的多次调用返回同一个字典对象，内容不变。
- 调用总是成功；从不引发异常。

- 推导的实际行为：

执行后，`__call__` 方法返回实例属性 `self._data` 的值，该属性预期为字典。`self._data` 的状态保持不变。形式化：
`\result == self._data`

- 代码证据：

```
第 1 行:    def __call__(self) -> dict:
第 2 行:        return self._data
```

- 触发条件：

代码直接返回 ``self._data``，没有过滤掉模型默认值，导致非显式配置的值可能出现在输出中。

Bug validator

- 触发器摘要：`_LayeredSource.call` 直接返回 `self._data`，没有过滤模型默认值，但 `self._data` 仅从 TOML 和环境变量填充，因此输出中永远不会出现模型默认值。
- Probe 标准输出：

```
NOT CONFIRMED — 没有模型默认值泄露；仅存在 TOML 值
Result: {'nested': {'count': 99}, 'llm': {'api_key': 'sk-88694ac2d6d84bef9b903056966b9d56',
'base_url': 'https://api.deepseek.com', 'backend': 'opencode', 'name': 'deepseek-v4-pro',
'effort': '', 'provider': 'deepseek'}, 'erlang': {'command': 'elp', 'timeout_s': '180'}}
```

INCR-MISMATCH-003 — `config-py--_LayeredSource::__init__`

- 人工审计：**SPEC 错误**。
- Validator：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：`config-py/_LayeredSource::__init__.py`。
- Reasoner 结果：`logic_verification_results/config-py/_LayeredSource::__init__.json`。
- 详细报告：`config-py--_LayeredSource::__init__.md`。
- Probe：`probe_config-py--_LayeredSource::__init__.py`。

完整生成 SPEC

```
[SPEC]
单元: config.py

_LayeredSource.__init__(self, settings_cls, path) -> None

前置条件：
- settings_cls 是 BaseSettings 的子类，具有通过 pydantic Field 注解声明的类型化字段
- path 是一个 pathlib.Path

后置条件：
- 将实例作为可调用对象调用会返回一个字典，其顶层键为字符串节名称，值为映射字符串字段名称到其值的字典
- 当 path 指向一个包含有效 TOML 的现有常规文件时，从文件解析出的每个键值对都存在于返回的字典中
- 当 path 不指向一个现有常规文件时，返回的字典中不会出现任何源自文件的条目，并且一条包含文件名和绝对路径的诊断消息会写入标准错误输出
- 对于每个已设置的支持的进程环境变量，返回的字典包含对应的（节，字段）条目；环境来源的条目会替换同一字段的任何文件来源条目
- 一个字段如果在返回的字典中没有条目——既没有来自文件也没有来自任何匹配的环境变量——则该字段在返回的字典中不存在，并且 pydantic 会将其解析为在 settings 模型上声明的 Field 默认值
[SPEC]
```

• SPEC 声称：

- 将实例作为可调对象调用会返回一个字典，其顶层键为字符串名称，值为映射字符串字段名称到其值的字典
 - 当 path 指向一个包含有效 TOML 的现有常规文件时，从文件解析出的每个键值对都存在于返回的字典中
 - 当 path 不指向一个现有常规文件时，返回的字典中不会出现任何源自文件的条目，并且一条包含文件名和绝对路径的诊断消息会写入标准错误输出
 - 对于每个已设置的支持的进程环境变量，返回的字典包含对应的（节， 字段）条目；环境来源的条目会替换同一字段的任何文件来源条目
 - 一个字段如果在返回的字典中没有条目——既没有来自文件也没有来自任何匹配的环境变量——则该字段在返回的字典中不存在，并且 pydantic 会将其解析为在 settings 模型上声明的 Field 默认值

• 推导的实际行为：

正常终止时（未引发异常）：

- 父类构造函数 `super().__init__(settings_cls)` 已成功完成，存储提供的 settings 类以供 settings 源后续使用。
- `self._data` 是一个根据以下规则组装的字典：
 1. 初始为空字典。
 2. 如果 `path` 指向一个现有常规文件（`path.is_file()` 为 `true`），则该字典被替换为通过 `tomllib.loads(path.read_text())` 将文件内容解析为 TOML 的结果。如果 `path` 不是一个文件，则向 `sys.stderr` 输出一条警告消息（"FM-Agent: {path.name} not found at {path}; using built-in defaults."），同时字典保持为空。
 3. 对于映射 `_ENV_MAP` 中的每个条目 `(env_name, (section, field))`，如果环境变量 `env_name` 存在（即 `os.environ.get(env_name)` 不为 `None`），则将其值存储在字典中 `data[section][field]` 下。如果 `data` 尚未拥有键 `section`，则在设置字段之前通过 `setdefault` 创建一个新的空字典。
- 该方法返回 `None`。

异常终止时（由 `super().__init__`、`path.is_file()`、`path.read_text()` 或 `tomllib.loads` 中的任何一个引发异常）：

- 异常传播出方法，实例 `self` 处于部分初始化状态。父构造函数可能已经完成（如果异常发生在此调用之后），但 `self._data` 未被赋值（或者如果 `__init__` 是在一个已经初始化的对象上调用，则保留任何原有值）。不会打印警告消息（除非异常发生在 `print` 调用之后，但实践中，文件读取/解析期间的异常会阻止进入 `print` 分支）。

形式化逻辑（Hoare风格）：

设 `_ENV_MAP` 为从环境变量名到（section, field）元组的映射。

定义 `_D_init` 为初始数据字典：

若 `path.is_file()` 且无异常发生，则 `_D_init = tomllib.loads(path.read_text())`；

否则 `_D_init = {}`（当路径不是文件时输出到 `stderr`）。

则在正常终止后：

```
self._data = _D_init { sec : { fld : os.environ[env] | (env, (sec, fld)) ENV_MAP
os.environ.get(env) None }}
```

其中 `` 通过嵌套更新合并字典：对于每个（env, (sec, fld)），如果 `_D_init` 没有键 `sec`，则添加为 `{}`；然后将 `fld` 设置为环境值。若发生异常 `E`，则后置条件为 `E` 被引发且 `self._data` 未定义。

• 代码证据：

```
第 11 行:         print(
                f"FM-Agent: {path.name} not found at {path}; using built-in defaults.",
                file=sys.stderr,
            )
```

• 触发条件：

规范要求诊断消息包含绝对路径，但代码打印给定的路径对象，该对象可能是相对路径，从而未能标识绝对路径。

Bug validator

- 触发摘要：当路径不存在时，诊断信息打印的是给定路径（可能为相对路径），而非规范要求的绝对路径。
- 探测标准输出：

```
CONFIRMED - diagnostic prints relative path instead of absolute path | stderr: 'FM-Agent:
nonexistent_config_test.toml not found at nonexistent_config_test.toml; using built-in
defaults.\n'
```

INCR-MISMATCH-004 — main-py--run_pipeline

- 人工审计：契约待确认。
- Validator: **confirmed**；尝试次数：1。
- 原始函数与 SPEC: `main-py/run_pipeline.py`。
- Reasoner 结果: `logic_verification_results/main-py/run_pipeline.json`。
- 详细报告: `main-py--run_pipeline.md`。
- Probe: `probe_main-py--run_pipeline.py`。

完整生成 SPEC

```
[SPEC]
单元: main.py

run_pipeline(proj_dir, resume, required_source_files, domain_knowledge_files, submodules,
one_phase, extra_call_edges_path, only_spec) -> None

前置条件:
- proj_dir 是一个非 None 的字符串; 如果它没有指向一个已存在的目录, 函数会打印诊断信息并调用 sys.exit(1)
- 如果 proj_dir 是一个目录, 但其中没有任何被流水线识别的扩展名文件, 函数会打印诊断信息并调用 sys.exit(1)
- resume 是一个真/假值; 当其为真且 proj_dir 下存在 fm_agent/ 时, 之前已完成的流水线工作将被保留, 仅执行剩余的工作
- domain_knowledge_files 为 None 或一个可迭代对象, 其中包含指向已存在 Markdown 文件的路径字符串
- submodules 为 None 或一个非空的可迭代对象, 其中包含相对于 proj_dir 的子目录名; 若提供, 则只处理这些子目录
- one_phase 是一个真/假值
- extra_call_edges_path 为 None 或一个指向 JSON 文件的路径, 该文件按引擎约定定义了补充的调用图边
- only_spec 是一个真/假值

后置条件:
- 成功 (正常返回): 整个流水线已对 proj_dir 下的所有源文件执行完毕
- 如果 only_spec 为真: 提取出的调用图中的每个函数都在其提取出的函数文件中预先添加了一个行为规约 ([SPEC] 块); 不执行任何验证或缺陷确认
```

Reasoner 差异

- SPEC 声明:

```
- 成功 (正常返回): 整个流水线已对 proj_dir 下的所有源文件执行完毕
- 如果 only_spec 为真: 提取出的调用图中的每个函数都在其提取出的函数文件中预先添加了一个行为规约 ([SPEC] 块); 不执行任何验证或缺陷确认
- 如果 only_spec 为假: 先生成规约, 然后每个有规约的函数都在 fm_agent/logic_verification_results/ 中有一个验证结果, 每个 MISMATCH 都在 fm_agent/bug_validation/ 中有一个缺陷确认报告
- proj_dir 下的 fm_agent/ 工作目录会被创建并填充; proj_dir 下 fm_agent/ 之外的文件不会被修改
- 如果 resume 为真且 fm_agent/ 存在, 之前已完成的阶段不会被重新执行; 如果 resume 为假或 fm_agent/ 不存在, 在开始前会移除所有先前的 fm_agent/ 内容
- 用户的领域知识文件在任何流水线阶段执行之前, 会被暂存到 fm_agent/spec_prompts/domain_context/user_knowledge/ 中
- 如果没有找到任何需要验证的函数 (空文件列表), 函数会提前返回, 不生成规约
- 在所有已配置的重试之后出现不可恢复的阶段失败: 打印诊断信息, 指明失败的阶段和跟踪目录, 然后调用 sys.exit(1)
- 流水线阶段依次执行: phases.json 生成 → 领域上下文生成 → 函数提取 → 规约生成 → (可选) 验证 → 缺陷确认
- 函数会为每个主要阶段的转换向 stdout 输出状态消息
- 在 only_spec 模式下, 最终摘要不会打印已确认缺陷的数量
```

- 推导的实际行为:

```
如果 os.path.isdir(proj_dir) 为 False, 程序打印一条包含 'proj_dir does not exist or is not a directory' 的错误信息并通过 sys.exit(1) 终止。否则, 如果 _has_source_code(proj_dir, submodules) 返回 False, 程序打印一条说明未找到源代码文件的错误信息并通过 sys.exit(1) 终止。否则函数继续: work_dir 被设为 os.path.join(proj_dir, 'fm_agent'), input_dir 设为 os.path.join(work_dir, 'extracted_functions'), output_dir 设为 os.path.join(work_dir, 'logic_verification_results'), script_dir 设为当前文件所在目录。extra_call_edges 通过 load_call_edges 从 extra_call_edges_path 加载 (返回一个字典或 None)。如果 resume 为真且 work_dir 存在, resume 保持为真并打印一条恢复消息; 如果 resume 为真但 work_dir 不存在, resume 变为 False。如果 resume 为假 (包括调整后), 则调用 _clean_previous_run(work_dir), 若存在 work_dir 树则移除。然后 os.makedirs(work_dir, exist_ok=True) 确保 work_dir 存在。stage_domain_knowledge_files 将 domain_knowledge_files (如果提供) 中任何已存在的 Markdown 文件复制到 work_dir/spec_prompts/domain_context/user_knowledge/ 中, 并返回一个项目相对路径列表; 如果该列表非空, 打印一条日志消息。然后流水线执行: _run_generate_phases (在不可恢复失败时可能调用 sys.exit(1))、_post_process_phases (如有必要会修改 phases.json)、_run_generate_domain_context (在不
```

可恢复失败时可能调用 `sys.exit(1)`、`collect_file_names` 以及 `generate_topdown_layers`。随后，对每个阶段和层级调用 `_run_spec_generation_batch` 在提取的函数文件中生成 [SPEC] 和 [INFO] 块，使用 `is_file_ready` 跳过已就绪的文件。验证由 `streaming_reasoner` 执行，除非 `only_spec` 为真，在此情况下验证被跳过。如果 `one_phase` 为真，仅处理第一个阶段；否则处理所有阶段。任何未处理的 I/O 或其他异常会向上传播。正常完成后，`work_dir` 包含完整的流水线产物：`phases.json`、领域上下文文件、`topdown` 层文件、已添加规约的提取函数文件，以及验证结果（如果未跳过）。程序不返回值。

形式化地，令 R 为满足前置条件的初始状态。则后置条件 Q 为：

```
Q (isdir(proj_dir) PrintedError('proj_dir does not exist') ExitWith(1))
  (isdir(proj_dir) _has_source_code(proj_dir, submodules) PrintedError('No source code
files found') ExitWith(1))
  (isdir(proj_dir) _has_source_code(proj_dir, submodules)
    (work_dir = proj_dir / 'fm_agent' ... [all assignments] ...
      (resume_effective = (resume isdir(work_dir) ? resume : False))
      (resume_effective clean(work_dir)) mkdir(work_dir)
      domain_knowledge_relpaths = stage(proj_dir, work_dir, domain_knowledge_files)
      (domain_knowledge_relpaths [] PrintedMsg)
      (generate_phases_succeeds ExitWith(1))
      (domain_context_succeeds ExitWith(1))
      (normal_termination (phases_artifacts specs_artifacts (only_spec ? no_verification :
verification_artifacts)))
    )
  )
```

- 代码证据：

第 27-32 行：`resume` 处理仅打印消息并可能将 `resume` 设为 `False`，但不会跳过后续调用的流水线阶段。即使 `resume` 为 `True` 且 `fm_agent/` 存在，代码仍无条件地继续执行 `_run_generate_phases`、`_run_generate_domain_context` 等，导致先前已完成的阶段被重新执行。

- 触发条件：

规约要求“如果 `resume` 为真且 `fm_agent/` 存在，先前已完成的阶段不会被重新执行”。条件 A 描述流水线总是运行 `_run_generate_phases` 及后续阶段，在恢复时没有任何检查来跳过它们。因此，对于 `resume=True` 且 `fm_agent/` 已存在的输入，代码会重新执行阶段生成（可能还有其他阶段），违反了规约。

Bug validator

- 触发摘要：`run_pipeline` 无条件调用 `generate_topdown_layers()`，而该函数没有 `resume` 参数，导致当 `resume=True` 且 `fm_agent/` 存在时，先前已完成的阶段被重新执行。
- Probe 标准输出：

CONFIRMED – `generate_topdown_layers` 缺少 `resume` 参数（参数列表=`['proj_dir', 'phase_numbers', 'extra_call_edges']`），在 `run_pipeline`（源文件第 113 行）中被调用时没有 `resume` 守卫，违反规约：“先前已完成的阶段不会被重新执行”。对比：`_run_generate_phases` 具有 `resume` 参数（`['proj_dir', 'work_dir', 'script_dir', 'is_incremental', 'resume', 'submodules']`）并在内部使用了 `_resume_skip`。

src/cli_backend-py

INCR-MISMATCH-005 — src--cli_backend-py--build_agent_command

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/cli_backend-py/build_agent_command.py`。
- Reasoner 结果：`logic_verification_results/src/cli_backend-py/build_agent_command.json`。
- 详细报告：`src--cli_backend-py--build_agent_command.md`。
- Probe：`probe_src--cli_backend-py--build_agent_command.py`。

完整生成 SPEC


```
[SPEC]
单元：fm_agent/extracted_functions/src/cli_backend-py/build_agent_command.py

build_agent_command(model, prompt, cwd, files=None, backend=None, effort=None) -> AgentCommand

前置条件：
- model 是一个非空字符串，用于标识一个LLM模型
- prompt 是一个非空字符串
- cwd 是一个指向已存在目录的路径
- files 要么是 None，要么是由文件路径字符串组成的列表（可以为空）
- backend 要么是 None，要么是一个字符串，标识期望的CLI后端
- effort 要么是 None，要么是一个字符串

后置条件：
- 有效的后端通过以下方式确定：当 backend 参数非 None 时，将其规范化（解析别名）；当 backend 为 None 时，读取配置的默认模型后端；哨兵值 "auto" 会解析为具体的后端
- 当有效后端不是一个受支持的CLI后端时，抛出携带标识不受支持的后端的消息的 ValueError
- 返回一个 AgentCommand，其 argv 是一个非空的参数字符串列表，当通过 subprocess 以解析为绝对路径的 cwd 作为工作目录执行时，这些参数会调用有效后端以使用 model 处理 prompt
- 当 files 为非空列表时，prompt 文本和每个列出文件的内容会合并到 AgentCommand 的 stdin 字段中；files 中的每个文件路径会作为上下文附加到后端调用中
- 当 files 为 None 或空列表时，AgentCommand 的 stdin 字段为 None
- 当提供 effort 且非空，或 effort 为 None 且配置的默认 effort 已设置且非空时，推理 effort 级别会包含在后端调用参数中
- AgentCommand 的 backend 字段记录有效后端的规范名称
[SPEC]
```

Reasoner 差异

- SPEC 声明：

```
- 有效的后端通过以下方式确定：当 backend 参数非 None 时，将其规范化（解析别名）；当 backend 为 None 时，读取配置的默认模型后端；哨兵值 "auto" 会解析为具体的后端
- 当有效后端不是一个受支持的CLI后端时，抛出携带标识不受支持的后端的消息的 ValueError
- 返回一个 AgentCommand，其 argv 是一个非空的参数字符串列表，当通过 subprocess 以解析为绝对路径的 cwd 作为工作目录执行时，这些参数会调用有效后端以使用 model 处理 prompt
- 当 files 为非空列表时，prompt 文本和每个列出文件的内容会合并到 AgentCommand 的 stdin 字段中；files 中的每个文件路径会作为上下文附加到后端调用中
- 当 files 为 None 或空列表时，AgentCommand 的 stdin 字段为 None
- 当提供 effort 且非空，或 effort 为 None 且配置的默认 effort 已设置且非空时，推理 effort 级别会包含在后端调用参数中
- AgentCommand 的 backend 字段记录有效后端的规范名称
```

- 推导实际行为：

设 B、M、P、D、F、E 分别表示形式参数 backend、model、prompt、cwd、files、effort。定义如下：

1. N = 若 B 非 None 则 _normalize_backend(B)，否则 resolve_model_backend()
2. R = 若 N == "auto" 则 resolve_model_backend()，否则 N

若 R ∈ {"codex-cli", "claude-cli"}，则抛出 ValueError。

否则，令：

```
C = os.path.abspath(D)
S = _compose_stdin(P, 若 F 非 None 则 F 否则 [])
    （根据规格，当 F 为 None 或空时 S 为 None，否则为字符串。）
M' = M.strip()                （即使 M 非空，也可能为空）
E' = (若 E 非 None 则 E，否则 cli_effort()).strip()
```

若 R = "codex-cli"：

```
argv = ["codex", "exec", "--sandbox", "danger-full-access",
        "--dangerously-bypass-approvals-and-sandbox",
        "--skip-git-repo-check", "-C", C]
+ (若 M' != "" 则 ["--model", M']，否则 [])
+ (若 E' != "" 则 ["-c", f'model_reasoning_effort="{E'}'], 否则 [])
+ ["-"]
返回一个值 r，满足 r.argv = argv, r.stdin = S, r.backend = "codex-cli"
```

若 R = "claude-cli"：

```
argv = ["claude", "-p", "--output-format", "text",
        "--no-session-persistence", "--dangerously-skip-permissions",
        "--permission-mode", "bypassPermissions", "--add-dir", C]
+ (若 M' != "" 则 ["--model", M']，否则 [])
+ (若 E' != "" 则 ["--effort", E']，否则 [])
返回一个值 r，满足 r.argv = argv, r.stdin = S, r.backend = "claude-cli"
```


没有其他副作用。

- 代码证据:

第12行: 为 `codex-cli` 构造的 `argv` 未将 `files` 作为上下文附加; 第28行: 为 `claude-cli` 构造的 `argv` 未将 `files` 作为上下文附加; 要求每个文件路径作为上下文附加到后端调用中的需求未在函数中任何位置实现。

- 触发条件:

规格要求在 `files` 为非空列表时, 每个文件路径应作为上下文附加到后端调用 (即, 出现在 `argv` 参数中)。代码仅将文件内容合并到 `stdin` 中, 但从不会将文件路径添加到 `argv` 中。对于给定的输入 (`files=['a.txt', 'b.txt']`), 返回的 `AgentCommand.argv` 缺少任何与文件相关的标志, 例如 `'--file a.txt'`, 违反了规范。

Bug validator

- 触发摘要: 当 `files=['a.txt', 'b.txt']` 时, `AgentCommand.argv` 缺少文件路径标志, 违反了要求每个文件路径作为上下文附加到后端调用的规范。
- Probe 标准输出:

CONFIRMED — 所有后端 `argv` 中均缺少文件路径: `codex-cli: argv` 中缺少文件路径 → 违反规范 → CONFIRMED | `claude-cli: argv` 中缺少文件路径 → 违反规范 → CONFIRMED

INCR-MISMATCH-006 — `src--cli_backend-py--cli_effort`

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/cli_backend-py/cli_effort.py`。
- Reasoner 结果: `logic_verification_results/src/cli_backend-py/cli_effort.json`。
- 详细报告: `src--cli_backend-py--cli_effort.md`。
- Probe: `probe_src--cli_backend-py--cli_effort.py`。

完整生成 SPEC

```
[SPEC]
单元: fm_agent/extracted_functions/src/cli_backend-py/cli_effort.py

cli_effort() -> str

前置条件:
- `settings.llm.effort` 是一个字符串

后置条件:
- 返回 `settings.llm.effort` 的值, 去除前导和尾随空白字符
- 如果该值为空或仅由空白字符组成, 则返回空字符串
[SPEC]
```

Reasoner 差异

- SPEC 声明:

- 返回 `settings.llm.effort` 的值, 去除前导和尾随空白字符
- 如果该值为空或仅由空白字符组成, 则返回空字符串

- 推导的实际行为:

该函数返回一个字符串，该字符串是原始 ``settings.llm.effort`` 去除前导和尾随空白字符后的结果。原始的 ``settings.llm.effort`` 保持不变。形式化描述：设函数入口处 ``old = settings.llm.effort``。则返回值 ``r`` 满足 ``r = old.strip()`` ``isinstance(r, str)`` ``settings.llm.effort = old``。

- 代码证据：

第 2 行: `return settings.llm.effort.strip()`

- 触发条件：

代码返回了对当前 `settings.llm.effort` 值调用 `.strip()` 的结果。如果该值是 `bytes` 对象（例如 `b' hello '`），`.strip()` 返回 `bytes` 对象（`b'hello'`），这不是字符串。规范要求函数返回字符串（例如对于仅含空白字符的输入返回空字符串），因此返回 `bytes` 值违反了类型预期。

Bug validator

- 触发摘要：`settings.llm.effort` 是一个 `bytes` 对象（`b' hello '`）；`.strip()` 返回 `bytes`（`b'hello'`）而不是 `str`，违反了规范的返回类型保证。
- Probe 标准输出：

CONFIRMED — 实际: `b'hello'` (类型: `bytes`) | 期望的 `str`: `'hello'`

INCR-MISMATCH-007 — `src--cli_backend-py--resolve_model_backend`

- 人工审计：**SPEC 错误**。
- Validator: **confirmed**；尝试次数: **1**。
- 原始函数与SPEC: `src/cli_backend-py/resolve_model_backend.py`。
- Reasoner 结果: `logic_verification_results/src/cli_backend-py/resolve_model_backend.json`。
- 详细报告: `src--cli_backend-py--resolve_model_backend.md`。
- 探针: `probe_src--cli_backend-py--resolve_model_backend.py`。

完整生成 SPEC

```
[SPEC]
单元: src/cli_backend.py

resolve_model_backend() -> str

前置条件:
- settings.llm.backend 包含一个字符串值（配置的模型后端）
- 进程环境可能包含或可能不包含名为 FM_AGENT_HOST 或 FM_AGENT_CLIENT 的变量，其值为任意字符串
- 进程环境可能包含或可能不包含以下任一标记：
  CLAUDE_PLUGIN_ROOT、CLAUDE_CODE_ENTRYPOINT、CODEX_HOME、
  CODEX_SANDBOX 或 CODEX_EXECUTION_MODE

后置条件:
- 返回一个规范的 backend 标识符字符串：即 "opencode"、
  "codex-cli" 或 "claude-cli" 中的一个
- 返回的 backend 首先通过 _normalize_backend 对
  settings.llm.backend 进行规范化确定；如果结果不是
  "auto"，则立即返回该结果
- 当 settings.llm.backend 的规范化值为 "auto" 时，backend
  按照固定的优先级顺序检查环境标记来确定：
  1. 检查 FM_AGENT_HOST 或 FM_AGENT_CLIENT（以设置的为准）
    是否包含 "claude" 或 "codex" 子串（不区分大小写）
  2. 是否存在任何 Claude 特定的环境变量
    （CLAUDE_PLUGIN_ROOT、CLAUDE_CODE_ENTRYPOINT）
  3. 是否存在任何 Codex 特定的环境变量
    （CODEX_HOME、CODEX_SANDBOX、CODEX_EXECUTION_MODE）
- 按此优先级顺序第一个匹配的标记确定返回的 backend：
```

Claude 标记对应 "claude-cli", Codex 标记对应 "codex-cli"

- 当没有标记匹配时, 返回 "codex-cli" (默认回退值)
- 相同的输入 (settings.llm.backend 值和环境状态) 始终产生相同的输出 (相对于调用时的输入是一个纯函数)

[SPEC]

Reasoner 差异

- SPEC 声明:

- 返回一个规范的 backend 标识符字符串: 即 "opencode"、"codex-cli" 或 "claude-cli" 中的一个
- 返回的 backend 首先通过 _normalize_backend 对 settings.llm.backend 进行规范化确定; 如果结果不是 "auto", 则立即返回该结果
- 当 settings.llm.backend 的规范化值为 "auto" 时, backend 按照固定的优先级顺序检查环境标记来确定:
 1. 检查 FM_AGENT_HOST 或 FM_AGENT_CLIENT (以设置的为准) 是否包含 "claude" 或 "codex" 子串 (不区分大小写)
 2. 是否存在任何 Claude 特定的环境变量 (CLAUDE_PLUGIN_ROOT、CLAUDE_CODE_ENTRYPOINT)
 3. 是否存在任何 Codex 特定的环境变量 (CODEX_HOME、CODEX_SANDBOX、CODEX_EXECUTION_MODE)
- 按此优先级顺序第一个匹配的标记确定返回的 backend: Claude 标记对应 "claude-cli", Codex 标记对应 "codex-cli"
- 当没有标记匹配时, 返回 "codex-cli" (默认回退值)
- 相同的输入 (settings.llm.backend 值和环境状态) 始终产生相同的输出 (相对于调用时的输入是一个纯函数)

- 推导的实际行为:

该函数返回一个字符串, 即根据配置和环境解析出的规范 backend 标识符。
设 normalized = _normalize_backend(settings.llm.backend)。
若 normalized != 'auto', 则结果为 normalized。
否则, 当 normalized == 'auto' 时, 令 H = (os.environ.get('FM_AGENT_HOST') or os.environ.get('FM_AGENT_CLIENT') or '').lower()。
若 H 中包含 'claude', 或者环境变量 CLAUDE_PLUGIN_ROOT 或 CLAUDE_CODE_ENTRYPOINT 被设置为非空值, 则结果为 'claude-cli'。
在所有其他情况下 (包括 H 中包含 'codex' 但不包含 'claude' 时、CODEX_HOME、CODEX_SANDBOX 或 CODEX_EXECUTION_MODE 中任一被设置时, 或者没有任何环境提示时), 结果为 'codex-cli'。

- 代码证据:

```
第2行:     backend = _normalize_backend(settings.llm.backend)
第3行:     if backend != "auto":
第4行:         return backend
```

- 触发条件:

规范要求函数返回规范 backend 标识符 'opencode'、'codex-cli' 或 'claude-cli' 之一。
而代码在 normalized 不是 'auto' 时直接返回 _normalize_backend 的结果。
根据给定的 _normalize_backend 行为, 若输入不是可识别的别名, 它将原样返回输入。
因此, 对于类似 'foobar' 的输入, 函数会返回 'foobar', 这并不是允许的标识符之一, 从而违背了规范。

Bug validator

- 触发条件摘要: 将 settings.llm.backend 设置为无法识别的值 'foobar' —— _normalize_backend 将其原样传递, 导致 resolve_model_backend 返回非规范标识符。
- 探针标准输出:

CONFIRMED — actual: 'foobar' | expected: one of ['claude-cli', 'codex-cli', 'opencode']

- 人工审计：**SPEC** 错误。
- Validator：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：`src/domain_knowledge-py/collect_domain_knowledge_paths.py`。
- Reasoner 结果：`logic_verification_results/src/domain_knowledge-py/collect_domain_knowledge_paths.json`。
- 详细报告：`src--domain_knowledge-py--collect_domain_knowledge_paths.md`。
- Probe：`probe_src--domain_knowledge-py--collect_domain_knowledge_paths.py`。

完整生成 SPEC

```
[SPEC]
单元：src/domain_knowledge.py

collect_domain_knowledge_paths(cli_paths, base_dir, fallback_base_dir) -> list[str]

前置条件：
- cli_paths 为 None 或一个可迭代的类路径字符串对象（可能嵌套）
- base_dir 是一个引用某个现有目录的字符串
- fallback_base_dir 为 None 或一个引用某个现有目录的字符串

后置条件：
- 返回一个解析后的绝对文件路径字符串列表，每个路径对应一个不同的领域知识 Markdown 文件
- 返回的列表包含来自两个来源的路径：FM_AGENT_DOMAIN_KNOWLEDGE 环境变量（按 os.pathsep 拆分）以及扁平化后的 cli_paths
- 空字符串或未设置的 FM_AGENT_DOMAIN_KNOWLEDGE 环境变量不贡献任何路径
- cli_paths 中为 None 或空的条目不贡献任何路径
- 返回列表中的每个路径都是绝对路径；如果源路径是相对路径，则先基于 base_dir 解析，若解析后不存在对应文件，则再基于 fallback_base_dir 作为第二基目录解析
- 返回的列表中不包含重复条目
- 若未找到任何有效的领域知识路径，则返回空列表
[SPEC]
```

Reasoner 差异

- SPEC 声明：

```
- 返回一个解析后的绝对文件路径字符串列表，每个路径对应一个不同的领域知识 Markdown 文件
- 返回列表中的路径来自两个来源：FM_AGENT_DOMAIN_KNOWLEDGE 环境变量（按 os.pathsep 拆分）以及扁平化后的 cli_paths
- FM_AGENT_DOMAIN_KNOWLEDGE 环境变量为空或未设置时，不贡献任何路径
- cli_paths 中为 None 或空的条目不贡献任何路径
- 返回列表中的每个路径都是绝对路径；若源路径为相对路径，则基于 base_dir 解析；若该解析结果不是现有文件，则再以 fallback_base_dir 为次级基目录进行解析
- 返回列表中不含重复条目
- 若未找到有效的领域知识路径，则返回空列表
```

- 推导的实际行为：

该函数返回一个绝对规范化文件路径字符串的列表。它首先构建一个列表 `paths`，由以下内容拼接而成：(a) 若环境变量 `settings.runtime.domain\_knowledge\_paths` 是非空字符串，则将其按 `os.pathsep` 拆分得到的结果，否则为空列表；以及 (b) 从 `cli\_paths` 中经过去除嵌套、`None` 和空值后扁平化的路径字符串列表，保持顺序。然后调用 `resolve\_domain\_knowledge\_paths(paths, base\_dir, fallback\_base\_dir)`，该函数对每个路径执行以下操作：展开 `~` 为用户主目录，若是相对路径则基于 `base\_dir` 解析（若相对路径经 `base\_dir` 解析后无法定位，则回退到 `fallback\_base\_dir`），检查解析后的路径是否作为常规文件存在且具有可识别的领域知识 Markdown 扩展名；如果任何路径未通过上述检查，则引发 `ValueError`。若无错误发生，函数返回一个去除重复项（根据构建顺序保留首次出现的项）的解析后绝对规范化路径列表。若 `paths` 列表为空，函数返回空列表。`base\_dir` 和 `fallback\_base\_dir`（若非 `None`）是预先存在的目录。

形式化描述：令 $E = (\text{settings.runtime.domain_knowledge_paths} \text{ 不为 } None \text{ 且 } \text{str}(\text{settings.runtime.domain_knowledge_paths}).\text{strip}() \text{ 不为 } '') ? \text{ 拆分 } (\text{str}(\text{settings.runtime.domain_knowledge_paths}), \text{os.pathsep}) : []$ ；令 $C = _flatten_paths(cli_paths)$ ；令 $P = E + C$ 。则返回 $R = resolve_domain_knowledge_paths(P, base_dir, fallback_base_dir)$ 。 R 满足：对于 P 中的每个 p ，若 $resolve(p)$ 失败，则引发 $ValueError$ ；否则， $R = distinct_canonical([resolve(p) \text{ for } p \text{ in } P])$ ，保持顺序，其中 $resolve(p)$ 展开用户主目录，对相对路径使用

base_dir 或 fallback_base_dir，并要求 p 是一个存在且具有有效 Markdown 扩展名的常规文件；如果 P 为空，则 R = []。

- 代码证据：

```
第 6-10 行： return resolve_domain_knowledge_paths(  
    paths,  
    base_dir=base_dir,  
    fallback_base_dir=fallback_base_dir,  
)
```

- 触发条件：

当任何源路径无法解析为有效的领域知识 Markdown 文件时，代码会引发 ValueError，但规范要求对无效路径静默忽略，仅将有效的领域知识路径包含在返回列表中（若未找到则返回空列表）。这是一个在错误处理行为上的具体不匹配之处。

Bug validator

- 触发摘要：在 cli_paths 中传入一个不存在的文件路径会导致 ValueError，而非按照规范静默跳过该路径。
- Probe 标准输出：

```
已确认 - 对无效路径引发了 ValueError（规范要求静默跳过）：domain knowledge file does not exist:  
/nonexistent_xyz_file_that_does_not_exist.md
```

INCR-MISMATCH-009 — src--domain_knowledge-py--resolve_domain_knowledge_paths

- 人工审计：契约待确认。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：src/domain_knowledge-py/resolve_domain_knowledge_paths.py。
- Reasoner 结果：logic_verification_results/src/domain_knowledge-py/resolve_domain_knowledge_paths.json。
- 详细报告：src--domain_knowledge-py--resolve_domain_knowledge_paths.md。
- Probe：probe_src--domain_knowledge-py--resolve_domain_knowledge_paths.py。

完整生成 SPEC

```
[SPEC]  
单元：src/domain_knowledge.py  
  
resolve_domain_knowledge_paths(paths, base_dir, fallback_base_dir=None) -> list[str]  
  
前置条件：  
- paths 是一个可迭代的类路径字符串，可能包含嵌套结构（内部可能含有路径字符串的可迭代对象）  
- base_dir 是一个引用已存在目录的字符串  
- fallback_base_dir 为 None，或者是一个引用已存在目录的字符串  
  
后置条件：  
- 返回一个绝对文件路径字符串列表，每一个路径对应一个唯一、有效的领域知识 markdown 文件  
- 每个输入条目会先展开用户主目录，并从任何嵌套结构中展平  
- 若展平后的条目是绝对路径，则直接使用；否则，将其基于 base_dir 进行解析  
- 当 fallback_base_dir 不为 None 且基于 base_dir 解析出的路径在文件系统中不存在时，会基于 fallback_base_dir 作为次级基路径进行解析；若 fallback 候选路径也不存在，则使用 base_dir 候选路径（并在随后进行检查）  
- 对于每个解析得到的路径（即第一个存在的候选路径，或当没有路径存在时使用 base_dir 候选路径）：  
    * 若该路径在文件系统中不存在，则抛出 ValueError，异常消息中包含原始输入路径  
    * 若该路径存在但不是普通文件，则抛出 ValueError，异常消息中包含原始输入路径  
    * 若该路径存在但其文件扩展名不在 VALID_DOMAIN_KNOWLEDGE_EXTENSIONS 中，则抛出 ValueError，异常消息中  
列出允许的扩展名及原始输入路径  
- 返回的列表中不含重复项；去重基于真实的（规范化）路径进行  
- 返回的列表保留输入条目中首次出现的相对顺序  
- 若未提供任何路径（或在处理后所有条目因去重被过滤，但需注意无效条目会引发错误，因此该情况仅适用于输入为空或仅包
```

含已见过的有效文件的重复项)，则返回空列表 [SPEC]

Reasoner 差异

- SPEC 宣称：

- 返回一个绝对文件路径字符串列表，每个路径对应一个不同且有效的领域知识 Markdown 文件
- 每个输入条目都会展开用户主目录，并从任意嵌套结构中扁平化
- 若展开后的条目是绝对路径，则直接使用；否则基于 `base_dir` 解析
- 当 `fallback_base_dir` 不为 `None`，且基于 `base_dir` 解析的路径在文件系统中不存在时，以 `fallback_base_dir` 为次级基目录解析该条目；若回退候选也不存在，则使用 `base_dir` 候选（并在之后检查）
- 对每个解析后的路径（第一个存在的候选；若均不存在，则为 `base_dir` 候选）：
 - * 若路径在文件系统中不存在，则引发 `ValueError`，消息中指明原始输入路径
 - * 若路径存在但不是普通文件，则引发 `ValueError`，消息中指明原始输入路径
 - * 若路径存在但扩展名不在 `VALID_DOMAIN_KNOWLEDGE_EXTENSIONS` 中，则引发 `ValueError`，消息中列出允许的扩展名和原始输入路径
- 返回列表中不含重复条目；去重依据真实（规范化）路径
- 返回列表保留输入条目首次出现的相对顺序
- 若未提供路径（或处理后所有条目均被去重过滤；但无效条目会引发错误，因此这里只适用于空输入或仅含已出现有效文件的重复项），则返回空列表

- 推导的实际行为：

该函数返回一个字符串列表 `resolved`，其中：

- 每个元素都是一个绝对路径，该路径存在、为普通文件且其小写扩展名在 `VALID_DOMAIN_KNOWLEDGE_EXTENSIONS` 中。
- 对于每个由 `flatten_paths(paths)` 产生的原始路径字符串 `raw`，函数通过以下方式确定候选路径：令 `e = os.path.expanduser(raw)`，如果 `e` 是绝对路径，则 `candidates = [e]`；否则 `candidates = [os.path.join(base_dir, e)] + ([os.path.join(fallback_base_dir, e)]` 如果 `fallback_base_dir` 不为 `None`)。实际考虑的候选路径为 `c = next((c for c in candidates if os.path.exists(c)), candidates[0])`，最终路径为 `p = os.path.abspath(c)`。如果 `p` 存在、为文件、具有允许的扩展名且 `os.path.realpath(p)` 尚未被用于任何其他 `raw` 而选择过，则将 `p` 追加到 `resolved` 中；若存在性、文件类型或扩展名检查中的任一项失败，函数将抛出 `ValueError`。
- 返回的列表包含这些最终路径，并以其唯一真实路径首次被遇到的顺序排列。形式化表示为：
`resolved = [p_i | i <- [0..len(F)-1], 令 raw_i = F[i], e_i = os.path.expanduser(raw_i), cand_i = (若 os.path.isabs(e_i) 则为 [e_i], 否则为 [os.path.join(base_dir, e_i)]), 并在 fallback_base_dir 不为 None 时追加 [os.path.join(fallback_base_dir, e_i)]), c_i = cand_i 中第一个存在的候选 (若均不存在则为 cand_i[0]), p_i = os.path.abspath(c_i), 其中 os.path.exists(p_i)、os.path.isfile(p_i)、os.path.splitext(p_i)[1].lower() 属于 VALID_DOMAIN_KNOWLEDGE_EXTENSIONS, 且不存在 j < i 使 os.path.realpath(p_j) = os.path.realpath(p_i)]`。

- 代码证据：

第 22 行：`path = os.path.abspath(path)`

- 触发条件：

代码在存在性验证之前对候选路径调用了 `os.path.abspath`，进行了词法规范化（例如折叠 `'..'`），但未考虑符号链接。若原始候选路径存在，但规范化后的字符串不存在，则代码会错误地因文件不存在而引发 `ValueError`，而规范要求在对所有检查直接使用候选路径。

Bug validator

- 触发摘要：包含 `'..'` 的相对路径穿过了一个符号链接：`os.path.abspath` 将其词法规范化为一个不存在的路径，导致虚假的 `ValueError`，即使原始候选路径在文件系统上存在。
- Probe 输出：

```
CONFIRMED — 对已存在文件抛出 ValueError: domain knowledge file does not exist: link/../doc.md
actual file path: /tmp/fm_probe_0_sv1xj0/doc.md
actual file exists: True
abspath'd path: /tmp/fm_probe_0_sv1xj0/base/doc.md
```


- 人工审计：推理误判。
- 验证器：**not_confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：**src/domain_knowledge-py/stage_domain_knowledge_files.py**。
- Reasoner 结果：**logic_verification_results/src/domain_knowledge-py/stage_domain_knowledge_files.json**。
- 详细报告：**src--domain_knowledge-py--stage_domain_knowledge_files.md**。
- 探针：**probe_src--domain_knowledge-py--stage_domain_knowledge_files.py**。

完整生成 SPEC

```
[SPEC]
单元：src/domain_knowledge.py

stage_domain_knowledge_files(proj_dir, work_dir, markdown_paths=None) -> list[str]

前置条件：
- proj_dir 是文件系统中存在的目录路径。
- work_dir 是 fm_agent/ 工作区目录路径。
- markdown_paths 为 None 或可迭代的文件路径字符串（相对或绝对路径）。

后置条件：
- 当 markdown_paths 为假值（None 或空）时：暂存目录
  <work_dir>/spec_prompts/domain_context/user_knowledge/ 不会被修改；
  任何先前暂存的文件都将被保留。这支持断点续运行。
- 当 markdown_paths 为真值且非空时：暂存目录将被原子性地替换，
  以包含解析所得的 Markdown 文件的精确副本，以及一个记录哪些源文件被暂存的清单文件。
- 替换是原子的：先填充一个临时目录，然后原子性地交换到位；
  并发读取者要么看到完整的旧状态，要么看到完整的新状态。
- 返回一个经过排序的、相对于项目的路径字符串列表，每个字符串都以
  "fm_agent/" 为前缀，表示当前在工作目录下暂存的所有领域知识文件。
- 返回的路径无论平台如何，都使用 "/" 作为路径分隔符。
[SPEC]
```

Reasoner 差异

- SPEC 声明：

- 当 markdown_paths 为假值（None 或空）时：暂存目录
 <work_dir>/spec_prompts/domain_context/user_knowledge/ 不会被修改；
 任何先前暂存的文件都将被保留。这支持断点续运行。
- 当 markdown_paths 为真值且非空时：暂存目录将被原子性地替换，
 以包含解析所得的 Markdown 文件的精确副本，以及一个记录哪些源文件被暂存的清单文件。
- 替换是原子的：先填充一个临时目录，然后原子性地交换到位；
 并发读取者要么看到完整的旧状态，要么看到完整的新状态。
- 返回一个经过排序的、相对于项目的路径字符串列表，每个字符串都以
 "fm_agent/" 为前缀，表示当前在工作目录下暂存的所有领域知识文件。
- 返回的路径无论平台如何，都使用 "/" 作为路径分隔符。

- 推导的实际行为：

如果 markdown_paths 为假值（None 或空的可迭代对象），该函数返回
list_staged_domain_knowledge_relpaths(work_dir) 的结果，且不修改文件系统。暂存目录
(work_dir/USER_KNOWLEDGE_REL_DIR) 及其内容在调用前后保持不变。

如果 markdown_paths 为真值且未发生异常，该函数将原子性地用一组新的领域知识文件替换暂存目录：

- resolve_domain_knowledge_paths 将提供的路径解析为绝对、有效的文件路径列表（否则引发 ValueError）。
- 每个解析出的源文件都被复制到一个临时目录（<target_dir>.tmp）下，使用唯一的安全名称（通过
_safe_staged_name）。
- 在临时目录中写入一个 JSON 清单文件（USER_KNOWLEDGE_MANIFEST），其中包含一个有序的对象列表，每个对象带有
'source_path'（原始解析的绝对路径）和 'staged_path'（相对于项目根的路径，以 'fm_agent/' 为前缀，使用 '/'
分隔符）。
- 然后将临时目录原子性地移动以替换最终目标目录（work_dir/USER_KNOWLEDGE_REL_DIR），若先前的暂存目录存在则将其删除。
- 该函数返回 list_staged_domain_knowledge_relpaths(work_dir) 返回的、对应于新暂存文件的排序后的相对路径列表。

如果 resolve_domain_knowledge_paths 引发 ValueError（例如由于文件不存在、非普通文件或无效扩展名），该异常
将传播出去，暂存目录不会发生任何变化（但可能会留下一个临时目录作为副作用）。

如果在原子重命名之前发生任何其他异常（例如文件操作期间的 OSError），暂存目录保持不变，但临时目录可能会残留在磁盘
上。成功返回后，暂存目录精确包含清单所描述的文件，没有临时目录的残留。

```

形式化表示：
assert markdown_paths is None or hasattr(markdown_paths, '__iter__')
if not markdown_paths:
    result = list_staged_domain_knowledge_relpaths(work_dir)
    assert result == list_staged_domain_knowledge_relpaths(work_dir) # 幂等
    return result
else:
    resolved = resolve_domain_knowledge_paths(markdown_paths, base_dir=proj_dir,
fallback_base_dir=os.getcwd())
    # 如果 resolve_domain_knowledge_paths 引发 ValueError，函数不会返回；暂存目录未修改（除了 tmp）。
    # 否则，resolved 是现有的、具有允许扩展名的普通文件的绝对规范路径列表。

    target_dir = os.path.join(work_dir, USER_KNOWLEDGE_REL_DIR)
    tmp_dir = target_dir + '.tmp'

    # 清理先前的 tmp_dir
    shutil.rmtree(tmp_dir, ignore_errors=True)
    os.makedirs(tmp_dir, exist_ok=True)

    used_names = set()
    entries = []
    for source_path in resolved:
        target_name = _safe_staged_name(source_path, used_names)
        target_path = os.path.join(tmp_dir, target_name)
        shutil.copy2(source_path, target_path)
        rel_to_work = os.path.join(USER_KNOWLEDGE_REL_DIR, target_name).replace(os.sep, '/')
        entries.append({
            'source_path': source_path,
            'staged_path': f'fm_agent/{rel_to_work}'
        })

    manifest_path = os.path.join(tmp_dir, USER_KNOWLEDGE_MANIFEST)
    with open(manifest_path, 'w', encoding='utf-8') as f:
        json.dump({'files': entries}, f, indent=2, ensure_ascii=False)

    parent_dir = os.path.dirname(target_dir)
    os.makedirs(parent_dir, exist_ok=True)

    # 原子替换
    shutil.rmtree(target_dir, ignore_errors=True)
    os.replace(tmp_dir, target_dir)

    result = list_staged_domain_knowledge_relpaths(work_dir)
    assert all(p.startswith('fm_agent/') for p in result)
    return result

```

- 代码证据：

第 8 行：return list_staged_domain_knowledge_relpaths(work_dir)；第 38 行：return list_staged_domain_knowledge_relpaths(work_dir)

- 触发条件：

规范要求返回的路径无论平台如何都使用 '/' 作为路径分隔符，但代码返回的是 list_staged_domain_knowledge_relpaths 的原始输出，该输出可能使用特定于操作系统的分隔符（例如在 Windows 上为反斜杠）。没有执行任何转换。

Bug validator

- 触发摘要：规范要求路径分隔符无论平台如何都是 '/'；list_staged_domain_knowledge_relpaths 已在第 124 行执行了 replace(os.sep, '/') 操作，因此不存在缺陷。
- 探针标准输出：

```

--- 测试 1:Linux 原生 (os.sep='/') ---
list_staged: ['fm_agent/spec_prompts/domain_context/user_knowledge/test.md']
stage empty: ['fm_agent/spec_prompts/domain_context/user_knowledge/test.md']
stage full: ['fm_agent/spec_prompts/domain_context/user_knowledge/input.md']
通过：Linux 原生测试 - 所有路径均使用 '/' 分隔符

--- 测试 2:打补丁 os (os.sep='\\') ---
patched list_staged: ['fm_agent/spec_prompts/domain_context/user_knowledge/test.md']
通过：打补丁测试 - replace(os.sep, '/') 正确转换了反斜杠

```

NOT CONFIRMED 代码已通过 `list_staged_domain_knowledge_relpaths` 在第 124 行的 `replace(os.sep, '/')` 调用将路径转换为 `'/'` 分隔符。

`stage_domain_knowledge_files` 的两个返回路径（第 137 行和第 171 行）都委托给 `list_staged_domain_knowledge_relpaths`，后者处理了转换。

src/entry_reasoning_pipeline.py

INCR-MISMATCH-011 — src--entry_reasoning_pipeline.py--_extracted_file_to_source_rel

- 人工审计：契约待确认。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：src/entry_reasoning_pipeline.py/_extracted_file_to_source_rel.py。
- Reasoner 结果：logic_verification_results/src/entry_reasoning_pipeline.py/_extracted_file_to_source_rel.json。
- 详细报告：src--entry_reasoning_pipeline.py--_extracted_file_to_source_rel.md。
- Probe：probe_src--entry_reasoning_pipeline.py--_extracted_file_to_source_rel.py。

完整生成 SPEC

```
[SPEC]
单元：fm_agent/extracted_functions/src/entry_reasoning_pipeline-
py/_extracted_file_to_source_rel.py

_extracted_file_to_source_rel(extracted_rel) -> str

前置条件：
- extracted_rel 是一个相对路径字符串，其最后一个路径组件是函数文件名，并且该路径至少包含一个目录组件，该组件
是通过将源文件名中的最后一个点替换为连字符后跟已知语言扩展名（“提取目录组件”）得到的。提取布局使得提取目录组件在最
后一个连字符之后的后缀与 EXT_TO_LANG 中的某个键匹配。

后置条件：
- 返回通过从右向左扫描路径组件（跳过文件名）找到提取目录组件所获得的源文件相对路径。一旦找到，该组件的最后一个连
字符及其后面的扩展名将替换为点号和扩展名（例如，"loader-cpp" -> "loader.cpp"）。生成的文件名前会加上任何前置的
目录前缀（提取目录组件之前的组件），并且函数文件组件将被丢弃。如果没有找到提取目录组件（即没有组件其连字符后缀在
EXT_TO_LANG 中），则回退到使用直接父目录：其最后一个连字符替换为点号（如果连字符位于第一个字符之后），并且结果将
加上该父目录的父目录前缀，如果没有祖父目录则原样使用。
- 返回的路径使用操作系统原生的路径分隔符（os.sep）。

[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 返回通过从右向左扫描路径组件（跳过文件名）找到提取目录组件所获得的源文件相对路径。一旦找到，该组件的最后一个连字符及其后面的扩展名将替换为点号和扩展名（例如，"loader-cpp" -> "loader.cpp"）。生成的文件名前会加上任何前置的目录前缀（提取目录组件之前的组件），并且函数文件组件将被丢弃。如果没有找到提取目录组件（即没有组件其连字符后缀在 EXT_TO_LANG 中），则回退到使用直接父目录：其最后一个连字符替换为点号（如果连字符位于第一个字符之后），并且结果将加上该父目录的父目录前缀，如果没有祖父目录则原样使用。

- 返回的路径使用操作系统原生的路径分隔符（os.sep）。

- 推导的实际行为：

该函数返回一个相对路径字符串 `source_rel`，它是提取布局映射下与 `extracted_rel` 对应的源文件。如果第 13-19 行的循环找到一个组件 `comp = parts[i]`（其中 `i` 的范围是从 `len(parts)-2` 向下到 `0`），该组件在索引 `h>0` 处包含连字符且 `comp[h+1:]` 是 `EXT_TO_LANG` 中的键，则返回的字符串为 `os.path.join(src_dir, source_base)`（若 `src_dir` 为空则为 `source_base`），其中 `src_dir = os.sep.join(parts[:i])`，`source_base = comp[:h] + '.' + comp[h+1:]`。在这种情况下，`source_rel` 正是丢弃 `extracted_rel` 中提取目录组件之后的所有组件，并将提取目录组件 `<base>-<ext>` 替换为 `<base>.<ext>` 的结果。如果未找到此类组件，第 21-26 行的回退逻辑返回 `os.path.join(src_dir, source_base)`（或 `source_base`），其中

```
`func_dir` = os.path.dirname(extracted_rel)`, `src_dir` = os.path.dirname(func_dir)`, `dir_name` = os.path.basename(func_dir)`, `h` = dir_name.rfind('-')`, 如果 `h > 0` 则 `source_base` = dir_name[:h] + '.' + dir_name[h+1:]`, 否则为 `dir_name`。在给定的前置条件下（即 `extracted_rel` 至少包含一个提取目录组件，其最后一个连字符后的后缀与 `EXT_TO_LANG` 中的键匹配），早期返回总是执行，返回的路径满足以下形式性质：  
设 `P` = split(extracted_rel, sep)`, 其中 `sep` 是操作系统路径分隔符。  
那么存在一个索引 `i`, 满足 `0 ≤ i ≤ len(P)-2`, 使得对于 `c = P[i]`, `h = c.rfind('-')`, 有 `h > 0 ∧ c[h+1:] ∈ EXT_TO_LANG`, 并且对于所有满足 `i < j ≤ len(P)-2` 的 `j`, 要么 `P[j]` 不包含 `-`, 要么 `P[j].rfind('-')+1: ∉ EXT_TO_LANG`。  
返回的值 `R` 满足 `R = join(P[:i], c[:h] + '.' + c[h+1:])`, 其中 `join` 为 `os.sep.join`, 但如果 `P[:i]` 为空, 结果则仅为 `c[:h] + '.' + c[h+1:]`。
```

- 代码证据:

```
第 16 行: if hyphen > 0 and comp[hyphen + 1:] in EXT_TO_LANG:
```

- 触发条件:

该规范要求将最后一个连字符后的后缀在 EXT_TO_LANG 中的任何组件视为提取目录，而不要求连字符索引大于 0。第 16 行中代码的条件 'hyphen > 0' 排除了像 '-cpp' 这样连字符位于开头的组件。对于输入 '-cpp/func.cpp', 规范会将 '-cpp' 识别为提取目录并返回 '.cpp', 但代码的循环跳过了它，回退到直接父目录并返回 '-cpp', 违反了规范。

Bug validator

- 触发条件摘要: hyphen>0 守卫条件错误地排除了以连字符开头的目录组件（例如 -cpp）被识别为提取目录，导致回退返回组件名而不是正确的源文件名。
- Probe 标准输出:

```
CONFIRMED — actual: '-cpp' | expected: '.cpp'
```

INCR-MISMATCH-012 — src--entry_reasoning_pipeline-py--_fqn_to_ident

- 人工审计: 推理误判。
- 验证器: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: src/entry_reasoning_pipeline-py/_fqn_to_ident.py。
- Reasoner 结果: logic_verification_results/src/entry_reasoning_pipeline-py/_fqn_to_ident.json。
- 详细报告: src--entry_reasoning_pipeline-py--_fqn_to_ident.md。
- 探针: probe_src--entry_reasoning_pipeline-py--_fqn_to_ident.py。

完整生成 SPEC

```
[SPEC]
单元: fm_agent/extracted_functions/src/entry_reasoning_pipeline-py/_fqn_to_ident.py

_fqn_to_ident(fqn) -> str

前置条件:
- fqn 是由 ":" 分隔的非空字符串组件

后置条件:
- 返回从 fqn 中去除路径前缀（包括源文件组件）后得到的类限定函数标识符
- 当某组件由非空基名、一个连字符以及一个被识别为源文件语言扩展名的后缀组成时，该组件被识别为源文件组件
- 当 fqn 包含多于一个源文件组件时，最右侧的组件决定前缀截断位置
- 当 fqn 不包含源文件组件时，原样返回 fqn 的最后一个组件
- 返回的字符串非空
[SPEC]
```

Reasoner 差异

- SPEC 声明:

- 返回从 `fqn` 中去除路径前缀（包括源文件组件）后得到的类限定函数标识符
- 当某组件由非空基名、一个连字符以及一个被识别为源文件语言扩展名的后缀组成时，该组件被识别为源文件组件
- 当 `fqn` 包含多于一个源文件组件时，最右侧的组件决定前缀截断位置
- 当 `fqn` 不包含源文件组件时，原样返回 `fqn` 的最后一个组件
- 返回的字符串非空

- 推导的实际行为：

设 `parts = fqn.split("::")`, `n = len(parts)`。设 `i` 为区间 `[0, n-1]` 中最大的下标，使得存在整数 `pos > 0`，满足 `parts[i][pos] == '-'` 且 `parts[i][pos+1:]` 是 `EXT_TO_LANG` 中的一个键。如果这样的 `i` 存在，则 `result = "::".join(parts[i+1:])`（当 `i == n-1` 时结果可能为空字符串）。否则，`result = parts[-1]`。

- 代码证据：

第 15 行： `return "::".join(parts[i + 1:])`

- 触发条件：

当最右侧的源文件组件是 `FQN` 的最后一个组件时，`parts[i+1:]` 为空，因此代码返回空字符串，违反了返回字符串必须非空的规范要求。

Bug validator

- 触发摘要：当最右侧的源文件组件是最后一个 `FQN` 组件时，`parts[i+1:]` 为空，所以 `"::".join([])` 返回空字符串，违反了返回值非空的要求。
- 探针标准输出：

```
CONFIRMED
测试 3 (src::storage-cpp):      actual='' | expected='storage-cpp'
测试 4 (storage-cpp):          actual='' | expected='storage-cpp'
测试 1,2,5 (无缺陷用例):      全部通过
```

INCR-MISMATCH-013 — `src--entry_reasoning_pipeline-py--_select_functions_by_source`

- 人工审计：推理误判。
- Validator：not confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/entry_reasoning_pipeline-py/_select_functions_by_source.py`。
- Reasoner 结果：`logic_verification_results/src/entry_reasoning_pipeline-py/_select_functions_by_source.json`。
- 详细报告：`src--entry_reasoning_pipeline-py--_select_functions_by_source.md`。
- Probe：`probe_src--entry_reasoning_pipeline-py--_select_functions_by_source.py`。

完整生成 SPEC

```
[SPEC]
单元：fm_agent/extracted_functions/src/entry_reasoning_pipeline-py/_select_functions_by_source.py

_select_functions_by_source(proj_dir, entry_func, end_funcs, extra_call_edges=None) ->
(dict[str, set[str]], dict[str, set[str]])

前置条件：
- proj_dir 是一个存在的目录（项目根目录）
- entry_func 是一个非空的完全限定函数名字字符串
- end_funcs 是一个包含零个或多个完全限定函数名字字符串的可迭代对象
- extra_call_edges，当提供时，通过 _build_call_graph 识别的格式提供补充调用边
```

后置条件：

- proj_dir 永远不会被修改；所有修改都发生在一个临时的同级目录中，该目录在此函数返回之前被销毁
- 返回一个元组 (all_by_source, keep_by_source)，其中：
 - all_by_source 是一个字典，将每个源文件相对路径映射到可从该源文件中提取的 ALL 函数名的集合
 - keep_by_source 是一个字典，将每个源文件相对路径映射到在静态调用图中从 entry_func 传递可达的函数名的集合；当 end_funcs 非空时，该集合进一步限制为位于至少一条从 entry_func 到某个 end_funcs 成员的调用链路径上的函数名
- 在以下情况下引发 ValueError：
 - 在 proj_dir 下未找到可提取的源文件
 - 在 proj_dir 下未找到可提取的函数
 - entry_func 不在提取的函数之中
 - end_funcs 非空，且调用图中从 entry_func 无法到达任何 end_funcs 成员
- 当提供 extra_call_edges 时，其补充边会贡献给用于可达性分析的调用图

[SPEC]

Reasoner 差异

- SPEC 声明：

- proj_dir 永远不会被修改；所有修改都发生在一个临时的同级目录中，该目录会在函数返回前被销毁
- 返回元组 (all_by_source, keep_by_source)，其中：
 - all_by_source 是一个字典，将每个源文件相对路径映射到可从该源文件提取的全部函数名称集合
 - keep_by_source 是一个字典，将每个源文件相对路径映射到静态调用图中从 entry_func 传递可达的函数名称集合；当 end_funcs 非空时，该集合进一步限制为至少位于一条从 entry_func 到 end_funcs 某个成员的调用链路径上的函数名称
- 以下情况会引发 ValueError：
 - 在 proj_dir 下找不到可提取的源文件
 - 在 proj_dir 下找不到可提取的函数
 - entry_func 不在已提取函数中
 - end_funcs 非空，且调用图中从 entry_func 无法到达 end_funcs 的任何成员
- 提供 extra_call_edges 时，其补充边会加入用于可达性分析的调用图

- 推导的实际行为：

执行完代码块后，以下情况恰有一者为真：

1. **异常传播路径。**
 - 如果 `_make_run_copy(proj_dir, sel_dir)` 引发异常，该异常将传播；`proj_dir` 未被修改，`sel_dir` 可能不存在或仅为部分。
 - 如果 `_enumerate_source_files(sel_dir)` 返回空列表，将引发 `ValueError` 并附带表明无可提取源文件的消息；`proj_dir` 未被修改，`sel_dir` 存在且为 `proj_dir` 的副本（截至枚举时）。
 - 如果任何后续操作（`shutil.rmtree`、`os.makedirs`、`open`、`json.dump`、`try_codegraph_init`、`run_extraction`、`_collect_phase_files`）引发异常，该异常将传播；`proj_dir` 未被修改，`sel_dir/work_dir` 下的中间状态可能存在。
2. **正常流程路径。**
 - 未引发任何异常。`proj_dir` 保持未被修改。
 - `sel_dir`（名称为 `proj_dir + '.fm-entry-select'`）存在，并包含调用时 `proj_dir` 的完整副本。
 - 在 `sel_dir` 内部，目录 `fm_agent`（`work_dir`）存在且不含以往的提取内容（任何之前的 `fm_agent/` 已被删除）。
 - `work_dir/phases.json` 包含 JSON 对象 `{ "phases": [ { "phase": 0, "name": "all", "modules": [ { "name": "all", "source_files": source_files } ] } ] }`，其中 `source_files` 为 `_enumerate_source_files(sel_dir)` 返回的非空可提取源文件路径列表。
 - 如果能为 `sel_dir` 构建代码图索引，则已完成初始化（`try_codegraph_init`）；否则提取将在没有索引的情况下继续进行。
 - `run_extraction(sel_dir, work_dir, force=True)` 已完成，将提取出的函数文件写入 `work_dir/extracted_functions/`。
 - `phase_files` 绑定到 `_collect_phase_files(work_dir, phase)` 的结果，这是一个包含 `(extracted_file_relative_path, module_name)` 元组的列表。此列表可能为空。
 - 执行点紧接在条件 `not phase_files` 求值之后。变量 `source_files` 存活且非空，`sel_dir`、`work_dir`、`phase` 在作用域内。

形式上，令 `SrcCopy(d)`（`sel_dir` 处为 `d` 的副本 `proj_dir` 未变）。则正常结果满足：

```
proj_dir ExistingDirectory entry_func NonEmptyFQN end_funcs Iterable[FQN]
(source_files = _enumerate_source_files(sel_dir) source_files )
(phase = {phase:0,name:all,modules:[{name:all,source_files: source_files}]}
(work_dir = sel_dir + /fm_agent)
FilesExist(work_dir/phases.json) jsonContent(work_dir/phases.json) = {phases:[phase]}
(try_codegraph_init(sel_dir) has succeeded or skipped without error)
(post-run_extraction: work_dir/extracted_functions exists with extracted artifacts)
(phase_files = _collect_phase_files(work_dir, phase) : List[(file, module)])
```


如果任一步骤发生异常，回溯将反映该异常。

- 代码证据：

第 40 行: `if not phase_files:` (及其后缺失的 `return` 语句)

- 触发条件：

代码块从未返回所需的元组 (`all_by_source`, `keep_by_source`) ; 在抵达第 40 行后函数执行结束并返回 `None` , 违反了规约。

Bug validator

- 触发摘要: Bug 主张声称函数执行完毕而未返回任何内容, 但源代码检查确认在空 `phase_files` 保护中有显式的 `raise` 语句, 且在函数末尾有显式的 `return` 语句。
- Probe 标准输出:

NOT CONFIRMED — `has_return=True`, `found_guard=True`, `has_raise=True`

src/env_check-py

INCR-MISMATCH-014 — `src--env_check-py--_check_codegraph_version`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/env_check-py/_check_codegraph_version.py`。
- Reasoner 结果: `logic_verification_results/src/env_check-py/_check_codegraph_version.json`。
- 详细报告: `src--env_check-py--_check_codegraph_version.md`。
- Probe: `probe_src--env_check-py--_check_codegraph_version.py`。

完整生成 SPEC

```
[SPEC]
单元: src/env_check.py

_check_codegraph_version(config) -> (bool, str | None)

前置条件:
- config 通过 settings.codegraph.version 提供对 codegraph 版本字符串的访问, 并通过 settings.codegraph.bin_dir 提供二进制目录路径

后置条件:
- 当配置的 codegraph 版本, 在去除空白字符和前置的 "v" 前缀后为空字符串时, 返回 (True, None) — 没有固定版本, 因此跳过验证
- 否则, 尝试通过以 --version 标志执行已安装的 codegraph 二进制文件并捕获其标准输出来获取其版本字符串
- 当无法获取版本字符串 (二进制文件丢失、不可执行或超时) 时, 返回 (False, message), 消息中会标识已配置的二进制目录, 并指示用户重新运行 ./install.sh
- 当获取到的版本字符串不等于已配置的固定版本 (去除空白字符及任何前置的 "v" 前缀后) 时, 返回 (False, message), 消息中会说明已安装版本与固定版本, 并指示用户重新运行 ./install.sh
- 当获取到的版本字符串等于已配置的固定版本时, 返回 (True, None)
- 绝不抛出异常: 所有错误路径均返回 (False, message) 并附上人类可读的描述
[SPEC]
```

Reasoner 差异

- SPEC 声明:

- 当配置的 codegraph 版本，在去除空白字符和前置的 "v" 前缀后为空字符串时，返回 (True, None) 没有固定版本，因此跳过验证
- 否则，尝试通过以 --version 标志执行已安装的 codegraph 二进制文件并捕获其标准输出来获取其版本字符串
- 当无法获取版本字符串（二进制文件丢失、不可执行或超时）时，返回 (False, message)，消息中会标识已配置的二进制目录，并指示用户重新运行 ./install.sh
- 当获取到的版本字符串不等于已配置的固定版本（去除空白字符及任何前置的 "v" 前缀后）时，返回 (False, message)，消息中会说明已安装版本与固定版本，并指示用户重新运行 ./install.sh
- 当获取到的版本字符串等于已配置的固定版本时，返回 (True, None)
- 绝不抛出异常：所有错误路径均返回 (False, message) 并附上人类可读的描述

- 推导的实际行为：

在执行 _check_codegraph_version(config) 之后，函数返回一个元组 (status, msg)，其中 status 若已安装的 codegraph 版本与固定版本匹配（或未固定任何版本）则为 True，否则为 False。其行为定义如下：

- 令 w = config.settings.codegraph.version.strip().removeprefix('v')。
- 若 w 为空，则返回 (True, None)。
- 否则，令 cmd = _codegraph_cmd()。依据其规范，若与 codegraph 对应的文件存在且可执行，cmd 为 config.settings.codegraph.bin_dir 下的一个绝对路径；否则为字符串 'codegraph'（通过 PATH 解析）。
- 尝试通过执行 subprocess.run([cmd, '--version'], capture_output=True, text=True, timeout=10) 来获取已安装的版本。若成功，令 got = strip(stdout)。若发生任何 OSError 或 subprocess.SubprocessError，令 got = ''。
- 令 bin_dir = os.path.expanduser(config.settings.codegraph.bin_dir)（仅用于消息中）。
- 若 got == ''，返回 (False, 'codegraph (pinned v{w}) is not installed at {bin_dir} run ./install.sh (C/C++ extraction falls back to the regex extractor otherwise).')。
- 否则若 got != w，返回 (False, 'codegraph {got} is installed but v{w} is pinned in fm-agent.toml re-run ./install.sh to install the pinned build.')
- 否则 (got == w)，返回 (True, None)。

形式化：

```
config :
  let w = strip(removeprefix(config.settings.codegraph.version, 'v'))
  result =
    if w == '' then (True, None)
    else let cmd = _codegraph_cmd()
         let got = if out : subprocess.run([cmd, '--version'], capture_output=True, text=True,
         timeout=10) completes successfully out = it.stdout.strip()
         then out else '' (包括 OSError 及 subprocess.SubprocessError 情形)
         let bin_dir = os.path.expanduser(config.settings.codegraph.bin_dir)
         if got == '' then (False, 'codegraph (pinned v{w}) is not installed at {bin_dir} run
         ./install.sh (C/C++ extraction falls back to the regex extractor otherwise).')
         else if got != w then (False, 'codegraph {got} is installed but v{w} is pinned in fm-
         agent.toml re-run ./install.sh to install the pinned build.')
         else (True, None)
```

- 代码证据：

第19行: if not got:

- 触发条件：

代码将空的版本字符串 (got='') 与执行二进制文件失败同等对待，均返回“未安装”错误。而规范要求，若二进制文件成功执行（即使输出为空），获取到的版本字符串（空）应与固定版本进行比较。空字符串不等于非空的固定版本，因此按照规范的正确行为应是返回版本不匹配错误，而非缺少二进制文件的错误。

Bug validator

- 触发摘要：当 codegraph 二进制文件成功执行但返回空标准输出时，空字符串在 'if not got:' 中被视为假值，导致代码报告“未安装”错误，而非规范所要求的版本不匹配错误。
- Probe 标准输出：

CONFIRMED — actual: (False, 'codegraph (pinned v1.2.3) is not installed at /tmp/bug_probe_env_check_codegraph — run ./install.sh (C/C++ extraction falls back to the regex extractor otherwise).') | expected: 版本不匹配错误，而非“未安装”错误

INCR-MISMATCH-015 — src--extract-py--run_extraction

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: 2。
- 原始函数与 SPEC: `src/extract-py/run_extraction.py`。
- Reasoner 结果: `logic_verification_results/src/extract-py/run_extraction.json`。
- 详细报告: `src--extract-py--run_extraction.md`。
- Probe: `probe_src--extract-py--run_extraction.py`。

完整生成 SPEC

```
[SPEC]
单元: src/extract.py

run_extraction(proj_dir, work_dir=None, force=False, verbose=False) -> (int, int)

前置条件:
- proj_dir 是一个指向已存在目录的路径
- work_dir (如果 work_dir 为 None, 则使用 proj_dir) 包含一个 phases.json 文件, 其
  结构包含一个 phases 列表, 每个 phase 含有 modules, 其中各 source_files
  条目是相对于 proj_dir 的路径

后置条件:
- 对 phases.json 中列出的每一个源文件, 只要满足 (a) 其文件扩展名被识别为受支持的语言,
  且 (b) 不匹配测试文件的启发式规则, 就从中提取一个函数
- 每个提取出的函数以一个单独的文件写入 work_dir/extracted_functions/ 下; 输出路径的
  构造方式是将源文件名中最后一个点替换为连字符以形成一个目录, 然后将经过规范化的函数名
  附上原始扩展名放入其中
- 一个已经存在且包含 [SPEC] 标记行和 [INFO] 标记行的输出文件将保持不变, 并计为已跳过,
  除非 force 为 True
- 所有提取完成后, 输出树中的每一个函数文件恰好包含一个函数体 (已校验)
- 返回 (written_count, skipped_count): 新写入的函数文件数量以及已跳过且已带有 spec 的文
  件的数量, 两者均为非负
```

Reasoner 差异

- SPEC 声称:

- 对 phases.json 中列出的每一个源文件, 只要满足 (a) 其文件扩展名被识别为受支持的语言, 且 (b) 不匹配测试文件的启发式规则, 就从中提取一个函数
- 每个提取出的函数以一个单独的文件写入 work_dir/extracted_functions/ 下; 输出路径的构造方式是将源文件名中最后一个点替换为连字符以形成一个目录, 然后将经过规范化的函数名附上原始扩展名放入其中
- 一个已经存在且包含 [SPEC] 标记行和 [INFO] 标记行的输出文件将保持不变, 并计为已跳过, 除非 force 为 True
- 所有提取完成后, 输出树中的每一个函数文件恰好包含一个函数体 (已校验)
- 返回 (written_count, skipped_count): 新写入的函数文件数量以及已跳过且已带有 spec 的文件的数量, 两者均为非负

- 推导的实际行为:

成功执行后 (无异常):

1. ``phases_path`` 存在, 并且已成功读取为 JSON 对象 ``phases_data``。
2. ``source_files`` 是通过遍历 ``phases_data['phases'][*]['modules'][*]['source_files'][*]` 获取的所有相对源文件路径的列表。
3. ``registry_langs`` 是 ``batch_extract_all(proj_dir)`` 返回的元组的第二个元素, 表示在项目中检测到的语言集合。
4. 对于 ``source_files`` 中的每个 ``src_rel``:
 - 如果 ``_is_test_file(src_rel)`` 返回 ``True``, 则跳过该文件 (无输出, 不贡献于 ``skipped``)。
 - 否则, 构建 ``src_path = os.path.join(proj_dir, src_rel)``。如果 ``src_path`` 不存在, 记录一条警告并跳过该文件 (无输出, 不贡献于 ``skipped``)。
 - 否则, 通过 ``src_rel`` 的文件扩展名 (通过外部映射) 检测语言 ``lang_key``, 并调用 ``funcs = extract_functions_from_file(src_path, lang_key)``, 返回一个 ``(func_name, func_source)`` 二元组的列表。
 - 对于 ``funcs`` 中的每个元素:
 - * 输出文件名由 ``_safe_filename(func_name, ext)`` 生成, 其中 ``ext`` 为 ``src_rel`` 的扩展名。

- * 完整的输出路径为 ``os.path.join(output_base, output_filename)``，其中 ``output_base = os.path.join(work_dir, 'extracted_functions')``（如果该目录不存在则创建）。
 - * 如果 ``force`` 为 ``False`` 且输出路径已作为一个常规文件存在，则**跳过**该函数：``skipped`` 增 1，现有文件不被覆盖。
 - * 否则（``force`` 为 ``True`` 或输出文件不存在），将函数源文本写入输出文件（若存在则覆盖），``written`` 增 1。
5. 处理完所有源文件后，调用 ``_validate_extraction(output_base, registry_langs)``。它返回一个 ``output_base`` 下不包含恰好一个函数体的文件列表；函数可能根据该列表记录或累积警告，但**不会**引发异常。
6. 该函数返回一个元组 ``(written, skipped)``，其中：
- ``written`` 是本次调用中写入 ``output_base`` 下的函数体总数。
 - ``skipped`` 是那些因为 ``force=False`` 且其对应的输出文件已存在而**没有**写入的函数体总数。

形式化地，令：

- ``D = { src_rel in source_files | not _is_test_file(src_rel) and os.path.exists(os.path.join(proj_dir, src_rel)) }``
- 对每个 ``src_rel in D``，令 ``L_src = extract_functions_from_file(os.path.join(proj_dir, src_rel), lang_key_src)`` 为提取出的函数列表。
- 定义 ``F = { (out_path, func_source) | src_rel in D, (name, body) in L_src, out_path = join(output_base, _safe_filename(name, ext_src)) }``。
- 则 ``written = |{ (out_path, src) in F : force=True or not exists(out_path) }|`` 且 ``skipped = |{ (out_path, src) in F : force=False and exists(out_path) }|``。
- ``output_base`` 下的所有其他文件均不受影响，仅由该过程创建或覆盖的文件除外。

- 代码证据：

第 30 行以及之后可见代码块：当输出文件已存在时，循环跳过写入，而未检查 [SPEC]/[INFO] 标记。

- 触发条件：

条件 A 在 `force=False` 时跳过其输出文件已存在的任何函数，但规范要求仅当现有文件同时包含 [SPEC] 和 [INFO] 标记行时才跳过。示例输出文件缺少这些标记，因此规范要求覆盖它，而代码却跳过了它。

Bug validator

- 触发摘要：is_file_ready 要求严格顺序恰好包含 2 个 SPEC + 2 个 INFO 标记，因此一个只包含 1 个 SPEC + 1 个 INFO 的文件被视为未就绪，从而被覆盖，这违反了规范中所说的“同时包含 [SPEC] 标记行和 [INFO] 标记行”就应足以跳过的要求。
- Probe 标准输出：

```
Extraction complete: 1 written, 0 skipped.
CONFIRMED - is_file_ready returned False for 1+1 markers, causing overwrite (written=1,
skipped=0) when spec says file with both SPEC+INFO markers should be skipped. content changed
from 99 to 42
```

src/file_utils.py

INCR-MISMATCH-016 — src--file_utils.py--_get_phase_files

- 人工审计：SPEC 错误。
- 验证器：已确认；尝试次数：1。
- 原始函数与 SPEC：src/file_utils.py/_get_phase_files.py。
- Reasoner 结果：logic_verification_results/src/file_utils.py/_get_phase_files.json。
- 详细报告：src--file_utils.py--_get_phase_files.md。
- 探针：probe_src--file_utils.py--_get_phase_files.py。

完整生成 SPEC

[SPEC]
单元：src/file_utils.py/_get_phase_files.py

```
_get_phase_files(phases_data, phase_num, input_dir) -> list[str]
```

前置条件：

- phases_data 是一个符合 phases.json 架构的字典：它拥有一个 "phases" 键，其值是一个阶段信息字典的列表，每个字典至少包含一个 "phase" 字段（整数阶段编号）和一个 "modules" 字段。
- phase_num 是一个整数，且与 phases_data["phases"] 中恰好一个字典的 "phase" 字段匹配。如果没有任何阶段字典的 "phase" 字段匹配，将引发 StopIteration。
- 匹配阶段中的每个模块字典有一个 "source_files" 键，其值是一个使用 "/" 分隔符的字符串路径的可迭代对象。
- input_dir 是一个已存在的目录路径，在其下按照引擎的目录布局惯例存储了提取函数文件。

后置条件：

- 返回一个相对路径字符串列表，每个路径都是从 input_dir 到位于提取函数子目录下的一个普通文件的路径。
- 每个返回的路径源自 phase_num 所标识阶段的模块中声明的源文件；从源文件路径到其提取函数子目录的映射遵循引擎惯例：源文件基础名称中最后一个 "." 被替换为 "-", 并将结果名称用作 input_dir 下与源文件的目录部分合并后的子目录。
- 在 input_dir 下对应提取函数子目录不存在的源文件不会向结果中添加任何条目（静默跳过）。
- 在每个提取函数子目录内部，包含的普通文件按文件名字典序排序出现。
- 结果中路径的整体顺序保持不变：phases_data["phases"] 的迭代顺序、匹配阶段内模块的迭代顺序以及每个模块内 source_files 的迭代顺序。
- 当匹配阶段没有模块、没有源文件或其任何源文件没有现存的提取函数目录时，返回的列表可能为空。

[SPEC]

Reasoner 差异

- SPEC 声明：

- 返回一个相对路径字符串列表，每个路径都是从 input_dir 到位于提取函数子目录下的一个普通文件的路径。
- 每个返回的路径源自 phase_num 所标识阶段的模块中声明的源文件；从源文件路径到其提取函数子目录的映射遵循引擎惯例：源文件基础名称中最后一个 "." 被替换为 "-", 并将结果名称用作 input_dir 下与源文件的目录部分合并后的子目录。
- 在 input_dir 下对应提取函数子目录不存在的源文件不会向结果中添加任何条目（静默跳过）。
- 在每个提取函数子目录内部，包含的普通文件按文件名字典序排序出现。
- 结果中路径的整体顺序保持不变：phases_data["phases"] 的迭代顺序、匹配阶段内模块的迭代顺序以及每个模块内 source_files 的迭代顺序。
- 当匹配阶段没有模块、没有源文件或其任何源文件没有现存的提取函数目录时，返回的列表可能为空。

- 推导实际行为：

如果存在 `phase == phase\_num` 的阶段字典，函数返回一个相对文件路径（字符串）的列表，这些路径是从 `input\_dir` 到所有存在于对应提取目录中的普通文件的路径。否则，引发 `StopIteration`；如果文件系统操作失败，其他异常（例如 `OSError`）可能会传播。
形式化描述：

```
p ∈ phases_data["phases"] : p["phase"] = phase_num
令 M = {p | p ∈ phases_data["phases"] ∧ p["phase"] = phase_num} (按前置条件为单元素集)。
对于每个 module ∈ M["modules"], 对于每个 src_file ∈ module["source_files"] :
    令 base = basename(src_file), ext_idx = base.rfind("."),
        subdir = (base[:ext_idx] + "-" + base[ext_idx+1:]) 若 ext_idx ≠ 0 否则 base,
        extracted_dir = join(input_dir, dirname(src_file), subdir)。
    如果 is_dir(extracted_dir), 则对于 os.walk(extracted_dir) 中的每个 (root, dirs, files) (自顶向下, 任意顺序),
        对于每个 fname ∈ sorted(files):
            令 fpath = join(root, fname)。
            如果 is_file(fpath), 则将 relpath(fpath, input_dir) 附加到结果列表。
返回最终列表 (在给定文件系统状态下是确定性的)。
```

否则（没有匹配的阶段），引发 `StopIteration`。

- 代码证据：

```
第 20 行: for root, _dirs, fnames in os.walk(extracted_dir):
第 21 行: for fname in sorted(fnames):
第 22 行: fpath = os.path.join(root, fname)
第 23 行: if os.path.isfile(fpath):
第 24 行: phase_files.append(os.path.relpath(fpath, input_dir))
```

- 触发条件：

规范要求提取函数子目录中包含的所有普通文件按文件名排序出现。代码使用 `os.walk` 并按目录对文件排序，因此当存在子目录时，整体列表并非全局排序。反例中代码返回 `['dir/file-cpp/z.txt', 'dir/file-cpp/sub/a.txt']`，而规范要求 `['dir/file-cpp/sub/a.txt', 'dir/file-cpp/z.txt']`。

Bug validator

- 触发总结：当提取函数子目录包含嵌套子目录时，`os.walk` 会先产生根级文件再产生子目录中的文件，导致最终文件列表未按规范要求进行全局按文件名排序。
- 探针标准输出：

已确认 — 实际 (`os.walk` 顺序): `['src/file-cpp/z.txt', 'src/file-cpp/sub/a.txt']` | 期望 (规范顺序): `['src/file-cpp/sub/a.txt', 'src/file-cpp/z.txt']`

src/generate_topdown_layers-py

INCR-MISMATCH-017 — src--generate_topdown_layers-py--_collect_phase_files

- 人工审计：契约待确认。
- Validator: **confirmed**；尝试次数：1。
- 原始函数与 SPEC: `src/generate_topdown_layers-py/_collect_phase_files.py`。
- Reasoner 结果: `logic_verification_results/src/generate_topdown_layers-py/_collect_phase_files.json`。
- 详细报告: `src--generate_topdown_layers-py--_collect_phase_files.md`。
- Probe: `probe_src--generate_topdown_layers-py--_collect_phase_files.py`。

完整生成 SPEC

```
[SPEC]
单元：src/generate_topdown_layers-py/_collect_phase_files.py

_collect_phase_files(proj_dir, phase_data) -> list[tuple[str, str]]

前置条件：
- proj_dir 是一个指向已存在目录的路径
- phase_data 是一个字典，可能包含 "modules" 键；若存在，其值是一个可迭代的模块字典，每个模块字典包含 "name"（字符串）以及可选的 "source_files"（字符串相对路径的可迭代对象）

后置条件：
- 返回一个 (file_path, module_name) 对组成的列表，其中 module_name 是 phase_data 中某个模块的 "name"
- 对于每个模块中声明的每个源文件：通过将最后一个 "." 替换为 "-" 来去除源文件的基名扩展名（例如，"loader.cpp"
→ "loader-cpp"），由此得到的目录名被解析到 proj_dir/extracted_functions/ 下，并与源文件的父目录并列
- 在这样一个目录中找到的每个常规文件都会被收集到结果中，并与声明该源文件的模块名称配对
- 磁盘上不存在的目录会被跳过，不会抛出错误
- 当 phase_data 中没有 "modules" 键、模块列表为空，或磁盘上不存在任何提取出的函数目录时，返回空列表
- 返回的列表在不同调用间不保证顺序
[SPEC]
```

Reasoner 差异

- SPEC 声明：

```
- 返回一个 (file_path, module_name) 对组成的列表，其中 module_name 是 phase_data 中某个模块的 "name"
- 对于每个模块中声明的每个源文件：通过将最后一个 "." 替换为 "-" 来去除源文件的基名扩展名（例如，"loader.cpp"
→ "loader-cpp"），由此得到的目录名被解析到 proj_dir/extracted_functions/ 下，并与源文件的父目录并列
- 在这样一个目录中找到的每个常规文件都会被收集到结果中，并与声明该源文件的模块名称配对
- 磁盘上不存在的目录会被跳过，不会抛出错误
- 当 phase_data 中没有 "modules" 键、模块列表为空，或磁盘上不存在任何提取出的函数目录时，返回空列表
- 返回的列表在不同调用间不保证顺序
```


- 推导的实际行为：

该函数返回一个列表 `results`，其满足：

`results = [(file_path, module_name)` 对于 `phase_data.get('modules', [])` 中的每个模块，若模块包含 `'source_files'`，对于模块 `['source_files']` 中的每个 `src_file`，当 `os.path.isdir(func_dir)` 为真时，对于通过 `os.walk` 递归遍历 `func_dir` 找到的每个常规文件（`os.path.isfile`），其路径 `file_path` 被加入]。

这里，如果 `src_dir(os.path.dirname(src_file))` 非空，则 `func_dir = os.path.join(proj_dir, 'extracted_functions', src_dir, dir_name)`，否则为 `os.path.join(proj_dir, 'extracted_functions', dir_name)`。`dir_name` 由 `os.path.basename(src_file)` 推导而来：如果最后一个点的位置 `> 0`，则 `dir_name = basename[:last_dot] + '-' + basename[last_dot+1:]`；否则 `dir_name = basename`。

`file_path` 是提取目录树内每个文件的绝对路径（基于 `proj_dir`）；`module_name` 是字符串 `module['name']`。`results` 中的顺序遵循模块的迭代顺序，然后是按 `source_files` 的顺序，对于每个目录则是 `os.walk` 返回的文件顺序（深度优先、自顶向下）。该函数无副作用，且在前置条件下不抛出异常。

- 代码证据：

```
第13行: last_dot = src_base.rfind(".")
第14行: if last_dot > 0:
第15行:     dir_name = src_base[:last_dot] + "-" + src_base[last_dot + 1:]
第17行: dir_name = src_base
```

- 触发条件：

当源文件的基名以一个点开头时（例如 `".hidden"`），最后一个点的位置为 `0`，条件 `last_dot > 0` 失败，导致 `dir_name` 保持为 `".hidden"`。规范会将最后一个 `"."` 替换为 `"-"`，从而生成 `"-hidden"`。这导致代码查找目录 `".hidden"` 而非期望的 `"-hidden"`，遗漏了本应包含的文件。

Bug validator

- 触发摘要：当源文件的基名以一个点开头时（例如 `".hidden"`），`last_dot` 等于 `0`，守卫条件 `last_dot > 0` 失败，因此代码查找目录 `".hidden"` 而非规范正确的 `"-hidden"`。
- Probe 标准输出：

```
CONFIRMED — 实际：
{'/tmp/probe_collect_phase_files_cztzyoif/extracted_functions/.hidden/buggy_file.py'} | 规范期望
（在 -hidden/ 中查找）：{'/tmp/probe_collect_phase_files_cztzyoif/extracted_functions/-
hidden/correct_file.py'} | 代码因 last_dot>0 在 last_dot==0 时为假而查找了 .hidden/ 目录
```

INCR-MISMATCH-018 — `src--generate_topdown_layers-py--_strip_comments_from_source`

- 人工审计：推理误判。
- Validator: **not_confirmed**；尝试次数：2。
- 原始函数与 SPEC: `src/generate_topdown_layers-py/_strip_comments_from_source.py`。
- Reasoner 结果: `logic_verification_results/src/generate_topdown_layers-py/_strip_comments_from_source.json`。
- 详细报告: `src--generate_topdown_layers-py--_strip_comments_from_source.md`。
- Probe: `probe_src--generate_topdown_layers-py--_strip_comments_from_source.py`。

完整生成 SPEC

```
[SPEC]
单元：src/generate_topdown_layers-py/_strip_comments_from_source.py

_strip_comments_from_source(text, lang_key) -> str

前置条件：
```

- text 是一个包含源代码的字符串，可以为空
- lang_key 是一个标识编程语言的字符串

后置条件：

- 返回与 text 长度相同（字符数相同）的字符串
- 输入中处于注释区域内的每个字符位置，都会在输出中替换为空格字符（' '）；
注释区域依据特定语言的注释语法定义：
 - * 当 lang_key 对应的语言配置具有 comment_prefix "#" 时：
注释区域从不在字符串字面量内的 '#' 字符开始，延伸到同一行末尾，并包含该 '#' 字符。
 - * 当 lang_key 对应的语言配置具有 comment_prefix "/" 时（或找不到 lang_key 而默认采用 "/" 时）：行注释区域从 "/" 延伸到行尾；块注释区域从 "/*" 延伸到下一个 "*/"（不嵌套），起止标记均属于注释区域。
 - * 注释区域内的换行字符（\n）绝不会被替换，保持不变。
- 输入中处于字符串字面量区域内的每个字符位置，都会在输出中替换为空格。
字符串字面量区域包含其定界引号以及引号之间的全部字符。
定界符可以是：
 - * 单引号（'...'）、双引号（"..."），
 - * 三单引号（'''...'''）、三双引号（"""..."""）。反斜杠转义处理：反斜杠（\）及其紧随字符（若有）被视为非定界字符，并替换为空格；若紧随字符是换行符，则只替换反斜杠，换行符保留。
- 既不位于注释区域、也不位于字符串字面量区域内的任何字符位置均原样保留（包括空白符、标点、标识符、关键字等）。
- 未替换字符的顺序和位置保持不变；返回字符串的长度与输入 text 完全相同。

[SPEC]

Reasoner 差异

- SPEC 声称：

- 返回与 text 长度相同（字符数相同）的字符串
- 输入中处于注释区域内的每个字符位置，都会在输出中替换为空格字符（' '）；
注释区域依据特定语言的注释语法定义：
 - * 当 lang_key 对应的语言配置具有 comment_prefix "#" 时：
注释区域从不在字符串字面量内的 '#' 字符开始，延伸到同一行末尾，并包含该 '#' 字符。
 - * 当 lang_key 对应的语言配置具有 comment_prefix "/" 时（或找不到 lang_key 而默认采用 "/" 时）：行注释区域从 "/" 延伸到行尾；块注释区域从 "/*" 延伸到下一个 "*/"（不嵌套），起止标记均属于注释区域。
 - * 注释区域内的换行字符（\n）绝不会被替换，保持不变。
- 输入中处于字符串字面量区域内的每个字符位置，都会在输出中替换为空格。
字符串字面量区域包含其定界引号以及引号之间的全部字符。
定界符可以是：
 - * 单引号（'...'）、双引号（"..."），
 - * 三单引号（'''...'''）、三双引号（"""..."""）。反斜杠转义处理：反斜杠（\）及其紧随字符（若有）被视为非定界字符，并替换为空格；若紧随字符是换行符，则只替换反斜杠，换行符保留。
- 既不位于注释区域、也不位于字符串字面量区域内的任何字符位置均原样保留（包括空白符、标点、标识符、关键字等）。
- 未替换字符的顺序和位置保持不变；返回字符串的长度与输入 text 完全相同。

- 推导的实际行为：

代码块执行完毕后，输入源文本已被处理：所有字符串字面量（单引号、双引号及三引号字符串，并处理反斜杠转义字符）中的字符都被替换为空格。变量 'result' 是一个与 'text' 长度相同的字符列表。对于每个索引 j，若原始 'text' 中第 j 个字符位于字符串字面量内（依据该语言的引号规则和处理逻辑），则 result[j] 等于空格 ' '；否则 result[j] 等于 text[j]。索引 'i' 等于 len(result)，表示扫描循环已完成。变量 'lang_cfg'、'comment_prefix' 和 'is_hash_comment' 保存推导出的语言配置。形式化表示：
len(result) = len(text) AND (forall j in [0, len(text)-1] : (in_string_literal(text, j) -> result[j] = ' ') and (not in_string_literal(text, j) -> result[j] = text[j])) AND i = len(result) AND is_hash_comment = (comment_prefix = '#') AND comment_prefix = (LANG_CONFIG.get(lang_key, {})).get('comment_prefix', '/') AND lang_cfg = LANG_CONFIG.get(lang_key, {})

- 代码证据：

第 59-64 行（井号注释处理）要么未执行，要么不影响 A 所描述的行为。根据 A，代码不会屏蔽注释，这违反了 B 中要求将注释区域替换为空格的规定。

- 触发条件：

条件 A 声明仅屏蔽字符串字面量，字符串字面量之外的字符原样保留。规范 B 要求同时屏蔽注释区域（替换为空格）。任何包含注释的输入（例如 '# comment\n'）都会使 A 将注释字符原样保留，而 B 要求这些字符为空格。这构成直接违反。

Bug validator

- 触发摘要：验证工具声称该函数只屏蔽字符串字面量而不屏蔽注释。对 Python #、C++ // 和 C /* */ 注释的实证测试表明，所有注释类型都会被正确替换为空格。
- Probe 标准输出：

```
=== 测试 1:Python 井号注释 ===
通过:True
输入:  'x = 1  # this is a comment\n'
结果:  'x = 1                                \n'
预期:  'x = 1                                \n'

=== 测试 2:Python 字符串和注释 ===
通过:True
输入:  'print("hello")  # greet\n'
结果:  'print(          )          \n'
预期:  'print(          )          \n'

=== 测试 3:C++ // 注释 ===
通过:True
输入:  'int x = 1; // comment\n'
结果:  'int x = 1;          \n'
预期:  'int x = 1;          \n'

=== 测试 4:C 块注释 ===
通过:True
输入:  'int /* block */ x;\n'
结果:  'int          x;\n'
预期:  'int          x;\n'

=== 测试 5:未知语言的 # (不是注释) ===
通过:True
输入:  'code # not a comment for unknown lang\n'
结果:  'code # not a comment for unknown lang\n'
预期:  'code # not a comment for unknown lang\n'

=== 测试 6:字符串中的反斜杠转义和注释 ===
通过:True
输入:  'print("hello\\"world") # comment\n'
结果:  'print(          )          \n'

=====
Python # 注释:通过
字符串和 # 注释:通过
C++ // 注释:通过
块 /* */ 注释:通过
未知语言 #:通过
反斜杠转义:通过

Bug 未确认:函数能够正确屏蔽字符串字面量和注释。
NOT CONFIRMED
```

src/git-py

INCR-MISMATCH-019 — src--git-py--frozen_worktree

- 人工审计：契约待确认。
- 验证器：**confirmed**；尝试次数：1。
- 原始函数与 SPEC：[src/git-py/frozen_worktree.py](#)。
- Reasoner 结果：[logic_verification_results/src/git-py/frozen_worktree.json](#)。
- 详细报告：[src--git-py--frozen_worktree.md](#)。
- Probe：[probe_src--git-py--frozen_worktree.py](#)。

[SPEC]
单元：src/git.py

```
frozen_worktree(proj_dir, exclude=("fm_agent",), copy_excluded=True) -> yields str
```

前置条件：

- proj_dir 是一个文件系统路径；它可能是一个 git 仓库，也可能不是；可能包含提交，也可能不包含。
- exclude 是一个字符串可迭代对象，命名了要排除在 git 快照提交之外的 proj_dir 子目录。
- copy_excluded 是一个布尔值。

后置条件：

- 在系统临时目录下创建一个新的、唯一的临时目录。其名称以 "fm_agent_wt_" 开头，后跟 proj_dir 的基本名称。
 - 当 proj_dir 是一个具有可访问 HEAD 提交的 git 仓库时：
 - 使用一个私有的 git 索引 (GIT_INDEX_FILE)，以确保 proj_dir 的真实索引和工作树不会被修改。
 - 快照提交捕获进入函数时 proj_dir 的完整状态：
HEAD 树 + 所有已跟踪的修改 + 所有未跟踪的文件，并且将 exclude 中的每个路径从快照提交树中移除。
 - 该提交成为一个分离的 git 工作树，检出到临时目录下的 "snapshot" 子目录中。yield 的路径就是这个 snapshot 子目录。
 - 如果 copy_excluded 为真值：对于 exclude 中的每个名称，如果对应的子目录存在于 proj_dir 中且在快照工作树中相对于同一路径还不存在，则将该子目录递归复制（保留符号链接）到快照工作树中。
 - 当 proj_dir 不是 git 仓库或没有可访问 HEAD 时：
 - 执行从 proj_dir 到 snapshot 子目录的普通递归目录复制，使用 copytree，并将 exclude 中的每个名称作为忽略模式传入，同时保留符号链接。yield 的路径就是这个 snapshot 子目录。
 - 如果 copy_excluded 为真值：在相同的条件（与 git 路径情况相同）下将排除的子目录复制到快照工作树中。
 - 快照工作树的绝对路径会打印到 stdout，同时打印平台相关的删除指令，引用 "git worktree remove" (git 路径) 或 "rm -rf" (非 git 路径)。
 - 快照工作树及其父临时目录在上下文管理器退出后仍然保留；不会执行自动清理。
 - 如果任何 git 或文件系统操作失败（例如 git 命令返回非零、目录不可写），相应的 subprocess.CallProcessError 或 OSError 将传播给调用者。
- [SPEC]

Reasoner 差异

- SPEC 声称：

- 在系统临时目录下创建一个新的、唯一的临时目录。其名称以 "fm_agent_wt_" 开头，后跟 proj_dir 的基本名称。
- 当 proj_dir 是一个具有可访问 HEAD 提交的 git 仓库时：
 - 使用一个私有的 git 索引 (GIT_INDEX_FILE)，以确保 proj_dir 的真实索引和工作树不会被修改。
 - 快照提交捕获进入函数时 proj_dir 的完整状态：
HEAD 树 + 所有已跟踪的修改 + 所有未跟踪的文件，并且将 exclude 中的每个路径从快照提交树中移除。
 - 该提交成为一个分离的 git 工作树，检出到临时目录下的 "snapshot" 子目录中。yield 的路径就是这个 snapshot 子目录。
- 如果 copy_excluded 为真值：对于 exclude 中的每个名称，如果对应的子目录存在于 proj_dir 中且在快照工作树中相对于同一路径还不存在，则将该子目录递归复制（保留符号链接）到快照工作树中。
- 当 proj_dir 不是 git 仓库或没有可访问 HEAD 时：
 - 执行从 proj_dir 到 snapshot 子目录的普通递归目录复制，使用 copytree，并将 exclude 中的每个名称作为忽略模式传入，同时保留符号链接。yield 的路径就是这个 snapshot 子目录。
 - 如果 copy_excluded 为真值：在相同的条件（与 git 路径情况相同）下将排除的子目录复制到快照工作树中。
- 快照工作树的绝对路径会打印到 stdout，同时打印平台相关的删除指令，引用 "git worktree remove" (git 路径) 或 "rm -rf" (非 git 路径)。
- 快照工作树及其父临时目录在上下文管理器退出后仍然保留；不会执行自动清理。
- 如果任何 git 或文件系统操作失败（例如 git 命令返回非零、目录不可写），相应的 subprocess.CallProcessError 或 OSError 将传播给调用者。

- 推导的实际行为：

在函数体执行到并包括 yield 语句（暂停生成器）之后，以下条件成立：

- 通过 tempfile.mkdtemp 在系统临时目录内创建了一个唯一的临时目录基路径；其绝对路径存储在变量 `base` 中。
- 变量 `wt` 保存着绝对路径 `os.path.join(base, 'snapshot')`，这是一个现已存在的目录。

****如果 `proj\_dir` 是一个至少有一个提交的 git 仓库**（即 `git -C proj\_dir rev-parse --verify HEAD` 成功）：**

- * 一个私有的 git 索引文件被创建于 `os.path.join(base, 'index')`。
- * HEAD 树被读入该索引（`git read-tree HEAD`）。
- * 所有工作树修改（已跟踪的编辑和未跟踪的文件，遵循 `.gitignore`）通过 `git add -A` 暂存。
- * 对于 `exclude` 中的每个名称，执行了 `git rm -r --cached --quiet --ignore-unmatch -- <name>`，如果存在则那些条目从私有索引中移除。
- * 从索引写入一个树对象（`git write-tree`）。
- * 使用该树、父提交 HEAD 和消息 `fm\_agent snapshot` 创建了一个新的提交对象（`git commit-tree`）。
- * 在 `wt` 处添加了一个引用该提交的分离工作树（`git worktree add --detach <wt> <snap>`）。
- * `wt` 是一个有效的 git 检出，包含已提交状态加上未提交的编辑和未跟踪的文件，**排除** 任何被 gitignore **或

**** 其名称列在 `exclude` 中的路径。**

****否则**（`proj\_dir` 不是 git 仓库或没有提交）：**

- * `shutil.copytree(proj\_dir, wt, ignore=shutil.ignore\_patterns(\*exclude), symlinks=True)` 成功

```
执行。
* `wt` 是 `proj_dir` 的普通目录副本，保留符号链接，但省略任何名称与 `exclude` 中的模式匹配的文件或子目录。
* 记录了一条 INFO 级别的日志消息，表明已执行复制。

**如果 `copy_excluded` 为 True：**
* 对于 `exclude` 中的每个名称，如果 `os.path.join(proj_dir, name)` 作为一个目录存在 **并且**
`os.path.join(wt, name)` **不** 已存在，则其内容通过 `shutil.copypath(src, dst, symlinks=True)` 复制到快照中。
* 因此，快照在物理上包含了那些来自 `proj_dir` 的目录的原始内容，即使它们被排除在 git 提交（或初始复制）之外。

- 两条消息被打印到 stdout：一条指示快照路径 `wt`，另一条包含删除指令（如果使用了 git 则使用 `git worktree remove`，否则使用 `rm -rf`）。
- 生成器在 yield 点暂停；调用者接收到 `wt` 作为 yielded 的值。
- 不会对临时目录或快照执行清理；它们在 yield 之后继续存在于磁盘上。

**形式化总结：**
设 `ProjDir` 为绝对路径。假设 `ProjDir` 存在。
- `base = tempfile.mkdtemp(...)` `wt = base + '/snapshot'` `IsDir(wt)`
- `(IsGitRepo(ProjDir) HasCommit(ProjDir, HEAD))`
  `wt` 是一个 git 工作树，内容 = `Committed(ProjDir) WorkingTreeChanges(ProjDir) \
(GitIgnored(ProjDir) exclude)`
- `(IsGitRepo(ProjDir) HasCommit(ProjDir, HEAD))`
  `wt` 是一个普通目录，内容 = `Content(ProjDir) \ { p | patexclude: Match(pat, baseName(p)) }`
- `copy_excluded` `nameexclude: (IsDir(ProjDir+'/'+name) Exists(wt+'/'+name))`
`Content(wt+'/'+name) = Content(ProjDir+'/'+name)`
- 输出到 stdout 的行描述了 `wt` 和删除方法。
- 生成器 yield `wt` 并暂停。
```

- 代码证据：

```
第 45 行： _git("add", "-A", env=env)
```

- 触发条件：

代码使用 `git add -A` 会跳过匹配 .gitignore 模式的文件，因此被 gitignore 忽略的未跟踪文件不会被包含在快照中。但规范要求捕获所有未跟踪文件，不例外 .gitignore。

Bug validator

- 触发摘要：git add -A 会跳过匹配 .gitignore 模式的未跟踪文件，因此被 gitignore 忽略的未跟踪文件会从快照中遗漏，尽管规范要求捕获所有未跟踪文件。
- Probe 标准输出：

```
[Pipeline] 快照创建于：/tmp/fm_agent_wt_testrepo_vqrce874/snapshot
[Pipeline] 运行结束后快照保留。使用以下命令删除：git -C /tmp/bug_probe_nuyn9g_/testrepo worktree
remove --force /tmp/fm_agent_wt_testrepo_vqrce874/snapshot
CONFIRMED — 被 gitignore 忽略的未跟踪文件 'test.secret' 在快照中缺失。present=False ,
normal_untracked_present=True , tracked_present=True
```

INCR-MISMATCH-020 — src--git-py--frozen_worktree::_git

- 人工审计：推理误判。
- 验证器：confirmed；尝试次数：1。
- 原始函数与 SPEC：src/git-py/frozen_worktree::_git.py。
- 推理器结果：logic_verification_results/src/git-py/frozen_worktree::_git.json。
- 详细报告：src--git-py--frozen_worktree::_git.md。
- 探测脚本：probe_src--git-py--frozen_worktree::_git.py。

完整生成 SPEC

```
[SPEC]
单元：src/git.py

_git(*args, **kwargs) -> str
```

前置条件：

- `proj_dir` 是外围作用域中的一个文件系统路径，指向一个目录。
- `*args` 是零个或多个位置字符串参数，构成 `git` 子命令及其操作数（例如 `"add"`、`"-A"`）。
- `**kwargs` 是零个或多个关键字参数，转发给子进程调用，与强制文本模式输出捕获和非零退出码报错的默认参数合并。

后置条件：

- 以 `proj_dir` 为根目录（等同于 `"git -C proj_dir"` 接每个位置参数按顺序）作为子进程执行一个 `git` 命令。
- 子进程的标准输出和标准错误输出不会出现在父进程的标准输出或标准错误流中。
- 如果子进程以非零退出码终止，则抛出 `subprocess.CalledProcessError`，其属性记录所调用的命令、返回码以及捕获的 `stdout` 和 `stderr` 字符串。
- 如果子进程以退出码零终止，则返回已去除所有前导和尾随空白字符（空格、制表符、换行符、回车符）的捕获 `stdout` 字符串。
- 子进程继承父进程的环境，并可通过 `**kwargs` 中传入的 `env` 关键字参数进行修改。

[SPEC]

Reasoner 差异

- SPEC 声称：

- 以 `proj_dir` 为根目录（等同于 `"git -C proj_dir"` 接每个位置参数按顺序）作为子进程执行一个 `git` 命令。

- 子进程的标准输出和标准错误输出不会出现在父进程的标准输出或标准错误流中。

- 如果子进程以非零退出码终止，则抛出 `subprocess.CalledProcessError`，其属性记录所调用的命令、返回码以及捕获的 `stdout` 和 `stderr` 字符串。

- 如果子进程以退出码零终止，则返回已去除所有前导和尾随空白字符（空格、制表符、换行符、回车符）的捕获 `stdout` 字符串。

- 子进程继承父进程的环境，并可通过 `**kwargs` 中传入的 `env` 关键字参数进行修改。

- 推导实际行为：

在调用 ``_git`` 时传入参数 ``*args`` 和 ``**kwargs``，并给定外围作用域中的目录路径 ``proj_dir`` 后：

- 如果 ``**kwargs`` 包含 ``'check'``、``'capture_output'`` 或 ``'text'`` 中的任意一个键，则函数调用将因 ``subprocess.run`` 中重复的关键字参数而引发 ``TypeError``；否则，

- 该函数构造命令列表 ``['git', '-C', proj_dir] + list(args)`` 并调用 ``subprocess.run(cmd, check=True, capture_output=True, text=True, **kwargs)``。

- 如果此次调用引发 ``CalledProcessError``（因 `git` 命令以非零退出码退出），则该异常原样传播。

- 如果引发任何其他异常（例如找不到 ``git`` 时的 ``FileNotFoundError``），该异常同样传播。

- 如果调用正常完成，则返回一个 ``CompletedProcess`` 对象 ``cp``，其 ``stdout`` 为字符串（因为 ``capture_output=True`` 且 ``text=True``）。函数接着返回 ``cp.stdout.strip()``，即去除首尾空白后的字符串。

形式化地：

令 ``cmd = ['git', '-C', proj_dir] + list(args)``。

令 ``conflict = {'check', 'capture_output', 'text'} ∩ keys(kwargs)``。

则：

- 若 ``conflict ≠ ∅``：调用引发 ``TypeError``。

- 若 ``conflict = ∅``：

- 若 ``subprocess.run(cmd, check=True, capture_output=True, text=True, **kwargs)`` 引发异常 ``E``（``E`` 可能是 ``CalledProcessError`` 或其他），则 ``_git`` 引发 ``E``。

- 否则，令 ``cp`` 为返回的 ``CompletedProcess``（``cp`` 必有 ``cp.returncode == 0``，因为 ``check=True`` 否则会引发异常）。然后 ``_git`` 返回 ``cp.stdout.strip()``。

- 代码证据：

```
第2行：    return subprocess.run(  
第3行：        ["git", "-C", proj_dir, *args],  
第4行：        check=True, capture_output=True, text=True, **kwargs,  
第5行：    ).stdout.strip()
```

- 触发条件：

该代码硬编码了 `check=True`、`capture_output=True`、`text=True`，但同时将 `**kwargs` 传递给 `subprocess.run`。如果 `kwargs` 包含 `'check'`、`'capture_output'` 或 `'text'` 中的任何一个键，对 `subprocess.run` 的调用将因重复的关键字参数而引发 `TypeError`，这与需要为任何有效的位置参数和 `env` 关键字参数执行 `git` 命令的规格不符。例如，`_git('status', capture_output=True)` 会引发 `TypeError`，而规格要求其捕获输出并返回去除空白的 `stdout` 或在失败时引发 `CalledProcessError`。

- 触发摘要: `_git()` 硬编码了 `check=True`、`capture_output=True`、`text=True`, 但将 `**kwargs` 传递给 `subprocess.run`; 在 `kwargs` 中传入这些键中的任何一个都会因重复的关键字参数而引发 `TypeError`。
- 探测标准输出:

CONFIRMED – 重复的 'capture_output' 引发 `TypeError: <MagicMock name='run' id='136370205191552'> 获取了关键字参数 'capture_output' 的多个值`

src/incremental_reasoner-py

INCR-MISMATCH-021 — src--incremental_reasoner-py--_extracted_files_by_method

- 人工审计: **SPEC 错误**。
- 验证器: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/incremental_reasoner-py/_extracted_files_by_method.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/_extracted_files_by_method.json`。
- 详细报告: `src--incremental_reasoner-py--_extracted_files_by_method.md`。
- Probe: `probe_src--incremental_reasoner-py--_extracted_files_by_method.py`。

完整生成 SPEC

[SPEC]
单元: `fm_agent/extracted_functions/src/incremental_reasoner-py/_extracted_files_by_method.py`

`_extracted_files_by_method(func_dir)`

前置条件:
- `func_dir` 是一个文件系统路径 (可能是一个存在的目录, 也可能不是)

后置条件:
- 返回一个可变的、类似字典的映射, 从字符串键 (函数名) 映射到绝对文件系统路径的列表, 每个列表包含一个或多个条目
- 当 `func_dir` 不是一个存在的目录时, 返回一个空映射 (没有任何键)
- 当 `func_dir` 是一个存在的目录时, 通过递归下降可从 `func_dir` 访问到的每个普通文件, 都根据一个或两个键进行索引, 使用文件的基名去掉最后一个点分隔的扩展名作为基础标识符 (即“stem”):
- 文件的绝对路径始终被追加到与完整 `stem` 相等的键对应的列表中
- 此外, 当 `stem` 包含至少一个“:”分隔符时, 最后一个“:”之后的子字符串 (裸方法名) 被用作第二个键, 同样的绝对路径也被追加到该键对应的列表中
- 当 `stem` 中不包含“:”分隔符时, 仅使用 `stem` 本身作为键
- 每个键的列表中的绝对路径的顺序反映了遍历过程中遇到相应文件的顺序
- 访问映射中不存在的键会返回一个空列表 (而不是引发错误), 并且对返回的列表进行修改不会影响映射
[SPEC]

Reasoner 差异

- SPEC 声称:

- 返回一个可变的、类似字典的映射, 从字符串键 (函数名) 映射到绝对文件系统路径的列表, 每个列表包含一个或多个条目
- 当 `func_dir` 不是一个存在的目录时, 返回一个空映射 (没有任何键)
- 当 `func_dir` 是一个存在的目录时, 通过递归下降可从 `func_dir` 访问到的每个普通文件, 都根据一个或两个键进行索引, 使用文件的基名去掉最后一个点分隔的扩展名作为基础标识符 (即“stem”):
- 文件的绝对路径始终被追加到与完整 `stem` 相等的键对应的列表中
- 此外, 当 `stem` 包含至少一个“:”分隔符时, 最后一个“:”之后的子字符串 (裸方法名) 被用作第二个键, 同样的绝对路径也被追加到该键对应的列表中
- 当 `stem` 中不包含“:”分隔符时, 仅使用 `stem` 本身作为键
- 每个键的列表中的绝对路径的顺序反映了遍历过程中遇到相应文件的顺序
- 访问映射中不存在的键会返回一个空列表 (而不是引发错误), 并且对返回的列表进行修改不会影响映射

- 推导的实际行为:

如果 `func_dir` 不是一个存在的目录, 函数返回一个空的 `defaultdict(list)` (`len(result) == 0`, 或等价地, 对于所有键 `k`, `result[k] == []`)。如果 `func_dir` 是一个存在的目录, 并且在 `os.walk` 过程中没有发生文件系统错误, 函数返回一个 `defaultdict(list)` 结果, 其中键是从找到的文件名中提取的 `stem` 或裸名称, 对于递归遍历中的每个文件 `fn`, 其绝对路径 `p` 被追加到 `result[stem]`, 并且如果 `stem` 包含 “:”, 也追加到 `result[bare]` (其中 `bare =`

stem.split('::')[-1] 且 bare != stem)。形式化地，设 F 为从 os.walk(func_dir) 得到的 (absolute_path, stem, bare) 元组的集合（多重集），其中 stem = fn[:fn.rfind('.')] 如果 '.' 在 fn 中，否则为 fn，而 bare = stem.split('::')[-1] 如果 '::' 在 stem 中，否则为 None。那么 result 是一个 defaultdict(list)，满足：(1) 对于 F 中的所有 (p, s, b)，p 是 result[s] 的一个元素，并且如果 b 不是 None 且 b != s，则 p 是 result[b] 的一个元素；(2) 对于任何键 k，result[k] 只包含这些路径，并且按照遍历过程中添加的顺序排列；(3) 对于所有未出现在 F 的任何 (stem, bare) 中的键 k，result[k] == []。如果 os.walk 抛出异常（例如 OSError、PermissionError），该异常将被传播，函数不会正常返回。

- 代码证据：

```
第 11 行： index = defaultdict(list)
第 27 行： return index
```

- 触发条件：

函数返回一个 collections.defaultdict(list)，当 __getitem__ 被调用时，它会自动为任何缺失的键存储一个新的列表。这意味着，如果调用者访问一个缺失的键并修改返回的列表（例如，追加一个元素），映射将被永久修改，加入该新键和元素，这与要求“对返回的列表进行修改不会影响映射”相矛盾。为了满足规范，映射需要为缺失的键返回一个副本或只读视图，或者避免使用具有副作用的默认工厂。

Bug validator

- 触发摘要：在返回的 defaultdict(list) 上访问一个缺失的键并修改返回的列表，会永久地将键值对添加到映射中，违反了规范中关于对返回列表的修改不得影响映射的要求。
- Probe 标准输出：

BUG CONFIRMED (测试 1)：访问缺失的键 'nonexistent_key' 并修改返回的列表修改了映射。初始键数：0，最终键数：1。规范要求：对返回列表的修改不得影响映射。
BUG CONFIRMED (测试 2)：在存在文件的情况下，访问缺失的键 'NonExistentFunction' 并修改返回的列表修改了映射。之前的键：['Bar', 'Foo']，之后的键：['Bar', 'Foo', 'NonExistentFunction']。规范要求：对返回列表的修改不得影响映射。
CONFIRMED

INCR-MISMATCH-022 — src--incremental_reasoner-py--_reconcile_extracted_dir

- 人工审计：SPEC 错误。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：src/incremental_reasoner-py/_reconcile_extracted_dir.py。
- Reasoner 结果：logic_verification_results/src/incremental_reasoner-py/_reconcile_extracted_dir.json。
- 详细报告：src--incremental_reasoner-py--_reconcile_extracted_dir.md。
- Probe：probe_src--incremental_reasoner-py--_reconcile_extracted_dir.py。

完整生成 SPEC

[SPEC]
单元：src/incremental_reasoner-py/_reconcile_extracted_dir.py

_reconcile_extracted_dir(proj_dir, abs_src) -> None

前置条件：

- proj_dir 是项目根目录的绝对路径
- abs_src 是 项目目录 proj_dir 内一个源文件的绝对路径

后置条件：

- 设 (func_dir, ext) 为将 abs_src 映射到提取函数目录和源扩展名的二元组，映射规则与 run_extraction 使用的命名约定相同。
 - 如果 func_dir 不是磁盘上已存在的目录，则不发生任何文件系统更改，函数直接返回。
- 否则，确定 abs_src 的预期提取函数文件集合：
 - * 当 abs_src 在磁盘上存在且其文件扩展名映射到项目语言注册表可识别的语言时，预期文件由 abs_src 当前的函数跨距 (span) 导出。每个跨距的去重标识符构成一个预期文件名：如果 ext 非空，则为标识符添加 "<ext>" 后缀；如果 ext 为空，则直接使用标识符。跨距边界使用与 run_extraction 相同的后端（当 codegraph 索引文件时使用 codegraph，否则使用正则表达式）计算。

* 当 `abs_src` 在磁盘上不存在，或者其扩展名不可识别时，预期文件集合为空。

- 递归遍历 `func_dir` 所能到达的每一个文件，如果其绝对路径与任何预期文件路径都不匹配，则将其删除。预期文件保留，其内容不变。
- 文件删除完成后，移除 `func_dir` 的每个子目录（不包括 `func_dir` 本身），条件是它既不含文件也不含子目录。

[SPEC]

Reasoner 差异

- SPEC 声明：

- 设 $(\text{func_dir}, \text{ext})$ 为将 `abs_src` 映射到提取函数目录和源扩展名的二元组，映射规则与 `run_extraction` 使用的命名约定相同。

如果 `func_dir` 不是磁盘上已存在的目录，则不发生任何文件系统更改，函数直接返回。

- 否则，确定 `abs_src` 的预期提取函数文件集合：
- * 当 `abs_src` 在磁盘上存在且其文件扩展名映射到项目语言注册表可识别的语言时，预期文件由 `abs_src` 当前的函数跨距（`span`）导出。每个跨距的去重标识符构成一个预期文件名：如果 `ext` 非空，则为标识符添加 `".<ext>"` 后缀；如果 `ext` 为空，则直接使用标识符。跨距边界使用与 `run_extraction` 相同的后端（当 `codegraph` 索引文件时使用 `codegraph`，否则使用正则表达式）计算。
- * 当 `abs_src` 在磁盘上不存在，或者其扩展名不可识别时，预期文件集合为空。
- 递归遍历 `func_dir` 所能到达的每一个文件，如果其绝对路径与任何预期文件路径都不匹配，则将其删除。预期文件保留，其内容不变。
- 文件删除完成后，移除 `func_dir` 的每个子目录（不包括 `func_dir` 本身），条件是它既不含文件也不含子目录。

- 推导实际行为：

如果在 ``os.remove`` 或 ``os.rmdir`` 期间引发异常，函数将终止并抛出该异常；文件系统将处于部分修改状态（在失败点之前已执行部分删除）。在正常终止（无异常）情况下，下列条件成立：

设 $(\text{fd}, \text{ext}) = \text{_src_rel_to_func_dir}(\text{proj_dir}, \text{abs_src})$ 。

如果在调用前 `fd` 不是目录，文件系统保持不变。否则，

设 `lang = EXT_TO_LANG.get(ext)`；

设 `Valid =` 如果 `lang` 为 `None` 或 `abs_src` 在前置状态中不是文件，则为空集；否则为

$\{ \text{os.path.abspath}(\text{os.path.join}(\text{fd}, \text{ident}) + ('.' + \text{ext} \text{ if } \text{ext} \text{ else } '')) \mid \text{ident} \in \{ \text{name} \mid (\text{name}, _, _) \in \text{_function_spans}(\text{abs_src}, \text{lang}, \text{proj_dir}) \} \}$ 。

定义 `Keep = \{ p \mid \text{pre.file}(p) \wedge p \in \text{Valid} \}`。

调用后，对任意路径 `p`：

（文件）如果 `p` 位于 `fd` 之下，则 $\text{post.file}(p) \Leftrightarrow p \in \text{Keep}$ ；

否则 $\text{post.file}(p) \Leftarrow \text{pre.file}(p)$ 。

（目录）如果 `p` 位于 `fd` 之下且 $p \neq \text{fd}$ ，则 $\text{post.dir}(p) \Leftarrow (\text{pre.dir}(p) \wedge \exists q \in \text{Keep} \text{ 使得 } p \text{ 是 } q \text{ 的真祖先})$ ；

如果 $p = \text{fd}$ ，则 $\text{post.dir}(p)$ 成立；

如果 `p` 不位于 `fd` 之下，则 $\text{post.dir}(p) \Leftarrow \text{pre.dir}(p)$ 。

（`fd` 下不存在空子目录，可能 `fd` 本身除外；`fd` 之外的目录不受影响。）

- 代码证据：

第 26 行：`os.remove(abs_path)`

- 触发条件：

规范 B 要求删除 `func_dir` 下的每个非预期文件。当 `os.remove` 引发异常（例如 `PermissionError`）时，函数立即终止，将非预期文件 `'stale.py'` 留在磁盘上。这违反了规范，因为所需的删除未完成。

Bug validator

- 触发摘要：在非可写子目录中的文件上 `os.remove()` 引发 `PermissionError`，导致函数立即终止，文件系统处于部分修改状态 — 一个孤儿文件已被删除，另一个仍存在。
- Probe 标准输出：

CONFIRMED — `PermissionError` 已引发；`stale_undeletable.txt` 仍保留在磁盘上，而 `stale_deletable.txt` 已被移除。文件系统处于部分状态（删除未完成）。

- 人工审计：契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/incremental_reasoner-py/_remove_stale_extracted.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/_remove_stale_extracted.json`。
- 详细报告: `src---incremental_reasoner-py--_remove_stale_extracted.md`。
- Probe: `probe_src---incremental_reasoner-py--_remove_stale_extracted.py`。

完整生成 SPEC

```
[SPEC]
_remove_stale_extracted(proj_dir, modified_functions) -> None

前置条件：
- proj_dir 是项目根目录的绝对路径，其下存在子目录 fm_agent/，该子目录下有 extracted_functions/ 子目录和一个 phases.json 文件。
- modified_functions 是一个字典，其键为绝对源码文件路径（该函数不使用其值）。

后置条件：
- 对于每一个作为 modified_functions 的键或通过 phases.json 加载的所有 phase 中 “source_files” 条目列出的绝对源码文件路径，fm_agent/extracted_functions/ 下与该源码文件关联的 extracted-function 树将根据当前 codegraph 输出进行协调。任何不再对应某个当前源码函数的 extracted 函数文件或目录（包括当源码文件本身不存在时）均被删除，且所有空的父目录均被剪除。
- fm_agent/extracted_functions/ 下那些对应于不在 modified_functions 键和 phases.json 条目并集中的源码文件的文件和目录保持不变。
[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 对于每一个作为 modified_functions 的键或通过 phases.json 加载的所有 phase 中 “source_files” 条目列出的绝对源码文件路径，fm_agent/extracted_functions/ 下与该源码文件关联的 extracted-function 树将根据当前 codegraph 输出进行协调。任何不再对应某个当前源码函数的 extracted 函数文件或目录（包括当源码文件本身不存在时）均被删除，且所有空的父目录均被剪除。

- fm_agent/extracted_functions/ 下那些对应于不在 modified_functions 键和 phases.json 条目并集中的源码文件的文件和目录保持不变。

- 推导的实际行为：

执行后，fm_agent/ 下的 extracted-functions 子目录树已更新，使得对于属于并集 $S = \text{modified_functions.keys()} \cup (\text{如果加载 phases.json 成功}) \text{所有 phases 内 modules 中 source_files 条目派生的绝对路径的每个源文件路径}$ ，对应于该源文件的 extracted 目录（如果存在）仅包含根据该源文件的函数跨度当前所期望的文件，任何陈旧的 extracted 文件已被删除，且空的子目录已被剪除。对于不在 S 中的任何源文件路径，其 extracted 目录未作任何更改。无其他副作用发生。形式逻辑：令 $M = \text{dom(modified_functions)}$ 。令 $P = \emptyset$ （如果在加载 phases.json 时发生了 `OSError`、`ValueError` 或 `KeyError`）；否则 $P = \{ \text{os.path.abspath(os.path.join(proj_dir, rel))} \mid \text{phase} \in \text{phases_data['phases']}, \text{module} \in \text{phase['modules']}, \text{rel} \in \text{module['source_files']} \}$ 。则 $S = M \cup P$ 。对于每个 $\text{abs_src} \in S$ ，已应用 `_reconcile_extracted_dir(proj_dir, abs_src)` 的效果，即： abs_src 的 extracted 目录（如果不存在）保持不变；否则，其内容现在等于从当前源代码派生的期望的 extracted 文件路径集合，且所有陈旧的文件/目录已被移除。对于 $\text{abs_src} \notin S$ ，其 extracted 目录（如果存在）不变。函数返回 `None`。

- 代码证据：

```
第 23 行： for abs_src in srcs:
第 24 行：     _reconcile_extracted_dir(proj_dir, abs_src)
```

- 触发条件：

该代码仅对每个单独的源文件调用 `_reconcile_extracted_dir`。当所有子目录被删除时，可能会留下由多个特定于源文件的目录共享的空父目录（例如 `a/`）。规范明确要求剪除所有空的父目录，而代码并未保证这一点。

- 触发摘要: `_remove_stale_extracted` 对每个源文件调用 `_reconcile_extracted_dir`, 它会移除陈旧文件, 但从不移除 `func_dir` 自身或剪除其上方的空父目录; 当多个已删除的源文件共享一个共同的父目录时, 会留下空目录。
- Probe 标准输出:

已确认 — 留下空的函数目录 `a-py` 和 `b-py`; 父目录 `pkg/` 未被剪除。规范要求剪除所有空的父目录。

INCR-MISMATCH-024 — `src--incremental_reasoner-py--_update_specs_for_intent`

- 人工审计: 推理误判。
- Validator: **not_confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/incremental_reasoner-py/_update_specs_for_intent.py`。
- Reasoner 结果: `logic_verification_results/src/incremental_reasoner-py/_update_specs_for_intent.json`。
- 详细报告: `src--incremental_reasoner-py--_update_specs_for_intent.md`。
- Probe: `probe_src--incremental_reasoner-py--_update_specs_for_intent.py`。

完整生成 SPEC

```
[SPEC]
单元: src/incremental_reasoner.py

_update_specs_for_intent(proj_dir, work_dir, developer_intent, changed_functions,
relevant_rel_files, extra_call_edges=None) -> list[str]

前置条件:
- proj_dir 是一个存在的目录路径
- work_dir 是一个目录路径, 其中包含 extracted_functions/ 子目录, 该子目录内有提取的函数文件, 还有一个 spec_prompts/ 子目录, 其中包含领域上下文和批处理提示元数据
- developer_intent 是一个非空字符串, 描述开发者的修改目标
- changed_functions 是一个字典, 将绝对源文件路径映射到字典, 每个字典至少包含键 "added" 和 "modified", 其值是在源代码中出现的函数名列表
- relevant_rel_files 是一个字符串列表, 每个字符串是从 extracted_functions/ 目录出发的相对路径, 标识一个用于初始播种的提取函数文件
- extra_call_edges, 当不为 None 时, 提供静态分析之外补充的调用者到被调用者边

后置条件:
- 返回一个提取函数相对路径 (相对于 extracted_functions/ 目录) 的列表, 这些文件的 [SPEC] 块、[INFO] 块或两者都
```

Reasoner 差异

- SPEC 声称:

```
- 返回一个提取函数相对路径 (相对于 extracted_functions/ 目录) 的列表, 这些文件的 [SPEC] 块、[INFO] 块或两者都
    在此次调用中被创建或修改; 列表按字典序排序
- 当归并后的种子集 (changed_functions 中 "added" 和 "modified" 分类下的所有函数名, 加上所有从 relevant_rel_files 派生的全限定名) 为空时, 返回空列表
- 对于完全限定名属于种子集 of 每个函数:
    * 如果该函数的提取文件没有预先存在的 [SPEC] 块, 则由一个外部过程在开发者意图和调用者上下文的指引下生成一个新的行为 [SPEC] 块 (以及可选的 [INFO] 被调用者期望块), 然后插入到文件开头; 块之后的原始函数源代码保持不变
    * 如果存在预先存在的 [SPEC] 块, 则由一个外部过程根据 developer_intent 对其进行重新评估; 如果评估确定规范必须更改, 则替换该 [SPEC] 块 (如果 [INFO] 块也必须更改, 则可能同时替换); 源代码保持与其输入形式完全相同
- 函数按照调用者先于被调用者的拓扑轮次进行处理: 在每一轮中, 仅当某个函数的调用者 (根据调用图) 已被处理或不在当前待处理集中时, 该函数才有资格被处理; 如果某个函数的任何调用者仍处于待处理状态, 则该函数被推迟到后续轮次
- 当函数的 [SPEC] 被新生成或更新时:
    * 该函数的每个调用者都会针对新 [SPEC] 调和其关于该函数的 [INFO] 条目, 调和由外部过程完成; 仅当调和产生的 [INFO] 块与现有块不同时, 才覆写调用者文件
    * 函数更新后的 [INFO] 块中列出的被调用者, 如果匹配到之前未检查的全限定名, 则会被加入待处理前沿, 并在后续轮次中接受规范检查
- 在存在循环依赖的情况下, 即每个待处理函数都至少有一个同样待处理的调用者时, 将恰好选择一个待处理函数来打破循环, 以确保向前推进
- 每个被写入文件的源代码部分 (前导 [SPEC] 和 [INFO] 注释块之后的所有内容) 与修改前从该文件读出的源代码逐字节完全相同
- 不在 work_dir/extracted_functions/ 之外创建、删除或修改任何文件
```

- 推导的实际行为：

如果第 41-80 行调用的任何函数（例如 `_file_to_fqn`, `_topdown_ordered_fqns`）抛出异常，则该函数以该异常终止，并且因为在此代码块中没有文件写入，所有 `work_dir/extracted_functions/` 下的文件内容与第 41 行开始时的内容保持一致。

否则（正常流程），令 `preSeed` 为第 41 行之前 ``seed`` 中已有的 `FQN` 集合，并令 `preFiles` 为该时刻的文件状态。定义 `added = { _file_to_fqn(os.path.join(extracted_dir, rel), work_dir) | rel ∈ relevant_rel_files }`。在第 80 行之后，有两种情况：

1. 如果 `preSeed ∪ added` 为空，则函数立即返回 `[]`。提取函数文件未变：对于所有 $p \in AFiles$, $Files(p) = preFiles(p)$ 。函数正常终止。
2. 如果 `preSeed ∪ added` 非空，则执行继续超过第 80 行。变量 ``seed`` 持有 `preSeed ∪ added`, ``topdown`` 和 ``order_index`` 由相应的调用计算，嵌套函数 ``_plan_spec_update`` 被定义但未被调用。尚未执行任何文件写入，因此对于所有 $p \in AFiles$, $Files(p) = preFiles(p)$ 。函数尚未返回。

正式地，使用给定前置条件中的记号：

令 `OldEF_41` 为在第 41 行开始时 `Files` 在 `extracted_dir` 下路径上的限制。令 `seed_41` 为那一刻种子变量的值。令 `AddFQN = { _file_to_fqn(os.path.join(extracted_dir, rel), work_dir) | rel ∈ relevant_rel_files }`。

在第 41-80 行执行之后（无异常）：

```
(seed_41 ∪ AddFQN = ∅) ⇒
  result = [] ∧ ∀ fqn ∈ FQNs . NewEF_41(FQNMap[fqn]) = OldEF_41(FQNMap[fqn]) ∧ 函数返回。
(seed_41 ∪ AddFQN ≠ ∅) ⇒
  seed = seed_41 ∪ AddFQN ∧ topdown = _topdown_ordered_fqns(work_dir,
extra_call_edges=extra_call_edges)
  ∧ order_index = { fqn ↦ i | (i, fqn) ∈ enumerate(topdown) }
  ∧ _plan_spec_update 被定义 ∧ ∀ fqn ∈ FQNs . NewEF_41(FQNMap[fqn]) = OldEF_41(FQNMap[fqn])
∧ 函数尚未返回。
```

如果任何被调用的辅助函数抛出异常 `E`，则该函数以 `E` 终止，且 $\forall fqn \in FQNs . NewEF_41(FQNMap[fqn]) = OldEF_41(FQNMap[fqn])$ 。

- 代码证据：

```
第 41 行： seed.add(_file_to_fqn(os.path.join(extracted_dir, rel), work_dir))
```

- 触发条件：

种子集仅从 ``relevant_rel_files`` 中填充；``changed_targets``（来自 `'added'` 和 `'modified'` 分类的函数）从未被添加到种子集中。因此，当 ``relevant_rel_files`` 为空但 ``changed_targets`` 非空时，代码错误地返回一个空列表，违背了所有变更函数都必须被处理的要求。

Bug validator

- 触发摘要：种子集同时从 `changed_targets`（通过 `seed.update`）和 `relevant_rel_files` 中填充；逻辑验证器遗漏了 `seed.update(changed_targets.keys())` 调用。
- Probe 标准输出：

```
NOT CONFIRMED – 在 _update_specs_for_intent 中发现了 seed.update(changed_targets.keys());
changed_targets 与 relevant_rel_files 一起被添加到种子集中。
```

INCR-MISMATCH-025 — `src--incremental_reasoner-py--collect_relevant_function_scope`

- 人工审计：推理误判。
- Validator：**not_confirmed**；尝试次数：1。
- 原始函数与 SPEC：`src/incremental_reasoner-py/collect_relevant_function_scope.py`。
- Reasoner 结果：`logic_verification_results/src/incremental_reasoner-py/collect_relevant_function_scope.json`。
- 详细报告：`src--incremental_reasoner-py--collect_relevant_function_scope.md`。

- Probe: `probe_src--incremental_reasoner-py--collect_relevant_function_scope.py`。

完整生成 SPEC

```
[SPEC]
单元：fm_agent/extracted_functions/src/incremental_reasoner-py/collect_relevant_function_scope.py

collect_relevant_function_scope(proj_dir, developer_intent, changed_functions, range=None) ->
list[str]

前置条件：
- proj_dir 是一个项目目录的路径，其 fm_agent/ 子目录下包含 phases.json（其中有一个 "phases" 列表，
  每个 phase 对象包含一个 "modules" 列表）以及 extracted_functions/ 目录
- developer_intent 是一个非空字符串，描述修改目标
- changed_functions 是一个字典，将绝对源文件路径映射到另一个字典，后者在 "added"、"modified"、
  "removed" 键下包含字符串列表值
- range 为 None 或一个非负整数

后置条件：
- 返回一个路径列表，每个路径相对于 extracted_functions/ 目录，按与 developer_intent 的相关性降序排列；
  相关性相同的路径按字典序排列
- 每个返回的路径都指向 extracted_functions/ 下一个存在的常规文件
- 当 range 不为 None 时，返回的列表长度 ≤ range
- 当 phases.json 未定义任何模块，或相关性评估未选中任何模块时，返回空列表
- 模块被选中的条件是：其自然语言描述（如 phases.json 中记录）被评估为与开发者意图相关，
  或者该模块包含至少一个源文件，其路径（相对于 proj_dir）与 changed_functions 中的某个键匹配
- 在每个选中的模块内，仅当源文件的内容被评估为与开发者意图相关时，该源文件才会被纳入，
  但 changed_functions 中存在的所有源文件无条件纳入
- 当无法获得模块级的文件相关性评估时，该模块中的所有源文件都会被纳入
- 在每个纳入的源文件中，其抽取函数的集合中，那些相关性得分（由基于 developer_intent 的启发式信号
  计算得出）在该文件中排名靠前的函数会被纳入
- 当无法获得某个纳入源文件的文件级函数排名时，该文件中的所有抽取函数都会被纳入
- 映射到同一源码级函数的多个抽取函数文件会被去重，仅保留相关性得分最高的那个

[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 返回一个路径列表，每个路径相对于 extracted_functions/ 目录，按与 developer_intent 的相关性降序排列；相关性相同的路径按字典序排列
- 每个返回的路径都指向 extracted_functions/ 下一个存在的常规文件
- 当 range 不为 None 时，返回的列表长度 ≤ range
- 当 phases.json 未定义任何模块，或相关性评估未选中任何模块时，返回空列表
- 模块被选中的条件是：其自然语言描述（如 phases.json 中记录）被评估为与开发者意图相关，或者该模块包含至少一个源文件，其路径（相对于 proj_dir）与 changed_functions 中的某个键匹配
- 在每个选中的模块内，仅当源文件的内容被评估为与开发者意图相关时，该源文件才会被纳入，但 changed_functions 中存在的所有源文件无条件纳入
- 当无法获得模块级的文件相关性评估时，该模块中的所有源文件都会被纳入
- 在每个纳入的源文件中，其抽取函数的集合中，那些相关性得分（由基于 developer_intent 的启发式信号计算得出）在该文件中排名靠前的函数会被纳入
- 当无法获得某个纳入源文件的文件级函数排名时，该文件中的所有抽取函数都会被纳入
- 映射到同一源码级函数的多个抽取函数文件会被去重，仅保留相关性得分最高的那个

- 推导的实际行为：

令 L 为 `collect_relevant_function_scope` 返回的列表。该函数不会修改其任何参数或文件系统（它只读取文件并写入日志）。

在给定的前置条件下（`proj_dir` 包含 `fm_agent/phases.json` 和 `fm_agent/extracted_functions`，`developer_intent` 为非空字符串，`changed_functions` 具有所需的结构，`range` 为 `None` 或非负整数），以下行为成立：

如果从 `phases.json` 中泛化出的模块列表为空（所有 `phase` 中均无 `module`），则 `L = []`。

否则，函数执行三趟处理：

1. 模块选择：一个 LLM 选择与 `developer_intent` 相关的模块子集。
2. 文件选择：对于每个选中的模块，`opcode` 选择其源文件的子集。
3. 函数选择：对于每个选中的文件，`rank_functions_in_file` 对函数进行排名，并返回一个按得分降序排列的字典列表。通过 `_extracted_files_by_method`，每个相关函数被映射到一个或多个相对于 `proj_dir/fm_agent/extracted_functions` 的抽取函数文件路径。这些路径的并集，按原始降序得分排序（保持每个文件内的顺序，并按模块-文件相关性降序合并文件），形成序列 S。

如果 S 为空（未选中任何模块、文件或函数），则 L = []。

如果 range 为 None，则 L = S。

如果 range 为一个非负整数，则 L = S[:range]（最多取前 range 个路径）。

因此：L 是一个字符串列表，每个字符串是 extracted_functions 目录下的一个相对路径，L 按与 developer_intent 的相关性递减排序。当选择过程未产生任何结果时，列表为空。

- 代码证据：

```
第 1 行: def collect_relevant_function_scope(proj_dir, developer_intent, changed_functions,
range=None):
第 40 行:         # Pass 1: module selection. The module descriptions are already parsed from
phases.json
```

- 触发条件：

规范要求，当一个模块包含至少一个源文件，且该文件的相对路径与 changed_functions 中的某个键匹配时，即使其描述未被评估为相关，该模块也应被选中。但代码仅对模块描述进行基于 LLM 的相关性评估，并未纳入 changed_functions 这一条件。在这个反例中，具有不相关描述的模块包含一个存在于 changed_functions 中的文件，因此它本应被选中，但代码忽略了它，从而返回了空列表，而非所要求的非空结果。

Bug validator

- 触发摘要：Bug 声明声称 changed_functions 条件未被纳入；对代码第 1158-1162 行的检查显示 or any() 子句是存在的，而探针证实即使 LLM 评估返回空时，它仍会选中包含已更改文件的模块。
- Probe 标准输出：

```
NOT CONFIRMED - changed_functions criterion IS incorporated: selected modules=['util_module'],
expected=['util_module']
```

INCR-MISMATCH-026 — `src--incremental_reasoner-py--run_incremental_pipeline`

- 人工审计：推理误判。
- Validator：not_confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/incremental_reasoner-py/run_incremental_pipeline.py`。
- Reasoner 结果：`logic_verification_results/src/incremental_reasoner-py/run_incremental_pipeline.json`。
- 详细报告：`src--incremental_reasoner-py--run_incremental_pipeline.md`。
- Probe：`probe_src--incremental_reasoner-py--run_incremental_pipeline.py`。

完整生成 SPEC

```
[SPEC]
单元：src/incremental_reasoner-py/run_incremental_pipeline.py

run_incremental_pipeline(proj_dir, intent_file_path, old_commit_id,
                        domain_knowledge_files=None, submodules=None,
                        one_phase=False, extra_call_edges_path=None)

-> list[str] | None
```

前置条件：

- proj_dir 是一个路径，指向一个包含受版本控制的源代码的现有目录；fm_agent/ 是该目录内的一个可写子目录。
- intent_file_path 是一个字符串；当为非空且指向一个常规文件时，其内容描述开发者的修改目标。
- old_commit_id 是一个字符串，标识 proj_dir 的 git 仓库中的一个先前的提交。
- domain_knowledge_files，当不为 None 时，是一个 Markdown 文件路径列表，提供项目特定的领域上下文。
- submodules，当不为 None 时，是一个子目录路径列表，用于限定分析范围，这些子目录均在 proj_dir 内。
- one_phase 是一个布尔值（默认为 False），控制是否将所有源文件放入单一的阶段。
- extra_call_edges_path，当不为 None 时，是一个指向 JSON 文件的路径，该文件定义了补充的调用图边。

后置条件：

- 如果 proj_dir 没有之前的全量运行基线（fm_agent/phases.json 不存在或 fm_agent/extracted_functions/ 在给定 submodules 的情况下不完整），则将整个流程委托给全量运行，通过 run_pipeline() 使用相同的参数，并返回

None。

- 如果 `intent_file_path` 未指向现有的常规文件，或者文件内容在去除空白后为空，则记录一条错误并返回 `None`，而不修改任何项目或 `fm_agent/` 文件。
- 在产生任何新的输出之前，移除 `fm_agent/logic_verification_results/` 和 `fm_agent/bug_validation/` 下的所有文件，并移除 `fm_agent/` 中前缀为 `"select_relevant_"`、`"relevant_"` 和 `"spec_update_"` 的增量范围选择与规格更新产出物。
- 从当前工作树重新生成 `fm_agent/phases.json`。
- 从当前代码中重新提取每个函数，然后对于其函数体在 `old_commit_id` 与当前工作树之间完全相同的每个函数，恢复先前运行中捕获的 `[SPEC]` 和 `[INFO]` 块。

Reasoner 差异

- SPEC 声称：

- 如果 `proj_dir` 没有之前的全量运行基线（`fm_agent/phases.json` 不存在或 `fm_agent/extracted_functions/` 在给定 `submodules` 的情况下不完整），则将整个流程委托给全量运行，通过 `run_pipeline()` 使用相同的参数，并返回 `None`。
- 如果 `intent_file_path` 未指向现有的常规文件，或者文件内容在去除空白后为空，则记录一条错误并返回 `None`，而不修改任何项目或 `fm_agent/` 文件。
- 在产生任何新的输出之前，移除 `fm_agent/logic_verification_results/` 和 `fm_agent/bug_validation/` 下的所有文件，并移除 `fm_agent/` 中前缀为 `"select_relevant_"`、`"relevant_"` 和 `"spec_update_"` 的增量范围选择与规格更新产出物。
- 从当前工作树重新生成 `fm_agent/phases.json`。
- 从当前代码中重新提取每个函数，然后对于其函数体在 `old_commit_id` 与当前工作树之间完全相同的每个函数，恢复先前运行中捕获的 `[SPEC]` 和 `[INFO]` 块。
- 为每个已修改的源代码文件生成一个映射，映射到自 `old_commit_id` 以来添加、修改或删除的函数名称集合；并删除已移除函数对应的提取函数文件。
- 生成一个按相关性排名的提取函数相对路径列表，这些函数的实现被判定为与开发者意图相关。
- 对于每个发生了更改（添加或修改）或出现在相关性排名列表中的函数，重新评估其 `[SPEC]` 和/或 `[INFO]` 块是否需要更新以反映当前的代码和意图；当被调用者的 `[SPEC]` 发生变化时，将更新传播到每个调用者的 `[INFO]` 块。将被修改了规格的文件集合写入 `fm_agent/incremental_updated_specs.json`。
- 在受影响的子集上运行验证：每个更改的函数、每个规格被更新的函数，以及每个调用了规格被更新的被调用者的函数。返回一个已排序的提取函数相对路径列表，对于这些路径，推理器报告了规格与代码不匹配（`MISMATCH` 判决）并且随后的错误验证确认了违规。当没有确认的违规时，返回一个空列表。
- 不修改 `proj_dir` 内除 `fm_agent/` 之外的任何文件。

- 推导的实际行为：

****正常终止路径：****

- 如果 `check_last_run_existence(proj_dir, submodules)` 返回 `False`：该函数发出警告日志消息（"未检测到之前的全量运行..."），调用 `run_pipeline(proj_dir, domain_knowledge_files=..., submodules=..., ...)`（该调用执行完整流程并在 `fm_agent/` 下生成产出物），然后立即返回 `None`。
- 如果 `check_last_run_existence` 返回 `True`，但位于 `intent_file_path` 的意图文件不存在或去除空白后为空：该函数记录错误（"意图文件 ... 不存在" 或 "为空"）并立即返回 `None`。
- 否则（上次运行存在且意图文件有效）：函数记录发现之前的运行，记录 "[2/10 阶段] 正在加载开发者意图..."，读取非空的开发者意图并绑定至 `developer_intent`，记录 "意图已加载(%d 个字符)"，然后移除陈旧目录 `output_dir` 和 `<work_dir>/bug_validation`（如果它们存在，每移除一个目录记录一行日志）。执行在设置 `developer_intent` 并移除陈旧产出物之后，继续至第 80 行之后；此时尚未返回值。

代码块 ****总是**** 执行的额外日志副作用（除非异常阻止到达）：

- 发出一个 70 个 `'='` 字符组成的分隔线（第 41 行）。
- 发出消息 "[1/10 阶段] 正在检查是否有之前的全量运行可作比较..."。
- 在 `check_last_run_existence` 的 `True` 分支中：" -> 发现之前的全量运行；继续进行增量分析。" 被发出。

****异常路径：****

- 如果 `check_last_run_existence` 引发异常，它会立即传播到调用者；到该点为止发出的任何日志消息（分隔线和 "[1/10 阶段] ..." 消息）会保留。
- 如果意图文件的打开或读取引发异常，它会传播；"[2/10 阶段] ..." 日志以及之前的阶段1日志会保留。
- 如果 `run_pipeline` 引发异常，异常会传播，并且 `run_pipeline` 在失败点之前造成的所有副作用，与之前的日志及 `run_pipeline` 调用本身一起，会被持久化；函数不会到达 `return`。
- 其他操作（`shutil.rmtree`、`os.path.isdir`、`logging.info` 等）假设在给定前置条件下不会引发异常。

****形式逻辑：****

设 $\backslash(\text{PRE})$ 表示刚好在第 41 行之前的前置条件状态，包括：

- `work_dir`、`input_dir`、`output_dir`、`extra_call_edges`、`staged_knowledge` 已按前述说明绑定。
- 增量日志已配置。
- 下列日志记录已在此之前发出（由于之前的代码）：如果 `staged_knowledge` 为真，发出包含 Markdown 文件数量的行；一个 70 个 `'='` 字符的分隔行；"增量流水线开始"；项目目录；意图文件路径；基线提交 ID；可能还有子模块范围。
- 到目前为止没有异常发生。

设 $\backslash(\text{POST})$ 为代码块（第 4180 行）结束后的状态。 $\backslash(\text{POST})$ 满足：

$$\begin{aligned} &\backslash[\\ &\backslash\text{begin}\{\text{aligned}\} \\ &\& \text{PRE} \backslash\text{and} \backslash\backslash \\ &\& (\backslash\text{text}\{\text{normal}\backslash_exit\} \backslash\text{implies} \backslash\backslash \\ &\& \backslash\text{quad} \backslash\text{exists} \backslash, \backslash\text{text}\{\text{separator}\backslash_log\}, \backslash\text{text}\{\text{stage1}\backslash_log\}, \backslash\text{text}\{\text{result}\backslash_log\}, \backslash\text{text}\{\text{etc.}\} \end{aligned}$$

```

\text{ 由此块发出} \\
& \quad \land \; ; \; ( \text{check\_path} = \text{false} \implies \\
& \quad \text{run\_pipeline\_called}(proj\_dir, \dots) \land \text{return\_value} = \text{None} \\
& \\
& \quad \land \text{full\_pipeline\_side\_effects}(proj\_dir) \\
& \quad \land \text{log\_record\_exists}(\text{warning, "未检测到之前的全量运行..."}) \\
& \quad ) \land \; ; \; ( \text{check\_path} = \text{true} \implies \\
& \quad ( \text{intent\_file\_invalid} \implies \\
& \quad \quad \text{log\_record\_exists}(\text{error, "意图文件 ..."} ) \land \text{return\_value} \\
& \quad = \text{None} ) \\
& \quad ) \land \; ; \; ( \text{intent\_file\_valid} \implies \\
& \quad \quad \text{developer\_intent} = \text{non\_empty\_content} \\
& \quad \quad \land \; ; \; \text{stale\_dirs\_removed}(output\_dir, \\
& \quad \quad work\_dir/\text{bug\_validation}) \\
& \quad \quad \quad \text{no\_return} \; ; \; \text{执行继续} \\
& \quad ) \\
& \quad ) \\
& \quad ) \\
& \quad \land \; ; \; ( \text{exception\_exit} \implies \\
& \quad \quad \text{propagated\_exception} \land \text{partial\_effects\_preserved\_up\_to\_failure} \\
& \quad ) \\
& \quad ) \\
\end{aligned}
\]

```

其中：

- (check_path) 是 `check_last_run_existence` 的返回值 (`True/False`)。
- $(\text{intent_file_invalid})$ 表示文件不存在或去除空白后为空。
- $(\text{stale_dirs_removed} \dots)$ 表示如果某个目录存在，它被递归删除并且记录了一行日志。

- 代码证据：

```

第 77 行：    for stale_dir in (output_dir, os.path.join(work_dir, "bug_validation")):
第 78 行：        if os.path.isdir(stale_dir):
第 79 行：            shutil.rmtree(stale_dir, ignore_errors=True)
第 80 行：            logging.info(" -> removed stale results dir %s.", stale_dir)

```

- 触发条件：

规格要求在产生新输出之前，移除增量范围选择和规格更新的产出物（前缀为 `'select_relevant_'`、`'relevant_'` 和 `'spec_update_'` 的文件）。代码仅移除了两个目录，而忽略了要求的基于前缀的文件删除操作，因此如果存在这些文件，一个有效的运行会使系统处于违反后续增量步骤前提条件的状态。

Bug validator

- 触发总结：引用的 `code_evidence` 仅展示了目录删除行，而遗漏了紧随其后的产出物删除代码（第 231-243 行），该代码通过 `glob.glob + os.remove` 删除匹配 `select_relevant_`、`relevant_` 和 `spec_update_*` 模式的文件 — 规格得到了满足。
- Probe 标准输出：

未确认 — 代码包含所需的带前缀文件删除：源中存在全部 6 个 `glob` 模式

src/languages/c-py

INCR-MISMATCH-027 — src--languages--c-py--batch_extract

- 人工审计：推理误判。
- Validator: **confirmed**；尝试次数：1。
- 原始函数与 SPEC: `src/languages/c-py/batch_extract.py`。
- Reasoner 结果: `logic_verification_results/src/languages/c-py/batch_extract.json`。
- 详细报告: `src--languages--c-py--batch_extract.md`。
- Probe: `probe_src--languages--c-py--batch_extract.py`。

```
[SPEC]
单元：src/languages/c-py/batch_extract.py

batch_extract(proj_dir) -> dict

前置条件：
- proj_dir 是一个指向包含 C 源文件的现有目录的路径

后置条件：
- 返回一个字典，将绝对文件路径映射到由 (function_name, function_body) 元组组成的列表，这些元组对应于使用 codegraph 分析从项目中提取的每个 C 函数
- 当 codegraph 初始化失败时，返回空字典 {}
[SPEC]
```

Reasoner 差异

- SPEC 断言：

```
- 返回一个字典，将绝对文件路径映射到由 (function_name, function_body) 元组组成的列表，这些元组对应于使用 codegraph 分析从项目中提取的每个 C 函数
- 当 codegraph 初始化失败时，返回空字典 {}
```

- 推导出的实际行为：

```
该函数返回一个字典。如果 CodeGraphExtractor.from_proj_dir(proj_dir) 返回一个有效的提取器对象 C（即不为 None），则结果等于 C.get_functions_by_file('c', proj_dir)：一个字典，其键为绝对文件路径（字符串），值为（函数名，函数体）元组的列表，覆盖 proj_dir 下所有可成功读取的 C 文件；无法读取的文件会被省略。如果初始化失败（C 为 None），则结果为空字典 {}。形式化描述：令 C = CodeGraphExtractor.from_proj_dir(proj_dir)，则 result = (若 C 不为 None 则为 C.get_functions_by_file('c', proj_dir)，否则为 {})。
```

- 代码证据：

```
第 4 行: return cg.get_functions_by_file("c", proj_dir) if cg else {}
```

- 触发条件：

```
代码使用 cg 的真值性 (if cg) 来检查初始化是否成功，但 from_proj_dir 的规范仅保证其返回一个已初始化的 CodeGraphExtractor 或 None；它并不保证返回的对象是真值。一个已初始化的提取器可能被评估为假（例如，如果它定义了 __len__ 并返回 0），导致函数错误地返回 {}，从而违反初始化成功时应返回提取到的函数的要求。
```

Bug validator

- 触发摘要：batch_extract 使用了真值性检查 (if cg) 而非显式的 None 检查，因此一个非 None 但为假值的 CodeGraphExtractor 会错误地返回 {} 而非提取到的函数。
- 探针标准输出：

```
CONFIRMED — 实际值：{} | 期望值：{'/fake/path.c': [('main', 'int main(void) {}\\n')]}
```

INCR-MISMATCH-028 — src--languages--c-py--function_spans

- 人工审计：SPEC 错误。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：src/languages/c-py/function_spans.py。
- Reasoner 结果：logic_verification_results/src/languages/c-py/function_spans.json。

- 详细报告: `src--languages--c-py--function_spans.md`。
- Probe: `probe_src--languages--c-py--function_spans.py`。

完整生成 SPEC

```
[SPEC]
单元 : src/languages/c-py/function_spans.py

function_spans(proj_dir: str, filepath: str) -> list[tuple[str, int, int]] | None

前置条件 :
- proj_dir 是一个现有项目目录的路径
- filepath 是一个带有 “.c” 扩展名的 C 源文件路径

后置条件 :
- 当无法从 proj_dir 初始化代码图实例时, 返回 None
- 否则返回一个 (function_name, start_idx, end_idx) 元组的列表, 对应于 filepath 处的 C 源文件中定义的每个函数,
    其中 start_idx 和 end_idx 是从 0 开始的包含行号
[SPEC]
```

Reasoner 差异

- SPEC 声称:

```
- 当无法从 proj_dir 初始化代码图实例时, 返回 None
- 否则返回一个 (function_name, start_idx, end_idx) 元组的列表, 对应于 filepath 处的 C 源文件中定义的每个函数,
    其中 start_idx 和 end_idx 是从 0 开始的包含行号
```

- 推导的实际行为:

```
执行后, 该函数返回 `None` 或一个 `(name, start_idx, end_idx)` 元组的列表。形式上, 令 `cg = CodeGraphExtractor.from_proj_dir(proj_dir)`。如果 `cg is None`, 则返回值为 `None`。否则, 返回值为 `cg.get_function_spans("c", filepath)`。因此, 当且仅当 `proj_dir` 的代码图不可用或 `get_function_spans` 调用返回 `None` 时 (例如, 语言键无法识别、文件未索引、无定义或路径解析失败), 整体返回值才为 `None`。如果返回值不是 `None`, 则它是一个 Python 列表 `L`, 其中每个元素是一个元组 `(name: str, start_idx: int, end_idx: int)`, 使得 `0 <= start_idx <= end_idx`, 这些元组表示在 `filepath` 指向的 C 源文件中检测到的所有函数/方法定义, `name` 是类限定标识符, 并且 `L` 按升序 `start_idx` 排序。调用方在返回值之外看不到任何副作用。
```

- 代码证据:

```
第8行: return cg.get_function_spans("c", filepath) if cg else None
```

- 触发条件:

```
规范指出, 只有在无法从 proj_dir 初始化代码图实例时才返回 None; 否则必须返回 (可能为空的) 函数元组列表。然而, 代码在 get_function_spans 因非初始化失败的原因返回 None 时 (例如, 文件没有定义) 也返回 None。这违反了规范, 因为对于有效的 proj_dir 和一个没有函数的 C 文件, 代码返回 None 而不是空列表。
```

Bug validator

- 触发摘要: 对于有效的 proj_dir 和一个没有函数的 C 文件, `function_spans` 返回 `None` 而不是空列表, 因为 `get_function_spans` 对没有定义的文件返回 `None`, 而代码直接传递了该值。
- Probe 标准输出:

```
CONFIRMED — actual: None | expected: []
```


INCR-MISMATCH-029 — src--languages--codegraph-py--
CodeGraphExtractor::get_functions_by_file

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：src/languages/codegraph-py/CodeGraphExtractor::get_functions_by_file.py。
- Reasoner 结果：logic_verification_results/src/languages/codegraph-py/CodeGraphExtractor::get_functions_by_file.json。
- 详细报告：src--languages--codegraph-py--CodeGraphExtractor::get_functions_by_file.md。
- Probe：probe_src--languages--codegraph-py--CodeGraphExtractor::get_functions_by_file.py。

完整生成 SPEC

```
[SPEC]
单元：src/languages/codegraph.py

CodeGraphExtractor.get_functions_by_file(lang_key: str, proj_dir: str = None) -> dict

前置条件：
- lang_key 是一个字符串
- proj_dir 是一个目录的字符串路径，或者为 None
- 接收者拥有一个已初始化的、可供读取的 codegraph 数据库

后置条件：
- 返回一个 dict，其键是绝对文件系统路径（str），值是（str，str）元组的列表
- 每个元组由一个函数标识符和对应函数体的完整源代码文本组成
- 每个函数体以一个换行符（"\n"）结尾
- 对于给定文件，元组按函数定义在源文件中的行号升序排列
- 函数标识符是带类限定的；当同一文件中多个函数具有相同标识符时，首次出现保留原始名称，后续每次出现追加一个从 1
  开始的数字后缀
- 当 lang_key 不是可识别的语言标识符时，返回的 dict 为空
- 无法打开读取的源文件会从结果中省略；不抛出错误
- 当提供 proj_dir 时，数据库中存储的文件路径会相对于 proj_dir 解析以产生绝对键

[SPEC]
```

Reasoner 差异

- SPEC 声称：

```
- 返回一个 dict，其键是绝对文件系统路径（str），值是（str，str）元组的列表
- 每个元组由一个函数标识符和对应函数体的完整源代码文本组成
- 每个函数体以一个换行符（"\n"）结尾
- 对于给定文件，元组按函数定义在源文件中的行号升序排列
- 函数标识符是带类限定的；当同一文件中多个函数具有相同标识符时，首次出现保留原始名称，后续每次出现追加一个从 1
  开始的数字后缀
- 当 lang_key 不是可识别的语言标识符时，返回的 dict 为空
- 无法打开读取的源文件会从结果中省略；不抛出错误
- 当提供 proj_dir 时，数据库中存储的文件路径会相对于 proj_dir 解析以产生绝对键
```

- 推导的实际行为：

```
该函数返回一个字典 `result`，满足：如果 `_CG_LANG.get(lang_key)` 为假值（None 或空列表），则 `result` 为
空字典。否则，令 `cg_langs = _CG_LANG[lang_key]`。函数打开到 `self._db` 的只读数据库连接，查询 `nodes`
表中 `kind` 为 'function' 或 'method' 且 `language` 在 `cg_langs` 中的行，排序方式为按 `file_path` 再
按 `start_line` 升序。每一行为一个元组 `(name, qualified_name, file_path, start_line, end_line)`。行
按 `file_path` 分组，保持查询顺序。对于每个不同的 `file_path`：
- 如果 `proj_dir` 不为 None，则计算 `abs_path = os.path.join(proj_dir, file_path)`，否则为
  `file_path`。
- 如果打开 `abs_path` 进行读取引发 `OSError`，则跳过该文件（`result` 中无条目）。
- 否则，将文件所有行读入 `all_lines`。
- 对于该文件中每个函数条目，按 `start_line` 顺序处理：
  1. 计算 `ident = _extraction_ident(name, qualified_name)`，一个确定性的文件系统安全字符串。
```

2. 在该文件内统计出现次数（从 0 开始）。首次出现保留裸`ident`；后续出现变为`{ident}\_{count}`，其中`count`是之前的出现次数。

3. 提取函数体行：切片`all\_lines[start\_line-1 : end\_line]`（基于 1 的索引，包含边界）。将它们拼接为一个字符串。如果结果字符串不以换行符结尾，则追加一个。

4. 将元组`(deduped\_ident, body)`追加到文件的列表中。

- `result[abs\_path]`被设置为该列表，保持提取顺序。

函数关闭数据库连接后返回`result`。不发生数据库修改；后置条件覆盖正常终止（没有来自数据库或文件系统的未处理异常）。

形式化：

```

```
令 cg_langs = _CG_LANG.get(lang_key)。
如果 cg_langs 为 None 或 cg_langs == []：
 result = {}
否则：
 conn = sqlite3.connect(self._db); cur = conn.cursor()
 cur.execute(query, cg_langs) // query 查询 (name, qualified_name, file_path, start_line,
end_line)
 rows = cur.fetchall()
 conn.close()
 将 rows 按 by_file 分区：dict 映射 file_path 到按出现顺序的 (name, qualified_name, start_line,
end_line) 列表。
 result = {}
 对于 keys(by_file) 中的每个 file_path，按它们在 rows 中出现的顺序遍历：
 abs_path = os.path.join(proj_dir, file_path) 如果 proj_dir 不为 None，否则为 file_path
 如果 open(abs_path) 引发 OSError：继续
 否则：
 all_lines = 文件的行列表
 ident_counts = {}
 file_funcs = []
 对于 by_file[file_path] 中的每个 (name, qualified_name, start_line, end_line)：
 ident = _extraction_ident(name, qualified_name)
 cnt = ident_counts.get(ident, 0)
 ident_counts[ident] = cnt + 1
 deduped_ident = 如果 cnt == 0 则为 ident，否则为 f"{ident}_{cnt}"
 body = "".join(all_lines[start_line-1 : end_line])
 如果 body 不以 "\n" 结尾：body += "\n"
 file_funcs.append((deduped_ident, body))
 result[abs_path] = file_funcs
 返回 result
```
```

- 代码证据：

```
第 32 行：abs_path = os.path.join(proj_dir, file_path) if proj_dir else file_path
```

- 触发条件：

规范 B 要求当提供 proj_dir 时，返回的 dict 键为绝对文件系统路径。代码使用 os.path.join(proj_dir, file_path)，但并没有确保结果是绝对的；传入一个相对的 proj_dir 会产生相对键，违反了规范。

Bug validator

- 触发摘要：向 get_functions_by_file 传入一个相对的 proj_dir，会在返回的 dict 中产生相对文件系统路径键，而不是规范所要求的绝对路径。
- Probe 标准输出：

```
CONFIRMED — 规范要求 dict 键为绝对路径，但传入相对 proj_dir='.././probe_cg_bq8naau0' 产生了相对键：
['.././probe_cg_bq8naau0/test_module.py']，而非预期的绝对键
'/tmp/probe_cg_bq8naau0/test_module.py'
```

INCR-MISMATCH-030 — src--languages--codegraph-py--_bare_function_name

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与规范：src/languages/codegraph-py/_bare_function_name.py。

- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_bare_function_name.json`。
- 详细报告: `src--languages--codegraph-py--_bare_function_name.md`。
- 探针: `probe_src--languages--codegraph-py--_bare_function_name.py`。

完整生成 SPEC

```
[SPEC]
单元: src/languages/codegraph.py

_bare_function_name(name: str) -> str

前置条件:
- name 是一个字符串, 可能是一个原始函数名、作用域限定名、
  装饰函数签名 (函数指针或指针返回语法)、运算符重载名或空字符串

后置条件:
- 返回一个字符串, 包含从 name 中提取的裸函数标识符,
  按以下顺序规则执行:

1. 去除首尾空白。如果结果为空, 则返回 ""。

2. 确定一个“尾部”字符串:
  - 初始时 tail = name (去除空白后)。
  - 如果 tail 包含 ":", 则将 tail 设置为最后一个 ":" 之后的子字符串,
    去除前导空白。
  - 否则如果 tail 包含 ".", 则将 tail 设置为最后一个 "." 之后的子字符串,
    去除前导空白。

3. 运算符重载检测 (应用于 tail):
  如果 tail 以 "operator" 开头:
  - 令 rest = tail[len("operator"):].lstrip()
  - 如果 rest 以 "[" 开头, 返回 "operator[]"。
  - 如果 rest 以 "(" 开头, 返回 "operator()"。
  - 如果 rest 匹配模式 "new" 后可选地跟随空白和 "[" 空白 "]", 若存在括号则返回 "operator new[]", 否则返回 "operator new"。
  - 如果 rest 匹配模式 "delete" 后可选地跟随空白和 "[" 空白 "]", 若存在括号则返回 "operator delete[]", 否则返回 "operator delete"。
  - 否则, 从 rest 中收集连续的属于集合
    + - * / % & | ^ ~ ! = < > , 的字符, 返回 "operator" + 收集的符号。

4. 如果没有产生运算符结果, 则在原始去除空白后的名称 (尾部修改之前) 上尝试以下正则匹配:
  a. `(?:^|:|\.)(\w+)$` - 返回最右侧的标识符组件
    (单词字符序列), 前面有字符串开头、":" 或 "."。
  b. `(\s*\s*(\w+)\s*\s*)` - 返回函数指针表达式 (如 "(*func)(...)") 中的标识符。
  c. `(\s*\s*(\w+)\s*)` - 返回前导 "*" 之后的标识符 (指针返回语法)。
  d. `^(\w+)$` - 返回前导的单词字符序列。

5. 如果上述都不匹配, 则原样返回去除空白后的 name。

- 因为提取模式使用 \w+, 模板参数尖括号 (<...>) 和括号括起的参数/参数列表会隐式从返回的标识符中排除,
  除非是运算符名, 它们会作为运算符表示的一部分被显式包含。

[SPEC]
```

Reasoner 差异

- 规范声明:

```
- 返回一个字符串, 包含从 name 中提取的裸函数标识符,
  按以下顺序规则执行:

1. 去除首尾空白。如果结果为空, 则返回 ""。

2. 确定一个“尾部”字符串:
  - 初始时 tail = name (去除空白后)。
  - 如果 tail 包含 ":", 则将 tail 设置为最后一个 ":" 之后的子字符串,
    去除前导空白。
  - 否则如果 tail 包含 ".", 则将 tail 设置为最后一个 "." 之后的子字符串,
    去除前导空白。

3. 运算符重载检测 (应用于 tail):
  如果 tail 以 "operator" 开头:
  - 令 rest = tail[len("operator"):].lstrip()
  - 如果 rest 以 "[" 开头, 返回 "operator[]"。
  - 如果 rest 以 "(" 开头, 返回 "operator()"。
  - 如果 rest 匹配模式 "new" 后可选地跟随空白和 "[" 空白 "]", 若存在括号则返回 "operator new[]", 否
```

则返回 "operator new"。

- 如果 rest 匹配模式 "delete" 后可选地跟随空白和 "[" 空白 "]"，若存在括号则返回 "operator delete[]"，否则返回 "operator delete"。
- 否则，从 rest 中收集连续的属于集合 + - * / % & | ^ ~ ! = < > , 的字符，返回 "operator" + 收集的符号。

4. 如果没有产生运算符结果，则在原始去除空白后的名称（尾部修改之前）上尝试以下正则匹配：

- a. ``(?:^|::|\.)(\w+)$`` 返回最右侧的标识符组件（单词字符序列），前面有字符串开头、 "::" 或 "."。
- b. ``\(\s*\s*(\w+)\s*\)`` 返回函数指针表达式（如 `"(*func)(...)"`）中的标识符。
- c. ``\*\s*(\w+)`` 返回前导 "*" 之后的标识符（指针返回语法）。
- d. ``^(\w+)`` 返回前导的单词字符序列。

5. 如果上述都不匹配，则原样返回去除空白后的 name。

- 因为提取模式使用 `\w+`，模板参数尖括号 (`<...>`) 和括号括起的参数/参数列表会隐式从返回的标识符中排除，除非是运算符名，它们会作为运算符表示的一部分被显式包含。

- 推导的实际行为：

该函数返回一个字符串 r，即从输入 name 中提取的裸函数标识符。令 `s = name.strip()`。如果 s 为空，则 `r = ''`。否则，定义 tail：如果 s 中既不包含 "::" 也不包含 "."，则 `tail = s`；否则 `tail = (s.rsplit('::', 1)[1] if '::' in s else s.rsplit('.', 1)[1]).lstrip()`。如果 tail 以 'operator' 开头，那么：令 `rest = tail[8:].lstrip()`；如果 rest 以 '[' 开头，则 `r = 'operator[]'`；否则如果 rest 以 '(' 开头，则 `r = 'operator()'`；否则如果 `re.fullmatch(r'new(?:\s*\[\s*\])?', rest)` 匹配，则 `r = 'operator new[]'` 如果 '[' 在 rest 中，否则 `r = 'operator new'`；否则如果 `re.fullmatch(r'delete(?:\s*\[\s*\])?', rest)` 匹配，则 `r = 'operator delete[]'` 如果 '[' 在 rest 中，否则 `r = 'operator delete'`；否则，令 sym 为 rest 中最长的前缀，仅由集合 `"+-*/%&|^~!=<>,"` 中的字符组成；如果 sym 非空，则 `r = 'operator' + sym`；否则回落。如果到此时还没有返回，那么：如果 `re.search(r'(?:^|::|\.)(\w+)$', s)` 匹配，则 `r =` 捕获的单词；否则如果 `re.match(r'\(\s*\s*(\w+)\s*\)', s)` 匹配，则 `r =` 捕获的标识符；否则如果 `re.match(r'\*\s*(\w+)', s)` 匹配，则 `r =` 捕获的标识符；否则如果 `re.match(r'^(\w+)', s)` 匹配，则 `r =` 捕获的单词；否则 `r = s`。结果始终是一个没有前导/尾随空白的字符串，表示一个简单的标识符或一个运算符名（例如 `'operator=='`、`'operator new[]'`）。在所有情况下，函数都会终止且不会引发异常。

- 代码证据：

```
第 41 行:if symbol:
第 42 行:return "operator" + "".join(symbol)
```

- 触发条件：

当 tail 以 'operator' 开头但 rest 不包含连续的运算符符号（例如 'Foo'）时，规范要求返回 'operator'（收集零个符号并连接的结果）。代码仅在 symbol 非空时返回；否则会回落到对原始名称的正则匹配，从而返回 'operatorFoo' 而不是 'operator'。

Bug validator

- 触发摘要：当 tail 以 'operator' 开头但 rest 不包含连续的运算符符号（例如 'operatorFoo'）时，代码回落到正则匹配并返回 'operatorFoo'，而不是规范要求的 'operator'。
- 探针标准输出：

CONFIRMED — 实际值: 'operatorFoo' | 预期值: 'operator'

INCR-MISMATCH-031 — `src--languages--codegraph-py--_codegraph_cmd`

- 人工审计：契约待确认。
- 验证器：**confirmed**；尝试次数：1。
- 原始函数与 SPEC：`src/languages/codegraph-py/_codegraph_cmd.py`。
- Reasoner 结果：`logic_verification_results/src/languages/codegraph-py/_codegraph_cmd.json`。
- 详细报告：`src--languages--codegraph-py--_codegraph_cmd.md`。
- Probe：`probe_src--languages--codegraph-py--_codegraph_cmd.py`。

[SPEC]
单元：src/languages/codegraph.py

`_codegraph_cmd() -> str`

前置条件：

- settings.codegraph.bin_dir 是一个字符串，指定一个目录路径，可能以波浪号（~）开头，表示当前用户的主目录。

后置条件：

- 返回一个适合作为可执行命令名的字符串。
- 当在将配置的 bin_dir 中的前导波浪号扩展为用户主目录后得到的目录中存在一个名为“codegraph”的文件，并且该文件为当前进程的有效用户设置了执行权限位时，返回该文件的绝对文件系统路径。
- 当该文件不存在或缺少执行权限位时，返回裸字符串“codegraph”，将解析延迟到调用进程的 PATH 环境变量所指定的目录。
- 从不引发异常。

[SPEC]

Reasoner 差异

- SPEC 声明：

- 返回一个适合作为可执行命令名的字符串。
 - 当在将配置的 bin_dir 中的前导波浪号扩展为用户主目录后得到的目录中存在一个名为“codegraph”的文件，并且该文件为当前进程的有效用户设置了执行权限位时，返回该文件的绝对文件系统路径。
 - 当该文件不存在或缺少执行权限位时，返回裸字符串“codegraph”，将解析延迟到调用进程的 PATH 环境变量所指定的目录。
 - 从不引发异常。

- 推导实际行为：

该函数返回波浪号扩展后的 `settings.codegraph.bin_dir` 目录下的可执行文件“codegraph”的绝对路径（如果该文件存在且具有可执行权限（`os.X_OK`））；否则返回裸命令字符串“codegraph”。形式化地：设 `expanded = os.path.expanduser(settings.codegraph.bin_dir)`，设 `path = os.path.join(expanded, 'codegraph')`；则返回值 `r` 满足 `r = path` 如果 `os.access(path, os.X_OK)`，否则 `r = 'codegraph'`。

- 代码证据：

```
第 13 行： local = os.path.join(bin_dir, "codegraph")
第 14 行： return local if os.access(local, os.X_OK) else "codegraph"
```

- 触发条件：

代码在返回组合路径之前未将其转换为绝对路径，因此当配置的 `bin_dir` 是相对路径时，返回的路径是相对的，这违反了规范关于返回绝对文件系统路径的要求。

Bug validator

- 触发摘要：当 settings.codegraph.bin_dir 是一个相对路径，且在该路径下存在一个可执行的“codegraph”文件时，_codegraph_cmd() 返回相对路径而非绝对路径。
- 探针标准输出：

```
CONFIRMED — actual (relative): '../..probe_codegraph_cmd_yy10d4pr/codegraph' | expected
(absolute): '/tmp/probe_codegraph_cmd_yy10d4pr/codegraph'
```

- 验证器：已确认；尝试次数：1。
- 原始函数与 SPEC：src/languages/codegraph-py/_extraction_ident.py。
- Reasoner 结果：logic_verification_results/src/languages/codegraph-py/_extraction_ident.json。
- 详细报告：src--languages--codegraph-py--_extraction_ident.md。
- Probe：probe_src--languages--codegraph-py--_extraction_ident.py。

完整生成 SPEC

```
[SPEC]
单元：src/languages/codegraph.py

_extraction_ident(name: str, qualified_name: str) -> str

前置条件：
- name 与 qualified_name 是从 codegraph 数据库中为单个函数或方法节点获取的字符串

后置条件：
- 返回一个由字面量 "::" 连接的一个或多个组件构成的字符串
- 当 qualified_name 非空且其后缀等于 name 时，返回字符串的前导组件（除最后一个之外的所有组件）按顺序对应于从 qualified_name 中 name 之前的前缀提取的范域限定符
- 当 qualified_name 为空或其尾缀不等于 name 时，返回的字符串恰好由一个组件构成
- 返回字符串的最后一个组件派生自 name
- 返回字符串的任何组件均不包含在任何文件系统中作为目录分隔符的字符
- 返回字符串的任何组件均不包含原始数据库列值中可能存在的签名语法、指针语法或模板语法
- 相同的 (name, qualified_name) 对始终产生相同的返回字符串
- qualified_name 中使用的分隔符 ( "." 与 "::" ) 不会影响返回字符串中范域组件的集合或顺序
[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 返回一个由字面量 "::" 连接的一个或多个组件构成的字符串
- 当 qualified_name 非空且其后缀等于 name 时，返回字符串的前导组件（除最后一个之外的所有组件）按顺序对应于从 qualified_name 中 name 之前的前缀提取的范域限定符
- 当 qualified_name 为空或其尾缀不等于 name 时，返回的字符串恰好由一个组件构成
- 返回字符串的最后一个组件派生自 name
- 返回字符串的任何组件均不包含在任何文件系统中作为目录分隔符的字符
- 返回字符串的任何组件均不包含原始数据库列值中可能存在的签名语法、指针语法或模板语法
- 相同的 (name, qualified_name) 对始终产生相同的返回字符串
- qualified_name 中使用的分隔符 ("." 与 "::") 不会影响返回字符串中范域组件的集合或顺序

- 推导的实际行为：

该函数返回一个字符串 `r`，计算方式为 `r = '::'.join(canonicalize(_bare_function_name(p)) for p in _qualified_parts(name, qualified_name))`。设 `Q = _qualified_parts(name, qualified_name)`；则 `Q` 是一个至少包含一个非空字符串的列表，且最后一个元素等于 `name`。对于 `Q` 中的每个元素 `q`，定义 `s = canonicalize(_bare_function_name(q))`。每个 `s` 是一个字符串，不包含任何在文件系统路径组件中无效的字符；若 `_bare_function_name` 返回空字符串（例如当 `q` 全由空白字符组成时），`s` 可能为空。返回字符串 `r` 是将结果字符串 `s1, s2, ..., sn` 以 `::'` 交错拼接而成。因此，`r` 的形式为 `s0 + '::' + s1 + '::' + ... + s_{n-1}`。当 `qualified_name` 非空且以 `name` 为后缀时，来自 `qualified_name` 前缀的范域限定符组件会作为 `Q` 的首部元素被包含；否则 `Q = [name]`。返回值具有确定性，仅依赖于 `name` 与 `qualified_name`。

- 代码证据：

```
第 15 行： return "::".join(
第 16 行：         canonicalize(_bare_function_name(p))
第 17 行：         for p in _qualified_parts(name, qualified_name)
第 18 行：     )
```

- 触发条件：

代码并未过滤 `_bare_function_name` 可能产生的空字符串。对于输入中 `qualified_name` 包含一个仅由空白字符组成的范域限定符（例如 `'::MyClass::func'`），`_bare_function_name` 会为该限定符返回空字符串，导致最终拼接得到

的字符串为 '::MyClass::func'。规格要求返回字符串由以 '::' 连接的一个或多个（非空）组件构成，但因为前导的空组件，结果以 '::' 开头，违反了这一要求。

Bug validator

- 触发摘要：仅由空白字符组成的范域限定符组件（例如 'foo:::func'）会使 `_bare_function_name` 返回空字符串，这些空字符串经过 `canonicalize` 后未被过滤，在结果中产生空的 '::' 分隔组件，从而违反规格中组件非空的要求。
- Probe 标准输出：

```
CONFIRMED - 在 _extraction_ident 输出中发现空组件
[仅空格中间限定符] name='func', qualified_name='foo:::func'
  actual:  'foo:::func'
  split by '::': ['foo', '', 'func'] (包含空组件!)
[空格+制表符限定符] name='baz', qualified_name='X:::  baz'
  actual:  'X:::  baz'
  split by '::': ['X', '', '', 'baz'] (包含空组件!)
```

INCR-MISMATCH-033 — `src--languages--codegraph-py--_node_fqn_map`

- 人工审计：**SPEC 错误**。
- Validator: **confirmed**；尝试次数：**1**。
- 原始函数与 SPEC: `src/languages/codegraph-py/_node_fqn_map.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_node_fqn_map.json`。
- 详细报告: `src--languages--codegraph-py--_node_fqn_map.md`。
- Probe: `probe_src--languages--codegraph-py--_node_fqn_map.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/codegraph-py/_node_fqn_map.py

_node_fqn_map(cur, cg_langs) -> dict

前置条件：
- cur 是一个数据库游标，连接到包含 nodes 表的 codegraph 数据库，该表包含 id、name、file_path、start_line、kind 和 language 列
- cg_langs 是一个非空的语言键字符串列

后置条件：
- 返回一个字典，将每个节点的 id 映射到其完全限定函数名（FQN）
- 该映射精确包含 nodes 表中 kind 为 'function' 或 'method' 且 language 属于给定 cg_langs 值之一的行，按 (file_path ASC, start_line ASC) 排序
- 每个 FQN 由节点的 file_path 以及经过规范化、去重后的函数名派生，规范约定为：路径各部分用 "::" 连接，且父目录部分中的源文件扩展名替换为连字符
  - 函数名规范化会剥离尖括号模板参数，并将运算符重载名称规范化为安全的标识符形式
  - 当 N > 1 个节点共享相同的 file_path 和规范化名称时，结果排序中的第一个节点接收不带后缀的规范化名称，随后的每个节点（从 1 开始的第 k 个）接收带 _k 后缀的规范化名称
  - 当查询没有匹配行时返回空字典
- 去重规则和排序与 get_functions_by_file 使用的规则一致，确保此处为节点分配的 FQN 与为同一节点生成的提取函数文件所接收的 FQN 相匹配
[SPEC]
```

Reasoner 差异

- SPEC 声明：

```
- 返回一个字典，将每个节点的 id 映射到其完全限定函数名（FQN）
- 该映射精确包含 nodes 表中 kind 为 'function' 或 'method' 且 language 属于给定 cg_langs 值之一的行，按 (file_path ASC, start_line ASC) 排序
- 每个 FQN 由节点的 file_path 以及经过规范化、去重后的函数名派生，规范约定为：路径各部分用 "::" 连接，且父目录部分中的源文件扩展名替换为连字符
  - 函数名规范化会剥离尖括号模板参数，并将运算符重载名称规范化为安全的标识符形式
  - 当 N > 1 个节点共享相同的 file_path 和规范化名称时，结果排序中的第一个节点接收不带后缀的规范化名称，随后的
```

每个节点（从 1 开始的第 k 个）接收带 `_k` 后缀的规范化名称

- 当查询没有匹配行时返回空字典
- 去重规则和排序与 `get_functions_by_file` 使用的规则一致，确保此处为节点分配的 FQN 与为同一节点生成的提取函数文件所接收的 FQN 相匹配

- 推导的实际行为：

如果不发生异常：该函数返回一个字典 ``result``，满足：设 ``Q`` 为通过 ``cur.execute`` 执行 ``SELECT id, name, qualified_name, file_path, start_line FROM nodes WHERE kind IN ('function','method') AND language IN (?,...?)`` 后获得的行按 ``file_path, start_line`` 升序排列的有序列表，参数为 ``cg_langs``。对于 ``Q`` 中顺序的每一行 ``(id, name, qualified_name, file_path, _)``，令 ``ident = _extraction_ident(name, qualified_name)``，``key = (file_path, ident)``。定义一个计数器函数 ``occ(key, i) = |{ j < i | key_j = key }|``（从 0 开始的出现索引）。那么 ``deduped = ident`` 如果 ``occ(key, i) = 0``，否则 ``f"{ident}_{occ(key,i)}"``。然后 ``result[id] = _fqn_for(file_path, deduped)``。迭代完所有行后，``result`` 精确包含 ``{r.id for r in Q}`` 作为键，没有其他条目。游标 ``cur`` 已被完全取出（该查询没有剩余行）。没有其他可变状态被修改。如果引发异常（例如 SQL 错误、提取错误或辅助函数的异常），异常会传播；不执行显式回滚或清理，``cur`` 的状态和任何部分构建的 ``result`` 对调用者来说丢失。

- 代码证据：

```
第 26 行: deduped = ident if c == 0 else f"{ident}_{c}"
```

- 触发条件：

规范要求第 k 次出现（1 索引）应后缀 `_k`，而代码使用从零开始的计数器，导致第二次出现得到 `_1`，第三次出现得到 `_2` 等，违反了定义的去重规则。

Bug validator

- 触发摘要：0-based 计数器 'c' 直接用作后缀；规范要求对第 k 次重复使用 1 索引的 `_k` —— 第二次出现得到 `_1` 而不是 `_2`。
- Probe 标准输出：

```
CONFIRMED — off-by-one 去重后缀复现。
预期（规范正确）：{1: 'src::util-c::helper', 2: 'src::util-c::helper_2', 3: 'src::util-c::helper_3'}
实际（有错误）：    {1: 'src::util-c::helper', 2: 'src::util-c::helper_1', 3: 'src::util-c::helper_2'}
Buggy pattern matches:    True
```

INCR-MISMATCH-034 — `src--languages--codegraph-py--_qualified_parts`

- 人工审计：**SPEC 错误**。
- Validator: **confirmed**；尝试次数：1。
- 原始函数与 SPEC: `src/languages/codegraph-py/_qualified_parts.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_qualified_parts.json`。
- 详细报告: `src--languages--codegraph-py--_qualified_parts.md`。
- Probe: `probe_src--languages--codegraph-py--_qualified_parts.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/codegraph.py

_qualified_parts(name: str, qualified_name: str) -> list

前置条件：
- name 和 qualified_name 均为字符串

后置条件：
```

- 返回一个非空字符串的列表
- 所返回列表的最后一个元素等于 `name`
- 当 `qualified_name` 非空且以 `name` 作为后缀时，最后一个元素之前的元素是范围限定符组成部分，它们从 `qualified_name` 中 `name` 之前的前缀中提取，并按 `::` 或 `.` 分割
- 当 `qualified_name` 为空或不以 `name` 作为后缀时，返回的列表为 `[name]`
- 在 `qualified_name` 中用作范围分隔符的字符 (`.` 或 `::`) 不影响返回列表中各组成部分的集合或顺序
- 相同的 `(name, qualified_name)` 对始终产生相同的返回列表

[SPEC]

Reasoner 差异

- SPEC 声明：

- 返回一个非空字符串的列表
- 所返回列表的最后一个元素等于 `name`
- 当 `qualified_name` 非空且以 `name` 作为后缀时，最后一个元素之前的元素是范围限定符组成部分，它们从 `qualified_name` 中 `name` 之前的前缀中提取，并按 `::` 或 `.` 分割
- 当 `qualified_name` 为空或不以 `name` 作为后缀时，返回的列表为 `[name]`
- 在 `qualified_name` 中用作范围分隔符的字符 (`.` 或 `::`) 不影响返回列表中各组成部分的集合或顺序
- 相同的 `(name, qualified_name)` 对始终产生相同的返回列表

- 推导的实际行为：

函数返回一个列表 `L`，其行为如下。令 `q = qualified_name.strip()`。如果 `q` 为空字符串或不以 `name` 结尾，则 `L = [name]`。否则，令 `scope = q[:-len(name)].rstrip(':.')`。如果 `scope` 为空字符串，则 `L = [name]`。否则，`L = [p for p in re.split(r'::|\.', scope) if p != ''] + [name]`。不会引发任何异常，因为输入是字符串，且对这些字符串的操作是安全的。

- 代码证据：

第 14 行：`q = (qualified_name or "").strip()`

- 触发条件：

规范要求从原始的 `qualified_name`（不进行去除空白处理）中提取范围前缀，但代码在第 14 行通过 `strip` 去除了空白。对于 `qualified_name=' bar::foo'`，规范会得出 `[' bar', 'foo']`，因为前缀 `' bar::'` 按 `::` 分割会得到 `' bar'`，而代码去除空白后将其变为 `'bar::foo'` 并返回 `['bar', 'foo']`，这违反了返回列表必须反映原始 `qualified_name` 前缀的范围组成部分的要求。

Bug validator

- 触发摘要：函数在提取范围组成部分之前去除了 `qualified_name` 两端的空白，因此前缀中的前导空白丢失：对于 `qualified_name=' bar::foo'` 且 `name='foo'`，实际返回 `['bar', 'foo']`，而非规范预期的 `[' bar', 'foo']`。
- Probe 标准输出：

CONFIRMED — actual: `['bar', 'foo']` | expected: `[' bar', 'foo']`

INCR-MISMATCH-035 — `src--languages--codegraph-py--_warn_on_codegraph_version_mismatch`

- 人工审计：契约待确认。
- Validator: **confirmed**；尝试次数：1。
- 原始函数与 SPEC: `src/languages/codegraph-py/_warn_on_codegraph_version_mismatch.py`。
- Reasoner 结果: `logic_verification_results/src/languages/codegraph-py/_warn_on_codegraph_version_mismatch.json`。

- 详细报告：`src--languages--codegraph-py--_warn_on_codegraph_version_mismatch.md`。
- Probe: `probe_src--languages--codegraph-py--_warn_on_codegraph_version_mismatch.py`。

完整生成 SPEC

[SPEC]
单元：src/languages/codegraph-py/_warn_on_codegraph_version_mismatch.py

_warn_on_codegraph_version_mismatch(cmd: str) -> None

前置条件：

- cmd 是一个非空字符串，标识位于系统 PATH 上的可执行命令。
- settings.codegraph.version 是一个字符串（可能为空或仅包含空白字符）。

后置条件：

- 返回 None；绝不引发异常。
- 该函数不产生任何外部可观察的副作用，除非以下所有条件均满足：
 - (a) 配置的 codegraph 版本，在去除首尾空白并移除任何前导 "v" 前缀后，非空；
 - (b) 执行由 cmd 指派的命令并附带参数 "--version" 作为子进程成功运行，且对其捕获的标准输出进行首尾空白去除后产生的输出非空；
 - (c) 该输出在各自去除空白及任何前导 "v" 前缀后与配置的版本不相等。
- 当条件 (a)、(b)、(c) 全部满足时：会以 WARNING 严重级别发出一条日志记录，其消息同时标识从命令输出获取的版本字符串以及来自 fm-agent.toml 的配置版本字符串。

[SPEC]

Reasoner 差异

- SPEC 声明：

- 返回 None；绝不引发异常。
- 该函数不产生任何外部可观察的副作用，除非以下所有条件均满足：
 - (a) 配置的 codegraph 版本，在去除首尾空白并移除任何前导 "v" 前缀后，非空；
 - (b) 执行由 cmd 指派的命令并附带参数 "--version" 作为子进程成功运行，且对其捕获的标准输出进行首尾空白去除后产生的输出非空；
 - (c) 该输出在各自去除空白及任何前导 "v" 前缀后与配置的版本不相等。
- 当条件 (a)、(b)、(c) 全部满足时：会以 WARNING 严重级别发出一条日志记录，其消息同时标识从命令输出获取的版本字符串以及来自 fm-agent.toml 的配置版本字符串。

- 推导实际行为：

函数完成且不引发任何异常。令 `V = settings.codegraph.version.strip().removeprefix('v')`。如果 `V` 为空，则不记录任何警告。否则，函数尝试以 `capture_output=True`、`text=True`、`timeout=10` 运行 `subprocess.run([cmd, '--version'])`。如果由此引发 `OSError` 或 `subprocess.SubprocessError`，则不记录警告。如果成功，令 `output =` 已结束进程的去除首尾空白后的标准输出。如果 `output` 非空且 `output != V`，则通过 `logging.warning` 发出一条 `WARNING` 日志记录，其格式字符串为 `'codegraph %r does not match the pinned %r (fm-agent.toml [codegraph].version); re-run install.sh to update.'`，参数为 `(output, V)`。否则，不记录警告。不产生其他副作用。

形式化描述：令 `V = strip(removeprefix(settings.codegraph.version, 'v'))`。后置条件为：
(`V!=''`) (`subprocess.run([cmd, '--version'], capture_output=True, text=True, timeout=10)` 引发 `OSError` 或 `SubprocessError`) (`令 stdout = strip(result.stdout); (stdout != stdout V)`)) 不发出指定消息的 `WARNING` 日志；
(`V != ''` 子进程成功 令 `s = strip(result.stdout); s != s V`) 发出指定消息和参数 `(s, V)` 的 `WARNING` 日志。

- 代码证据：

第6行: want = settings.codegraph.version.strip().removeprefix("v"); 第15行: if got and got != want:

- 触发条件：

代码仅从配置版本中移除前导 'v'，而未从命令输出中移除。规范要求两个字符串在比较前都要去除空白并移除任何前导 'v'。当归一化后两字符串匹配时（例如配置 'v1.0' 且输出 'v1.0'），代码错误地发出警告，违反了规范的条件（c）。

Bug validator

- 触发摘要：代码仅从配置版本移除前导 'v' 而未从命令输出移除，导致当两个版本在语义上相同时（例如配置 'v1.0'，输出 'v1.0'）产生虚假的 WARNING。
- 探针标准输出：

CONFIRMED — 尽管版本在归一化后匹配，仍发出了 WARNING："codegraph 'v1.0' does not match the pinned '1.0' (fm-agent.toml [codegraph].version); re-run install.sh to update."

INCR-MISMATCH-036 — src--languages--codegraph-py--try_codegraph_init

- 人工审计：实现缺陷候选。
- Validator: **confirmed**；尝试次数: 1。
- 原始函数与 SPEC: [src/languages/codegraph-py/try_codegraph_init.py](#)。
- Reasoner 结果: [logic_verification_results/src/languages/codegraph-py/try_codegraph_init.json](#)。
- 详细报告: [src--languages--codegraph-py--try_codegraph_init.md](#)。
- Probe: [probe_src--languages--codegraph-py--try_codegraph_init.py](#)。

完整生成 SPEC

```
[SPEC]
单元：src/languages/codegraph-py/try_codegraph_init.py

try_codegraph_init(proj_dir: str, force: bool = True) -> None

前置条件：
- proj_dir 是一个非空字符串，表示文件系统上的一个目录路径。
- force 是 True 或 False。

后置条件：
- 返回 None；绝不抛出异常。
- 当系统 PATH 上找不到 `codegraph` 可执行文件时：立即返回，不对 proj_dir 下的任何文件进行创建、修改或删除。
- 当 proj_dir/.codegraph/codegraph.db 存在且 force 为 False 时：立即返回；保留现有的索引文件及其父目录。
- 否则（force 为 True，或 proj_dir/.codegraph/codegraph.db 不存在）：
  - 如果 proj_dir/.codegraph/ 目录存在，在重建前将其删除（递归删除，忽略错误）。
  - 以 proj_dir 为工作目录执行 `codegraph init`。
  - 如果 `codegraph init` 以退出码 0 退出：返回后 proj_dir/.codegraph/codegraph.db 存在，并反映调用 `codegraph init` 时 proj_dir 的文件树。
  - 如果 `codegraph init` 以非零退出码退出：记录一条警告，其消息包含 stderr 的前 300 个字符；函数返回，proj_dir/.codegraph/ 的内容未指定。
[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 返回 `None`；绝不抛出异常。
- 当系统 `PATH` 上找不到 ``codegraph`` 可执行文件时：立即返回，不对 `proj_dir` 下的任何文件进行创建、修改或删除。
- 当 `proj_dir/.codegraph/codegraph.db` 存在且 `force` 为 `False` 时：立即返回；保留现有的索引文件及其父目录。
- 否则（`force` 为 `True`，或 `proj_dir/.codegraph/codegraph.db` 不存在）：
 - 如果 `proj_dir/.codegraph/` 目录存在，在重建前将其删除（递归删除，忽略错误）。
 - 以 `proj_dir` 为工作目录执行 ``codegraph init``。
 - 如果 ``codegraph init`` 以退出码 `0` 退出：返回后 `proj_dir/.codegraph/codegraph.db` 存在，并反映调用 ``codegraph init`` 时 `proj_dir` 的文件树。
 - 如果 ``codegraph init`` 以非零退出码退出：记录一条警告，其消息包含 `stderr` 的前 300 个字符；函数返回，`proj_dir/.codegraph/` 的内容未指定。

- 推导的实际行为：

函数返回后，关于项目目录 ``proj_dir`` 的以下后置条件成立：

1. 代码图索引数据库文件 ``db = os.path.join(proj_dir, '.codegraph', 'codegraph.db')`` 的存在性满足：

```
db_post_exists ( (db_pre_exists force = False) (cmd_available (force db_pre_exists)
exit_code = 0) )
```

其中

- `db_pre_exists` = 调用前 `os.path.exists(db)`，
 - `db_post_exists` = 调用后 `os.path.exists(db)`，
 - `cmd_available` = 由 ``_codegraph_cmd()`` 识别的 ``codegraph`` 命令在 `PATH` 上存在且不抛出 ``FileNotFoundError``，
 - `exit_code` = 如果运行 ``subprocess.run([cmd, 'init'], ...)`` 则为其返回码（成功为 `0`）。
2. 如果 ``force`` 为 `True` 且 ``db_pre_exists`` 为 `True`，则在尝试执行命令前通过 ``shutil.rmtree`` 删除了整个 ``codegraph`` 目录。
 3. 该函数绝不抛出异常；它优雅地处理缺失的可执行文件和非零退出码。
 4. 根据第 25、27、37、39-42 行，打印或记录了信息性消息。

- 代码证据：

```
第 19 行: if os.path.exists(db_path):
第 20 行:     if not force:
第 21 行:         return
第 24 行:     shutil.rmtree(codegraph_dir, ignore_errors=True)
```

- 触发条件：

代码在检查 `'codegraph'` 可执行文件是否存在（第 31 行）之前删除了现有的 `.codegraph` 目录（第 24 行）。当可执行文件缺失时，捕获了 `FileNotFoundError`（第 34-35 行）且函数返回，但删除操作已经发生。这违反了规范要求：当可执行文件未找到时，函数必须立即返回，不对 `proj_dir` 下的任何文件进行创建、修改或删除。

Bug validator

- 触发摘要：当 `force=True` 且 `.codegraph/codegraph.db` 存在，但 `codegraph` 可执行文件在 `PATH` 上未找到时，`shutil.rmtree()` 在可执行文件检查前删除了 `.codegraph` 目录，违反了当可执行文件缺失时不得有任何文件修改的规范要求。
- Probe 标准输出：

```
[Pipeline] 正在为当前工作树重建 codegraph 索引...
CONFIRMED - .codegraph 目录在检查 codegraph 可执行文件前已被删除。违反规范：规范要求当 codegraph 可执行文件未找到时，函数立即返回，不对 proj_dir 下的任何文件进行创建、修改或删除。
```

src/languages/cpp-py

- 人工审计：推理误判。
- 验证器：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：**src/languages/cpp-py/function_spans.py**。
- Reasoner 结果：**logic_verification_results/src/languages/cpp-py/function_spans.json**。
- 详细报告：**src--languages--cpp-py--function_spans.md**。
- Probe：**probe_src--languages--cpp-py--function_spans.py**。

完整生成 SPEC

```
[SPEC]
单元：src/languages/cpp-py/function_spans.py

function_spans(proj_dir: str, filepath: str) -> list | None

前置条件：
- proj_dir 是文件系统上项目目录的有效路径
- filepath 是一个字符串，标识项目内的一个 C++ 源文件

后置条件：
- 当代码图后端不可用或未索引该文件时返回 None，通知调用者回退到基于正则表达式的提取
- 否则返回一个由 (name, start_idx, end_idx) 元组组成的列表，每个元组通过函数名称及其所占用的源行范围来标识文件中的一个函数
- 在返回的每个元组中，start_idx 和 end_idx 是以0为起始的包含性行索引
- 返回的列表包含代码图后端在文件中检测到的每个函数
[SPEC]
```

Reasoner 差异

- SPEC 声称：

```
- 当代码图后端不可用或未索引该文件时返回 None，通知调用者回退到基于正则表达式的提取
- 否则返回一个由 (name, start_idx, end_idx) 元组组成的列表，每个元组通过函数名称及其所占用的源行范围来标识文件中的一个函数
- 在返回的每个元组中，start_idx 和 end_idx 是以0为起始的包含性行索引
- 返回的列表包含代码图后端在文件中检测到的每个函数
```

- 推导实际行为：

```
执行后，函数要么传播 `CodeGraphExtractor.from_proj_dir(proj_dir)` 引发的任何异常，要么返回一个值。如果没有异常发生，返回值按如下确定：令 `cg = CodeGraphExtractor.from_proj_dir(proj_dir)`。如果 `cg` 为假值（例如 `None`、`False`），则函数返回 `None`。否则，令 `spans = cg.get_function_spans('cpp', filepath)`。如果 `spans` 为 `None`，则函数返回 `None`；否则返回列表 `spans`，这是一个由三元组 `(name: str, start: int, end: int)` 组成的非空列表。每个元组描述了在给定 C++ 源文件中找到的一个函数定义，其中 `start` 和 `end` 是以0为起始的包含性行号。该函数不修改任何外部状态。形式化地，令 `R` 表示结果（异常或返回值），`E_from` 表示 `CodeGraphExtractor.from_proj_dir(proj_dir)` 抛出异常的事件。那么：(E_from → R = 该异常) ∧ (¬E_from ∧ cg 为假值 → R = None) ∧ (¬E_from ∧ cg 为真值 → (spans = cg.get_function_spans('cpp', filepath) ∧ (spans = None → R = None) ∧ (spans ≠ None → R = spans ∧ spans 是一个元组列表，每个元组形式为 (string, int, int)，语义如上所述))。
```

- 代码证据：

```
第 7 行：cg = CodeGraphExtractor.from_proj_dir(proj_dir)
第 8 行：return cg.get_function_spans("cpp", filepath) if cg else None
```

- 触发条件：

```
该函数未处理来自 `CodeGraphExtractor.from_proj_dir` 的异常。当 proj_dir 是一个无效目录时，from_proj_dir 可能引发异常（例如 FileNotFoundError），而非返回一个假值。规范要求代码图后端不可用时返回 None，以便调用者回退到正则表达式提取。通过传播异常，代码对于任何无效的 proj_dir 都违反了此规范。
```

Bug validator

- 触发摘要：传递 `None` 作为 `proj_dir` 会导致 `CodeGraphExtractor.from_proj_dir()` 通过 `os.path.abspath(None)` 引发 `TypeError`，该异常向上传播，而不是像规范要求的那样返回 `None`。
- `Probe` 标准输出：

CONFIRMED — 异常已传播：TypeError: expected str, bytes or os.PathLike object, not NoneType | 期望值：None

src/languages/erlang-py

INCR-MISMATCH-038 — src--languages--erlang-py--ElpClient::_handle_server_message

- 人工审计：契约待确认。
- 验证器：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：`src/languages/erlang-py/ElpClient::_handle_server_message.py`。
- 推理器结果：`logic_verification_results/src/languages/erlang-py/ElpClient::_handle_server_message.json`。
- 详细报告：`src--languages--erlang-py--ElpClient::_handle_server_message.md`。
- 探针：`probe_src--languages--erlang-py--ElpClient::_handle_server_message.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/erlang.py

ElpClient._handle_server_message(self, message: dict)

前置条件：
- self 是一个 ElpClient，其 __enter__ 已被调用（ELP 子进程正在运行，且 JSON-RPC 消息读取线程处于活动状态）
- message 是一个 dict，表示从 ELP 服务器接收到的已解析 JSON-RPC 消息

后置条件：
- 当 message.method 为 "elp/status" 时，将 self._status 更新为 message.params.status 的值；如果 params 字典缺失或没有 "status" 键，self._status 保持不变
- 当 message 缺少 "id" 字段，或 message.method 缺失或为假值时，不发送响应（该消息被视为通知）
- 当 message 同时具有非空的 "method" 和 "id"（服务器请求）时，发送一个 JSON-RPC 响应，其 jsonrpc 为 "2.0"，id 相同；结果值满足该方法的协议定义期望：
  - 对于工作区配置查询：结果是一个列表，其长度等于请求的配置项数量，每个元素为 null
  - 对于工作区文件夹查询：结果是一个单元素列表，包含具有项目根 URI 和目录名称的工作区文件夹描述符
  - 对于工作区编辑请求：结果表示编辑被拒绝（applied 为 false）
  - 对于客户端未处理的任何其他方法：结果为 null

[SPEC]
```

Reasoner 差异

- SPEC 声明：

```
- 当 message.method 为 "elp/status" 时，将 self._status 更新为 message.params.status 的值；如果 params 字典缺失或没有 "status" 键，self._status 保持不变
- 当 message 缺少 "id" 字段，或 message.method 缺失或为假值时，不发送响应（该消息被视为通知）
- 当 message 同时具有非空的 "method" 和 "id"（服务器请求）时，发送一个 JSON-RPC 响应，其 jsonrpc 为 "2.0"，id 相同；结果值满足该方法的协议定义期望：
  - 对于工作区配置查询：结果是一个列表，其长度等于请求的配置项数量，每个元素为 null
  - 对于工作区文件夹查询：结果是一个单元素列表，包含具有项目根 URI 和目录名称的工作区文件夹描述符
  - 对于工作区编辑请求：结果表示编辑被拒绝（applied 为 false）
  - 对于客户端未处理的任何其他方法：结果为 null
```

- 推导的实际行为：

执行 `_handle_server_message` 之后：如果传入消息的 `method` 字段等于 `'elp/status'`，客户端的 `_status` 属性将被设置为 `params.get('status')` 的值（可能为 `None`）。如果消息包含 `'id'` 且 `method` 字段为真值（非空字符串），则会

通过标准输入管道向 ELP 服务器发送一个 JSON-RPC 2.0 响应，该响应具有相同的 id 和计算的结果，使用传输协议框架；服务器将以其作为下一个输入接收。发送的结果是：对于 'workspace/configuration'，一个长度为 len(params.get('items', [])) 的 None 列表；对于 'workspace/workspaceFolders'，[{'uri': self.root_uri, 'name': os.path.basename(self.proj_dir)}]；对于 'workspace/applyEdit'，{'applied': False}；对于其他任何带有 id 的方法，None。如果消息是通知（没有 'id'）或 method 为假值，则不发送响应。ELP 子进程保持运行，消息读取线程继续活动。形式逻辑：令前状态满足 IsRunning(pre) ThreadActive(pre) pre_message = message。则后状态满足：IsRunning(post) ThreadActive(post) (self._status = pre_self._status (message.method = 'elp/status' self._status = message.params.get('status')))) (((('id' message.is_truthy(message.method)) sent_response : sent_response.jsonrpc = '2.0' sent_response.id = message.id writtenToStdin(sent_response) sent_response.result = RESULT(message.method, params, self))) 其中 RESULT = 方法, params, self. case method of 'workspace/configuration' [None | _ params.get('items', [])], 'workspace/workspaceFolders' [{'uri': self.root_uri, 'name': os.path.basename(self.proj_dir)}], 'workspace/applyEdit' {'applied': False}, other None.

- 代码证据：

```
第 4 行: if params is None:
第 5 行:     params = {}
第 7 行:     self._status = params.get("status")
```

- 触发条件：

代码无条件地将 self._status 设置为 params.get('status')，当 'status' 键缺失或 params 缺失时，该值为 None。规范要求在此场景下 self._status 保持不变。

Bug validator

- 触发摘要：当 'status' 键缺失时，params.get('status') 返回 None，覆盖 _status 而不是保持不变
- 探针标准输出：

CONFIRMED — actual: None | expected: 'INITIAL_VALUE'

INCR-MISMATCH-039 — `src--languages--erlang-py--ElpClient::_next_message`

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/languages/erlang-py/ElpClient::_next_message.py`。
- Reasoner 结果：`logic_verification_results/src/languages/erlang-py/ElpClient::_next_message.json`。
- 详细报告：`src--languages--erlang-py--ElpClient::_next_message.md`。
- Probe：`probe_src--languages--erlang-py--ElpClient::_next_message.py`。

完整生成 SPEC

[SPEC]
单元：src/languages/erlang.py

ElpClient._next_message(self, deadline: float)

前置条件：

- deadline 是一个用单调时钟值表示的时间点
- self 是一个 ElpClient，其消息读取线程处于活动状态，并正在将解析后的 JSON-RPC 消息写入内部队列

后置条件：

- 当在单调时钟到达 deadline 之前有消息可用时，返回来自服务器的下一个待处理解析后的 JSON-RPC 响应或通知消息，该消息为字典，其结构符合 JSON-RPC 2.0 规范
- 当在单调时钟到达 deadline 之前没有可用消息时，包括在调用时 deadline 已经过去的情况，引发 TimeoutError
- 当消息读取线程因不可恢复的异常而终止时，引发 RuntimeError；读取线程的原始异常被链接为 RuntimeError 的原因

[SPEC]

Reasoner 差异

- SPEC 声称：

- 当在单调时钟到达 `deadline` 之前有消息可用时，返回来自服务器的下一个待处理解析后的 JSON-RPC 响应或通知消息，该消息为字典，其结构符合 JSON-RPC 2.0 规范
- 当在单调时钟到达 `deadline` 之前没有可用消息时，包括在调用时 `deadline` 已经过去的情况，引发 `TimeoutError`
- 当消息读取线程因不可恢复的异常而终止时，引发 `RuntimeError`；读取线程的原始异常被链接为 `RuntimeError` 的原因

- 推导的实际行为：

方法执行后，恰好发生以下结果之一：

1. 正常返回：方法返回一个消息 ``msg``。``msg`` 不是 ``BaseException`` 的实例。它在 ``deadline`` 之前（即某个时刻 ``t`` 满足 ``t` <= deadline``）从 ``self._messages`` 中被移除。调用后的内部队列包含调用期间读取线程入队的除 ``msg`` 之外的所有元素。返回点的单调时钟满足 ``time.monotonic()` <= deadline``。
2. 引发 `TimeoutError`：引发 ``TimeoutError``。在 ``deadline`` 之前没有元素从 ``self._messages`` 中被移除。调用后的队列除了读取线程添加的任何新元素外，保持不变。引发异常时 ``time.monotonic()` >= deadline``。
3. 引发 `RuntimeError`：引发 ``RuntimeError``，并链接一个 ``BaseException`` ``e``。``e`` 在 ``deadline`` 之前从 ``self._messages`` 中被移除。调用后的队列包含调用期间读取线程入队的其他所有元素。移除发生在某个时刻 ``t`` 满足 ``t` <= deadline``。

活动的消息读取线程继续运行，并可能继续将解析后的 JSON-RPC 消息或异常入队到 ``self._messages`` 中。

形式化地，设：

- ``Q_pre`` 为方法进入时 ``self._messages`` 中元素的多重集。
- ``Q_post`` 为方法结束时（返回后或异常传播前）的多重集。
- ``T_pre`` = 进入时 ``time.monotonic()``，``T_post`` 为退出时。
- ``added`` 为读取线程在区间 ``[T_pre, T_post]`` 期间入队的元素的多重集。
- ``deadline`` 为给定的截止时间。

后置条件为析取：

- ``(return msg) [ msg BaseException msg (Q_pre added) Q_post = (Q_pre added) \ {msg} T_post deadline ]``
- ``(raise TimeoutError) [ Q_post = Q_pre added T_post deadline ]``
- ``(raise RuntimeError from e) [ e BaseException e (Q_pre added) Q_post = (Q_pre added) \ {e} T_post deadline ]``

- 代码证据：

第11行：`return message`

- 触发条件：

规范要求该方法返回一个符合 JSON-RPC 2.0 规范的字典。然而，代码返回队列中的任何非 `BaseException` 对象，而没有检查它是否是字典，因此在队列包含非字典的非异常值的任何输入下，违反了规范。

Bug validator

- 触发摘要：`_next_message` 返回队列中的任何非 `BaseException` 项而不检查它是否是字典，因此非字典的 JSON 值（如列表）被返回，违反了规范。
- Probe 标准输出：

CONFIRMED — actual: [1, 2, 3] (type: list) | expected: dict conforming to JSON-RPC 2.0

INCR-MISMATCH-040 — `src--languages--erlang-py--ElpClient::_send`

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/languages/erlang-py/ElpClient::_send.py`。

- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::_send.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::_send.md`。
- Probe: `probe_src--languages--erlang-py--ElpClient::_send.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/erlang.py

ElpClient._send(self, message: dict)

前置条件：
- self._proc 不为 None，self._proc.stdin 不为 None 且已打开、可供写入
- message 是一个表示可 JSON 序列化的 JSON-RPC 消息的字典

后置条件：
- 消息的 JSON 序列化形式通过 LSP 传输协议帧被传输到 ELP 子进程的标准输入
- 传输的帧由一个 Content-Length 头组成，其值为 UTF-8 编码的 JSON 载荷的字节长度，后跟一个 CRLF 空行，然后是 UTF-8 编码的 JSON 载荷本身
- 头部部分是 ASCII 编码的；载荷是紧凑的 JSON，键、值、冒号或逗号之间没有空白字符，所有非 ASCII 字符保留其原始形式
- 对于同一客户端实例上的其他并发 _send 调用，传输是原子的
- 返回时，完整的帧已递送到子进程的输入流（写入已被刷新到 OS 管道）
- 当 ELP 子进程未运行或其标准输入不可用时，抛出 RuntimeError
[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 消息的 JSON 序列化形式通过 LSP 传输协议帧被传输到 ELP 子进程的标准输入

- 传输的帧由一个 Content-Length 头组成，其值为 UTF-8 编码的 JSON 载荷的字节长度，后跟一个 CRLF 空行，然后是 UTF-8 编码的 JSON 载荷本身

- 头部部分是 ASCII 编码的；载荷是紧凑的 JSON，键、值、冒号或逗号之间没有空白字符，所有非 ASCII 字符保留其原始形式

- 对于同一客户端实例上的其他并发 _send 调用，传输是原子的

- 返回时，完整的帧已递送到子进程的输入流（写入已被刷新到 OS 管道）

- 当 ELP 子进程未运行或其标准输入不可用时，抛出 RuntimeError

- 推导的实际行为：

在给定的前置条件下执行后，方法完成且未引发异常。字典消息被序列化为 JSON 字符串（UTF-8 编码），以 'Content-Length' 头进行帧封装，在 self._write_lock 下原子地写入 self._proc.stdin，然后刷新。刷新确保所有缓冲数据被推送到子进程的控制台流（无条件地），并且如果日志流打开，也推送到日志流。方法返回后，self._proc.stdin 保持打开并可写，self._write_lock 被释放，整个帧已成功传输到子进程。形式化地，设 为前状态， 为后状态。则 满足：(1) 写入 self._proc.stdin 的控制台流的字节序列为 .console_stream frame，其中 frame = b'Content-Length: ' + len(payload_encoded).to_bytes() + b'\r\n\r\n' + payload_encoded，且 payload_encoded = json.dumps(message, ...).encode('utf-8')。(2) 如果与 self._proc.stdin 关联的可选日志流在 中打开，则其内容为 .log_stream frame；否则不变。(3) self._proc.stdin 的内部缓冲区为空。(4) self._write_lock 处于解锁状态。(5) self._proc.stdin 保持非 None 并可写。

- 代码证据：

```
第 2 行：         if self._proc is None or self._proc.stdin is None:
```

- 触发条件：

规范要求子进程的标准输入不可用时抛出 RuntimeError。代码仅检查 None，但 stdin 可能不可用，同时仍然是一个非 None 对象（例如已关闭或管道断裂），从而导致传播不同的异常而非 RuntimeError。

Bug validator

- 触发摘要：`stdin` 是一个非 `None` 对象但已关闭/不可用；`None` 检查通过，因此没有抛出 `RuntimeError`，而 `write/flush` 抛出 `ValueError` 代替
- `Probe` 标准输出：

```
CONFIRMED – 代码引发 ValueError("write to closed file"), 但规范要求 stdin 不可用时引发
RuntimeError。_proc.stdin 非 None (mock: <MagicMock name='mock.stdin' id='138086028551440'>) 但
stdin 已关闭/不可用。
```

INCR-MISMATCH-041 — `src--languages--erlang-py--ElpClient::_wait_for_response`

- 人工审计：**SPEC 错误**。
- Validator: **confirmed**；尝试次数: **1**。
- 原始函数与 SPEC: `src/languages/erlang-py/ElpClient::_wait_for_response.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/ElpClient::_wait_for_response.json`。
- 详细报告: `src--languages--erlang-py--ElpClient::_wait_for_response.md`。
- `Probe`: `probe_src--languages--erlang-py--ElpClient::_wait_for_response.py`。

完整生成 SPEC

```
[SPEC]
单元: src/languages/erlang.py

ElpClient._wait_for_response(self, request_id: int, deadline: float) -> Any

前置条件:
- self 是一个 ElpClient, 其底层 JSON-RPC 通信通道处于打开且可操作的状态。
- 一个对应给定整数 request_id 的请求之前已通过同一通道发送。
- deadline 是一个单调时间值, 表示与底层通道超时机制处于同一时钟域中的一个绝对时间点。

后置条件:
- 阻塞调用者, 按顺序从底层通道中消费消息, 直到收到一个带有匹配 id 字段的 JSON-RPC 响应。
- 当某条消息的 "id" 字段等于 request_id 且该消息缺少 "method" 字段时, 该消息被视为匹配的响应, 以此区分服务器的响应与服务器发起的请求和通知。
- 从服务器收到的非匹配响应消息将被转发以进行服务器初始化的处理, 并且不会导致该函数返回。
- 当底层通道关闭或在 deadline 之前未收到任何消息时: 引发 TimeoutError。
- 当匹配的响应包含 "error" 字段时:
    • 如果 error 是一个字典, 且其 "code" 字段等于瞬态内容修改错误代码: 则引发 _ContentModifiedError 并携带错误详情。
    • 否则: 引发 RuntimeError, 其消息中包含错误的描述。
- 当匹配的响应不包含 "error" 字段时: 返回该响应中 "result" 字段的值。
[SPEC]
```

Reasoner 差异

- SPEC 声明:

```
- 阻塞调用者, 按顺序从底层通道中消费消息, 直到收到一个带有匹配 id 字段的 JSON-RPC 响应。
- 当某条消息的 "id" 字段等于 request_id 且该消息缺少 "method" 字段时, 该消息被视为匹配的响应, 以此区分服务器的响应与服务器发起的请求和通知。
- 从服务器收到的非匹配响应消息将被转发以进行服务器初始化的处理, 并且不会导致该函数返回。
- 当底层通道关闭或在 deadline 之前未收到任何消息时: 引发 TimeoutError。
- 当匹配的响应包含 "error" 字段时:
    如果 error 是一个字典, 且其 "code" 字段等于瞬态内容修改错误代码: 则引发 _ContentModifiedError 并携带错误详情。
    否则: 引发 RuntimeError, 其消息中包含错误的描述。
- 当匹配的响应不包含 "error" 字段时: 返回该响应中 "result" 字段的值。
```

- 推导的实际行为:

如果在绝对单调时间 `deadline` 之前收到 JSON-RPC 响应消息 `m`, 满足 `m['id'] == request_id` 且 `'method'` 不在 `m` 中, 那么该方法要么在 `m` 不包含 `'error'` 字段或 `'error'` 为 `null` 时返回 `m['result']`, 要么在 `m['error']` 是一个字典且其键 `'code'` 等于 `_CONTENT_MODIFIED_ERROR` 时引发 `_ContentModifiedError(m['error'])`, 要么对于任何其他非空的 `'error'` 引发 `RuntimeError('ELP request failed: {error}')`。如果在 `deadline` 之前没有收到这样的匹配响应, 则引发 `TimeoutError`。在执行期间, 从服务器收到的每一条不满足匹配条件的消息都会被传递给

`self._handle_server_message`，该方法会根据已注册的处理器对其进行处理，适当地改变客户端状态，并且在正常操作下保证不会引发异常。底层通信通道保持打开且可操作状态，除非某个处理器导致致命异常，在这种情况下异常将不被捕获地传播。形式化地，令传入的消息流 `M` 为已解析的消息字典的有序序列。令 `P` 为 `M` 的最长前缀，其中不包含任何满足 `'id' == request_id` 且缺少 `'method'` 的消息，并且该前缀在 `deadline` 之前被完全消费。执行过程通过 `_handle_server_message` 处理 `P` 中的所有消息，产生中间状态 `S_i`。如果 `P` 之后的第一条消息 `m*` 满足匹配条件并在 `deadline` 之前到达，那么结果是 `return(m*.get('result'))` 或如上所述引发异常，客户端状态相应前进。如果在 `deadline` 之前不存在这样的 `m*`，结果是引发 `TimeoutError`，并且处理完 `P` 之后的状态即为最终状态。在所有情况下，不会对 `ElpClient` 或其通道产生其他副作用。

- 代码证据：

```
第 6 行: if error:
```

- 触发条件：

代码通过 `'if error:'` 检查 `'error'` 值的真值性，对于 `None`（以及其他类假值）结果为 `False`。规范要求任何包含 `'error'` 字段的匹配响应都应引发异常（如果不是特定的内容修改错误，则为 `RuntimeError`）。因此，对于 `error=null` 的响应，代码错误地返回了 `'result'` 而不是引发 `RuntimeError`。

Bug validator

- 触发摘要：带有 `error=null` 的响应导致真值性检查失败，返回结果而不是引发 `RuntimeError`。
- Probe 标准输出：

```
CONFIRMED — actual returned: 'some_value' | expected: RuntimeError (per spec)
```

INCR-MISMATCH-042 — `src--languages--erlang-py--ElpClient::initialize`

- 人工审计：契约待确认。
- Validator： **confirmed**；尝试次数： **1**。
- 原始函数与 SPEC： `src/languages/erlang-py/ElpClient::initialize.py`。
- Reasoner 结果： `logic_verification_results/src/languages/erlang-py/ElpClient::initialize.json`。
- 详细报告： `src--languages--erlang-py--ElpClient::initialize.md`。
- Probe： `probe_src--languages--erlang-py--ElpClient::initialize.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/erlang.py

ElpClient.initialize(self, bootstrap_path: str, bootstrap_source: str | None = None)

前置条件：
- self 是一个已调用 __enter__ 的 ElpClient (ELP 子进程正在运行，stdin/stdout 管道已打开，且 JSON-RPC 消息读取线程处于活动状态)
- bootstrap_path 是一个非空字符串，标识一个文件系统路径；如果 bootstrap_source 为 None，则 bootstrap_path 必须指向一个存在、可读的文本文件
- bootstrap_source (若提供) 是一个字符串，包含要使用的文档内容，用于替代从 bootstrap_path 读取

后置条件：
- 完成 LSP 初始化握手：发送带有客户端能力的 "initialize" 请求，然后发送 "initialized" 通知
- 在服务器上打开位于 bootstrap_path 的文档，其文本内容与 bootstrap_source 匹配 (若 bootstrap_source 为 None，则为 bootstrap_path 的文件内容)
- 阻塞直到服务器报告的状态表明其已达到运行状态，或在调用入口点起计的 self.timeout 秒内未达到时引发 TimeoutError
- 返回服务器 "initialize" 响应中的 "serverInfo" 子字典，或当响应缺失、不是字典或不包含 "serverInfo" 键时返回 None
- 当 ELP 子进程未运行 (stdin 不可用) 或 JSON-RPC 通道遇到不可恢复错误时引发 RuntimeError
- 当服务器未能在截止时间前达到运行状态或子进程停止生成消息时引发 TimeoutError

[SPEC]
```

- SPEC 声称：

```
- 完成 LSP 初始化握手：发送带有客户端能力的 "initialize" 请求，然后发送 "initialized" 通知
- 在服务器上打开位于 bootstrap_path 的文档，其文本内容与 bootstrap_source 匹配（若 bootstrap_source 为
None，则为 bootstrap_path 的文件内容）
- 阻塞直到服务器报告的状态表明其已达到运行状态，或在调用入口点起计的 self.timeout 秒内未达到时引发
TimeoutError
- 返回服务器 "initialize" 响应中的 "serverInfo" 子字典，或当响应缺失、不是字典或不包含 "serverInfo" 键时
返回 None
- 当 ELP 子进程未运行（stdin 不可用）或 JSON-RPC 通道遇到不可恢复错误时引发 RuntimeError
- 当服务器未能在截止时间前达到运行状态或子进程停止生成消息时引发 TimeoutError
```

- 推导的实际行为：

```
如果方法正常返回，则返回值是 (R.get('serverInfo') if type(R) == dict else None)，其中 R 是 JSON-RPC
'initialize' 响应的 'result' 字段；'initialize' 请求、'initialized' 通知以及针对 bootstrap_path 的
'textDocument/didOpen' 通知均已成功发送；while 循环处理服务器消息直到 str(self._status).lower() ==
'running'，因此 self._status 指示 'running'；在该状态之前接收到的所有服务器请求均已得到适当响应，且任何
'elp/status' 通知更新了 self._status。消息接收的截止时间在循环入口处固定（time.monotonic() +
self.timeout）。如果方法引发异常，则可能是 TimeoutError（来自 request 或 _next_message 超过
self.timeout）、RuntimeError（来自 request 错误、重试耗尽、notify 失败或读取线程异常）或 IOError（来自
open_document，当 bootstrap_source 为 None 且文件无法读取时），且客户端状态可能被部分修改（例
如，'initialize' 已发送但状态未达到 'running'）。形式化：(return(server_info) exception) (R: R =
request('initialize',...).result server_info = (R.get('serverInfo') if dict(R) else None)
sent('initialized') sent(didOpen(bootstrap_path,...)) (str(self._status).lower() = 'running')
m received_before('running'): handled(m)). 如果引发异常 e，则 e {TimeoutError, RuntimeError,
IOError} (some_side_effects none).
```

- 代码证据：

```
第 26 行： deadline = time.monotonic() + self.timeout
```

- 触发条件：

规范要求超时截止时间从调用入口点开始计算，但代码在 initialize 请求和 open_document 之后才设置截止时间，可能导致方法在应该超时的情况下成功。

Bug validator

- 触发摘要：截止时间在 request() 之后计算，而非在调用入口点；请求延迟导致有效超时窗口超出规范限制。
- Probe 标准输出：

```
CONFIRMED — 实际截止时间：6546.858 > 预期：6546.257（差值：0.601 秒，规范要求截止时间从调用入口点计算）
```

INCR-MISMATCH-043 — `src--languages--erlang-py--ElpClient::open_document`

- 人工审计：契约待确认。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：src/languages/erlang-py/ElpClient::open_document.py。
- Reasoner 结果：logic_verification_results/src/languages/erlang-py/ElpClient::open_document.json。
- 详细报告：src--languages--erlang-py--ElpClient::open_document.md。
- Probe：probe_src--languages--erlang-py--ElpClient::open_document.py。

[SPEC]
单元：src/languages/erlang.py

ElpClient.open_document(self, path: str, source: str | None = None)

前置条件：

- self 是一个 ElpClient，其底层的 JSON-RPC 通信通道已打开且可写
- path 是一个非空字符串，标识一个文件系统路径
- 当 source 为 None 时，path 必须解析到一个存在且可读的文本文件

后置条件：

- 向 ELP 服务器发送一条 “textDocument/didOpen” 通知，该通知的 textDocument 字段是一个字典，包含：
 - uri: 表示 path 实参的绝对 file:// URI
 - languageId: "erlang"
 - version: 1
 - text: 若提供了 source，则为其值；否则为 path 所指文件的 UTF-8 文本内容
- 当 source 为 None 且无法读取 path 所指文件时，底层的 IOError 将传播给调用者

[SPEC]

Reasoner 差异

- SPEC 声明：

- 向 ELP 服务器发送一条 “textDocument/didOpen” 通知，该通知的 textDocument 字段是一个字典，包含：
 - uri: 表示 path 实参的绝对 file:// URI
 - languageId: "erlang"
 - version: 1
 - text: 若提供了 source，则为其值；否则为 path 所指文件的 UTF-8 文本内容
- 当 source 为 None 且无法读取 path 所指文件时，底层的 IOError 将传播给调用者

- 推导实际行为：

ElpClient 底层的 JSON-RPC 通信通道保持打开且可写。一条 JSON-RPC 2.0 通知消息已被发送，其方法为 "textDocument/didOpen"。消息的 params 包含单一键 "textDocument"，其值为一个字典，包含以下键："uri" 设置为 `path` 解析出的绝对路径的 URI（即 `Path(path).resolve().as\_uri()`），"languageId" 设置为 "erlang"，"version" 设置为 1，"text" 设置为：若 `source` 不为 None，则为 `source` 提供的字符串内容；否则为以 UTF-8 文本读取（解码错误用替换字符处理）的解析路径所指文件的全部内容。局部变量 `document` 保存解析后的绝对 `Path` 对象，局部变量 `source` 保存已发送的文本内容（即原始实参或者文件内容）。没有返回值。形式化表示：(ch = self.communication_channel . is_open(ch) writable(ch)) (msg = notification(method: "textDocument/didOpen", params: { "textDocument": { uri: document.as_uri(), languageId: "erlang", version: 1, text: source_text } }) . transmitted(ch, msg)) 其中 document = resolve(Path(path)) source_text = (if source_arg None then source_arg else read_text(document, encoding="utf-8", errors="replace")) source_arg = 进入函数时原始的 `source` 绑定。

- 代码证据：

第 2 行： document = Path(path).resolve()

- 触发条件：

代码使用了 Path(path).resolve()，它会解析符号链接，因此生成的 URI 并不是规约所要求的原始 path 实参的表示。规约要求绝对 file:// URI 表示 path 实参，而不是其解析后的目标。

Bug validator

- 触发摘要：Path.resolve() 会跟随符号链接，因此当 path 为符号链接时，open_document 发送的是目标 URI，而不是 path 实参对应的 URI。
- 探针 stdout：

CONFIRMED — 实际 URI: 'file:///tmp/tmpn3s6sqex/real.erl' | 期望 URI: 'file:///tmp/tmpn3s6sqex/link.erl'

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/languages/erlang-py/ElpClient::request.py`。
- Reasoner 结果：`logic_verification_results/src/languages/erlang-py/ElpClient::request.json`。
- 详细报告：`src--languages--erlang-py--ElpClient::request.md`。
- Probe：`probe_src--languages--erlang-py--ElpClient::request.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/erlang.py

ElpClient.request(self, method: str, params: dict | list | None = None) -> Any

前置条件：
- self 是一个 ElpClient 实例，其底层的 JSON-RPC 通信通道处于打开且可用的状态
- method 是一个非空字符串
- params，当不为 None 时，是一个 JSON 可序列化的字典或列表

后置条件：
- 向服务器发送一个 JSON-RPC 2.0 请求，包含给定的 method 和 params（其中 None params 被视为空对象），并附带一个唯一的整数标识符，该标识符在同一客户端实例的连续调用中严格递增
- 阻塞调用方，直到服务器返回匹配该标识符的响应，或者从入口起经过的总时间达到 self.timeout 秒，以先发生者为准
- 成功时：返回匹配响应中的 "result" 字段的值
- 当服务器指示临时 ContentModified 错误时：最多重发请求达到固定的最大总尝试次数，从入口起总持续时间受 self.timeout 秒限制；当所有尝试均未成功而耗尽时，抛出标识失败方法的 RuntimeError
- 当从入口起经过 self.timeout 秒仍未有匹配响应到达时：抛出 TimeoutError
- 当服务器响应的错误语义未被重试策略覆盖时：抛出 RuntimeError
[SPEC]
```

Reasoner 差异

- SPEC 声称：

```
- 向服务器发送一个 JSON-RPC 2.0 请求，包含给定的 method 和 params（其中 None params 被视为空对象），并附带一个唯一的整数标识符，该标识符在同一客户端实例的连续调用中严格递增
- 阻塞调用方，直到服务器返回匹配该标识符的响应，或者从入口起经过的总时间达到 self.timeout 秒，以先发生者为准
- 成功时：返回匹配响应中的 "result" 字段的值
- 当服务器指示临时 ContentModified 错误时：最多重发请求达到固定的最大总尝试次数，从入口起总持续时间受 self.timeout 秒限制；当所有尝试均未成功而耗尽时，抛出标识失败方法的 RuntimeError
- 当从入口起经过 self.timeout 秒仍未有匹配响应到达时：抛出 TimeoutError
- 当服务器响应的错误语义未被重试策略覆盖时：抛出 RuntimeError
```

- 推导的实际行为：

执行后，如果 `_MAX_CONTENT_MODIFIED_RETRIES > 0`，则恰好发生以下结果之一：(i) 方法返回一个值 `v`，使得 `v` 等于具有匹配请求 `id` 的 JSON-RPC 响应的 `'result'` 字段，且在最终尝试中未引发临时 `content-modified` 错误。(ii) 抛出 `TimeoutError`，原因是在任何尝试中 `_wait_for_response` 调用超时（截止时间超过或通道关闭），或者在捕获 `_ContentModifiedError` 后的重试处理期间截止时间到期。(iii) 抛出 `RuntimeError`，原因是在任何尝试中从 `_wait_for_response` 接收到非临时服务器错误，或者所有 `_MAX_CONTENT_MODIFIED_RETRIES` 次尝试均导致临时 `content-modified` 错误且达到重试限制。(iv) 任何由 `_send` 或 `_wait_for_response` 抛出的其他异常（例如连接错误）会直接传播而未被捕获。如果 `_MAX_CONTENT_MODIFIED_RETRIES == 0`，则抛出带有消息 `'unreachable'` 的 `AssertionError`。正式地：(`v, return v` `response_of_final_attempt.result = v` `no _ContentModifiedError on final attempt`) (`raise TimeoutError ( attempt. _wait_for_response raised TimeoutError on attempt (caught _ContentModifiedError deadline time.monotonic() 0))`) (`raise RuntimeError (( attempt. _wait_for_response raised RuntimeError not subclass of _ContentModifiedError) ( attempts. _ContentModifiedError raised))`) (`raise AssertionError _MAX_CONTENT_MODIFIED_RETRIES == 0`)。通信通道状态在正常返回时保持打开；在异常情况下其状态未定义。

- 代码证据：

```
第 4 行:      for attempt in range(_MAX_CONTENT_MODIFIED_RETRIES):
第 26 行:          raise AssertionError("unreachable")
```

- 触发条件:

规范仅允许返回结果、抛出 `TimeoutError` 或抛出 `RuntimeError`。当 `_MAX_CONTENT_MODIFIED_RETRIES` 为 0 时，代码抛出 `AssertionError`，该异常不属于允许的结果。

Bug validator

- 触发摘要: 将 `_MAX_CONTENT_MODIFIED_RETRIES` 设置为 0 会导致 `range(0)` 生成零次迭代，直接进入并抛出 `AssertionError("unreachable")`，这违反了规范允许的结果（返回、`TimeoutError`、`RuntimeError`）。
- Probe 标准输出:

```
CONFIRMED - AssertionError 被触发 ( 规范违规 ) : unreachable
```

INCR-MISMATCH-045 — `src--languages--erlang-py--_SourceIndex::build`

- 人工审计: 实现缺陷候选。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/erlang-py/_SourceIndex::build.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/_SourceIndex::build.json`。
- 详细报告: `src--languages--erlang-py--_SourceIndex::build.md`。
- Probe: `probe_src--languages--erlang-py--_SourceIndex::build.py`。

完整生成 SPEC

```
[SPEC]
单元 : src/languages/erlang-py/_SourceIndex.py

_SourceIndex.build(cls, source: str) -> _SourceIndex

前置条件:
- source 是一个字符串

后置条件:
- 返回一个 _SourceIndex 实例，其内容完全从 source 派生
- 返回的索引将 source 表示为一个有序的行序列，其中行边界对应于 source 中的换行符位置
- 对于每一行，其在 source 中的第一个字符的字节偏移量可以从返回的索引中计算得出
- 返回的索引中所有行的总字节数等于 source 的长度
[SPEC]
```

Reasoner 差异

- SPEC 主张:

- 返回一个 `_SourceIndex` 实例，其内容完全从 `source` 派生
- 返回的索引将 `source` 表示为一个有序的行序列，其中行边界对应于 `source` 中的换行符位置
- 对于每一行，其在 `source` 中的第一个字符的字节偏移量可以从返回的索引中计算得出
- 返回的索引中所有行的总字节数等于 `source` 的长度

- 推导的实际行为:

该方法返回一个 ``cls`` 的实例（预期为 ``_SourceIndex`` 或其子类），该实例具有三个属性：``source`` 等于原始输入字符串 ``source``，``lines`` 是调用 ``source.splitlines(keepends=True)`` 得到的字符串列表，``line_offsets`` 是一个整数列表，其中每个元素表示对应行在原始 `source` 中起始位置的字符偏移量。形式化地，令 ``r`` 为返回的对象。则

`r.source == source, r.lines == source.splitlines(keepends=True), len(r.line_offsets) == len(r.lines)`, 并且对于所有满足 $0 \leq i < \text{len}(r.\text{lines})$ 的 i , 有 $r.\text{line_offsets}[i] == \sum_{j=0}^{i-1} \text{len}(r.\text{lines}[j])$ (当 $i=0$ 时, 求和定义为 0)。不会引发异常, 也不产生其他副作用。

- 代码证据:

第 2 行: `lines = source.splitlines(keepends=True)`

- 触发条件:

该规范要求行边界仅对应于换行符位置。对于 `source='hello\vworld'`, 垂直制表符 (`\v`) 不是标准的换行符, 但 `splitlines(keepends=True)` 会在此处拆分, 产生两行 (`'hello\v'` 和 `'world'`) 并相应计算行偏移量。这违反了返回的索引应将 `source` 表示为一个有序的行序列, 且行边界对应于换行符位置的要求。

Bug validator

- 触发摘要: 对于 `source='hello\vworld'`, `splitlines(keepends=True)` 在垂直制表符 (`\v`) 处错误地进行了拆分, 而该字符并非换行符, 导致生成 2 行而不是 1 行。
- Probe 标准输出:

```
CONFIRMED — 实际行数: 2 | 期望行数: 1
lines: ['hello\x0b', 'world']
line_offsets: [0, 6]
source: 'hello\x0bworld'
```

INCR-MISMATCH-046 — `src--languages--erlang-py--_analyze_project_uncached`

- 人工审计: **推理误判**。
- Validator: **not_confirmed**; 尝试次数: 3。
- 原始函数与 SPEC: `src/languages/erlang-py/_analyze_project_uncached.py`。
- Reasoner 结果: `logic_verification_results/src/languages/erlang-py/_analyze_project_uncached.json`。
- 详细报告: `src--languages--erlang-py--_analyze_project_uncached.md`。
- Probe: `probe_src--languages--erlang-py--_analyze_project_uncached.py`。

完整生成 SPEC

```
[SPEC]
单元: src/languages/erlang-py/_analyze_project_uncached.py

_analyze_project_uncached(proj_dir: str) -> ErlangAnalysis

前置条件:
- proj_dir 是一个非空字符串, 表示一个文件系统路径

后置条件:
- 返回一个 ErlangAnalysis 对象, 其 .functions 属性是一个字典,
  将每个 .erl 文件的绝对路径映射到一个 (function_id, source_text) 元组列表,
  其中 function_id 是规范的字符串标识符, source_text 是该函数的源代码
- 返回的 ErlangAnalysis 的 .edges 属性是一个字典,
  将 (function_id, caller_module) 元组映射到被调用方 function_id 的集合
- 返回的 ErlangAnalysis 的 .spans 属性是一个字典,
  将每个 .erl 文件的绝对路径映射到一个 (function_id, start_line, end_line) 元组列表,
  其中 start_line 和 end_line 是基于 1 的闭区间行号
- 返回的 ErlangAnalysis 的 .server_info 属性由 ELP 服务器初始化响应填充
- 当解析为绝对路径后, 以 proj_dir 为根的目录树中不存在 .erl 文件时,
  返回一个 ErlangAnalysis, 其所有三个字典属性为空
- 当 ELP 后台进程无法启动、LSP 通信通道失败或无法分析 proj_dir 处的项目时, 引发异常

[SPEC]
```

Reasoner 差异

- SPEC 声明:

- 返回一个 ErlangAnalysis 对象, 其 .functions 属性是一个字典, 将每个 .erl 文件的绝对路径映射到一个 (function_id, source_text) 元组列表, 其中 function_id 是规范的字符串标识符, source_text 是该函数的源代码
- 返回的 ErlangAnalysis 的 .edges 属性是一个字典, 将 (function_id, caller_module) 元组映射到被调用方 function_id 的集合
- 返回的 ErlangAnalysis 的 .spans 属性是一个字典, 将每个 .erl 文件的绝对路径映射到一个 (function_id, start_line, end_line) 元组列表, 其中 start_line 和 end_line 是基于 1 的闭区间行号
- 返回的 ErlangAnalysis 的 .server_info 属性由 ELP 服务器初始化响应填充
- 当解析为绝对路径后, 以 proj_dir 为根的目录树中不存在 .erl 文件时, 返回一个 ErlangAnalysis, 其所有三个字典属性为空
- 当 ELP 后台进程无法启动、LSP 通信通道失败或无法分析 proj_dir 处的项目时, 引发异常

- 推导的实际行为:

当函数正常终止时, 返回一个 ErlangAnalysis 对象; 否则会引发 TimeoutError 或 RuntimeError 并且不会返回任何值。

****正常终止 (无异常)****

1. 如果 `proj\_dir` 不包含任何 `.erl` 文件 (递归查找), 函数返回 `ErlangAnalysis(functions={}, edges={})`。不启动 ElpClient, 不读取任何文件。
2. 如果至少有一个 `.erl` 文件,
 - 收集 `os.path.abspath(proj\_dir)` 下的所有 `.erl` 文件到 `files` 中; 将其内容读入 `sources` 字典 (UTF-8, 错误替换)。
 - 创建 ElpClient 并进入上下文, 启动语言服务器子进程; 在 `with` 代码块结束后子进程停止, 客户端关闭。
 - 使用第一个文件及其源代码初始化服务器; 所有其他文件通过 `open\_document` 打开。
 - 对于每个文件:
 - * 构建源代码索引。
 - * 确定调用方模块名称。
 - * 从服务器请求文档符号。
 - * 处理具有有效范围的函数符号 (kind == FUNCTION_KIND)。格式错误或重复的符号 (在同一个文件内按规范函数 ID 判别) 会被静默跳过。对格式错误的符号记录警告。
 - * 从符号的 URI 和名称获得规范函数 ID。有效符号产生元组添加到 `functions` 字典中: `functions[function\_id]` 变成一个非空列表, 包含 `(caller\_module, source\_text\_of\_function)`, 其中源文本是根据函数定义的范围从原始文件中提取的。
 - * 类似地填充 `edges` 和 `spans` 将文件路径映射到 `[(function\_name, start\_line, end\_line)]`, 对应于每个识别的函数, `edges` 捕获调用者-被调用者关系。
 - 处理完所有文件后, 函数返回 `ErlangAnalysis(functions=functions, edges=edges, spans=spans)`。
 - 所有文件读取和服务器通信均成功; 因为 `\_function\_id` 抛出的 ValueError 被捕获并忽略, 所以没有未处理的 ValueError。

****异常情况****

在服务器初始化或符号请求期间, 可能引发未处理的 `TimeoutError` 或 `RuntimeError`。在这种情况下, `with` 代码块仍然确保 ElpClient 被清理 (服务器子进程终止), 但函数不返回值。

****形式逻辑****

- 令 P 为输入 `proj\_dir`, $F = \{ f \mid \text{_erlang_files}(\text{os.path.abspath}(P)).f \}$, R 为无异常时返回的值。
- R ErlangAnalysis。
 - 如果 $F = \emptyset$ 则 $R.functions = \{\}$ $R.edges = \{\}$ 。
 - 如果 $F \neq \emptyset$ 则:
 - functions, edges, spans 使得 $R.functions = functions$ $R.edges = edges$ $R.spans = spans$
 - $\text{fid} \in \text{keys}(functions)$ ($functions[\text{fid}]$ 是列表 L $L = []$ (cm, src) L , $\text{cm} = \text{_caller_module}(\text{path})$ 对于包含 fid 的文件, $\text{src} =$ 该函数范围的源代码索引子串)。
 - 路径 F , $\text{spans}[\text{path}] =$ 一个元组列表 (name, l1, l2), 对应该文件中定义的函数。
 - (edges 反映从 LSP 符号中提取的静态调用关系)。
 - 服务器子进程在 with 代码块期间运行, 并在返回之前已终止。

- 代码证据:

第 5 行: `return ErlangAnalysis(functions={}, edges={})`

- 触发条件:

当没有 .erl 文件存在时, 规范要求返回的 ErlangAnalysis 将所有三个字典属性 (functions、edges、spans) 都设为空; 代码返回的对象没有 spans 属性, 违反了规范。

- 触发摘要：当没有 .erl 文件时，ErlangAnalysis(functions={}, edges={}) 仍然通过数据类字段 default_factory=dict 拥有 spans={} —— 逻辑验证器在分析提取的函数时未看到类默认值，导致误报。
- Probe 标准输出：

```
WARNING:root:ELP Erlang analysis unavailable for /tmp/tmp8yhcvxf: simulated failure
NOT CONFIRMED - ErlangAnalysis(functions={}, edges={}) asdict 显示所有三个 : functions=True,
edges=True, spans=True。后备路径也正确 : functions=True, edges=True, spans=True。DataClass 字段
default_factory=dict 始终提供 spans={}。
```

INCR-MISMATCH-047 — src--languages--erlang-py--_elp_argv

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：src/languages/erlang-py/_elp_argv.py。
- Reasoner 结果：logic_verification_results/src/languages/erlang-py/_elp_argv.json。
- 详细报告：src--languages--erlang-py--_elp_argv.md。
- Probe：probe_src--languages--erlang-py--_elp_argv.py。

完整生成 SPEC

```
[SPEC]
单元 : src/languages/erlang-py/_elp_argv.py

_elp_argv() -> list[str]

前置条件 :
- 进程环境变量可供读取

后置条件 :
- 返回一个非空的字符串列表，表示用于启动 Erlang Language Platform 服务器子进程的参数向量
- 返回列表的最后一个元素是字符串 "server"
- 该返回值在多次调用之间是确定性的：在 ELP_COMMAND 环境变量值不变且操作系统平台（POSIX 与非 POSIX）相同的情况下，重复调用会返回同一个列表
- 当 ELP_COMMAND 环境变量变化时，返回的列表会反映新的命令
[SPEC]
```

Reasoner 差异

- SPEC 声称：

```
- 返回一个非空的字符串列表，表示用于启动 Erlang Language Platform 服务器子进程的参数向量
- 返回列表的最后一个元素是字符串 "server"
- 该返回值在多次调用之间是确定性的：在 ELP_COMMAND 环境变量值不变且操作系统平台（POSIX 与非 POSIX）相同的情况下，重复调用会返回同一个列表
- 当 ELP_COMMAND 环境变量变化时，返回的列表会反映新的命令
```

- 推导的实际行为：

```
该函数返回一个字符串列表，具体为 (shlex.split(settings.erlang.command.strip() or 'elp', posix=
(os.name != 'nt')) or ['elp']) + ['server'] 的结果。用自然语言描述：返回值 r 是一个非空列表，其最后一个元素是 'server'。前缀列表的获取方式是：取 settings.erlang.command 去除首尾空白之后的值，若为空则用 'elp' 作为默认值，然后通过 shlex.split 将其切分成词元（当 os.name 不是 'nt' 时启用 posix 模式），若切分结果为空列表则使用 ['elp']。形式化表达：
r = _elp_argv()
r = (令 c = settings.erlang.command.strip(), 令 cmd = 若 c != '' 则为 c 否则为 'elp', 令 parts = shlex.split(cmd, posix=(os.name != 'nt')), 若 parts 非空则为 parts 否则为 ['elp']) + ['server']。
```

- 代码证据：

```
第 2 行：command = settings.erlang.command.strip() or "elp"
```

- 触发条件：

当 `ELP_COMMAND` 未设置时，`settings.erlang.command` 可能为 `None`。在 `None` 上调用 `.strip()` 会引发 `AttributeError`，因此该函数无法按照规范要求返回一个字符串列表。

Bug validator

- 触发摘要：当 `settings.erlang.command` 为 `None` 时，调用 `.strip()` 会引发 `AttributeError`，而不是按照规范要求返回一个有效的 `argv` 列表。
- Probe 标准输出：

CONFIRMED — 当 `command` 为 `None` 时引发 `AttributeError: 'NoneType' object has no attribute 'strip'`

INCR-MISMATCH-048 — `src--languages--erlang-py--_source_for_range`

- 人工审计：推理误判。
- Validator：**not_confirmed**；尝试次数：1。
- 原始函数与 SPEC：`src/languages/erlang-py/_source_for_range.py`。
- Reasoner 结果：`logic_verification_results/src/languages/erlang-py/_source_for_range.json`。
- 详细报告：`src--languages--erlang-py--_source_for_range.md`。
- Probe：`probe_src--languages--erlang-py--_source_for_range.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/erlang-py/_source_for_range.py

_source_for_range(source, lsp_range) -> str

前置条件：
- source 是一个包含源代码文本的字符串
- lsp_range 是一个字典，包含键 'start' 和 'end'，每个都是一个字典，包含键 'line'（整数，从 0 开始）和 'character'（整数，从 0 开始）

后置条件：
- 返回 source 中从起始位置（含）到结束位置（不含）的子串
[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 返回 source 中从起始位置（含）到结束位置（不含）的子串

- 推导的实际行为：

如果 ``lsp_range['start']`` 在源顺序上不紧随 ``lsp_range['end']``，则函数返回 ``source`` 的子串，该子串从对应于含起始位置（行 ``lsp_range['start']['line']``，字符 ``lsp_range['start']['character']``）的字节偏移量开始，到对应于不含结束位置（行 ``lsp_range['end']['line']``，字符 ``lsp_range['end']['character']``）的字节偏移量结束。如果起始位置确实紧随结束位置，则行为未定义：函数可能引发异常（例如，``AssertionError`` 或 ``ValueError``）或返回任意结果。

- 代码证据：

第 3 行：`return _SourceIndex.build(source).source_for_range(lsp_range)`

- 触发条件：

规范要求对于任何输入都返回从起始到结束的子串，但当起始紧随结束时，代码的行为未定义。对于反例，起始字符 2 > 结束字符 1。代码可能引发异常或返回任意结果，而规范暗示返回一个空子串（当起始 > 结束时，从起始到结束（不含）的范围为空）。因此代码违反了规范。

Bug validator

- 触发摘要：同一行上的 LSP 范围，起始字符 2 > 结束字符 1；Python 切片 `source[start:end]` 返回空字符串，与规范暗示的空范围匹配。
- Probe 标准输出：

NOT CONFIRMED — 实际结果与预期匹配：''

INCR-MISMATCH-049 — `src--languages--erlang-py--_timeout_seconds`

- 人工审计：SPEC 错误。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/languages/erlang-py/_timeout_seconds.py`。
- Reasoner 结果：`logic_verification_results/src/languages/erlang-py/_timeout_seconds.json`。
- 详细报告：`src--languages--erlang-py--_timeout_seconds.md`。
- Probe：`probe_src--languages--erlang-py--_timeout_seconds.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/erlang-py/_timeout_seconds.py

_timeout_seconds() -> int

前置条件：
- 无（函数不需要参数）

后置条件：
- 返回一个  $\geq 1$  的整数，表示等待单个 LSP 操作的最大秒数
- 当环境变量 `ELP_TIMEOUT_SECONDS` 被设置为一个表示十进制整数  $n$  的字符串时，返回值为  $n$ （当  $n \geq 1$ ），当  $n < 1$  时为 1
- 当 `ELP_TIMEOUT_SECONDS` 未设置，或者被设置为一个不是有效十进制整数表示的字符串时，返回值为内置默认常量 `_DEFAULT_TIMEOUT_SECONDS`
[SPEC]
```

Reasoner 差异

- SPEC 声称：

```
- 返回一个  $\geq 1$  的整数，表示等待单个 LSP 操作的最大秒数
- 当环境变量 `ELP_TIMEOUT_SECONDS` 被设置为一个表示十进制整数  $n$  的字符串时，返回值为  $n$ （当  $n \geq 1$ ），当  $n < 1$  时为 1
- 当 `ELP_TIMEOUT_SECONDS` 未设置，或者被设置为一个不是有效十进制整数表示的字符串时，返回值为内置默认常量 `_DEFAULT_TIMEOUT_SECONDS`
```

- 推导的实际行为：

该函数返回一个整数，始终至少为 1。形式化地：`isinstance(result, int)  $\wedge$  result  $\geq$  1`。

- 代码证据：

```
第 1 行: def _timeout_seconds() -> int:
第 4 行:     return max(1, settings.erlang.timeout_s)
```

- 触发条件:

规范 (B) 规定当 `ELP_TIMEOUT_SECONDS` 缺失或无效时, 函数必须返回 `_DEFAULT_TIMEOUT_SECONDS`。代码从不读取环境变量, 也从不引用 `_DEFAULT_TIMEOUT_SECONDS`; 它只返回 `max(1, settings.erlang.timeout_s)`。即使配置加载器通常设置默认值, 函数自身的逻辑并未强制执行指定的回退, 因此存在某些状态 (例如 `settings.erlang.timeout_s != _DEFAULT_TIMEOUT_SECONDS`) 下输出违反 B 的情况。

Bug validator

- 触发摘要: `settings.erlang.timeout_s` 与 `_DEFAULT_TIMEOUT_SECONDS` 出现分歧 (30 对 180), 且 `ELP_TIMEOUT_SECONDS` 未设置; 函数返回 `settings` 值, 而非规范要求的默认常量
- Probe 标准输出:

CONFIRMED — actual: 30 | expected: 180

src/languages/go-py

INCR-MISMATCH-050 — `src--languages--go-py--batch_extract`

- 人工审计: 契约待确认。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/go-py/batch_extract.py`。
- Reasoner 结果: `logic_verification_results/src/languages/go-py/batch_extract.json`。
- 详细报告: `src--languages--go-py--batch_extract.md`。
- Probe: `probe_src--languages--go-py--batch_extract.py`。

完整生成 SPEC

```
[SPEC]
单元: src/languages/go-py/batch_extract.py

batch_extract(proj_dir) -> Dict[str, List[Tuple[str, str]]]

前置条件:
- proj_dir 是一个非空字符串路径, 指向包含 Go 源文件的项目根目录

后置条件:
- 返回一个字典, 其键为 Go 源文件的绝对文件路径 (字符串),
  值为从每个文件中提取出的所有函数定义的 (func_name, body) 元组组成的列表
- func_name 是包含规范化函数标识符的字符串; body 是包含函数定义完整源代码文本的字符串
- 当没有可用于 Go 的 codegraph 后端时, 返回空字典 {}
- 仅处理 proj_dir 内的 .go 源文件
[SPEC]
```

Reasoner 差异

- SPEC 声明:

- 返回一个字典, 其键为 Go 源文件的绝对文件路径 (字符串), 值为从每个文件中提取出的所有函数定义的 (func_name, body) 元组组成的列表
- func_name 是包含规范化函数标识符的字符串; body 是包含函数定义完整源代码文本的字符串
- 当没有可用于 Go 的 codegraph 后端时, 返回空字典 {}
- 仅处理 proj_dir 内的 .go 源文件

- 推导的实际行为：

如果 `CodeGraphExtractor.from_proj_dir(proj_dir)` 返回 `None`，则函数返回空字典 `{}`。否则，令 `cg` 为返回的实例；函数返回 `cg.get_functions_by_file('go', proj_dir)`，该调用返回一个字典，将绝对文件路径映射到 `(func_name, body)` 元组组成的列表，每个 `body` 以换行符结尾，元组按行号升序排列，重复的函数名会通过添加数字后缀进行消歧，无法读取的源文件会被静默跳过，如果 `'go'` 不是可识别的语言则返回空字典。形式化表示为：`ret = ({}` 如果 `cg` 为 `None` 则返回 `{}`，否则返回 `cg.get_functions_by_file('go', proj_dir)`)。

- 代码证据：

```
第 4 行: return cg.get_functions_by_file("go", proj_dir) if cg else {}
```

- 触发条件：

代码的行为并不保证仅处理 `proj_dir` 内的 `.go` 文件；`get_functions_by_file` 可能会返回 `proj_dir` 外的文件路径，违反规范。

Bug validator

- 触发摘要：当 `codegraph` 数据库包含带有父目录遍历 (`../`) 的 `file_path` 条目时，`get_functions_by_file` 通过 `os.path.join` 解析它们，而未验证路径是否保持在 `proj_dir` 内，导致返回的字典中出现 `proj_dir` 外的文件。
- Probe 标准输出：

```
CONFIRMED — spec 要求仅包含 proj_dir 内的文件，但返回的键中包含了 proj_dir 外的路径：
['/tmp/probe_batch_go_p07ik0lo/project/../outside/evil.go']。内部键：
['/tmp/probe_batch_go_p07ik0lo/project/pkg/handler.go']
```

INCR-MISMATCH-051 — `src--languages--go-py--function_spans`

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/languages/go-py/function_spans.py`。
- Reasoner 结果：`logic_verification_results/src/languages/go-py/function_spans.json`。
- 详细报告：`src--languages--go-py--function_spans.md`。
- Probe：`probe_src--languages--go-py--function_spans.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/go-py/function_spans.py

function_spans(proj_dir: str, filepath: str) -> list | None

前置条件：
- proj_dir 是一个非空字符串，引用一个项目目录。
- filepath 是一个字符串，标识该项目内的一个 Go 源文件。

后置条件：
- 对于在 filepath 处找到的每一个顶层函数定义，返回一个包含 (function_name, start_line, end_line) 元组的列表。
- start_line 和 end_line 为 0 基索引，且包含在内。
- 返回的列表按照函数在文件中出现的顺序排列。
- 当 codegraph 后端不可用或未索引 filepath 处的文件时，返回 None。
[SPEC]
```


- SPEC 声明:

- 对于在 `filepath` 处找到的每一个顶层函数定义, 返回一个包含 (`function_name`, `start_line`, `end_line`) 元组的列表。

- `start_line` 和 `end_line` 为 0 基索引, 且包含在内。
- 返回的列表按照函数在文件中出现的顺序排列。
- 当 `codegraph` 后端不可用或未索引 `filepath` 处的文件时, 返回 `None`。

- 推导出的实际行为:

自然语言描述: 如果 `CodeGraphExtractor.from_proj_dir(proj_dir)` 返回一个假值 (例如, 由于无法加载 `codegraph` 而返回 `None`), 该函数立即返回 `None`。否则, 它调用所获得实例的 `get_function_spans("go", filepath)` 并返回其结果。当 `codegraph` 未索引给定文件或该文件不包含任何函数定义时, 或者当 `get_function_spans` 的前置条件不满足时 (例如, `filepath` 不是绝对路径, 或者语言键未被识别), 该结果为 `None`。如果内部前置条件成立, 返回值为一个非 `None` 的 3 元组列表 (`name`, `start_idx`, `end_idx`), 其中 `name` 是类限定的函数/方法标识符, `start_idx` 和 `end_idx` 是从后端转换而来的 0 基包含行号, 并且列表按 `start_idx` 升序排序。不会有意抛出异常; 所有错误情况都通过 `None` 返回值来表示。形式化描述: 设 `cg = CodeGraphExtractor.from_proj_dir(proj_dir)`。如果 `not cg`: 返回 `None`。否则: `result = cg.get_function_spans("go", filepath)`。最终返回值满足: (`result = None`) ((语言键 "go" 受支持 `filepath` 是项目根目录内的绝对路径) (`result` 是一个列表 `L` `i`: `L[i] = (n_i, s_i, e_i)` `n_i` 字符串 `s_i`, `e_i` 0 `s_i` `e_i` (`j < i`: `s_j` `s_i`) 每个 `n_i` 是 `filepath` 所指文件中类限定的函数/方法名称))。如果这些附加条件不满足, `result` 可能为 `None` 或具有反映内部实现的未指定结构。

- 代码证据:

```
第 8 行: return cg.get_function_spans("go", filepath) if cg else None
```

- 触发条件:

该代码硬编码了语言键 'go', 导致它对于非 Go 文件 (但已被 `codegraph` 索引且包含顶层函数定义的文件) 返回 `None`, 违反了要求为任何此类文件返回定义列表的规范。

Bug validator

- 触发摘要: `function_spans` 在调用 `get_function_spans` 时硬编码了语言键 'go', 导致对于已由 `codegraph` 索引且包含函数定义的非 Go 文件返回 `None`。
- Probe 标准输出:

```
CONFIRMED — actual: None | expected: [('my_func', 0, 5)]
硬编码的 'go' 语言键导致 get_function_spans 遗漏了 Python 语言节点, 返回 None 而非已索引的函数范围。
```

src/languages/javascript-py

INCR-MISMATCH-052 — src--languages--javascript-py--function_spans

- 人工审计: 推理误判。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/javascript-py/function_spans.py`。
- Reasoner 结果: `logic_verification_results/src/languages/javascript-py/function_spans.json`。
- 详细报告: `src--languages--javascript-py--function_spans.md`。
- Probe: `probe_src--languages--javascript-py--function_spans.py`。

完整生成 SPEC

[SPEC]
单元: `src/languages/javascript-py/function_spans.py`

```
function_spans(proj_dir: str, filepath: str) -> list[tuple] | None
```

前置条件:

- proj_dir 是一个指向现有项目目录的路径。
- filepath 是该项目中一个 JavaScript 源文件的路径。

后置条件:

- 当项目的代码图后端不可用, 或者后端存在但未索引给定文件时, 返回 None。
- 否则返回一个由 (name, start_idx, end_idx) 元组组成的列表, 每个元组对应文件中定义的一个函数。
- start_idx 和 end_idx 是从 0 开始的闭区间行号。
- 列表按照出现顺序排序 (start_idx 升序)。
- 当文件中没有定义任何函数时, 列表为空。

[SPEC]

Reasoner 差异

- SPEC 声明:

- 当项目的代码图后端不可用, 或者后端存在但未索引给定文件时, 返回 None。
- 否则返回一个由 (name, start_idx, end_idx) 元组组成的列表, 每个元组对应文件中定义的一个函数。
- start_idx 和 end_idx 是从 0 开始的闭区间行号。
- 列表按照出现顺序排序 (start_idx 升序)。
- 当文件中没有定义任何函数时, 列表为空。

- 推导的实际行为:

如果 CodeGraphExtractor.from_proj_dir(proj_dir) 返回 None, 则 function_spans 返回 None。否则, 令 cg 为该返回的 CodeGraphExtractor; 如果 cg.get_function_spans("javascript", filepath) 返回 None, 则 function_spans 返回 None; 否则, 它返回包含 (name, start_idx, end_idx) 元组的列表, 且 start_idx 和 end_idx 是从 0 开始的闭区间行号。形式化表述: 令 $R = \text{function_spans}(\text{proj_dir}, \text{filepath})$, $C = \text{CodeGraphExtractor.from_proj_dir}(\text{proj_dir})$ 。则 $(C = \text{None} \wedge R = \text{None}) \vee ((C \neq \text{None} \wedge C.\text{get_function_spans}(\text{"javascript"}, \text{filepath}) = \text{None}) \wedge R = \text{None}) \vee ((C \neq \text{None} \wedge C.\text{get_function_spans}(\text{"javascript"}, \text{filepath}) \neq \text{None}) \wedge R = C.\text{get_function_spans}(\text{"javascript"}, \text{filepath}))$ 。

- 代码证据:

```
第 8 行: return cg.get_function_spans("javascript", filepath) if cg else None
```

- 触发条件:

规约要求返回的列表按 start_idx 升序排序。代码只是直接返回来自 cg.get_function_spans 的原始列表, 并未进行排序。get_function_spans 的后置条件并不保证顺序, 因此存在这样的有效输入: 代码图后端返回了一个无序列表, 这将导致违反规约。

Bug validator

- 触发摘要: function_spans 直接返回来自 cg.get_function_spans 的原始列表, 未按 start_idx 排序; 当后端返回无序的 spans 时, 规约中关于按 start_idx 升序的保证就会被破坏。
- Probe 标准输出:

```
CONFIRMED — 实际: [('func_c', 40, 52), ('func_a', 5, 18), ('func_b', 22, 35)] | 预期: [('func_a', 5, 18), ('func_b', 22, 35), ('func_c', 40, 52)]
```

src/languages/rust-py

INCR-MISMATCH-053 — src--languages--rust-py--batch_extract

- 人工审计: 推理误判。

- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/languages/rust-py/batch_extract.py`。
- Reasoner 结果: `logic_verification_results/src/languages/rust-py/batch_extract.json`。
- 详细报告: `src--languages--rust-py--batch_extract.md`。
- Probe: `probe_src--languages--rust-py--batch_extract.py`。

完整生成 SPEC

```
[SPEC]
单元: src/languages/rust-py/batch_extract.py

batch_extract(proj_dir) -> dict

前置条件:
- proj_dir 是一个字符串, 表示项目目录的文件系统路径

后置条件:
- 返回一个字典, 其中每个键是项目目录内或目录下的 Rust 源文件的绝对文件系统路径 (str)
- 每个值是一个非空的 (str, str) 元组列表: 第一个元素是该文件中声明的函数名, 第二个元素是函数体的完整源文本
- 包含 N 个已检测函数的源文件会在其值列表中产生 N 个条目
- 当给定项目没有可用的 Rust codegraph 后端时, 返回空字典
[SPEC]
```

Reasoner 差异

- SPEC 声称:

- 返回一个字典, 其中每个键是项目目录内或目录下的 Rust 源文件的绝对文件系统路径 (str)
- 每个值是一个非空的 (str, str) 元组列表: 第一个元素是该文件中声明的函数名, 第二个元素是函数体的完整源文本
- 包含 N 个已检测函数的源文件会在其值列表中产生 N 个条目
- 当给定项目没有可用的 Rust codegraph 后端时, 返回空字典

- 推导的实际行为:

该函数返回一个字典, 将项目目录 `proj_dir` 内 Rust 源文件的绝对文件路径映射到它们包含的函数定义列表, 列表元素为 `(function_name: str, function_body: str)` 元组。如果 `CodeGraphExtractor.from_proj_dir(proj_dir)` 失败 (返回 `None`), 函数返回空字典 `{}`。即使提取器初始化成功, 如果没有找到 Rust 源文件或所有此类文件都不可读, 字典也可能为空。形式上: `result = batch_extract(proj_dir) result dict ((CodeGraphExtractor.from_proj_dir(proj_dir) = None result = {}) (CodeGraphExtractor.from_proj_dir(proj_dir) None (k keys(result), k 是 proj_dir 中一个可读 Rust 文件的绝对路径 result[k] 是该文件中函数的 (name, body) 对构成的列表 (对于项目中的每个 Rust 源文件 f, 如果 f 可读, 则 result 中存在一个键为 abs(f)、值为相应列表的条目, 否则 f 被省略))))`。

- 代码证据:

```
第 4 行: return cg.get_functions_by_file("rust", proj_dir) if cg else {}
```

- 触发条件:

规范要求返回字典中的每个值都是非空的 (函数名, 函数体) 元组列表。代码直接返回 `get_functions_by_file` 的结果, 这可能会包含将可读文件映射到空列表的条目 (当未检测到函数时), 从而违反了非空要求。

Bug validator

- 触发摘要: `batch_extract` 直接将 `get_functions_by_file` 返回的空列表值传递出去而没有进行过滤, 违反了规范中每个值必须为非空元组列表的要求。
- Probe 标准输出:

CONFIRMED — `batch_extract` 未过滤空列表值。文件 `'/fake/proj/src/empty_mod.rs'` 映射为 `[]`, 但规范要求每个

INCR-MISMATCH-054 — `src--languages--rust-py--function_spans`

- 人工审计：**SPEC** 错误。
- 验证器：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：`src/languages/rust-py/function_spans.py`。
- Reasoner 结果：`logic_verification_results/src/languages/rust-py/function_spans.json`。
- 详细报告：`src--languages--rust-py--function_spans.md`。
- 探针：`probe_src--languages--rust-py--function_spans.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/rust.py

function_spans(proj_dir: str, filepath: str) -> list[tuple[str, int, int]] | None

前置条件：
- proj_dir 是指向项目目录的文件系统路径
- filepath 是项目中一个 Rust 源文件的路径

后置条件：
- 如果 codegraph 可用且索引了 filepath：返回一个 (function_name, start_line, end_line) 元组的列表，
  每个对应于文件中声明的顶层函数。每个 start_line 和 end_line 是 0 基、包含的行号，界定函数的源代码跨度。
- 如果 filepath 不包含任何顶层函数声明：返回空列表。
- 如果 codegraph 不可用或未索引 filepath：返回 None。None 返回向调用方表明应回退到基于正则表达式的提取。
[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 如果 codegraph 可用且索引了 filepath：返回一个 (function_name, start_line, end_line) 元组的列表，每个对应于文件中声明的顶层函数。每个 start_line 和 end_line 是 0 基、包含的行号，界定函数的源代码跨度。

- 如果 filepath 不包含任何顶层函数声明：返回空列表。

- 如果 codegraph 不可用或未索引 filepath：返回 None。None 返回向调用方表明应回退到基于正则表达式的提取。

- 推导的实际行为：

如果调用 `CodeGraphExtractor.from_proj_dir(proj_dir)` 抛出异常，则 `function_spans` 抛出该异常。否则，设 `cg` 为返回值。如果 `cg` 是 `None`，则函数返回 `None`。如果 `cg` 是 `CodeGraphExtractor` 实例，则在评估 `cg.get_function_spans('rust', filepath)` 时：如果该调用引发异常，`function_spans` 引发该异常；否则函数返回结果，该结果可能是 `None`（当后端不识别语言键 `'rust'` 或数据库中没有 `filepath` 的条目时）或一个 `(name, start_idx, end_idx)` 元组的列表，每个元组对应于 `filepath` 中找到的每个函数和方法定义，具有 0 基包含的行索引，按 `start_idx` 升序排列。该函数不会修改任何外部可观察状态，除了在 `from_proj_dir` 期间初始化的内部状态以及对 `codegraph` 后端的只读访问。

- 代码证据：

第 8 行：`return cg.get_function_spans("rust", filepath) if cg else None`

- 触发条件：

代码返回了来自 `cg.get_function_spans` 的未过滤列表，该列表包含所有函数和方法定义（如文档所述）。规范要求仅顶层函数声明；`impl` 块内的方法定义必须被排除。此输入同时包含一个顶层函数和一个方法，导致代码产生的输出包含了方法，违反了要求。

- 触发器摘要： `function_spans` 委托给 `get_function_spans`，后者返回 `'function'` 和 `'method'` 两种类型，但规范要求仅顶层函数声明； `impl` 块内的方法未经过滤而泄露。
- 探针标准输出：

已确认 – `function_spans` 返回了方法 `'Foo::bar'`（在 `impl` 块内）以及顶层函数 `'top_level'`。规范要求仅顶层函数。完整结果：`[('top_level', 0, 2), ('Foo::bar', 4, 6)]`

src/languages/typescript-py

INCR-MISMATCH-055 — src--languages--typescript-py--batch_extract

- 人工审计： **SPEC 错误**。
- 验证器： **confirmed**；尝试次数： **1**。
- 原始函数与 SPEC： `src/languages/typescript-py/batch_extract.py`。
- 推理器结果： `logic_verification_results/src/languages/typescript-py/batch_extract.json`。
- 详细报告： `src--languages--typescript-py--batch_extract.md`。
- 探针： `probe_src--languages--typescript-py--batch_extract.py`。

完整生成 SPEC

```
[SPEC]
单元：src/languages/typescript.py

batch_extract(proj_dir: str) -> dict[str, list[tuple[str, str]]]

前置条件：
- proj_dir 是一个指向项目目录的文件系统路径

后置条件：
- 若 codegraph 可用：返回一个字典，其键为 proj_dir 内 TypeScript 源文件的绝对文件路径，
  其值为列表，列表中的元素为对应文件中每个顶层函数声明的（函数名，函数体）元组。
- 每个函数名是函数声明的标识符。
- 每个函数体是函数定义的完整源代码文本。
- 若 proj_dir 中不包含任何带有顶层函数的 TypeScript 文件：
  返回空字典。
- 若 codegraph 不可用：返回空字典 {}。
[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 若 codegraph 可用：返回一个字典，其键为 proj_dir 内 TypeScript 源文件的绝对文件路径，其值为列表，列表中的元素为对应文件中每个顶层函数声明的（函数名，函数体）元组。
- 每个函数名是函数声明的标识符。
- 每个函数体是函数定义的完整源代码文本。
- 若 proj_dir 中不包含任何带有顶层函数的 TypeScript 文件：返回空字典。
- 若 codegraph 不可用：返回空字典 {}。

- 推导的实际行为：

该函数返回一个字典。令 `cg = CodeGraphExtractor.from_proj_dir(proj_dir)`。如果 `cg` 为 `None`，则结果为空字典 `{}`。否则结果为 `cg.get_functions_by_file("typescript", proj_dir)`。形式上：结果 `dict`。（`result = {}`（`cg None result = cg.get_functions_by_file("typescript", proj_dir)`））。结果的值（若存在）满足：对于每一个键 `k`（`str`），`result[k]` 是一个元组列表；每个元组（`name: str`，`body: str`）代表一个 TypeScript 函数。结果可能因为下列任一原因而为空： `CodeGraphExtractor` 初始化失败、语言 `"typescript"` 未被识别、或者在 `proj_dir` 下未找到可提取函数的 TypeScript 源文件。

- 代码证据:

```
第 4 行: return cg.get_functions_by_file("typescript", proj_dir) if cg else {}
```

- 触发条件:

规范 (B) 声明值必须仅包含顶层函数定义。而实现委托给了 ``get_functions_by_file`` , 其文档化的后置条件并未将结果限制为顶层函数。因此, 若项目目录下的某个文件包含嵌套函数, 其输出将包含这些嵌套声明, 从而无法满足要求 B。

Bug validator

- 触发摘要: `batch_extract` 委托给 `get_functions_by_file`, 后者返回所有函数, 包括嵌套函数, 但规范要求仅包含顶层函数定义。
- 探针标准输出:

```
CONFIRMED - batch_extract 返回了嵌套函数 (违反规范)。  
实际名称: ['exportData', 'formatItem']  
预期名称: ['exportData']  
嵌套函数 'formatItem' 根据规范声明不应出现。
```

src/llm_client-py

INCR-MISMATCH-056 — src--llm_client-py--_inject_targets

- 人工审计: **推理误判**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/llm_client-py/_inject_targets.py`。
- Reasoner 结果: `logic_verification_results/src/llm_client-py/_inject_targets.json`。
- 详细报告: `src--llm_client-py--_inject_targets.md`。
- Probe: `probe_src--llm_client-py--_inject_targets.py`。

完整生成 SPEC

```
[SPEC]  
单元: src/llm_client.py  
_inject_targets() -> list[str]  
前置条件:  
- 无 (不需要参数)  
后置条件:  
- 返回一个非空字符串列表, 表示用于将用户ID元数据注入请求体的配置目标  
- 返回值由环境变量 INJECT_HOST 决定: 若设置了该变量, 则将其值解析为逗号分隔列表, 每一段去除首尾空白, 并丢弃任何去除空白后为空的段  
- 当 INJECT_HOST 未设置或为空时, 返回空列表  
- 返回列表中元素的相对顺序与 INJECT_HOST 中对应各段的顺序一致  
[SPEC]
```

Reasoner 差异

- SPEC 声明:

```
- 返回一个非空字符串列表, 表示用于将用户ID元数据注入请求体的配置目标  
- 返回值由环境变量 INJECT_HOST 决定: 若设置了该变量, 则将其值解析为逗号分隔列表, 每一段去除首尾空白, 并丢弃任何去除空白后为空的段  
- 当 INJECT_HOST 未设置或为空时, 返回空列表  
- 返回列表中元素的相对顺序与 INJECT_HOST 中对应各段的顺序一致
```


- 推导的实际行为：

自然语言：函数 `_inject_targets()` 访问全局或模块级对象 ``settings.inject.hosts``。如果该属性链存在，则返回一个非空字符串列表。``settings.inject.hosts`` 的值首先经过表达式 ``(settings.inject.hosts or '')`` 的处理，因此如果它为 `None`、空字符串或任何其他假值，则用空字符串替代。得到的结果字符串按逗号分割，每个部分去除首尾空白，任何去除空白后变为空的部分都被丢弃。返回的列表包含这些去空白后的非空部分。如果属性链不存在（即 ``settings``、``settings.inject`` 或 ``settings.inject.hosts`` 未定义），则会引发 `AttributeError`。形式逻辑：设 `S = settings.inject.hosts`，当 `settings`、`settings.inject` 和 `settings.inject.hosts` 均存在；否则 `S` 未定义。若 `S` 已定义，则 `result = COMPREHENSION{ s.strip() | for each s in (S or '').split(',') if s.strip() != '' }`。若 `S` 未定义，则函数引发 `AttributeError`。在成功的情况下，对所有 `e ∈ result`，`e` 是字符串且 `len(e) > 0`。

- 代码证据：

```
第 2 行: return [s.strip() for s in (settings.inject.hosts or "").split(",") if s.strip()]
```

- 触发条件：

该函数从 `settings.inject.hosts` 读取主机列表，但规范要求从环境变量 `INJECT_HOST` 读取。当 `INJECT_HOST` 已设置但 `settings.inject.hosts` 不存在或包含不同的数据时，该函数要么引发错误，要么返回不正确的列表，违反了规范。

Bug validator

- 触发摘要： `INJECT_HOST` 环境变量被设置为逗号分隔的主机，但 `settings.inject.hosts` 为空；函数返回 `[]` 而不是解析出的环境变量值
- 探针标准输出：

```
CONFIRMED — 从 settings.inject.hosts 读取而非 INJECT_HOST 环境变量 | 实际: [] | 期望: ['alpha', 'beta', 'gamma']
```

INCR-MISMATCH-057 — `src--llm_client-py--_messages_to_anthropic`

- 人工审计：**SPEC 错误**。
- Validator：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：`src/llm_client-py/_messages_to_anthropic.py`。
- Reasoner 结果：`logic_verification_results/src/llm_client-py/_messages_to_anthropic.json`。
- 详细报告：`src--llm_client-py--_messages_to_anthropic.md`。
- 探测：`probe_src--llm_client-py--_messages_to_anthropic.py`。

完整生成 SPEC

```
[SPEC]
单元: src/llm_client.py

_messages_to_anthropic(messages) -> (str, list)

前置条件:
- messages 是一个字典列表，每个字典可选地包含键 "role" 和 "content"。

后置条件:
- 返回一个对 (system_text, anthropic_messages)，其中:
  - anthropic_messages 是一个字典列表，每个字典恰好包含键 "role" 和 "content"，包含所有角色为 "user" 或 "assistant" 的输入消息，按原始相对顺序排列。
  - 对于 "content" 值为字符串的输入消息，它原样传递。当 "content" 是一个字典列表（内容块）时，它被替换为一个单一字符串，该字符串由列表中每个字典的 "text" 值通过换行符连接而成。列表中缺少 "text" 键的字典在该位置贡献一个空字符串。
  - 当没有输入消息的角色为 "system" 时，system_text 为空字符串。当恰好存在一条 system 角色的消息时，system_text 就是其（可能被展平后的）内容字符串，不做任何空白字符修剪（无前后空格去除）。当存在多条 system 角色的消息时，system_text 是将它们（可能展平后的）内容字符串按顺序连接，中间用 "\n\n" 连接，然后对整个连接结果去除前后空白字符。
```

- 角色既不是 "system"、"user" 也不是 "assistant" 的消息将从两个输出中排除。
[SPEC]

Reasoner 差异

- SPEC 声称：

- 返回一个对 (system_text, anthropic_messages)，其中：
 - anthropic_messages 是一个字典列表，每个字典恰好包含键 "role" 和 "content"，包含所有角色为 "user" 或 "assistant" 的输入消息，按原始相对顺序排列。
 - 对于 "content" 值为字符串的输入消息，它原样传递。当 "content" 是一个字典列表（内容块）时，它被替换为一个单一字符串，该字符串由列表中每个字典的 "text" 值通过换行符连接而成。列表中缺少 "text" 键的字典在该位置贡献一个空字符串。
 - 当没有输入消息的角色为 "system" 时，system_text 为空字符串。当恰好存在一条 system 角色的消息时，system_text 就是其（可能被展平后的）内容字符串，不做任何空白字符修剪（无前后空格去除）。当存在多条 system 角色的消息时，system_text 是将它们（可能展平后的）内容字符串按顺序连接，中间用 "\n\n" 连接，然后对整个连接结果去除前后空白字符。
 - 角色既不是 "system"、"user" 也不是 "assistant" 的消息将从两个输出中排除。

- 推导的实际行为：

该函数返回一个元组 (system_text, out)。令 processed_content(m) 定义为：如果 m.get('content', '') 是字符串，则使用它；否则（content 是一个列表），将其展平为一个字符串，通过将 content 中每个字典的 c.get('text', '') 用换行符连接而成。system_text 从所有满足 m.get('role') == 'system' 的输入消息 m 构建：如果没有这样的消息，system_text 为 ''；如果恰好一条，system_text 为 processed_content(m)（不进行额外的空白修剪）；如果多于一条，system_text 是将所有那些消息的 processed_content 按顺序用分隔符 '\n\n\n' 连接，再去除前后空白字符后的结果。out 是一个字典列表 {'role': m['role'], 'content': processed_content(m)}，保留了输入中那些角色为 'user' 或 'assistant' 的消息的相对顺序。该函数无副作用。

- 代码证据：

```
第 13 行： system_text = (system_text + "\n\n" + content).strip() if system_text else content
```

- 触发条件：

当存在多条 system 消息时，代码在每次连接后都去除空白字符，导致内部空白丢失。规范要求先用 '\n\n' 连接所有内容，然后仅对最终结果进行修剪。在输入 [{'role': 'system', 'content': ' a '}, {'role': 'system', 'content': ' b '}] 下，代码产生 'a \n\n b'，而规范要求 'a \n\n b '，两者不同。

Bug validator

- 触发摘要：当存在 3 条或更多 system 消息时，代码在第 92 行迭代执行的 .strip() 会过早地移除中间消息内容的后置空白，而规范要求首先连接所有内容，然后仅对最终结果去除空白字符。
- Probe 标准输出：

```
CONFIRMED — 漏洞已重现。失败项：  
[三个 system (关键测试)]  
  实际结果： 'a\n\n b\n\nc'  
  预期结果： 'a\n\n b \n\nc'  
[四个 system]  
  实际结果： 'x \n\n y\n\n z\n\nw'  
  预期结果： 'x \n\n y\n\n z \n\nw'
```

INCR-MISMATCH-058 — src--llm_client-py--_metadata_body

- 人工审计：推理误判。
- Validator: **confirmed**；尝试次数：1。
- 原始函数与 SPEC: src/llm_client-py/_metadata_body.py。
- Reasoner 结果: logic_verification_results/src/llm_client-py/_metadata_body.json。

- 详细报告：`src--llm_client-py--_metadata_body.md`。
- Probe: `probe_src--llm_client-py--_metadata_body.py`。

完整生成 SPEC

```
[SPEC]
单元：src/llm_client.py

_metadata_body() -> dict

前置条件：
- 无（不接受任何参数）

后置条件：
- 返回一个字典，其顶层恰好有一个键 "metadata"，其值是一个嵌套字典，其中包含一个 "user_id" 键
- 与 "user_id" 关联的值是一个稳定、一致的字符串，在同一安装环境的多次调用中标识当前环境
[SPEC]
```

Reasoner 差异

- SPEC 声明：

```
- 返回一个字典，其顶层恰好有一个键 "metadata"，其值是一个嵌套字典，其中包含一个 "user_id" 键
- 与 "user_id" 关联的值是一个稳定、一致的字符串，在同一安装环境的多次调用中标识当前环境
```

- 推导的实际行为：

```
返回：一个字典对象 `result`。后置条件：`isinstance(result, dict) and ('metadata' in result) and
(len(result) == 1) and isinstance(result['metadata'], dict) and ('user_id' in
result['metadata']) and (len(result['metadata']) == 1) and isinstance(result['metadata']
['user_id'], str) and (len(result['metadata']['user_id']) > 0)`。自然语言：该函数始终返回一个结构为
`{"metadata": {"user_id": s}}` 的字典，其中 `s` 是一个非空字符串（调用 `_stable_user_id()` 的结果）。
```

- 代码证据：

```
第2行: return {"metadata": {"user_id": _stable_user_id()}}
```

- 触发条件：

```
该函数依赖 `_stable_user_id()`，当 `settings.inject.id` 为真值时，`_stable_user_id()` 返回该值。因为该
设置可能在调用之间发生变化，输出的 `user_id` 并不保证稳定，这与规格说明相冲突。
```

Bug validator

- 触发摘要：在两次调用 `_metadata_body()` 之间改变 `settings.inject.id` 会导致返回的 `user_id` 发生变化，违反了规格说明中的稳定性保证。
- Probe 标准输出：

```
CONFIRMED — 在设置变更后的多次调用中，user_id 发生了变化。
第一次调用 (settings.inject.id=''): user_id='stable-user-or-session-id-xxxxxxx123'
第二次调用 (settings.inject.id='mutated-user-id-abc123'): user_id='mutated-user-id-abc123'
规格要求：user_id 是一个稳定、一致的字符串，在同一安装环境的多次调用中标识当前环境。
缺陷：settings.inject.id 是可变的，因此 _stable_user_id()（由 _metadata_body 调用）可能在调用之间返回
不同的值。
```

- 人工审计：契约待确认。

- 验证器: `confirmed`; 尝试次数: `1`。
- 原始函数与 SPEC: `src/llm_client-py/_retry_create.py`。
- Reasoner 结果: `logic_verification_results/src/llm_client-py/_retry_create.json`。
- 详细报告: `src--llm_client-py--_retry_create.md`。
- 探针: `probe_src--llm_client-py--_retry_create.py`。

完整生成 SPEC

```
[SPEC]
单元: src/llm_client-py/_retry_create.py

_retry_create(client, model, messages) -> (str, dict)

前置条件:
- client 可以为给定模型发送对话消息并返回文本响应
- model 是非空字符串
- messages 是包含 "role" 和 "content" 键的消息字典列表

后置条件:
- 从成功的 LLM 调用返回 (response_text, usage_metadata_dict) 元组
- response_text 是 LLM 返回的文本内容
- usage_metadata_dict 将令牌使用键映射到 LLM 响应中的数字计数, 或者当没有使用数据报告时为空字典
- 当 CLI 后端处于活跃状态时, LLM 交互被委派给外部代理
- 对于直接客户端调用, Anthropic 系列模型使用专用的原生 Anthropic 端点; 所有其他模型使用标准的 chat-completions 端点
- 可恢复的错误 (速率限制、服务器不可用和其他短暂的 HTTP/中间件故障) 会以增加的延迟进行重试, 延迟受每个类别最大重试次数限制
- 不可恢复的错误 (提供商拒绝的格式错误请求) 立即传播而不重试
- 当可恢复错误耗尽重试预算时引发 RuntimeError
[SPEC]
```

Reasoner 差异

- SPEC 声称:

```
- 从成功的 LLM 调用返回 (response_text, usage_metadata_dict) 元组
- response_text 是 LLM 返回的文本内容
- usage_metadata_dict 将令牌使用键映射到 LLM 响应中的数字计数, 或者当没有使用数据报告时为空字典
- 当 CLI 后端处于活跃状态时, LLM 交互被委派给外部代理
- 对于直接客户端调用, Anthropic 系列模型使用专用的原生 Anthropic 端点; 所有其他模型使用标准的 chat-completions 端点
- 可恢复的错误 (速率限制、服务器不可用和其他短暂的 HTTP/中间件故障) 会以增加的延迟进行重试, 延迟受每个类别最大重试次数限制
- 不可恢复的错误 (提供商拒绝的格式错误请求) 立即传播而不重试
- 当可恢复错误耗尽重试预算时引发 RuntimeError
```

- 推导的实际行为:

如果 ``is_cli_backend_enabled()`` 为真, 函数返回 ``run_agent_for_messages(model, messages)`` 的结果, 且不抛出任何在此函数内处理的异常。否则, 函数重复尝试获取给定模型和消息的补全, 针对速率限制 (HTTP 429 或 `RateLimitError`) 以指数退避重试最多 `_MAX_RATE_LIMIT_RETRIES` 次, 针对短暂故障 (HTTP 5xx、其他异常) 以指数退避重试最多 `_MAX_LLM_RETRIES` 次。如果在重试耗尽之前补全成功, 函数返回元组 `(text, usage)`, 其中 `text` 是助手响应内容的字符串, `usage` 是令牌使用详情的字典 (如果没有可用使用情况则为空字典)。如果发生 `BadRequestError` 或状态码为 400 的 `HTTPError`, 函数立即引发该异常。如果速率限制重试耗尽, 引发 `RuntimeError`。如果短暂重试耗尽, 引发 `RuntimeError`。在最后一次允许的重试之后发生的任何其他异常都会被包装为 `RuntimeError`。形式上, 对于 ``_retry_create(client, model, messages)`` 的每次执行 `E`: `E` 要么以返回值 `R` 终止, 要么引发异常 `X`。(E 返回 `R`) `[(is_cli_backend_enabled() R = run_agent_for_messages(model, messages)) (is_cli_backend_enabled() t: str, u: dict . R = (t, u) valid_response(t, u))]`。(E 引发 `X`) `[X {BadRequestError, HTTPError(400), RuntimeError}]`, 其中 `RuntimeError` 表示速率限制或短暂重试耗尽。没有其他结果, 并且函数遵守所描述的重试限制和休眠间隔。

- 代码证据:

第 60 行: `except Exception as exc:`

- 触发条件:

代码捕获所有 `Exception` 实例并将其视为短暂错误进行重试。由无效消息类型引起的 `TypeError` 不是可恢复的短暂错误；规范要求仅对可恢复错误（速率限制、服务器不可用等）进行重试。像格式错误的输入这样的不可恢复错误应立即传播而不重试，但是代码会对它们进行重试并最终引发 `RuntimeError`。

Bug validator

- 触发摘要：传入导致 `client.chat.completions.create()` 内部发生 `TypeError` 的输入会触发捕获所有异常的 `except Exception` 处理器，该处理器将其作为短暂错误重试 5 次而不是立即传播。
- 探针标准输出：

```
WARNING:root:LLM 错误 (TypeError: 无效的消息类型：期望 dict 列表，得到 str)，休眠 7.8 秒（尝试 1）
WARNING:root:LLM 错误 (TypeError: 无效的消息类型：期望 dict 列表，得到 str)，休眠 12.9 秒（尝试 2）
WARNING:root:LLM 错误 (TypeError: 无效的消息类型：期望 dict 列表，得到 str)，休眠 22.5 秒（尝试 3）
WARNING:root:LLM 错误 (TypeError: 无效的消息类型：期望 dict 列表，得到 str)，休眠 41.7 秒（尝试 4）
确认 - TypeError 被重试（5 次，重试预算为 5）而不是立即传播；最终结果为 RuntimeError: LLM 请求在 5 次重试后
失败：无效的消息类型：期望 dict 列表，得到 str
```

INCR-MISMATCH-060 — `src--llm_client-py--_stable_user_id`

- 人工审计：推理误判。
- Validator: **confirmed**；尝试次数：1。
- 原始函数与 SPEC: `src/llm_client-py/_stable_user_id.py`。
- Reasoner 结果: `logic_verification_results/src/llm_client-py/_stable_user_id.json`。
- 详细报告: `src--llm_client-py--_stable_user_id.md`。
- Probe: `probe_src--llm_client-py--_stable_user_id.py`。

完整生成 SPEC

```
[SPEC]
单元: src/llm_client.py

_stable_user_id() -> str

前置条件:
- 无（不接受参数）

后置条件:
- 当 `settings.inject.id` 的值为真值时返回该值
- 当 `settings.inject.id` 为假值（空或 None）时返回预定义的静态默认值 `_DEFAULT_INJECT_USER_ID`
- 在所有情况下返回的字符串均非空
[SPEC]
```

Reasoner 差异

- SPEC 声明：

```
- 当 `settings.inject.id` 的值为真值时返回该值
- 当 `settings.inject.id` 为假值（空或 None）时返回预定义的静态默认值 `_DEFAULT_INJECT_USER_ID`
- 在所有情况下返回的字符串均非空
```

- 推导的实际行为：

该函数如果 ``settings.inject.id`` 为真值（根据 Python 布尔转换）则返回其值，否则返回 ``_DEFAULT_INJECT_USER_ID``。不会修改外部状态。形式化地：设 ``ret`` 为返回值，则 ``ret` = settings.inject.id`` 若 ``bool(settings.inject.id)`` 为真，否则为 ``_DEFAULT_INJECT_USER_ID``，且所有模块级对象保持不变。

- 代码证据：

第 2 行: `return settings.inject.id or _DEFAULT_INJECT_USER_ID`

- 触发条件:

规范指出在所有情况下返回的字符串均非空,这意味着函数必须始终返回一个字符串。然而,当 `settings.inject.id` 为真值但非字符串(例如整数 5)时,代码会原样返回该非字符串值,违反了返回值必须为字符串的要求。

Bug validator

- 触发摘要: 当 `settings.inject.id` 为真值但非字符串(例如整数 5)时, Python 的 `or` 操作符会原样返回非字符串值而非字符串,违反了规范中关于返回值始终为非空字符串的要求。
- 探测标准输出:

CONFIRMED – actual type: int, value: 5 | expected type: str

src/opencode_trace-py

INCR-MISMATCH-061 — src--opencode_trace-py--_opencode_provider_config

- 人工审计: **推理误判**。
- Validator: **confirmed**; 尝试次数: 1。
- 原始函数与 SPEC: `src/opencode_trace-py/_opencode_provider_config.py`。
- Reasoner 结果: `logic_verification_results/src/opencode_trace-py/_opencode_provider_config.json`。
- 详细报告: `src--opencode_trace-py--_opencode_provider_config.md`。
- Probe: `probe_src--opencode_trace-py--_opencode_provider_config.py`。

完整生成 SPEC

```
[SPEC]
单元: src/opencode_trace.py

_opencode_provider_config() -> dict | None

前置条件:
- 全局 settings 对象已加载,其 .llm 属性提供了对 LLM 配置字段 api_key、base_url、name、provider 和 api_style 的访问。

后置条件:
- 当 api_key、base_url、name 或 provider 中任意一项为假值(即空字符串、None 或在布尔上下文中被评估为 False)时,返回 None。
- 当 api_key、base_url、name 和 provider 全部为真值时,返回一个字典,当将其放入 OpenCode 配置中时,定义一个有效的 provider。
- 返回的字典嵌套在 "provider" 键下,该键的值由 provider 设置的值作为键,并包含一个 npm 适配器包名、一个基础 URL、一个模型名称和一个 API 密钥引用。
- npm 适配器根据 api_style 设置进行选择:当 api_style 为 "anthropic" 时使用 Anthropic SDK 包,否则使用 OpenAI 兼容 SDK 包。
- API 密钥以环境变量引用({env:LLM_API_KEY})的形式指定,而非直接使用密钥值。
- 该函数没有副作用:不修改任何全局状态,不执行 I/O,也不修改任何传入的参数。
- 在正常操作下不会引发异常。
[SPEC]
```

Reasoner 差异

- SPEC 声称:

- 当 `api_key`、`base_url`、`name` 或 `provider` 中任意一项为假值(即空字符串、None 或在布尔上下文中被评估为 False)时,返回 None。
- 当 `api_key`、`base_url`、`name` 和 `provider` 全部为真值时,返回一个字典,当将其放入 OpenCode 配置中时,定义

- 一个有效的 `provider`。
 - 返回的字典嵌套在 `"provider"` 键下，该键的值由 `provider` 设置的值作为键，并包含一个 `npm` 适配器包名、一个基础 URL、一个模型名称和一个 API 密钥引用。
 - `npm` 适配器根据 `api_style` 设置进行选择：当 `api_style` 为 `"anthropic"` 时使用 `Anthropic SDK` 包，否则使用 `OpenAI 兼容 SDK` 包。
 - API 密钥以环境变量引用（`{env:LLM_API_KEY}`）的形式指定，而非直接使用密钥值。
 - 该函数没有副作用：不修改任何全局状态，不执行 I/O，也不修改任何传入的参数。
 - 在正常操作下不会引发异常。

- 推导的实际行为：

函数 `_opencode_provider_config()` 不会修改任何全局状态（例如 `settings` 对象保持不变）。其返回值按如下方式确定：如果 `settings.llm.api_key`、`settings.llm.base_url`、`settings.llm.name` 或 `settings.llm.provider` 中任意一个为假值（`None`、空字符串等），则函数返回 `None`。否则，返回一个恰好包含一个顶层键 `'provider'` 的字典，其值是一个恰好包含一个键 `settings.llm.provider` 的字典。该内部字典拥有键 `'npm'`、`'options'` 和 `'models'`。`'npm'` 的值为 `'@ai-sdk/anthropic'`（如果 `settings.llm.api_style` 等于字符串 `'anthropic'`），否则为 `'@ai-sdk/openai-compatible'`。`'options'` 的值是一个包含 `'baseUrl'`（设置为 `settings.llm.base_url`）和 `'apiKey'`（设置为字符串字面量 `'{env:LLM_API_KEY}'`）的字典。`'models'` 的值是一个字典，其唯一键为 `settings.llm.name`，对应值为空字典 `{}`。形式化表示为：

```
let llm = settings.llm in
((llm.api_key llm.base_url llm.name llm.provider) result = None)
((llm.api_key llm.base_url llm.name llm.provider)
  result = { 'provider': { llm.provider: {
    'npm': '@ai-sdk/anthropic' if llm.api_style = 'anthropic' else '@ai-sdk/openai-compatible',
    'options': { 'baseUrl': llm.base_url, 'apiKey': '{env:LLM_API_KEY}' },
    'models': { llm.name: {} }
  } } })
```

- 代码证据：

```
第 16 行: llm = settings.llm
第 17 行: if not (llm.api_key and llm.base_url and llm.name and llm.provider):
```

- 触发条件：

代码试图在一个可能为 `None` 的值（`settings.llm`）上访问属性，从而引发 `AttributeError`，而不是按照规范要求所需字段缺失/为假值时返回 `None`。这违反了规范中关于不引发异常且在此类字段缺失/假值时返回 `None` 的要求。

Bug validator

- 触发摘要：当 `settings.llm` 为 `None` 时，函数解引用 `None.api_key`，引发 `AttributeError`，而不是按照规范要求返回 `None`。
- Probe 标准输出：

```
CONFIRMED — AttributeError: 'NoneType' object has no attribute 'api_key'
Expected: None (当字段缺失时返回 None)
Actual: AttributeError('NoneType' object has no attribute 'api_key')
```

INCR-MISMATCH-062 — `src--opencode_trace-py--_start_opencode_process`

- 人工审计：契约待确认。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/opencode_trace-py/_start_opencode_process.py`。
- Reasoner 结果：`logic_verification_results/src/opencode_trace-py/_start_opencode_process.json`。
- 详细报告：`src--opencode_trace-py--_start_opencode_process.md`。
- Probe：`probe_src--opencode_trace-py--_start_opencode_process.py`。

[SPEC]

单元：src/opcode_trace.py

```
_start_opcode_process(proj_dir, work_dir, event_id, command, trace_log_path) ->
(subprocess.Popen, threading.Thread, threading.Thread | None)
```

前置条件：

- proj_dir 是文件系统上一个已存在目录的路径
- work_dir 是文件系统上一个已存在目录的路径
- event_id 是一个非空、唯一标识本次追踪事件的字符串
- command 是一个 AgentCommand 或一个非空的字符串列表，构成一个合法的 CLI 调用
- trace_log_path 是位于 work_dir 下的文件系统路径，其父目录已存在

后置条件：

- 将 command 作为子进程启动，其工作目录为 proj_dir，其环境变量由 work_dir 和 event_id 派生，其 stdout 和 stderr 合并到单个管道中以供捕获
- 当且仅当 command 携带非 None 的 stdin 文本时，子进程通过已连接的 stdin 管道接收输入；否则 stdin 不与子进程连接
- 子进程的文本流编码为 UTF-8，解码错误时使用替换处理，保证在读取输出时不会出现 UnicodeDecodeError
- 启动一个后台守护线程，将子进程的合并输出实时复制到 trace_log_path，确保子进程写入的每一个字节都被记录
- 如果 command 携带非 None 的 stdin 文本，则启动一个后台守护线程，将相应文本写入子进程的 stdin 管道然后关闭；如果 command 未携带 stdin 文本，则不启动 stdin 写入线程
- 返回一个元组 (process_handle, log_thread, stdin_thread)，其中 log_thread 总是一个已启动的 threading.Thread，而 stdin_thread 要么是一个已启动的 threading.Thread，要么是 None
- 所有启动的线程均为守护线程：它们不会阻止调用进程退出
- 尚未对子进程进行等待；其退出码尚不可用

[SPEC]

Reasoner 差异

- SPEC 声明：

- 将 command 作为子进程启动，其工作目录为 proj_dir，其环境变量由 work_dir 和 event_id 派生，其 stdout 和 stderr 合并到单个管道中以供捕获
- 当且仅当 command 携带非 None 的 stdin 文本时，子进程通过已连接的 stdin 管道接收输入；否则 stdin 不与子进程连接
- 子进程的文本流编码为 UTF-8，解码错误时使用替换处理，保证在读取输出时不会出现 UnicodeDecodeError
- 启动一个后台守护线程，将子进程的合并输出实时复制到 trace_log_path，确保子进程写入的每一个字节都被记录
- 如果 command 携带非 None 的 stdin 文本，则启动一个后台守护线程，将相应文本写入子进程的 stdin 管道然后关闭；如果 command 未携带 stdin 文本，则不启动 stdin 写入线程
- 返回一个元组 (process_handle, log_thread, stdin_thread)，其中 log_thread 总是一个已启动的 threading.Thread，而 stdin_thread 要么是一个已启动的 threading.Thread，要么是 None
- 所有启动的线程均为守护线程：它们不会阻止调用进程退出
- 尚未对子进程进行等待；其退出码尚不可用

- 推导实际行为：

成功完成后，函数返回一个 3 元组 `(proc, log\_thread, stdin\_thread)`，其中：

- `proc` 是一个 `subprocess.Popen` 实例，代表已启动的子进程，具有以下属性：
 - 命令参数：`command\_argv(command)`。
 - 工作目录：`proj\_dir`。
 - 环境变量：`\_opcode\_env(work\_dir, event\_id)`（是当前进程环境变量的超集）。
 - 标准输入：如果 `command\_stdin(command)` 不为 `None`，则使用管道；否则为 `None`。
 - 标准输出：管道。
 - 标准错误：与标准输出合并（`subprocess.STDOUT`）。
 - 文本模式已启用，采用 UTF-8 编码和 `'replace'` 错误处理。
 - 底层的操作系统级子进程已创建，可能正在运行，也可能已经终止；`proc.pid` 已设置。
- `log\_thread` 是一个已启动的 `threading.Thread` 对象。它是守护线程，执行 `\_copy\_opcode\_output(proc.stdout, trace\_log\_path)`，该线程会从子进程的 stdout 读取行，并将它们写入 `trace\_log\_path` 所指向的文件，直到流耗尽（EOF），然后刷新并关闭输出文件。
- `stdin\_thread` 为：
 - 如果 `command\_stdin(command)` 不为 `None`：一个已启动的 `threading.Thread` 对象，是守护线程，执行

`_write_command_stdin(proc.stdin, command_stdin(command))`。该线程会将 stdin 文本写入子进程的 stdin，刷新并关闭管道，此后无法再向 proc.stdin 写入。`

- 如果 `command_stdin(command)` 为 `None`：`None`。

返回之后，调用方不得从 `proc.stdout` 读取（因为由 `log_thread` 消费），如果 `stdin_thread` 不为 `None`，也不得向 `proc.stdin` 写入（因为该线程最终会关闭它）。子进程的输出最终会被记录到 `trace_log_path` 中，前提是守护线程未被进程退出过早终止。

形式化表示（设 `stdin_text = command_stdin(command)`，`args = command_argv(command)`）：

```
...
( result == (proc, log_thread, stdin_thread) )
( proc.pid == None )
( proc.args == args )
( proc.cwd == proj_dir )
( proc.env == _opencode_env(work_dir, event_id) )
( proc.stdout is a readable pipe )
( proc.stderr == subprocess.STDOUT )
( proc.text == True )
( proc.encoding == 'utf-8' )
( proc.errors == 'replace' )
( (stdin_text != None) (proc.stdin is a writable pipe)
  (stdin_thread 是一个 Thread, 其 target=_write_command_stdin,
    args=(proc.stdin, stdin_text), daemon=True, started=True) )
( (stdin_text == None) (proc.stdin == None) (stdin_thread == None) )
( log_thread 是一个 Thread, 其 target=_copy_opencode_output,
  args=(proc.stdout, trace_log_path), daemon=True, started=True )
...
```

- 代码证据：

第 7 行： `stdin=subprocess.PIPE if stdin_text is not None else None,`

- 触发条件：

当 `stdin_text` 为 `None` 时，代码设置 `stdin=None`，这会导致子进程继承父进程的 `stdin` 文件描述符，而不是关闭或断开它。这违反了规范中“否则 `stdin` 不与子进程连接”的要求。

Bug validator

- 触发摘要：当 `command` 没有 `stdin` 文本（`command_stdin` 返回 `None`）时，`subprocess.Popen` 收到 `stdin=None`，导致子进程继承父进程的 `stdin` 文件描述符，而不是按照规范断开它。
- Probe 输出：

CONFIRMED – subprocess.Popen received stdin=None, so the subprocess inherits the parent's stdin file descriptor instead of having stdin disconnected. Spec requires: 'otherwise stdin is not connected to the subprocess'.

src/pipeline_setup.py

INCR-MISMATCH-063 — src--pipeline_setup.py--_deduplicate_phases

- 人工审计：契约待确认。
- Validator： **confirmed**；尝试次数： 1。
- 原始函数与 SPEC： `src/pipeline_setup.py/_deduplicate_phases.py`。
- Reasoner 结果： `logic_verification_results/src/pipeline_setup.py/_deduplicate_phases.json`。
- 详细报告： `src--pipeline_setup.py--_deduplicate_phases.md`。
- 探针： `probe_src--pipeline_setup.py--_deduplicate_phases.py`。

完整生成 SPEC

```
[SPEC]
单元：src/pipeline_setup.py

_deduplicate_phases(phases_dir) -> dict
```

前置条件：

- phases_dir 是一个字符串路径，指向一个包含有效 phases.json 文件的目录。
- phases.json 符合流水线模式：一个 JSON 对象，带有一个 "phases" 数组；每个 phase 有 "phase" (int) 和 "modules" (数组)；每个 module 有 "name" (字符串) 和 "source_files" (字符串数组)。
- 每个 source_files 元素是从项目根目录开始的相对文件路径字符串。
- phases.json 文件可被当前进程读取和写入。

后置条件：

- phases.json 被重写为每个源文件路径最多出现在所有阶段中的一个模块里。
- 对于出现在多个模块中（跨相同或不同阶段）的任何源文件路径，只有首次出现——按阶段号升序，然后在每个阶段内按模块顺序——被保留；所有后续出现都从其各自模块的 source_files 列表中移除。
- 阶段和模块的集合不变：即使某个模块的 source_files 变为空，也不会移除任何阶段或模块。
- 阶段号、模块名称以及 phases.json 中阶段/模块的顺序被保留。
- 返回一个字典，键 "modified_modules" 映射到一个字典列表，每个字典对应至少删除一个源文件的模块。每个模块字典包含："phase" (int - 阶段号), "module" (str - 模块名), "removed_files" (字符串列表 - 被移除的去重文件路径), 以及 "source_files" (字符串列表 - 去重后模块中剩余的源文件)。
- 当所有模块间没有重复源文件时，返回 {"modified_modules": []}。

[SPEC]

Reasoner 差异

- SPEC 声称：

- phases.json 被重写为每个源文件路径最多出现在所有阶段中的一个模块里。
- 对于出现在多个模块中（跨相同或不同阶段）的任何源文件路径，只有首次出现——按阶段号升序，然后在每个阶段内按模块顺序——被保留；所有后续出现都从其各自模块的 source_files 列表中移除。
- 阶段和模块的集合不变：即使某个模块的 source_files 变为空，也不会移除任何阶段或模块。
- 阶段号、模块名称以及 phases.json 中阶段/模块的顺序被保留。
- 返回一个字典，键 "modified_modules" 映射到一个字典列表，每个字典对应至少删除一个源文件的模块。每个模块字典包含："phase" (int - 阶段号), "module" (str - 模块名), "removed_files" (字符串列表 - 被移除的去重文件路径), 以及 "source_files" (字符串列表 - 去重后模块中剩余的源文件)。
- 当所有模块间没有重复源文件时，返回 {"modified_modules": []}。

- 推导的实际行为：

如果函数返回一个值 R 且未引发异常，则以下属性成立：(1) 位于 os.path.join(phases_dir, 'phases.json') 的文件已被覆写为 JSON 对象 data，使得 data['phases'] 是与原始数据具有相同长度和顺序的列表；对于 data['phases'] 中的每个 phase p，p['phase'] 等于原始整数，p['modules'] 是按相同顺序的原始模块列表；对于 p['modules'] 中的每个 module m，m['name'] 保持不变，m['source_files'] 是原始 m['source_files'] 的子序列，仅包含那些不在集合 Seen 中的文件，Seen 是通过按 phase 升序和模块原始顺序迭代构建的，当文件首次遇到时 Seen 增长；因此，每个在原始数据中出现的源文件路径在 data 中恰好出现在一个模块的 source_files 列表中，具体是最初声明它的那个模块（按阶段，然后模块顺序），并且每个模块中保留文件的相对顺序被保留。(2) 没有添加、移除或重新排序任何阶段或模块。(3) R 是一个带有键 'modified_modules' 的字典；R['modified_modules'] 是一个对象列表，每个对象对应一个 source_files 列表发生变化的模块（即某些文件被移除），按相同的遍历顺序排列；每个对象具有：'phase'：整数阶段号，'module'：模块名字符串（如果缺失则为 ''），'removed_files'：移除的文件路径列表，以及 'source_files'：该模块的最终去重列表。所有发生变化的模块都被包含，未变化的模块被省略。(4) 副作用：对于找到的每个重复文件，调用 logging.info() 并附带指示文件路径、阶段和模块的消息。(5) 如果在 'with open(..., "w")' 块之前引发异常，原始文件保持不变且函数不返回。如果在最终写入期间引发异常，文件状态未指定（可能部分写入或被截断）且函数不返回。

- 代码证据：

```
第 32 行： removed_files = [sf for sf in original if sf not in deduped]
```

- 触发条件：

如果模块中原始的 source_files 列表包含该模块中完全移除的文件的重复条目，则 removed_files 可能包含重复的文件路径。规范要求被移除的文件列表去重，即每个不同的文件路径最多出现一次。

Bug validator

- 触发摘要：当模块的 `source_files` 包含一个被完全移除的文件的重复条目时，`removed_files` 保留这些重复项而不是去重。
- Probe 标准输出：

```
CONFIRMED - 模块_b 的 removed_files 包含重复项: actual=['a.py', 'a.py'] | expected (unique)= ['a.py']
```

INCR-MISMATCH-064 — `src--pipeline_setup-py--_phase_plan_complete`

- 人工审计：推理误判。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/pipeline_setup-py/_phase_plan_complete.py`。
- Reasoner 结果：`logic_verification_results/src/pipeline_setup-py/_phase_plan_complete.json`。
- 详细报告：`src--pipeline_setup-py--_phase_plan_complete.md`。
- 探针：`probe_src--pipeline_setup-py--_phase_plan_complete.py`。

完整生成 SPEC

```
[SPEC]
单元：src/pipeline_setup-py/_phase_plan_complete.py

_phase_plan_complete(work_dir) -> bool

前置条件：
- work_dir 是一个指向现有目录的字符串路径。

后置条件：
- 当文件 phases.json 在 work_dir 下存在、是一个常规文件、其内容解析为有效的 JSON 且符合所需模式（由 _phase_plan_schema_errors 确定）时，返回 True。
- 当 phases.json 在 work_dir 下不存在、不是常规文件、未能解析为有效 JSON 或不符合所需模式时，返回 False。
- 对于相同的文件系统状态，返回值是幂等的：使用相同的 work_dir 和相同的文件内容重复调用将产生相同的布尔结果。
[SPEC]
```

Reasoner 差异

- SPEC 声称：

- 当文件 `phases.json` 在 `work_dir` 下存在、是一个常规文件、其内容解析为有效的 JSON 且符合所需模式（由 `_phase_plan_schema_errors` 确定）时，返回 `True`。
- 当 `phases.json` 在 `work_dir` 下不存在、不是常规文件、未能解析为有效 JSON 或不符合所需模式时，返回 `False`。
- 对于相同的文件系统状态，返回值是幂等的：使用相同的 `work_dir` 和相同的文件内容重复调用将产生相同的布尔结果。

- 推导的实际行为：

在调用 `_phase_plan_complete(work_dir)` 完成执行后，没有发生任何副作用，也没有引发任何异常。返回值 `r` 满足：`r` 为 `True` 当且仅当通过 `os.path.join(work_dir, "phases.json")` 获得的文件存在、可读、是有效的 JSON，并且其解码内容完全符合所需模式（即 `_phase_plan_schema_errors` 返回空列表）；否则 `r` 为 `False`。用形式化术语表示：令 `p = os.path.join(work_dir, "phases.json")`。那么 $\text{result} = \text{True} \iff (\text{isfile}(p) \wedge \text{readable}(p) \wedge \text{valid_json}(\text{read}(p)) \wedge \text{schema_conforms}(\text{parse_json}(\text{read}(p))))$ ，这等价于 $\text{result} = \text{True} \iff \text{phase_plan_schema_errors}(p) = []$ 。

- 代码证据：

```
第 4 行： return not _phase_plan_schema_errors(phases_path)
```

- 触发条件：

规范要求 `phases.json` 必须是一个常规文件才能返回 `True`。代码完全委托给 `_phase_plan_schema_errors`，而根据其后置条件，该函数不检查文件是否为常规文件。如果存在一个非常规文件（例如 `FIFO`），该文件可读且包含符合模式的有效 JSON，那么 `_phase_plan_schema_errors` 会返回空列表，导致 `_phase_plan_complete` 返回 `True`，从而违反规范。

Bug validator

- 触发条件摘要：当 `phases.json` 是一个 `FIFO`（非常规文件）且包含符合模式的有效 JSON 时，`_phase_plan_complete` 返回 `True`，而不是规范要求的 `False`。
- 探针标准输出：

CONFIRMED — actual: True | expected: False

INCR-MISMATCH-065 — `src--pipeline_setup-py--_phase_plan_schema_errors`

- 人工审计：实现缺陷候选。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/pipeline_setup-py/_phase_plan_schema_errors.py`。
- Reasoner 结果：`logic_verification_results/src/pipeline_setup-py/_phase_plan_schema_errors.json`。
- 详细报告：`src--pipeline_setup-py--_phase_plan_schema_errors.md`。
- Probe：`probe_src--pipeline_setup-py--_phase_plan_schema_errors.py`。

完整生成 SPEC

```
[SPEC]
单元：src/pipeline_setup-py/_phase_plan_schema_errors.py

_phase_plan_schema_errors(phases_path) -> list[str]

前置条件：
- phases_path 是一个字符串。

后置条件：
- 返回一个人类可读的错误消息字符串列表。

- 当无法读取 phases_path 所指向的文件时（任何操作系统错误，包括文件不存在或权限被拒绝），返回的列表为非空，并且只包含一个描述操作系统错误的元素。

- 当 phases_path 所指向的文件可读，但其内容不是合法的 JSON 时，返回的列表为非空，并且只包含一个描述 JSON 解析错误的元素。

- 当文件可读且其内容是合法的 JSON 时，当且仅当解码后的 JSON 结构满足以下所有要求时，返回的列表为空：
  a) 顶层解码值是一个 JSON 对象（dict）。
  b) 该对象有一个键 "phases"，其值是一个 JSON 数组。
  c) "phases" 数组中的每一个元素都是一个 JSON 对象。
  d) "phases" 中的每一个对象都有一个键 "modules"，其值是一个 JSON 数组。
  e) "modules" 数组中的每一个元素都是一个 JSON 对象。
  f) "modules" 中的每一个对象都有一个键 "source_files"，其值是一个 JSON 数组。
  g) "source_files" 数组中的每一个元素都是一个 JSON 字符串。

- 当解码后的 JSON 违反了从（a）到（g）的任何要求时，返回的列表为非空。列表中的每个元素都使用 JSON-path 记法（方括号中包含从零开始的数组索引）精确描述一个违反项。当一个模块对象具有非空的 "name" 键时，该模块的违反消息会包含该 name 值以供识别。

- 该函数不会修改 phases_path 所指向的文件或任何其他持久状态。
- 无论输入如何，该函数总是在有限时间内返回。

[SPEC]
```

Reasoner 差异

- SPEC 声明：
- 返回一个人类可读的错误消息字符串列表。
- 当无法读取 phases_path 所指向的文件时（任何操作系统错误，包括文件不存在或权限被拒绝），返回的列表为非空，并

且只包含一个描述操作系统错误的元素。

- 当 `phases_path` 所指向的文件可读，但其内容不是合法的 JSON 时，返回的列表为非空，并且只包含一个描述 JSON 解析错误的元素。

- 当文件可读且其内容是合法的 JSON 时，当且仅当解码后的 JSON 结构满足以下所有要求时，返回的列表为空：
 - a) 顶层解码值是一个 JSON 对象 (dict)。
 - b) 该对象有一个键 "phases"，其值是一个 JSON 数组。
 - c) "phases" 数组中的每一个元素都是一个 JSON 对象。
 - d) "phases" 中的每一个对象都有一个键 "modules"，其值是一个 JSON 数组。
 - e) "modules" 数组中的每一个元素都是一个 JSON 对象。
 - f) "modules" 中的每一个对象都有一个键 "source_files"，其值是一个 JSON 数组。
 - g) "source_files" 数组中的每一个元素都是一个 JSON 字符串。

- 当解码后的 JSON 违反了从 (a) 到 (g) 的任何要求时，返回的列表为非空。列表中的每个元素都使用 JSON-path 记法（方括号中包含从零开始的数组索引）精确描述一个违反项。当一个模块对象具有非空的 "name" 键时，该模块的违反消息会包含该 name 值以供识别。

- 该函数不会修改 `phases_path` 所指向的文件或任何其他持久状态。
- 无论输入如何，该函数总是在有限时间内返回。

- 推导的实际行为：

该函数尝试读取并解析 `phases_path` 处的 JSON 文件。如果发生 `OSError`（例如，文件未找到、权限被拒绝），它将返回一个包含单个字符串的列表，该字符串描述了错误（格式为 `'phases.json could not be read: ...'`）。如果文件被读取但包含无效的 JSON，它将返回一个包含单个字符串的列表，该字符串描述了解码错误（格式为 `'phases.json is not valid JSON: ...'`）。否则，文件被成功解析为一个 Python 对象；然后该函数验证其结构，并返回一个人类可读的模式错误字符串列表（可能为空）。验证规则如下：（1）顶层值必须是一个字典，否则将报告错误；（2）该字典必须包含键 `"phases"`，且其值是一个列表，否则将报告错误；（3）该列表的每个元素必须是一个字典；（4）每个 `phase` 字典必须包含键 `"modules"`，且其值是一个列表；（5）`modules` 列表的每个元素必须是一个字典；（6）每个 `module` 字典必须包含键 `"source_files"`；（7）`"source_files"` 的值必须是一个列表，且该列表的每个元素必须是一个字符串。所有发现的错误按遍历顺序追加到结果列表中。如果没有发现错误，则返回的列表为空。该函数不修改任何全局状态或文件系统，并且如果成功打开了文件，它总是会关闭文件。没有未处理的异常传播到调用方。

- 代码证据：

```
第 3 行:     try:
第 4 行:         with open(phases_path, "r") as f:
第 5 行:             data = json.load(f)
第 6 行:     except OSError as exc:
第 7 行:         return [f"phases.json could not be read: {exc}"]
第 8 行:     except json.JSONDecodeError as exc:
第 9 行:         return [f"phases.json is not valid JSON: {exc}"]
```

- 触发条件：

规范要求无论输入如何，函数总是在有限时间内返回，但代码只捕获 `OSError` 和 `JSONDecodeError`。如果文件无法从文本解码（例如，无效的 UTF-8），则会导致 `UnicodeDecodeError`，而该异常未被处理，因此函数引发异常，从不返回列表，从而违反了规范。

Bug validator

- 触发摘要：打开包含无效 UTF-8 字节的文件会导致未处理的 `UnicodeDecodeError`，而不是返回错误列表。
- Probe 标准输出：

CONFIRMED – `_phase_plan_schema_errors` 引发了 `UnicodeDecodeError`，而不是返回一个列表（规范要求“无论输入如何，总是在有限时间内返回”）

INCR-MISMATCH-066 — `src--pipeline_setup-py--_phases_cover_current_sources`

- 人工审计：实现缺陷候选。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/pipeline_setup-py/_phases_cover_current_sources.py`。

- Reasoner 结果: `logic_verification_results/src/pipeline_setup-py/_phases_cover_current_sources.json`。
- 详细报告: `src--pipeline_setup-py--_phases_cover_current_sources.md`。
- Probe: `probe_src--pipeline_setup-py--_phases_cover_current_sources.py`。

完整生成 SPEC

```
[SPEC]
单元：src/pipeline_setup-py/_phases_cover_current_sources.py

_phases_cover_current_sources(phases_json, proj_dir, submodules=None) -> bool

前置条件：
- phases_json 是一个字符串路径
- proj_dir 是一个已存在的目录
- submodules 要么为 None，要么是一个子目录名字符串的可迭代对象

后置条件：
- 当以下所有条件均满足时返回 True：(a) phases_json 是一个可读文件，其内容可解析为有效 JSON；
  (b) 该 JSON 在所有阶段和模块中至少包含一个源文件条目；
  (c) JSON 中列出的每一个源文件路径在 proj_dir 下均可解析为一个已存在的文件；
  (d) 当 submodules 既不是 None 也不是空可迭代对象时，每一个列出的源文件路径都至少位于指定的一个子模块目录下
  (由 _is_under_submodules 判定)；
  (e) 由 submodules 所限定项目目录（当 submodules 为 None 时为整个 proj_dir）下的每一个源文件都出现在该
  JSON 中
- 当 (a)-(e) 中任意一项失败时返回 False
- JSON 中源文件路径里的反斜杠分隔符在路径比较和文件存在性解析时均被视为正斜杠
- 该函数不会创建、修改、删除或重命名任何文件或目录
[SPEC]
```

Reasoner 差异

- SPEC 声明：

```
- 当以下所有条件均满足时返回 True：(a) phases_json 是一个可读文件，其内容可解析为有效 JSON；
  (b) 该 JSON 在所有阶段和模块中至少包含一个源文件条目；
  (c) JSON 中列出的每一个源文件路径在 proj_dir 下均可解析为一个已存在的文件；
  (d) 当 submodules 既不是 None 也不是空可迭代对象时，每一个列出的源文件路径都至少位于指定的一个子模块目录下
  (由 _is_under_submodules 判定)；
  (e) 由 submodules 所限定项目目录（当 submodules 为 None 时为整个 proj_dir）下的每一个源文件都出现在该
  JSON 中
- 当 (a)-(e) 中任意一项失败时返回 False
- JSON 中源文件路径里的反斜杠分隔符在路径比较和文件存在性解析时均被视为正斜杠
- 该函数不会创建、修改、删除或重命名任何文件或目录
```

- 推导实际行为：

```
执行后，当且仅当以下所有条件均满足时，返回值才为 True：
(i) phases_json 处的文件被成功打开且解析为 JSON（无 OSError/ValueError）；
(ii) 解析后的数据产生一个非空的源文件路径集合 listed，其中的每个路径均已将反斜杠转换为正斜杠，
    这些路径是从 phases→modules→source_files 层级中提取的；
(iii) 若 submodules 不为 None，则 listed 中的每一个路径都满足 _is_under_submodules(sf, submodules)
      （即，将其中一个子模块字符串作为路径组成部分包含在路径中）；
(iv) listed 中的每一个路径都对应 proj_dir 下的一个已存在文件（即 os.path.exists(join(proj_dir,
sf)))；
(v) 由 _collect_project_source_files(proj_dir, submodules) 收集到的所有项目源文件的集合是 listed 的
子集。
若其中任一条件不满足，或读取/解析失败，则该函数返回 False。
不会修改任何可变状态。
形式化表示为：ret_val = True ⇔ (read_parse_success(phases_json) ∧ listed ≠ ∅ ∧ (submodules=None ∨
∀sf∈listed: _is_under_submodules(sf, submodules)) ∧ ∀sf∈listed:
os.path.exists(os.path.join(proj_dir, sf)) ∧ _collect_project_source_files(proj_dir, submodules) ⊆
listed)。
```

- 代码证据：

```
第 17 行: if any(not os.path.exists(os.path.join(proj_dir, sf)) for sf in listed):
```


- 触发条件：

存在性检查并未保证文件在 `proj_dir` 之下；绝对路径（或包含 `'..'` 的相对路径）可以指向 `proj_dir` 之外的文件，导致函数在规范要求返回 `False` 时返回 `True`。

Bug validator

- 触发摘要： `phases.json` 中的绝对文件路径通过 `os.path.join()` 绕过了 `os.path.exists()` 检查，从而允许接受 `proj_dir` 之外的文件，而规范要求这些文件必须位于 `proj_dir` 下。
- 探针标准输出：

```
CONFIRMED -- 实际: True | 期望: False | 绝对路径 /etc/hostname (存在) 通过 os.path.join(proj_dir,
'/etc/hostname') = '/etc/hostname' 绕过了存在性检查, 但规范要求返回 False, 因为文件不在 proj_dir 下。
```

INCR-MISMATCH-067 — `src--pipeline_setup-py--_run_generate_phases`

- 人工审计：实现缺陷候选。
- Validator： **confirmed**；尝试次数： **1**。
- 原始函数与 SPEC： `src/pipeline_setup-py/_run_generate_phases.py`。
- Reasoner 结果： `logic_verification_results/src/pipeline_setup-py/_run_generate_phases.json`。
- 详细报告： `src--pipeline_setup-py--_run_generate_phases.md`。
- Probe： `probe_src--pipeline_setup-py--_run_generate_phases.py`。

完整生成 SPEC

```
[SPEC]
单元: src/pipeline_setup-py/_run_generate_phases.py

_run_generate_phases(proj_dir, work_dir, script_dir, is_incremental=False, resume=False,
submodules=None) -> None

前置条件:
- proj_dir、work_dir 和 script_dir 指代已存在的目录路径
- is_incremental 是一个布尔值；当为真时，work_dir 下已有的 phases.json 会被原地更新，而不是从头重新生成
- resume 是一个布尔值
- submodules 为 None 或一个非空可迭代对象，包含相对于 proj_dir 的子目录名称字符串

后置条件:
- 正常返回时，phases.json 存在于 work_dir 下，并符合 phases.json 的模式
- 当 resume 为真并且 phases.json 已经满足管道的完整性条件时，函数返回而不产生或修改任何文件
- 当提供了 submodules 时：phases.json 涵盖 proj_dir 指定子目录下的所有源文件；这些子目录之外的源文件既不会被添加，也不需要存在
- 当 is_incremental 为真时：work_dir 下已存在的有效 phases.json 如果涵盖了所有当前源文件，即使其修改时间戳没有改变，也可以被接受而不做修改
- 如果在可配置的最大重试次数之后仍未生成或确认有效的 phases.json，函数会向 stdout 打印一条诊断信息，标识失败的阶段和跟踪目录，然后调用 sys.exit(1)
- 当非最终尝试产生有效 phases.json 失败时，函数不会调用 sys.exit(1) — 它会在固定的间隔后等待重试
[SPEC]
```

Reasoner 差异

- SPEC 声称：

```
- 正常返回时，phases.json 存在于 work_dir 下，并符合 phases.json 的模式
- 当 resume 为真并且 phases.json 已经满足管道的完整性条件时，函数返回而不产生或修改任何文件
- 当提供了 submodules 时：phases.json 涵盖 proj_dir 指定子目录下的所有源文件；这些子目录之外的源文件既不会被添加，也不需要存在
- 当 is_incremental 为真时：work_dir 下已存在的有效 phases.json 如果涵盖了所有当前源文件，即使其修改时间戳没有改变，也可以被接受而不做修改
- 如果在可配置的最大重试次数之后仍未生成或确认有效的 phases.json，函数会向 stdout 打印一条诊断信息，标识失败的阶段和跟踪目录，然后调用 sys.exit(1)
- 当非最终尝试产生有效 phases.json 失败时，函数不会调用 sys.exit(1) — 它会在固定的间隔后等待重试
```

- 推导实际行为:

自然语言:

在之前的 try 块正常完成 (即 run_opencode_traced 成功返回, prompt、prompt_file、command 已设置, 跟踪事件已记录, attempt = 1, 且 phases.json 可能存在也可能不存在) 的状态下, 从第 81 行起执行后, 三种互斥的结果之一会发生:

1. Break (第 103 行): phase_plan_ready 变为 True。phase_plan_errors 的值被计算为: 如果 phases.json 存在, 则为 $\backslash(\backslash\text{titit}\{\text{phase_plan_schema_errors}\}(\backslash\text{text}\{\text{phases_json}\})\backslash)$, 否则为 ["phases.json is missing"]。变量 failure 和 missing 未设置。外层循环退出; 继续执行该循环之后的代码。
2. Sys.exit (第 130 行): 如果 phase_plan_ready 为 False 且 $\backslash(\backslash\text{text}\{\text{attempt}\} \geq \backslash\text{text}\{\text{OPENCODE_MAX_RETRIES}\}\backslash)$, 程序打印错误信息并以退出码 1 终止。不再存在进一步的程序状态。
3. 重试 (第 123 行): 如果 phase_plan_ready 为 False 且 $\backslash(\backslash\text{text}\{\text{attempt}\} < \backslash\text{text}\{\text{OPENCODE_MAX_RETRIES}\}\backslash)$, 该块打印一条警告, 休眠 10 秒, 然后完成 (块外的代码将递增 attempt 并重新进入循环)。在这种情况下, phase_plan_ready 保持 False, failure 被设置为 "update phases.json" (如果 is_incremental 为真) 或 "produce phases.json" (否则), missing 被相应设置, phase_plan_errors 保留其计算出的列表。

形式逻辑:

令 $\backslash(\backslash\text{text}\{\text{attempt}\} = 1\backslash)$, $\backslash(\backslash\text{text}\{\text{phases_json}\}\backslash)$ 是一个路径, $\backslash(\backslash\text{text}\{\text{is_incremental}\}\backslash)$ 是一个布尔值, $\backslash(\backslash\text{text}\{\text{submodules}\}\backslash)$ 可能是一个非空列表或 None, $\backslash(\backslash\text{text}\{\text{prev_mtime}\}\backslash)$ 是一个浮点数, $\backslash(\backslash\text{text}\{\text{OPENCODE_MAX_RETRIES}\}\backslash)$ 是一个正整数常量。从环境中定义辅助谓词:
 $\backslash(\backslash\text{text}\{\text{exists}\}(\backslash\text{text}\{\text{phases_json}\})\backslash)$ 为真当且仅当该文件存在。
 $\backslash(\backslash\text{text}\{\text{cover}\}(\backslash\text{text}\{\text{phases_json}\}, \backslash\text{text}\{\text{proj_dir}\}, S)\backslash)$ 表示 $\backslash(\backslash\text{textit}\{\backslash\text{phases_cover_current_sources}\}(\backslash\text{text}\{\text{phases_json}\}, \backslash\text{text}\{\text{proj_dir}\}, S)\backslash)$ 。
 $\backslash(\backslash\text{text}\{\text{schema_errs}\}(\backslash\text{text}\{\text{phases_json}\})\backslash)$ 表示 $\backslash(\backslash\text{textit}\{\backslash\text{phase_plan_schema_errors}\}(\backslash\text{text}\{\text{phases_json}\})\backslash)$ 。
 $\backslash(\backslash\text{text}\{\text{mtime}\}(\backslash\text{text}\{\text{phases_json}\})\backslash)$ 是 $\backslash(\text{os.path.getmtime}(\backslash\text{text}\{\text{phases_json}\})\backslash)$ (仅在文件存在时有定义)。

那么后置条件 $\backslash(Q_{\{81-130\}}\backslash)$ 是三个情况的析取, 其中该块之后的状态恰好满足其中之一:

```
\(Q_{\{81-130\}} \equiv
\ ( \ \text{exists } e \in \backslash \ \text{if } \backslash \text{exists}(\backslash\text{text}\{\text{phases\_json}\}) \ \text{then }
\backslash\text{schema\_errs}(\backslash\text{text}\{\text{phases\_json}\}) \ \text{else } [\backslash\text{"phases.json is missing"}] \backslash ) , \backslash
r \in \backslash \{ \backslash\text{True}, \text{False} \} \backslash , \backslash f \in \backslash\text{strings} \backslash , \backslash m \in \backslash\text{strings} \backslash .
\backslash \bigl( e = \backslash\text{if } \backslash \text{exists}(\backslash\text{text}\{\text{phases\_json}\}) \ \text{then } \backslash\text{schema\_errs}
(\backslash\text{text}\{\text{phases\_json}\}) \ \text{else } [\backslash\text{"phases.json is missing"}] \bigr)
\backslash \land \bigl( \backslash\text{if } e = [] \ \land \backslash\text{submodules is not None and } \backslash \backslash\text{submodules}
\neq [] \ \text{then }
\quad r = \backslash\text{cover}(\backslash\text{text}\{\text{phases\_json}\}, \backslash\text{text}\{\text{proj\_dir}\}, \backslash\text{text}\{\text{submodules}\})
\backslash \quad \backslash\text{elif } e = [] \ \land \backslash\text{is\_incremental} \ \text{then }
\quad r = (\backslash\text{exists}(\backslash\text{text}\{\text{phases\_json}\}) \ \land \backslash\text{mtime}(\backslash\text{text}\{\text{phases\_json}\}) \neq
\backslash\text{prev\_mtime}) \ \vee \backslash\text{cover}(\backslash\text{text}\{\text{phases\_json}\}, \backslash\text{text}\{\text{proj\_dir}\})
\backslash \quad \backslash\text{elif } e = [] \ \land \neg \backslash\text{is\_incremental} \ \text{then }
\quad r = \backslash\text{True}
\backslash \quad \backslash\text{else }
\quad r = \backslash\text{False}
\backslash \bigr)
\backslash \land \bigl(
\quad (r = \backslash\text{True} \rightarrow \backslash\text{break 被执行} \backslash \land \backslash\text{phase\_plan\_ready} =
\backslash\text{True} \backslash \land \backslash\text{phase\_plan\_errors} = e)
\backslash \quad \backslash\text{lor } (r = \backslash\text{False} \backslash \land \backslash\text{attempt} \geq \backslash\text{OPENCODE\_MAX\_RETRIES}
\rightarrow \backslash\text{sys.exit}(1) \backslash \text{被调用, 程序终止})
\backslash \quad \backslash\text{lor } (r = \backslash\text{False} \backslash \land \backslash\text{attempt} < \backslash\text{OPENCODE\_MAX\_RETRIES}
\rightarrow
\backslash \quad \backslash\text{if is\_incremental then "update phases.json" else "produce phases.json"}
\backslash \quad \backslash\text{and } m = \backslash\text{if } e \neq [] \ \text{then "phases.json schema validation failed: "}
\backslash \text{mathbin{+}} \backslash\text{join}(e, " "; \backslash)
\backslash \quad \backslash\text{else if is\_incremental then "phases.json was not updated" else
"phases.json missing or invalid"}
\backslash \quad \backslash\text{and } \backslash\text{print/log 被执行, time.sleep(10) 被执行}
\backslash \quad \backslash\text{and } \backslash\text{phase\_plan\_ready} = \backslash\text{False} \backslash \land \backslash\text{phase\_plan\_errors} = e
\backslash \quad \backslash\text{and } \backslash\text{failure} = f \backslash \land \backslash\text{missing} = m
\backslash \quad \backslash\text{and } \backslash\text{该块结束, 在块外继续执行 (例如, 递增 attempt 并重试)})
\backslash \bigr)
\backslash)
```

- 代码证据:

```
第 95 行:         elif is_incremental:
第 96 行:             phase_plan_ready = (
第 97 行:                 os.path.getmtime(phases_json) != prev_mtime
第 98 行:                 or _phases_cover_current_sources(phases_json, proj_dir)
第 99 行:             )
```

- 触发条件:

当 `is_incremental` 为真时，仅因为文件修改时间发生变化，代码就将 `phase_plan_ready` 设为 `True`，即使 `_phases_cover_current_sources` 返回 `False`。规范要求仅在增量模式下，有效的 `phases.json` 涵盖所有当前源文件时才能被接受；仅时间戳变化并不满足该要求。

Bug validator

- 触发概要：当 `is_incremental=True` 且 `phases.json` 的 `mtime` 发生变化，但 `_phases_cover_current_sources` 返回 `False` 时，OR 运算符将 `phase_plan_ready` 设为 `True`，错误地接受了不完整的 `phases.json`。
- Probe 标准输出：

CONFIRMED — 有缺陷的条件 (OR) 产生 `phase_plan_ready=True`，但符合规范的条件 (AND) 产生 `phase_plan_ready=False`。 `phases.json` 已被修改 (`mtime` 已更改)，但仍未覆盖所有源文件 (缺少 `real_source.py`)。OR 错误地将其视为就绪。

错误位置： `src/pipeline_setup.py`，第1003-1007行

当前： `phase_plan_ready = (mtime_changed OR _phases_cover_current_sources(...))`

应为： `phase_plan_ready = (mtime_changed AND _phases_cover_current_sources(...))`

INCR-MISMATCH-068 — `src--pipeline_setup-py--_setup_outputs_complete`

- 人工审计： **SPEC 错误**。
- Validator： **confirmed**；尝试次数： **1**。
- 原始函数与 SPEC： `src/pipeline_setup-py/_setup_outputs_complete.py`。
- Reasoner 结果： `logic_verification_results/src/pipeline_setup-py/_setup_outputs_complete.json`。
- 详细报告： `src--pipeline_setup-py--_setup_outputs_complete.md`。
- Probe： `probe_src--pipeline_setup-py--_setup_outputs_complete.py`。

完整生成 SPEC

```
[SPEC]
单元：src/pipeline_setup-py/_setup_outputs_complete.py

_setup_outputs_complete(work_dir) -> bool

前置条件：
- work_dir 是一个有效的目录路径，该路径下可能存在也可能不存在流水线输出文件。

后置条件：
- 当 phases.json、engine_overview.txt 以及至少一个文件名匹配模式 phase_NN_types.txt (其中 NN 是一位或多位数字) 的文件都作为普通文件存在于 work_dir 下时，返回 True。
- 当三个所需输出类别中任一在 work_dir 中缺失时，返回 False。
[SPEC]
```

Reasoner 差异

- SPEC 声明：

- 当 `phases.json`、`engine_overview.txt` 以及至少一个文件名匹配模式 `phase_NN_types.txt` (其中 `NN` 是一位或多位数字) 的文件都作为普通文件存在于 `work_dir` 下时，返回 `True`。

- 当三个所需输出类别中任一在 `work_dir` 中缺失时，返回 `False`。

- 推导的实际行为：

函数返回后，布尔结果 `R` 满足：`R = True` 当且仅当 (a) 在 `work_dir` 下存在普通文件 “`phases.json`”，其内容能解析为有效的 JSON 并且符合要求的模式，以及 (b) “`engine_overview.txt`” 和至少一个匹配模式 “`phase_NN_types.txt`” (其中 `NN` 是至少一位数字) 的普通文件都存在于 `work_dir` 下。否则 `R = False`。 `work_dir` 路径保持不变。

- 代码证据：

- 触发条件:

如果 `phases.json` 不是有效的 JSON/不符合模式, 代码会返回 `False`, 但规范只要求文件存在。因此, 对于 `phases.json` 存在但 JSON 无效的输入, 代码返回 `False`, 而规范要求返回 `True`。

Bug validator

- 触发摘要: `phases.json` 存在但包含无效 JSON; `_phase_plan_complete` 会检查 JSON 有效性和模式, 但规范只要求文件存在, 因此 `_setup_outputs_complete` 返回 `False` 而不是 `True`。
- Probe 标准输出:

```
CONFIRMED -- actual: False | expected: True
```

src/prompts-py

INCR-MISMATCH-069 — `src--prompts-py--_generate_block_post_condition`

- 人工审计: **SPEC 错误**。
- Validator: **confirmed**; 尝试次数: **1**。
- 原始函数与 SPEC: `src/prompts-py/_generate_block_post_condition.py`。
- Reasoner 结果: `logic_verification_results/src/prompts-py/_generate_block_post_condition.json`。
- 详细报告: `src--prompts-py--_generate_block_post_condition.md`。
- Probe: `probe_src--prompts-py--_generate_block_post_condition.py`。

完整生成 SPEC

```
[SPEC]
单元: src/prompts-py/_generate_block_post_condition.py

_generate_block_post_condition(block, pre_condition, knowledge, language, trace_dir=None,
trace_meta=None)

前置条件:
- block 是一个非空字符串, 包含代码语句, 可能带有 "Line N:" 前缀
- pre_condition 是一个非空字符串, 描述在 block 开始执行之前假定成立的逻辑状态
- knowledge 是一个可选字符串, 提供可包含在提示中的额外上下文信息; 可以为空、None 或其他假值
- language 是一个非空字符串, 标识 block 中代码的源编程语言
- trace_dir 在不为 None 时, 是一个用于持久化跟踪记录的目录路径
- trace_meta 在不为 None 时, 是一个用于跟踪注释的元数据字典

后置条件:
- 返回一个字符串, 描述从给定的前置条件执行 block 后必须成立的最强后置条件, 涵盖 block 的所有执行路径, 包括正常顺序流、提前返回和异常退出
- 当无法根据给定输入确定后置条件时, 返回 None
- 返回的后置条件以自然语言表述, 适合后续对照规范后置条件进行逻辑蕴涵检查
[SPEC]
```

Reasoner 差异

- SPEC 声称:

- 返回一个字符串, 描述从给定的前置条件执行 block 后必须成立的最强后置条件, 涵盖 block 的所有执行路径, 包括正常顺序流、提前返回和异常退出
- 当无法根据给定输入确定后置条件时, 返回 None
- 返回的后置条件以自然语言表述, 适合后续对照规范后置条件进行逻辑蕴涵检查

- 推导的实际行为：

在正常执行（无异常）后，函数返回调用 `_llm_json_call` 产生的值，调用参数如下：`client = _llm_provider_client`，`model = REASONER_POST_CONDITION_MODEL`，`messages =` 包含两个字典的列表（一个系统消息，内容结合了专家角色描述和语言特定语义；一个用户消息，包含编程语言、前置条件、代码块以及可选的知识上下文），`parse_fn = _parse_post_condition_json`，`default = '{"post_condition": "non-empty string"}'`，`trace_dir =` 输入的 `trace_dir`，以及 `trace_meta =` 一个包含 `'purpose'`、`'summary'` 和任何输入的 `trace_meta` 合并后的字典。返回值要么是非空字符串（提取出的后置条件），要么是 `None`（如果 LLM 响应无法解析）。如果 `_llm_json_call` 引发异常，该异常会向上传播，函数异常终止。不会改变任何输入参数，唯一可能的副作用是如果提供了 `trace_dir`，会在其中持久化跟踪记录。

形式化表示：

```
let info_str = if (knowledge 为真值) then "\nAdditional context:\n" + knowledge else "" in
let messages = [
  {role: "system", content: "You are an expert in formal verification of " + language + " programs. Given a " + language + " code block and its pre-condition, generate the post-condition that describes the program state after the code block finishes execution. Cover all execution paths including early returns, exceptions, and normal flow-through. Apply " + language + "-specific semantics (ownership, lifetimes, error handling, etc.) as appropriate. Be precise and unambiguous. Express the post-condition in natural language and formal logic."},
  {role: "user", content: "Programming language: " + language + "\n\nPre-condition:\n" + pre_condition + "\n\nCode block:\n```\n" + language_lower + "\n" + block + "\n```\n" + info_str + "\nGenerate the post-condition. Return only a valid JSON object with this required field: {\"post_condition\": \"...\"}. Do not include Markdown, tags, or prose outside the JSON object."}
] in
let meta = {"purpose": "generate_block_post_condition", "summary": "Generated post-condition for code block"} + (trace_meta or {}) in
(result = _llm_json_call(_llm_provider_client, REASONER_POST_CONDITION_MODEL, messages, _parse_post_condition_json, "{\"post_condition\": \"non-empty string\"}", trace_dir, meta)
  (result String {None})
  (result None result is a non-empty string))

( Exception E : E is raised by _llm_json_call 当前函数引发 E)
```

- 代码证据：

第 28 行： `return _llm_json_call(...)`

- 触发条件：

规范要求当无法确定后置条件时函数返回 `None`，这意味着所有失败模式都应被优雅地处理。代码没有捕获来自 `_llm_json_call` 的异常，因此任何运行时错误（例如网络故障）都会导致未处理的异常，违反了规范。

Bug validator

- 触发摘要：调用 `_generate_block_post_condition`，同时让 `_llm_json_call` 引发 `RuntimeError`（模拟网络故障）——异常未捕获地传播，而不是如规范所要求地返回 `None`。
- Probe 标准输出：

```
CONFIRMED — exception propagated instead of returning None | exception type: RuntimeError |
message: Simulated LLM API failure — network error
```

INCR-MISMATCH-070 — `src--prompts-py--_parse_spec_check_json`

- 人工审计：**SPEC 错误**。
- Validator：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：`src/prompts-py/_parse_spec_check_json.py`。
- Reasoner 结果：`logic_verification_results/src/prompts-py/_parse_spec_check_json.json`。
- 详细报告：`src--prompts-py--_parse_spec_check_json.md`。
- Probe：`probe_src--prompts-py--_parse_spec_check_json.py`。

[SPEC]

单元：src/prompts-py/_parse_spec_check_json.py

`_parse_spec_check_json(response)`

前置条件：

- response 是一个非空字符串

后置条件：

- 如果 response 不是有效的 JSON 文本，则引发 ValueError
- 如果解析得到的 JSON 值不是映射（dict），则引发 ValueError
- 如果解析得到的映射未包含所有必需键："verdict"、"counterexample"、"offending_statements"、"reason"，

则引发 ValueError

- 如果 "verdict" 的值在转换为大写后既不是 "MATCH" 也不是 "MISMATCH"，则引发 ValueError
- 如果 "counterexample" 存在且非空值但不是字符串，则引发 ValueError
- 如果 "offending_statements" 存在且非空值但不是字符串，则引发 ValueError
- 如果 "reason" 不是字符串值，则引发 ValueError
- 对于 "MISMATCH" 判定：如果 counterexample、offending_statements 或 reason 任一为空或仅由空白字符组成，则引发 ValueError；否则返回一个元组（True，

offending_statements_with_leading_trailing_whitespace_removed, reason_with_whitespace_removed, data），其中 data 是已解析的字典，verdict 已转为大写，counterexample 设为去除首尾空白后的值，offending_statements 设为去除首尾空白后的值，reason 设为去除首尾空白后的值

- 对于 "MATCH" 判定：如果 counterexample 或 offending_statements 是非空字符串，则引发 ValueError；否则返回一个元组（False, None, None, data），其中 data 是已解析的字典，verdict 已转为大写，counterexample 设为 None，offending_statements 设为 None，reason 设为去除首尾空白后的值

[SPEC]

Reasoner 差异

- SPEC 声明：

- 如果 response 不是有效的 JSON 文本，则引发 ValueError
- 如果解析得到的 JSON 值不是映射（dict），则引发 ValueError
- 如果解析得到的映射未包含所有必需键："verdict"、"counterexample"、"offending_statements"、"reason"，则引发 ValueError

- 如果 "verdict" 的值在转换为大写后既不是 "MATCH" 也不是 "MISMATCH"，则引发 ValueError
- 如果 "counterexample" 存在且非空值但不是字符串，则引发 ValueError
- 如果 "offending_statements" 存在且非空值但不是字符串，则引发 ValueError
- 如果 "reason" 不是字符串值，则引发 ValueError
- 对于 "MISMATCH" 判定：如果 counterexample、offending_statements 或 reason 任一为空或仅由空白字符组成，则引发 ValueError；否则返回一个元组（True，

offending_statements_with_leading_trailing_whitespace_removed, reason_with_whitespace_removed, data），其中 data 是已解析的字典，verdict 已转为大写，counterexample 设为去除首尾空白后的值，offending_statements 设为去除首尾空白后的值，reason 设为去除首尾空白后的值

- 对于 "MATCH" 判定：如果 counterexample 或 offending_statements 是非空字符串，则引发 ValueError；否则返回一个元组（False, None, None, data），其中 data 是已解析的字典，verdict 已转为大写，counterexample 设为 None，offending_statements 设为 None，reason 设为去除首尾空白后的值

- 推导的实际行为：

给定一个非空字符串 `response`，函数 `\_parse\_spec\_check\_json` 要么引发 `ValueError`，要么返回元组 `(match\_flag, offending, reason, data)`。可能的结果为：

1. 如果 `\_load\_spec\_check\_json(response)` 引发 `JSONDecodeError`，则引发 `ValueError`，消息为 `spec-check response is not valid JSON: {exc}`。
2. 否则，令 `data` 为解析后的 JSON 对象。如果 `data` 不是字典（`not isinstance(data, dict)`），则引发 `ValueError`，消息为 `spec-check JSON must be an object`。
3. 如果 `data` 缺少任一必需字段 `verdict`、`counterexample`、`offending\_statements`、`reason`，则引发 `ValueError`，消息为 `spec-check JSON missing required field(s): ...`，并列缺失的键。
4. 否则，令 `verdict\_raw = data['verdict']`，`counterexample\_raw = data.get('counterexample')`，`offending\_statements\_raw = data.get('offending\_statements')`，`reason\_raw = data.get('reason')`。令 `verdict = verdict\_raw.upper()` if `isinstance(verdict\_raw, str)` else `verdict\_raw`。如果 `verdict not in ('MATCH', 'MISMATCH')`，则引发 `ValueError`，消息为 `spec-check JSON verdict must be MATCH or MISMATCH`。
5. 如果 `counterexample\_raw is not None and not isinstance(counterexample\_raw, str)`，则引发 `ValueError('spec-check JSON field counterexample must be a string or null')`。如果 `offending\_statements\_raw is not None and not isinstance(offending\_statements\_raw, str)`，则引发 `ValueError('spec-check JSON field offending\_statements must be a string or null')`。如果 `not isinstance(reason\_raw, str)`，则引发 `ValueError('spec-check JSON field reason must be a string')`。
6. 更新 `data['verdict'] = verdict`。
 - **MISMATCH 情形**（`verdict == 'MISMATCH'`）：


```
- 定义 `valid = lambda x: isinstance(x, str) and bool(x.strip())`。如果
`counterexample_raw`、`offending_statements_raw`、`reason_raw` 中任意一个未通过 `valid(x)` 检查，则引发
`ValueError('spec-check MISMATCH JSON missing non-empty field(s): ...')`，并列出不通过的字段。
- 否则，设置 `data['counterexample'] = counterexample_raw.strip()`，
`data['offending_statements'] = offending_statements_raw.strip()`，`data['reason'] =
reason_raw.strip()`。返回 `(True, data['offending_statements'], data['reason'], data)`。
- **MATCH 情形** (`verdict == 'MATCH')`：
- 如果 `valid(counterexample_raw)` 或 `valid(offending_statements_raw)` 为真，则引发
`ValueError('spec-check MATCH JSON must not include counterexample or offending_statements')`。
- 否则，设置 `data['counterexample'] = None`，`data['offending_statements'] = None`，
`data['reason'] = reason_raw.strip()`。返回 `(False, None, None, data)`。
```

形式化表述：

```
\[
\begin{aligned}
&\&\text{pre: } \text{response} \in \Sigma^+ \\\
&\&\text{post: } \left( \begin{aligned}
&\&(\neg \text{valid\_json}(\text{response}) \Rightarrow \text{raise ValueError}) \\\
&\&\text{and } (\text{valid\_json}(\text{response}) \text{ and } D = \text{parse}(\text{response}) \text{ and } \neg \text{isinstance}(D, \text{dict}) \\
&\&\Rightarrow \text{raise ValueError}) \\\
&\&\text{and } (\text{isinstance}(D, \text{dict}) \text{ and } \text{keys\_missing}(D) \Rightarrow \text{raise ValueError}) \\\
&\&\text{and } (\text{isinstance}(D, \text{dict}) \text{ and } \neg \text{keys\_missing}(D) \text{ and } \neg \text{valid\_verdict}(D) \Rightarrow \\
&\&\text{raise ValueError}) \\\
&\&\text{and } (\text{valid fields and verdict}) \text{ and } \neg \text{valid\_counterexample\_type}(D) \Rightarrow \\
&\&\text{raise ValueError}) \\\
&\&\text{and } (\text{valid fields and verdict}) \text{ and } \neg \text{valid\_offending\_type}(D) \Rightarrow \\
&\&\text{raise ValueError}) \\\
&\&\text{and } (\text{valid fields and verdict}) \text{ and } \neg \text{valid\_reason\_type}(D) \Rightarrow \\
&\&\text{raise ValueError}) \\\
&\&\text{and } (\text{valid types}) \text{ and } \text{verdict\_up} = \text{MISMATCH} \text{ and } \neg \\
&\&\text{all\_nonempty\_strings}(D) \Rightarrow \text{raise ValueError listing failing fields}) \\\
&\&\text{and } (\text{valid types}) \text{ and } \text{verdict\_up} = \text{MISMATCH} \text{ and } \text{all\_nonempty\_strings}(D) \\
&\&\Rightarrow \text{return } (\text{True}, \text{strip}(O), \text{strip}(R), D') \\\
&\&\text{and } (\text{valid types}) \text{ and } \text{verdict\_up} = \text{MATCH} \text{ and } (\text{nonempty\_str}(C) \text{ lor } \\
&\&\text{nonempty\_str}(O)) \Rightarrow \text{raise ValueError}) \\\
&\&\text{and } (\text{valid types}) \text{ and } \text{verdict\_up} = \text{MATCH} \text{ and } \neg \text{nonempty\_str}(C) \text{ and } \\
&\&\neg \text{nonempty\_str}(O) \Rightarrow \text{return } (\text{False}, \text{None}, \text{None}, D') \\
&\end{aligned} \right)
\end{aligned}
\]
```

其中谓词遵循 Python 代码的语义。

- 代码证据:

```
第 47 行: if _nonempty_string(counterexample) or _nonempty_string(offending_statements):
第 48 行:     raise ValueError(
第 49 行:         "spec-check MATCH JSON must not include counterexample or
offending_statements"
第 50 行:     )
```

- 触发条件:

规范声明：对于 MATCH 判定，如果 counterexample 或 offending_statements 是非空字符串，则引发 ValueError。一个仅包含空白字符的字符串（例如 ' '）是非空字符串（长度 > 0），因此应当引发 ValueError。代码中的 _nonempty_string 检查的是 bool(value.strip())，会将仅含空白字符的字符串视为空，不会引发错误，从而违反了规范。

Bug validator

- 触发摘要：仅含空白字符的 counterexample/offending_statements 字符串绕过了 MATCH 判定下的 ValueError 检查，因为 _nonempty_string 使用了 bool(value.strip()) 而非 len(value) > 0。
- Probe 标准输出：

CONFIRMED — _nonempty_string 将仅含空白字符视为空，但规范要求对任何非空字符串引发 ValueError。实际输出：返回了元组 (False, None, None)（无错误）| 预期：'ValueError'

- 人工审计：实现缺陷候选。
- 验证器：**confirmed**；尝试次数：**1**。
- 原始函数与 SPEC：`src/reasoner-py/_compute_brace_depth_per_line.py`。
- Reasoner 结果：`logic_verification_results/src/reasoner-py/_compute_brace_depth_per_line.json`。
- 详细报告：`src--reasoner-py--_compute_brace_depth_per_line.md`。
- Probe：`probe_src--reasoner-py--_compute_brace_depth_per_line.py`。

完整生成 SPEC

```
[SPEC]
单元：src/reasoner-py/_compute_brace_depth_per_line.py

_compute_brace_depth_per_line(lines) -> list[int]

前置条件：
- lines 是一个非空字符串列表，其中每个字符串包含一行函数体文本，不带 "Line N:" 前缀。
- 输入表示一个括号平衡的函数体，即每个 '}' 都在序列中更早位置有一个匹配的 '{'，因此累计深度永远不会变为负数。

后置条件：
- 返回一个整数列表，其长度等于 len(lines)。
- 对于每个索引 i，该整数是在应用以下排除规则之后，在 lines[0] 到 lines[i] (含) 之间遇到的 '{' 减去 '}' 字符的累计净计数：
    * 双引号字符串字面量内部的字符（由未转义的 '"' 界定，反斜杠转义下一个字符）被排除。
    * 单引号字符串字面量内部的字符（由未转义的 "'" 界定，相同反斜杠转义规则）被排除。
    * 从序列 "///" 出现在任何字面量外部的点开始，该行剩余的所有字符被排除。
    * 从序列 "/*" 出现在任何字面量外部的点开始，该行剩余的所有字符被排除（简化实现不会将块注释传播到后续行）。
- 由于括号平衡的前置条件，返回的每个整数都是非负的。
[SPEC]
```

Reasoner 差异

- SPEC 声称：

```
- 返回一个整数列表，其长度等于 len(lines)。
- 对于每个索引 i，该整数是在应用以下排除规则之后，在 lines[0] 到 lines[i] (含) 之间遇到的 '{' 减去 '}' 字符的累计净计数：
    * 双引号字符串字面量内部的字符（由未转义的 '"' 界定，反斜杠转义下一个字符）被排除。
    * 单引号字符串字面量内部的字符（由未转义的 "'" 界定，相同反斜杠转义规则）被排除。
    * 从序列 "///" 出现在任何字面量外部的点开始，该行剩余的所有字符被排除。
    * 从序列 "/*" 出现在任何字面量外部的点开始，该行剩余的所有字符被排除（简化实现不会将块注释传播到后续行）。
- 由于括号平衡的前置条件，返回的每个整数都是非负的。
```

- 推导的实际行为：

```
该函数返回一个列表 `depths`，满足：
- `len(depths) == len(lines)`。
- 对于每个  $i (0 \leq i < \text{len}(\text{lines}))$ ，令前缀深度  $d = 0$  (如果  $i == 0$ ) 否则为  $\text{depths}[i-1]$ 。然后  $\text{depths}[i]$  是从  $d$  开始扫描  $\text{lines}[i]$  后的深度，扫描过程从左到右处理字符，跳过：
    * 双引号字符串字面量（由 '"' 包围，反斜杠转义序列 '\\' 导致下一个字符被跳过），
    * 单引号字符串字面量（由 "'" 包围，相同的转义规则），
    * 以 `///<` 开头的行注释（跳过该行剩余部分），
    * 以 `/*` 开头的块注释：字符被跳过，直到在同一行找到 `*/`；如果未找到 `*/`，则跳过该行剩余部分。
  在这些跳过的区域之外，每个 `{` 使深度加 1，每个 `}` 使深度减 1。其他任何字符都不改变深度。
- 因为输入表示一个括号平衡的函数体（每个 '}' 在更早位置有匹配的 '{'，累计深度永不为负），处理最后一行后的最终深度为 0，即  $\text{depths}[-1] == 0$ 。形式上，令  $\text{scan}(s, d)$  为将上述规则应用于字符串  $s$  和起始深度  $d$  的结果。那么对于所有  $i$ ： $\text{depths}[i] = \text{scan}(\text{lines}[i], 0 \text{ if } i == 0 \text{ else } \text{depths}[i-1])$ ，且  $\text{depths}[-1] == 0$ 。
```

- 代码证据：

```
第 13 行：            if ch == '"':
第 14 行：                i += 1
第 15 行：                while i < len(line):
第 16 行：                    if line[i] == '\\':
第 17 行：                        i += 2
第 18 行：                    continue
```

```
第 19 行:         if line[i] == '':
第 20 行:             i += 1
第 21 行:             break
第 22 行:             i += 1
第 23 行:         continue
```

- 触发条件:

该规范要求，无论在行边界如何，双引号字符串字面量内部的字符都应被排除（字符串字面量由未转义的双引号界定，没有行限制）。代码的字符串扫描循环（第13-23行）仅扫描到当前行的末尾；如果在该行未找到闭合引号，字符串状态在下一行被重置。这导致多行字符串内部的花括号被错误计数，违反了规范。对于具体输入 `["", "{", "}"]`，代码生成的 `depths` 为 `[0, 1, 0]`，而规范要求为 `[0, 0, 0]`。

Bug validator

- 触发摘要：多行字符串未被追踪：第0行未终止的双引号导致后续行中的花括号被计数，而不是被排除。
- Probe 标准输出：

CONFIRMED — 实际：[0, 1, 0] | 预期：[0, 0, 0]

INCR-MISMATCH-072 — `src--reasoner-py--_split_into_blocks_braced`

- 人工审计：SPEC 错误。
- Validator：confirmed；尝试次数：1。
- 原始函数与 SPEC：`src/reasoner-py/_split_into_blocks_braced.py`。
- Reasoner 结果：`logic_verification_results/src/reasoner-py/_split_into_blocks_braced.json`。
- 详细报告：`src--reasoner-py--_split_into_blocks_braced.md`。
- Probe：`probe_src--reasoner-py--_split_into_blocks_braced.py`。

完整生成 SPEC

```
[SPEC]
单元：src/reasoner-py/reasoner.py

_split_into_blocks_braced(func, language) -> list[str]

前置条件：
- func 是一个非空字符串，包含一个函数体，每一行可以选择性地以 "Line N: " 作为前缀，后面跟源代码文本
- language 是一个标识源代码编程语言的字符串

后置条件：
- 返回一个非空的字符串列表，每个字符串是函数体的一个连续、不重叠的片段，出现的顺序与原函数体中的顺序相同
- 对于使用大括号界定语言的情况：连续返回片段之间的每一个边界都出现在一个其大括号嵌套深度等于函数体第一个有实际含义的大括号层级的深度的行。因此每个片段在括号结构上都是自包含的——它的开始和结束所处的嵌套深度与函数的入口深度相同
- 对于使用缩进来表示语法结构的语言（被该函数归类为类似 Python 的语言）：片段在尊重缩进级别变化的边界处进行分割，每个片段都是一个语法上连续的代码块
- 将所有返回的字符串按顺序使用换行符连接，所得到的结果在去除首尾空白后能够重建原始的函数体文本

[SPEC]
```

Reasoner 差异

- SPEC 声明：

- 返回一个非空的字符串列表，每个字符串是函数体的一个连续、不重叠的片段，出现的顺序与原函数体中的顺序相同
- 对于使用大括号界定语言的情况：连续返回片段之间的每一个边界都出现在一个其大括号嵌套深度等于函数体第一个有实际含义的大括号层级的深度的行。因此每个片段在括号结构上都是自包含的——它的开始和结束所处的嵌套深度与函数的入口深度相同
- 对于使用缩进来表示语法结构的语言（被该函数归类为类似 Python 的语言）：片段在尊重缩进级别变化的边界处进行分割，每个片段都是一个语法上连续的代码块
- 将所有返回的字符串按顺序使用换行符连接，所得到的结果在去除首尾空白后能够重建原始的函数体文本

- 推导实际行为：

该函数返回一个非空字符串列表 `blocks`。设 `preprocessed` 是通过对 `func` 进行如下操作得到的字符串：去除首尾空白，按行分割，移除每行中可能存在的 "Line N: " 前缀（其中 N 为数字），对剩余部分进行左剥离，并将结果使用换行符连接。那么 `'\n'.join(blocks) == preprocessed`。如果 `language.lower()` 在集合 `{"python"}` 中，或者大括号深度入口点为 0（即在逐行计算大括号深度后，未能找到第一个非零深度，且初始深度为零），那么 `blocks` 就是调用 `\_split\_into\_blocks(func)` 的结果。否则，令 `lines` 为预处理后的行列表，`depths = \_compute\_brace\_depth\_per\_line(lines)`，令 `entry` 为 `depths[0]` 如果 `depths[0] > 0`，否则为 `depths` 中第一个正值（如果不存在则为 0）。那么 `blocks` 就是通过以下算法对 `lines` 进行的分割：从索引 `i = 0` 开始。当 `i < len(lines)` 时，如果剩余行数 `len(lines) - i` 小于或等于 `2 \* GRANULARITY`，则将剩余所有行作为一个最终块添加并停止。否则，设置 `target = i + GRANULARITY`；搜索满足 `depths[j] == entry` 的最小索引 `j >= target`。如果存在这样的 `j`，则将行 `i` 到 `j`（包含）作为一个块添加，设置 `i = j + 1`；否则，将剩余所有行从 `i` 开始作为一个最终块添加并停止。任何块都不会是空的。

- 代码证据：

```
第 23 行: entry_depth = depths[0] if total > 0 else 0; 第 24-25 行: entry_depth = next((d for d
in depths if d > 0), 0); 第 40-43 行: for j in range(target, total): if depths[j] ==
entry_depth: split_point = j; break
```

- 触发条件：

该规格要求每个片段的开始和结束所处的大括号嵌套深度等于函数的入口深度。代码将 `entry\_depth` 计算为 `depths[0]` 如果为正数，否则为第一个正深度值。当 `depths[0]` 为 0 并且稍后存在一个正深度时，`entry\_depth` 就变成了那个正深度。然后，分割算法从 `i+GRANULARITY` 开始搜索深度等于 `entry\_depth` 的行。对于第一个块，第 0 行的深度为 0，但该块结束于深度等于 `entry\_depth` 的行。因此第一个块从深度 0 开始，结束于深度 `entry\_depth`，这违反了它应始于入口深度的要求。对于给定的反例（第二行放置左大括号的 C 函数体，`GRANULARITY=1`），代码在行 1 后进行分割，产生第一个块 ["int main()", "{"]，该块从深度 0 开始，结束于深度 1，这相对于入口深度并非句法自包含的。

Bug validator

- 触发总结：第二行放置左大括号的 C 函数体：`entry\_depth=1` 但第一个块从深度 0 开始，违反了片段应始于入口深度的规格要求。
- Probe 标准输出：

```
CONFIRMED — 第一个块从深度 0 开始，而不是入口深度 1。块: 'int main()\n{' | 所有块: ['int main()\n{' , '
return 0;\n}'] | 深度: [0, 1, 1, 0]
```

src/verification-py

INCR-MISMATCH-073 — src--verification-py--streaming_reasoner

- 人工审计：实现缺陷候选。
- Validator: **confirmed**；尝试次数: 1。
- 原始函数与 SPEC: `src/verification-py/streaming_reasoner.py`。
- Reasoner 结果: `logic_verification_results/src/verification-py/streaming_reasoner.json`。
- 详细报告: `src--verification-py--streaming_reasoner.md`。
- Probe: `probe_src--verification-py--streaming_reasoner.py`。

完整生成 SPEC

```
[SPEC]
单元: src/verification-py/streaming_reasoner.py

streaming_reasoner(input_dir, output_dir, file_list=None, proj_dir=None, work_dir=None,
poll_interval=2, spec_procs=None, already_processed=None, resume=False) -> set

前置条件:
- input_dir 是一个目录路径，其中包含已提取的函数文件，
  每个文件预期最终会前置 [SPEC]/[INFO] 块
- output_dir 是一个目录路径，用于写入验证结果 JSON 文件
```

- 若提供了 `file_list`，则必须为可迭代对象，其中包含 `input_dir` 下应当被处理的文件名；省略时，则处理 `input_dir` 下所有不以下划线开头的 `.py` 文件，并（当提供了 `already_processed` 且 `resume` 为 `True` 时）加上 `already_processed` 中存在的文件
- `proj_dir` 为项目根目录，在需要对 `MISMATCH` 进行缺陷验证时使用；可以为 `None`
- `work_dir` 可用于覆盖自动推导的工作目录；可以为 `None`
- `poll_interval` 为轮询新待处理文件之间的等待秒数
- `spec_procs` 为 `subprocess.Popen` 对象（或其列表）的可选引用，这些对象会为 `input_dir` 中的文件生成 `[SPEC]/[INFO]` 标记
- `already_processed` 是一个可选的集合，包含已成功验证过的相对文件路径；传入后，函数不会重新提交这些文件；若 `resume` 为 `True`，它还会将这些文件视为已提交（且已成功）
- `resume` 是一个布尔值；若为 `True` 且提供了 `already_processed`，函数会包含 `already_processed` 中的文件作为已成功验证的文件，并不再重新提交它们
- 函数假定：如果提供了 `spec_procs`，这些进程会随着时间推移为 `input_dir` 下的文件写入 `[SPEC]/[INFO]` 块；函数通过 `is_file_ready()` 来检测已就绪的文件

后置条件：

- 对于 `input_dir` 中（当提供了 `file_list` 时限定为该列表）所有 `is_file_ready()` 返回 `True` 的文件，均会恰好调用一次 `_verify_single_file` 进行提交；已在 `already_processed` 中的就绪文件不会重新提交
- 对每一个提交的文件，都会在 `output_dir` 下写入验证结果 `JSON`，且会镜像 `input_dir` 下的相对路径结构；结果中的 `verdict` 字段取值为以下之一：`"MATCH"`、`"MISMATCH"`、`"ERROR"`、`"SKIPPED"`
- 对于 `verdict` 为 `"MISMATCH"` 且 `proj_dir` 不为 `None` 的每个已验证文件，会通过 `_validate_single_bug` 提交一个缺陷验证任务；验证过程会在 `proj_dir/bug_validation/<bug_id>.result.json` 写入结果 `JSON`，其中 `bug_id` 通过剥离结果 `JSON` 路径中的 `fm_agent/logic_verification_results/` 前缀，移除 `".json"` 并将 `"/"` 替换为 `"--"` 得到
- 每个完成的验证都会打印进度输出：`MATCH` 和 `SKIPPED` 文件使用绿色对勾标记，经确认的缺陷使用红色叉号标记；每一行前缀为 `"[<N>/<total>] <relative_path>: <label>"`
- 当所有预期文件均已验证、所有推理 `future` 均已完成，且没有仍在进行中的验证 `future` 时，循环正常退出
- 当提供了 `spec_procs` 且每个进程都已通过 `_spec_task_done` 退出，但并非所有预期文件都已就绪时：函数会带警告退出。如果完全没有文件收到 `spec`，警告会说明未观察到 `[SPEC]/[INFO]` 标记；否则会报告有多少文件缺少 `spec`，并将每一个列为 `"[pending]"`
- 当发生 `KeyboardInterrupt` 时：所有进行中的推理和验证 `future` 会等待至完成，然后函数返回
- 如果 `proj_dir` 不为 `None`，则在所有处理结束后（正常退出、因 `spec` 停滞而提前退出、或中断），会调用 `_generate_validation_summary`，生成 `proj_dir/bug_validation/summary.json`
- 返回一个文件路径（`input_dir` 内的绝对路径）集合，这些文件已被成功验证；当提供了 `already_processed` 时，返回的集合也包含这些文件（即为其超集）。返回的集合是与 `already_processed` 不同的独立对象（一个副本）

Reasoner 差异

- SPEC 声明：

- 对于 `input_dir` 中（当提供了 `file_list` 时限定为该列表）所有 `is_file_ready()` 返回 `True` 的文件，均会恰好调用一次 `_verify_single_file` 进行提交；已在 `already_processed` 中的就绪文件不会重新提交
- 对每一个提交的文件，都会在 `output_dir` 下写入验证结果 `JSON`，且会镜像 `input_dir` 下的相对路径结构；结果中的 `verdict` 字段取值为以下之一：`"MATCH"`、`"MISMATCH"`、`"ERROR"`、`"SKIPPED"`
- 对于 `verdict` 为 `"MISMATCH"` 且 `proj_dir` 不为 `None` 的每个已验证文件，会通过 `_validate_single_bug` 提交一个缺陷验证任务；验证过程会在 `proj_dir/bug_validation/<bug_id>.result.json` 写入结果 `JSON`，其中 `bug_id` 通过剥离结果 `JSON` 路径中的 `fm_agent/logic_verification_results/` 前缀，移除 `".json"` 并将 `"/"` 替换为 `"--"` 得到
- 每个完成的验证都会打印进度输出：`MATCH` 和 `SKIPPED` 文件使用绿色对勾标记，经确认的缺陷使用红色叉号标记；每一行前缀为 `"[<N>/<total>] <relative_path>: <label>"`
- 当所有预期文件均已验证、所有推理 `future` 均已完成，且没有仍在进行中的验证 `future` 时，循环正常退出
- 当提供了 `spec_procs` 且每个进程都已通过 `_spec_task_done` 退出，但并非所有预期文件都已就绪时：函数会带警告退出。如果完全没有文件收到 `spec`，警告会说明未观察到 `[SPEC]/[INFO]` 标记；否则会报告有多少文件缺少 `spec`，并将每一个列为 `"[pending]"`
- 当发生 `KeyboardInterrupt` 时：所有进行中的推理和验证 `future` 会等待至完成，然后函数返回
- 如果 `proj_dir` 不为 `None`，则在所有处理结束后（正常退出、因 `spec` 停滞而提前退出、或中断），会调用 `_generate_validation_summary`，生成 `proj_dir/bug_validation/summary.json`

- 返回一个文件路径（`input_dir` 内的绝对路径）集合，这些文件已被成功验证；当提供了 `already_processed` 时，返回的集合也包含这些文件（即为其超集）。返回的集合是与 `already_processed` 不同的独立对象（一个副本）

- 推导的实际行为：

`validation_futures = (V0 @ S) \ Done`，其中，当且仅当 `verdict = "MISMATCH"` 且赋值 `validation_futures[vf] = ...` 在出现任何异常之前成功时，
`S = {(vf, (fpath, rel_path, result_json_rel, completed_count))}`，否则 `S = ∅`。
`Done = { f | f ∈ dom(V0 @ S) ∧ f.done() }`。
所有其他变量（`processed`、`submitted`、`reasoning_futures`、`completed_count`、`num_functions`、`executor`、`work_dir`、`proj_dir`、`resume`、`expected_files`、`poll_interval`、`spec_procs`、...）保持其代码块之前的值。
不会有异常传播出该代码块；日志、文件 I/O 和打印输出可能已发生，但不影响抽象状态。

- 代码证据：

```
第 133 行: `if _all_procs is not None and all(_spec_task_done(p) for p in _all_procs):`  
第 134 行: `unready = (expected_files or set()) - processed`  
第 135 行: `if unready and not reasoning_futures and not validation_futures:`
```

- 触发条件：

当所有 `spec_procs` 均已退出时触发的提前退出条件，并不检查未被处理的预期文件是否实际上已就绪（即是否具有所需的标记）。
如果文件在任何处理发生之前就已经包含 [SPEC] 和 [INFO] 标记，同时 `spec_procs` 列表为空或所有进程均已完成，循环会过早中断，导致就绪的文件未被提交。这违反了所有 `is_file_ready()` 返回 `True` 的文件必须恰好被提交给 `_verify_single_file` 一次的要求。

Bug validator

- 触发摘要：当所有 `spec_procs` 完成时的提前退出，不会对剩余的预期文件调用 `is_file_ready()`；就绪的文件被跳过，从未被提交验证。
- Probe 标准输出：

```
待验证的函数数量：2  
WARNING:root：规范生成进程已退出（退出码 [0]），但没有任何文件收到 [SPEC]/[INFO] 标记。  
CONFIRMED — 遗漏的就绪文件：['ready_file.py', 'ready_file2.py'] | 已处理：[]
```

产物位置

- 本轮增量 SPEC 更新列表：[incremental_updated_specs.json](#)。
- 全部逻辑结果：[logic_verification_results/](#)。
- Bug validator 汇总：[bug_validation/summary.json](#)。
- 逐条报告与 probe：[bug_validation/](#)。
- 完整运行日志：[/tmp/fm-agent-incremental-main-20260721.log](#)（位于本机 [/tmp](#)，不在 `target` 仓库内）。

人工复核追加：最可能的实际 Bug

复核口径与总括

本节不把 Bug validator 的 `confirmed` 直接等同于“FM-Agent 源码中存在真实 Bug”。`confirmed` 只说明 `probe` 构造出的输入能够实现偏离同一轮生成的 SPEC；如果 SPEC 擅自扩大了输入域、规定了并不存在的返回格式，或者 `probe` 依赖 `monkeypatch` 制造不可能状态，仍然可能是假阳性。

这里仅保留下列同时具备独立证据的候选：

- 1. 触发输入符合真实调用链，或可能由 LLM、并发写文件、环境缺失等正常运行条件产生；
- 2. 不依赖生成 SPEC，也能从源码注释、路径约束、协议语义或下游无保护调用推出错误后果；
- 3. 能指出具体的错误分支以及可观察影响，而不只是返回值与 SPEC 的文字不同。

按这个标准，73 个 mismatch 中目前最值得处理的是 **7 项：5 项高置信度、1 项中高置信度、1 项中等置信度**。其中只有 `_phase_plan_schema_errors` 的问题明确来自本轮比较区间内的新提交 `29a4c57`；其余大多是旧代码中的潜伏问题，因为“增量验证覆盖到某函数”并不等于“该 Bug 是两个版本之间新引入的”。

优先级	人工结论	对应 mismatch	置信度	核心后果
P0	phase 路径未限制在项目根目录内	INCR-MISMATCH-066	高	可读取项目外源码，并把提取结果写到 <code>fm_agent/</code> 之外
P1	phase schema 校验器接受会令下游崩溃的结构	INCR-MISMATCH-065	高	<code>malformedphases.json</code> 被视为完整，随后触发 <code>KeyError</code> ；非法 UTF-8 还会直接逃逸
P1	增量 phase 计划只要 mtime 改变就可绕过覆盖检查	INCR-MISMATCH-067	高	漏列当前源码的计划仍被接受，后续函数不提取、不验证
P1	codegraph 不可用时先删除旧索引	INCR-MISMATCH-036	高	本可继续使用的 <code>.codegraph/</code> 被不可恢复地丢弃，随后静默降级
P1	brace-depth 扫描没有保存跨行词法状态	INCR-MISMATCH-071	高	注释/续行字符串内的花括号污染分块，进而造成推理误报或漏报
P2	spec 进程结束检查存在最后一次扫描竞态	INCR-MISMATCH-073	中高	已写完 SPEC 的函数可能被留在 <code>pending</code> ，直到下次恢复才处理
P2	Erlang LSP 行索引错误地采用 Python 的广义 <code>splitlines()</code>	INCR-MISMATCH-045	中	含 VT/FF 等控制字符时，ELP range 映射到错误源码片段

ACTUAL-001（P0）— `phases.json` 可用绝对路径或 `..` 逃逸项目目录

- 对应报告：`INCR-MISMATCH-066` / `_phases_cover_current_sources`。
- 主要代码位置：`src/pipeline_setup.py:731-751`、`src/extract.py:715-776`。
- 人工结论：高置信度实际 Bug，而且实际影响比 `validator` 报告的“错误接受绝对路径”更严重。

独立依据如下：

- 1. `workflow` 明确要求 `source_files` 是相对项目根目录的路径（`pipeline_setup.py:879-884`），但 `schema` 校验只检查它是字符串。
- 2. `_phases_cover_current_sources()` 在 `pipeline_setup.py:749` 使用 `os.path.exists(os.path.join(proj_dir, sf))`。当 `sf` 是绝对路径时，`os.path.join()` 会丢弃 `proj_dir`；当 `sf` 是 `../outside.c` 时，也没有做 `realpath/commonpath` 边界校验。

3. 只要 `plan` 同时列出项目当前源码，最后的 `current_sources.issubset(listed)` 就会成立；多出来的项目外路径不会被拒绝。
4. 下游 `run_extraction()` 在 `extract.py:722` 再次直接 `join(proj_dir, src_rel)`，因此会读取项目外文件。更严重的是，`extract.py:742` 用绝对 `src_dir` 拼接 `output_base` 时，`output_base` 同样会被丢弃；随后 `os.makedirs()` 和 `open(..., "w")`（753、775 行）可能在 `fm_agent/` 之外创建目录和函数文件。

真实触发条件不是手工传入异常内部参数，而是 LLM 生成或更新 `phases.json` 时误写绝对路径/父目录路径。该文件本来就是非可信的模型产物，所以仅靠 `prompt` 约束不足以形成安全边界。

建议修复时同时做两层保护：在 `phase` 校验阶段拒绝绝对路径、空路径和任何解析后不位于 `realpath(proj_dir)` 下的路径；在 `run_extraction()` 写输出前再次验证 `src_path` 与 `out_dir` 的 `containment`，避免单点校验被绕过。

ACTUAL-002 (P1) — `_phase_plan_schema_errors()` 的“完整 schema”并不完整

- 对应报告：INCR-MISMATCH-065 / `_phase_plan_schema_errors`。
- 主要代码位置：`src/pipeline_setup.py:624-681`、`src/pipeline_setup.py:247-280`、`src/generate_topdown_layers.py:28-38`。
- 引入范围：校验器由本轮比较区间内的 29a4c57（`fix(setup): retry malformed phase plans`）加入。
- 人工结论：高置信度实际 Bug；`validator` 的非法 UTF-8 例子成立，但还存在更容易到达的结构漏检。

该函数声称验证 `required schema`，却只验证 `phases`、`modules` 和 `source_files` 的容器类型，没有要求：

- 每个 `phase` 必须有可用的 `phase` 编号；
- 每个 `module` 必须有字符串 `name`；
- `source_files` 必须是项目内、受支持且规范化的相对路径。

因此下面这种 JSON 会返回空错误列表，被 `_phase_plan_complete()` 当作完整计划：

```
{"phases": [{"modules": [{"source_files": ["src/a.py"]}]}]}
```

但后续 `_deduplicate_phases()` 会在 `pipeline_setup.py:266` 访问 `p["phase"]`，`generate_topdown_layers._collect_phase_files()` 会在 `generate_topdown_layers.py:37` 访问 `module["name"]`，都会对已经“通过 schema”的数据抛出 `KeyError`。这证明问题不依赖 LLM SPEC，而是校验器与其直接消费者的真实契约不一致。

原 `mismatch` 指出的编码分支也是真问题：`open(..., "r").json.load()` 可能抛 `UnicodeDecodeError`，当前只捕获 `OSError` 和 `JSONDecodeError`。`_run_generate_phases()` 在进入修复重试前就调用该函数，因此一个截断或非 UTF-8 的模型产物可直接中止 `pipeline`，而不是进入“报告 schema 错误并重试”的设计路径。

建议把 `phase`、`name`、路径规范以及必要字段一次性纳入 `schema`；读取异常至少覆盖 `UnicodeError`。校验后仍应让下游使用防御式 `.get()` 或给出带字段路径的明确错误，避免再次形成“校验器说合法、消费者却崩溃”的缝隙。

ACTUAL-003 (P1) — 增量 phase 计划的 mtime 与覆盖完整性被错误地写成 OR

- 对应报告：INCR-MISMATCH-067 / `_run_generate_phases`。
- 代码位置：`src/pipeline_setup.py:997-1007`。
- 人工结论：高置信度实际 Bug。

增量分支当前判定为：

```
phase_plan_ready = (
    os.path.getmtime(phases_json) != prev_mtime
    or _phases_cover_current_sources(phases_json, proj_dir)
)
```

`mtime` 改变只能证明文件被写过，不能证明更新后的计划覆盖当前源码。这里的两项实际上应满足两个不同必要条件：模型确实更新过旧计划，并且更新结果仍完整。当前 **OR** 允许模型只改一个描述、空白或删除某些 `source_files` 后立即通过。

下游 `run_extraction()` 仅遍历 `phases.json` 中列出的 `source_files` (`extract.py:703-715`)，没有一个全项目补扫步骤。因此被漏掉的源文件不会提取函数，也不会进入 `SPEC` 生成和逻辑验证；最终表现可能是“增量运行成功，但一部分改动完全没被验证”。这与增量模式自己的 `prompt`（要求添加新文件、移除已删除文件并保留仍准确条目）直接冲突。

该 **OR** 在比较基点 `95fd9ed...` 前已经存在；本轮之所以发现它，是因为相关 `setup` 函数重新进入了增量验证范围，并不代表它由当前 `merge` 新引入。

建议将“是否发生更新”和“更新后是否完整”分开记录；在需要强制本轮更新时使用 `mtime_changed and covers_current_sources`，若允许原计划本来就完整则显式设计成两个有名称的分支，不能用一个无区分的 **OR** 吞掉覆盖失败。

ACTUAL-004 (P1) — `try_codegraph_init(force=True)` 在确认可重建前删除旧数据库

- 对应报告：`INCR-MISMATCH-036 / try_codegraph_init`。
- 代码位置：`src/languages/codegraph.py:500-537`。
- 真实调用点示例：`src/incremental_reasoner.py:794-799`。
- 人工结论：高置信度实际 **Bug**。

当 `.codegraph/codegraph.db` 已存在且 `force=True` 时，代码在 526 行先执行 `shutil.rmtree(codegraph_dir)`，到 530-536 行才解析并运行可执行文件。如果配置的 `pinned binary` 不存在、`PATH` 中也没有 `codegraph`，`subprocess.run()` 抛 `FileNotFoundError` 后函数静默返回。此时旧数据库已经消失。

这与函数 `docstring` 中“未安装时静默跳过，让 `pipeline` 回退”的行为不等价：真正安全的跳过应不修改已有状态。对增量流程尤其明显，`run_incremental_pipeline` 在重新提取前调用该函数；一次环境变动或安装损坏会把可用索引删掉，随后 `C/C++` 等语言静默切换到精度较低的 `regex` 路径，使本轮函数边界和调用图与上一轮不一致。

建议先验证 `executable` 可执行，再把新 `index` 构建到临时目录；只有 `codegraph init` 成功后才原子替换旧目录。至少也应把旧目录先重命名为备份，并在启动失败或非零退出时恢复。

ACTUAL-005 (P1) — `brace-depth` 计算器没有跨行保存注释/字符串状态

- 对应报告：`INCR-MISMATCH-071 / _compute_brace_depth_per_line`。
- 代码位置：`src/reasoner.py:25-79`、下游使用位置 `src/reasoner.py:82-147`。
- 人工结论：高置信度实际 **Bug**。`validator` 使用“跨行字符串”作为例子略有语言差异，但跨行块注释给出了不依赖 `SPEC` 的标准 `C/C++` 反例。

函数在每一行开始时都重置 `i=0`，没有 `in_block_comment`、`in_string` 等跨行状态。遇到 `/*` 后只扫描当前行；如果本行没有 `*/`，下一行仍按普通代码处理。源码第 71 行的注释声称“`block comment spans lines` 时忽略其中花括号”，但实现实际上没有做到。

例如：

```
int f() {  
    /* comment starts  
    }  
    */  
    return 0;  
}
```

注释里的 `}` 会被 75-76 行当成真实闭括号，深度提前从 1 降到 0，函数末尾再降到 -1。

`_split_into_blocks_braced()` 随后把这份错误 `depth` 数组当成“安全语法边界”，可能在注释或嵌套块附近错误切分。`reasoner()` 又会按这些 `block` 串联 LLM 生成的 `post-condition`，所以影响不是显示层错误，而是会直接增加推理误报与漏报。

建议使用一次跨行词法扫描并显式维护状态；更稳妥的是按语言复用 `tree-sitter/tokenizer`，而不是继续扩充一个同时处理 C、C++、Java、Rust、JavaScript 等语言的简化字符扫描器。

ACTUAL-006 (P2) — spec producer 退出与 watcher 最后扫描之间存在竞态

- 对应报告：INCR-MISMATCH-073 / `streaming_reasoner`。
- 代码位置：扫描在 `src/verification.py:108-132`，提前退出在 `src/verification.py:205-226`。
- 人工结论：中高置信度实际并发 Bug，但 `validator` 的 `mock` 只是确定性模拟；真实窗口很窄。

每轮先扫描文件并调用 `is_file_ready()`，随后才检查所有 `spec_procs` 是否退出。如果 `producer` 恰好在某文件本轮 `readiness` 检查返回 `False` 后写完最后的 `[SPEC]/[INFO]` 标记并退出，那么同一轮的 207-226 行会看到“全部进程已结束、没有 `future`、仍有 `unprocessed` 文件”并直接 `break`。代码没有在退出前对 `unready` 再做一次 `readiness` 扫描。

因此 `probe` 中“第一次 `False`、随后 `True`”并非天然证明 Bug，但它对应一个真实可发生的进程时序，而不是不合法参数。结果是日志把已经 `ready` 的文件报告成“missing specs/pending”，当前 `invocation` 不再验证它；只有外层重试或下次 `--resume` 才可能补上。

建议 `producer` 全部退出后先进行一次最终稳定扫描；只有确认剩余文件仍不 `ready` 才退出。若 `producer` 采用非原子原地写入，还可要求文件 `mtime/size` 连续两个轮询周期稳定后再判断，避免读取半成品。

ACTUAL-007 (P2) — Erlang `_SourceIndex` 把非换行控制字符当成 LSP 新行

- 对应报告：INCR-MISMATCH-045 / `_SourceIndex.build`。
- 代码位置：`src/languages/erlang.py:320-355`，ELP range 的实际消费者在 `src/languages/erlang.py:505`。
- 人工结论：中等置信度实际 Bug；语义明确，但真实 Erlang 源码中出现原始 VT/FF 等字符的概率较低。

Python `str.splitlines()` 不只识别 `\n/\r/\n`，还会把垂直制表符 `\v`、换页符 `\f`、NEL 及若干 Unicode 分隔符当作行界。ELP/LSP 返回的 `line/UTF-16 character` 坐标却是编辑器文档行坐标。源码字符串或注释中若含原始 VT，`_SourceIndex.build()` 会多造一行，后续 `position_to_offset()` 从该点开始全部偏移，最终 `_source_for_range()` 截出错误函数体。

这个问题不像其他 Erlang mismatch 那样依赖人为破坏内部状态：`_SourceIndex` 的输入就是从磁盘读取的源码，控制字符仍是合法字符串内容。只是触发样本罕见，所以优先级低于 `phase` 路径、增量完整性和 `codegraph` 数据丢失。

建议按 LSP 文档实际支持的行结束符显式切分，并保留精确偏移；不要直接使用 Python 的“通用文本行”定义。

为什么其余 **confirmed** 暂未列为实际 Bug

其余多数 mismatch 至少存在一种明显的假阳性来源，例如：给 Pydantic **str** 字段塞入 **bytes**/整数、把 **None** 传给真实调用链始终提供路径的内部函数、把常量 monkeypatch 成 0、要求 **defaultdict** 不具备标准的缺键插入语义、规定所有后端文件必须出现在 **argv** 而忽略文件清单实际通过 **stdin** 传递，或继续使用旧 **SPEC** 认定一对 **[SPEC]/[INFO]** 就应当 **ready**。

这些条目不是断言“绝对无 Bug”，而是当前证据只证明实现不满足生成 **SPEC**，没有形成足以指导修复的真实产品契约。优先修复上面的 7 项后，再针对剩余候选补充真实 **caller**、合法配置模型和独立 **oracle**，会比按 **confirmed** 数量逐项修改更可靠。